

TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG
ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÀI GIẢNG HỌC PHẦN
HỌC SÂU VÀ ỨNG DỤNG

Nhóm tác giả:
PGS.TS. Nguyễn Thị Kim Anh
TS. Đinh Viết Sang
TS. Trịnh Anh Phúc
TS. Trần Việt Trung
TS. Nguyễn Kiên Hiếu
PGS.TS. Phạm Văn Hải
PGS.TS. Thân Quang Khoát

Lời nói đầu

Học sâu (Deep Learning - DL) là một nhánh của lĩnh vực Học máy (Machine Learning). Đây là một trong những lĩnh vực nóng bỏng nhất của kỷ nguyên Trí tuệ nhân tạo (TTNT - AI) hiện nay. Nó đã và đang giúp chúng ta tạo ra rất nhiều đột phá cho loài người, không những trong TTNT mà còn ở nhiều lĩnh vực khác. Các mô hình cỡ rất lớn (chứa hàng tỉ các tham số) đang giúp các tập đoàn lớn (chẳng hạn như Microsoft, Google, OpenAI, Meta) chiếm lĩnh các công nghệ mũi nhọn.

OpenAI mở màn kỷ nguyên của mô hình cực lớn khi giới thiệu GPT-3 vào năm 2020. Mô hình đó là một mạng nơ-ron (neural network) với hơn 175 tỉ tham số, và đã được huấn luyện trên một tập dữ liệu cực lớn. GPT-3 thể hiện khả năng vượt trội ở nhiều tác vụ khác nhau về ngôn ngữ tự nhiên, ví dụ viết, hỏi đáp, hiểu ngôn ngữ, sáng tác thơ, viết thư, ... Vào năm 2022, một loạt hệ thống TTNT ấn tượng đã được sinh ra, ví dụ Midjourney¹, Stable diffusion², DALL-E³, và Imagen. Các hệ thống này có khả năng vẽ những bức tranh cực kỳ sắc nét và chân thực, từ những gợi ý về nội dung bức tranh của người dùng. Chúng đã giúp chúng ta có thể vẽ tranh một cách đơn giản, chỉ bằng cách dùng vài từ khoá ngắn gọn để mô tả về nội dung của bức tranh cần vẽ. Khả năng đó đã giúp Midjourney chiến thắng các nghệ sĩ trong một cuộc thi tranh ở Mỹ.⁴ Đến cuối năm 2022, OpenAI giới thiệu hệ thống ChatGPT với thế giới, một bước phát triển lớn từ GPT-3 trước đó.⁵ ChatGPT là một hệ thống đa năng, có thể làm được nhiều việc khác nhau với chất lượng cao (đôi khi vượt qua con người). Có thể coi sự xuất hiện của ChatGPT là một bước đột phá rất lớn của TTNT và đã đưa loài người vào một kỷ nguyên hoàn toàn mới. Việc xuất hiện ChatGPT đã tạo ra một cú hích lớn, và mở màn cuộc đua giữa các tập đoàn công nghệ và các cường quốc trên thế giới. Năm 2023, Google giới thiệu hệ thống Gemini⁶ có thể học từ đa dạng nguồn dữ liệu để hiểu ngôn ngữ tự nhiên, hiểu

¹<https://www.midjourney.com/>

²<https://stability.ai/>

³<https://openai.com/index/dall-e-3/>

⁴<https://www.nytimes.com/2022/09/02/technology/ai-artificial-intelligence-artists.html>

⁵<https://chatgpt.com/>

⁶<https://blog.google/technology/ai/google-gemini-ai/>

tranh/ảnh/âm thanh, sinh code, sinh nhạc, sinh tóm tắt cho các tài liệu, nhận dạng ảnh, lập luận logic, chứng minh toán học, hỏi đáp, ... Năm 2024, ChatGPT tiếp tục được nâng cấp để có thể làm được nhiều tác vụ hơn với chất lượng cao. Cho đến nay Gemini hay ChatGPT có thể cung cấp dịch vụ cho nhiều ứng dụng khác nhau trong thực tế, giúp con người có thể làm những việc mà trước đây tưởng chừng không thể.

Sự phát triển ngoạn mục của TTNT trong khoảng 10 năm gần đây không thể có nếu thiếu các mô hình mạng nơron. Đây là một lớp mô hình có khả năng rất mạnh ở khả năng biểu diễn. Mỗi mạng nơron có thể chứa nhiều tầng (layers) để giúp nó có khả năng học để biểu diễn những đặc trưng cơ bản cho đến những đặc trưng trừu tượng của dữ liệu. Đó là một lý do mà trong thực tế người ta hay gọi cách học đó là “*Học sâu*” (**Deep Learning**). Đã có khá nhiều phương pháp huấn luyện và kiến trúc hiệu quả được đề xuất trong hơn một thập kỷ vừa qua, giúp mạng nơron có thể sử dụng dễ dàng trong thực tế. Cho đến nay, học sâu đã và đang được sử dụng rất nhiều trong thực tế.

Trong môn học này, người học sẽ được trang bị những kiến thức từ cơ bản cho đến những tiến bộ gần đây của Học sâu. Nội dung chính của môn học bao gồm:

- Chương 1: giới thiệu về học sâu
- Chương 2: giới thiệu về mạng nơron cơ bản
- Chương 3: Mạng tích chập
- Chương 4: Huấn luyện mạng nơron
- Chương 5: Phần cứng và phần mềm cho học sâu
- Chương 6: Một số ứng dụng học sâu trong Thị giác máy
- Chương 7: Mạng hồi quy
- Chương 8: Một số ứng dụng học sâu trong xử lý ngôn ngữ tự nhiên
- Chương 9: Các mô hình sinh
- Chương 10: Một số xu hướng mới trong học sâu

Bản soạn thảo này không thể tránh khỏi sai sót và nhầm lẫn nào đó. Nếu bạn đọc có ý kiến góp ý thì xin hãy gửi về địa chỉ: khoattq at soict dot hust dot edu dot vn

Hà Nội, ngày 15 tháng 09 năm 2025

Nhóm tác giả

Những kí hiệu

Trong cuốn sách này ta dùng những kí hiệu với các ý nghĩa xác định trong bảng dưới đây:

$\mathbb{N}$	Tập hợp số tự nhiên
$\mathbb{R}$	Tập hợp số thực
$[k]$	$\{1, 2, \dots, k\}$, tập hợp các số nguyên dương không vượt quá k
$\emptyset$	Tập hợp rỗng
$\mathbb{R}^n$	Không gian các vectơ thực n chiều
$\mathbf{x}$	Một vectơ trong không gian nhiều chiều
$\ \mathbf{x}\ $	Chuẩn 2 của vectơ $\mathbf{x}$, tức là $\ \mathbf{x}\ = \sqrt{x_1^2 + \dots + x_n^2}$ khi cho trước vectơ $\mathbf{x} = (x_1, \dots, x_n)$
$\mathbf{A}^T$	Chuyển vị của ma trận $\mathbf{A}$
$\langle \mathbf{x}, \mathbf{w} \rangle$	Tích vô hướng của hai vectơ $\mathbf{x}$ và $\mathbf{w}$ có cùng số chiều. Đôi khi ta có dùng ký hiệu $\langle \mathbf{x} \cdot \mathbf{w} \rangle$ hoặc $\mathbf{x}^T \mathbf{w}$.
$\Pr(X = a)$	Xác suất để biến ngẫu nhiên X nhận giá trị cụ thể a
$\mathbb{E}_x[y(x)]$	Kỳ vọng của hàm y theo phân bố của x
n	Số chiều của mỗi vectơ đầu vào
m	Số lượng mẫu trong tập huấn luyện
$\mathcal{D}$	Tập huấn luyện (tập học)
$\mathcal{T}$	Tập kiểm tra (tập kiểm tra)
ℓ hoặc L	Hàm mất mát, hàm lỗi

Mục lục

Lời nói đầu	3
Những kí hiệu	5
Mục lục	6
1 Giới thiệu về học sâu	13
1.1 Thế nào là học sâu?	13
1.2 Vì sao cần học sâu?	14
1.3 Tại sao tại thời điểm bây giờ mới bùng nổ các ứng dụng học sâu ? .	14
1.4 Nhắc lại một số kiến thức về học máy	16
1.4.1 Học có giám sát	16
1.4.2 Tập huấn luyện và tập kiểm tra	17
1.4.3 Hiện tượng học chưa đủ và học quá	17
1.4.4 Độ lệch (bias) và dao động (variance)	18
1.4.5 Bài toán phân loại tuyến tính	19
1.4.6 Hàm mục tiêu	20
1.4.7 Tối ưu hóa hàm mục tiêu	20
1.5 Giới thiệu công cụ và môi trường	21
1.5.1 Google Colab	21
1.5.2 Jupyter Notebook	22
1.5.3 Các thư viện phổ biến	22
1.6 Bài tập	23
2 Giới thiệu về mạng nơ ron nhân tạo	25
2.1 Mạng nơ ron nhân tạo	25
2.1.1 Mạng nơ ron nhân tạo mô phỏng bộ não con người	26

MỤC LỤC	7
2.1.2 Kiến trúc mạng nơ ron	29
2.1.3 So sánh giữa bộ não người và mạng nơ ron	30
2.2 Định lý xấp xỉ tổng quát	30
2.2.1 Ví dụ minh họa	30
2.2.2 Vì sao ta cần mạng nơ ron nhiều lớp ?	32
2.3 Giải thuật học lan truyền ngược	32
2.4 Bài tập	33
3 Giới thiệu về mạng tích chập	35
3.1 Lịch sử về mạng nơ ron tích chập	35
3.2 Mạng nơ ron tích chập	38
3.2.1 Lớp tích chập	39
3.2.2 Lớp giảm mẫu	42
3.2.3 Kiến trúc "bánh mì kẹp"	43
3.3 Một số mạng nơ ron nổi tiếng	44
3.3.1 LeNet-5	44
3.3.2 AlexNet	45
3.4 Tổng kết	45
3.5 Bài tập	46
4 Huấn luyện mạng nơ-ron	48
4.1 Activation functions in Neural Networks	48
4.1.1 Sigmoid	49
4.1.2 Tanh	50
4.1.3 ReLU (Rectified Linear Unit)	52
4.1.4 Leaky ReLU	53
4.1.5 Parametric ReLU	54
4.1.6 Softmax	54
4.1.7 Swish	55
4.2 Tiền xử lý dữ liệu	56
4.3 Khởi tạo trọng số	58
4.3.1 Khởi tạo các trọng số bằng 0 hay hằng số	59

4.3.2	Khởi tạo ngẫu nhiên	59
4.3.3	Khởi tạo Xavier	60
4.3.4	Khởi tạo He	62
4.4	Các kỹ thuật chuẩn hóa	62
4.4.1	Batch Normalization (BN)	62
4.4.2	Chuẩn hóa trọng số (Weight Normalization - WN)	66
4.4.3	Chuẩn hóa tầng (Layer Normalization - LN)	66
4.4.4	Chuẩn hóa mẫu (Instance Normalization - IN)	67
4.4.5	Chuẩn hóa nhóm (Group Normalization - GN)	67
4.5	Chiến lược thay đổi tốc độ học	68
4.6	Các thuật toán tối ưu cho mạng nơ-ron	72
4.6.1	Giảm gradient ngẫu nhiên (Stochastic Gradient Descent – SGD)	73
4.6.2	Giảm gradient ngẫu nhiên với momentum (SGD With Momentum)	74
4.6.3	Adagrad (Adaptive Gradient Descent)	76
4.6.4	RMS Prop (Root Mean Square Propagation)	77
4.6.5	Adam (Adaptive Moment Estimation)	78
4.7	Một số kỹ thuật chống overfitting	79
4.7.1	Dropout: Một kỹ thuật hiệu chỉnh mạnh đối với mạng sâu	79
4.8	Làm giàu dữ liệu (data augmentation)	80
4.9	Kỹ thuật kết hợp nhiều mô hình (model ensemble)	85
4.10	Kỹ thuật học tái sử dụng (transfer learning)	86
5	Phần cứng và phần mềm cho học sâu	93
5.1	Tổng quan	93
5.2	Phần cứng cho Học sâu (Hardware for DL)	94
5.2.1	So sánh CPU, GPU, và TPU	94
5.2.2	Thiết bị biên (Edge Devices)	95
5.3	Các nền tảng lập trình cho Học sâu	96
5.3.1	Sự phát triển và hợp nhất	96
5.3.2	PyTorch (do Facebook phát triển)	96
5.3.3	TensorFlow (do Google phát triển)	97

5.4	Công cụ Tăng tốc và Nén mạng	98
5.4.1	Tại sao cần tối ưu hóa?	98
5.4.2	Các kỹ thuật chính	98
5.4.3	Các bộ công cụ phổ biến	99
6	Một số ứng dụng học sâu trong Thị giác máy	100
6.1	Giới thiệu tổng quan về Thị giác máy tính	100
6.1.1	Thị giác máy tính là gì?	100
6.1.2	Tại sao nên học Thị giác máy tính?	101
6.2	Bài toán Phát hiện đối tượng (Object Detection)	102
6.2.1	Các bài toán cơ bản trong Thị giác máy	102
6.2.2	Ứng dụng của bài toán Phát hiện đối tượng	103
6.3	Mạng đề xuất vùng (Two-Stage Detectors)	103
6.3.1	Tiếp cận quét cửa sổ (Sliding Windows)	103
6.3.2	R-CNN (Region-based Convolutional Neural Network)	103
6.3.3	Fast R-CNN	104
6.3.4	Faster R-CNN	104
6.4	Mạng không đề xuất vùng (One-Stage Detectors)	105
6.4.1	YOLO (You Only Look Once)	105
6.4.2	SSD (Single Shot MultiBox Detector)	106
6.5	Các kiến trúc nâng cao (Đọc thêm)	107
6.5.1	RetinaNet và Focal Loss	107
6.5.2	CornerNet: Phát hiện đối tượng bằng các cặp góc	107
6.5.3	CenterNet: Phát hiện đối tượng như các điểm	108
6.6	Giới thiệu bài toán phân đoạn ảnh	109
6.6.1	Các bài toán thị giác máy	109
6.6.2	Phân đoạn ngữ nghĩa (Semantic Segmentation)	109
6.6.3	Ứng dụng của phân đoạn ảnh	110
6.7	Hướng tiếp cận cho bài toán phân đoạn ảnh	110
6.7.1	Trượt cửa sổ (Sliding Window)	110
6.7.2	Mạng Tích chập hoàn toàn (Fully Convolutional Network - FCN)	111

6.8	Lớp tăng độ phân giải (Upsampling)	111
6.8.1	Lớp Unpooling	111
6.8.2	Lớp Max Unpooling	112
6.8.3	Tích chập chuyển vị (Transposed Convolution)	112
6.9	Hàm mục tiêu (Loss Functions)	112
6.9.1	Hàm mục tiêu dựa trên phân phối (Distribution-based)	112
6.9.2	Hàm mục tiêu dựa trên vùng (Region-based)	113
6.9.3	Hàm mục tiêu kết hợp (Compound Loss)	114
6.9.4	Boundary Loss	114
6.10	Một số mạng phân đoạn ảnh tiêu biểu	114
6.10.1	FCN (Fully Convolutional Network)	114
6.10.2	U-Net	114
6.10.3	U-Net++	115
6.10.4	Mask R-CNN	115
7	Mạng hồi quy	116
7.1	Bài toán dự đoán chuỗi	116
7.2	Mạng hồi quy thông thường	117
7.3	Lan truyền ngược theo thời gian (Backpropagation Through Time - BPTT)	121
7.4	Mạng LSTM và GRU	123
8	Một số ứng dụng học sâu trong xử lý ngôn ngữ tự nhiên	127
8.1	Tổng quan về xử lý ngôn ngữ tự nhiên	127
8.2	Biểu diễn từ và văn bản	129
8.3	Word2vec	129
8.3.1	Mô hình skip-gram	130
8.3.2	Mô hình glove	132
8.4	Giới thiệu về bài toán dịch máy	135
8.4.1	Mô hình Character RNN	135
8.4.2	Giới thiệu về bài toán dịch máy	135
8.4.3	Mô hình dịch máy sâu	136

MỤC LỤC	11
8.4.4 Độ đo BLEU	137
8.4.5 Giải mã	139
8.4.6 Cơ chế chú ý	140
9 Các mô hình sinh	142
9.1 Tổng quan về Mô hình Sinh	142
9.1.1 Mô hình Sinh và Mô hình Phân biệt: Hai triết lý tiếp cận . .	142
9.1.2 Ứng dụng thực tiễn: Vượt ra ngoài ranh giới sáng tạo	143
9.2 Autoencoder: Nghệ thuật của sự nén và tái tạo	143
9.2.1 Kiến trúc Encoder-Decoder và "Nút cổ chai" thông tin	143
9.2.2 Hạn chế của AE trong vai trò mô hình sinh	144
9.3 Variational Autoencoder (VAE): Kiến tạo không gian ẩn có ý nghĩa	145
9.3.1 Giải pháp của VAE: Từ điểm đến phân phối	145
9.3.2 Hàm mất mát của VAE	145
9.3.3 Thủ thuật Đổi tham số (Reparameterization Trick)	146
9.4 Generative Adversarial Networks (GANs): Cuộc đối đầu sáng tạo . .	146
9.4.1 Lý thuyết trò chơi trong Học sâu	146
9.4.2 Những thách thức kinh điển trong huấn luyện GAN	147
9.4.3 Các kiến trúc GAN nâng cao và ứng dụng	147
10 Các xu hướng mới trong học sâu	150
10.1 Transformer	150
10.1.1 Lời mở đầu	150
10.1.2 Tại sao cần cơ chế chú ý	151
10.1.3 Cơ chế tự chú ý	152
10.1.4 Cơ chế mã hóa vị trí thông qua hàm Sin	154
10.1.5 Chú ý đa đầu (Multi-Headed Attention)	155
10.1.6 Kiến trúc Transformer encoder	157
10.1.7 Kiến trúc Transformer decoder	158
10.2 Graph neural network	161
10.2.1 Khái niệm về đồ thị và Dữ liệu dạng đồ thị	161
10.2.2 Mạng nơ ron đồ thị(GNN)	162

10.2.3	Một số thuật ngữ quan trọng	165
10.3	Nhúng đỉnh (Node Embedding)	166
10.3.1	Giới thiệu	166
10.3.2	Khái niệm	166
10.3.3	Các đặc điểm	167
10.3.4	Cách tiếp cận kỹ thuật	167
10.3.5	Shallow Encoder	167
10.3.6	Random Walk	168
10.3.7	Node2Vec	168
10.3.8	Hạn chế của các mô hình Node Embedding	168
10.4	Mạng đồ thị tự mã hoá (GAE)	169
10.4.1	Mạng tự mã hoá (AutoEncoder)	169
10.4.2	Ứng dụng mạng tự mã hoá trong miền đồ thị	169
10.5	GGNN	170
10.6	RGNN	170
10.6.1	Vấn đề đối với dữ liệu dạng tuần tự (Sequence data)	170
10.6.2	Mạng nơ ron hồi quy (RNNs)	171
10.6.3	Mạng nơ ron đồ thị hồi quy	172
10.7	Mạng nơ ron đồ thị tích chập (GCNN)	173
10.7.1	Mạng nơ ron tích chập (CNN)	173
10.7.2	Mạng nơ ron đồ thị tích chập	175
10.8	Thực nghiệm	176
	Danh mục từ khóa	187

Chương 1

Giới thiệu về học sâu

1.1	Thế nào là học sâu?	13
1.2	Vì sao cần học sâu?	14
1.3	Tại sao tại thời điểm bây giờ mới bùng nổ các ứng dụng học sâu ?	14
1.4	Nhắc lại một số kiến thức về học máy	16
1.5	Giới thiệu công cụ và môi trường	21
1.6	Bài tập	23

1.1 Thế nào là học sâu?

Định nghĩa: là một nhánh của học máy sử dụng mạng nơron nhân tạo để trích xuất đặc trưng tự động từ dữ liệu.

Theo định nghĩa này ta phải đặt trong khung cảnh chung của lĩnh vực Trí tuệ Nhân tạo (Artificial Intelligence) rồi nhánh con của nó là Học Máy (Machine Learning) rồi mới đến Học Sâu (Deep Learning). Trong mỗi phần ta cố gắng đưa ra các định nghĩa cô đọng nhưng cũng có tính phân biệt để người đọc có thể hình dung nội dung học phần sắp tới.

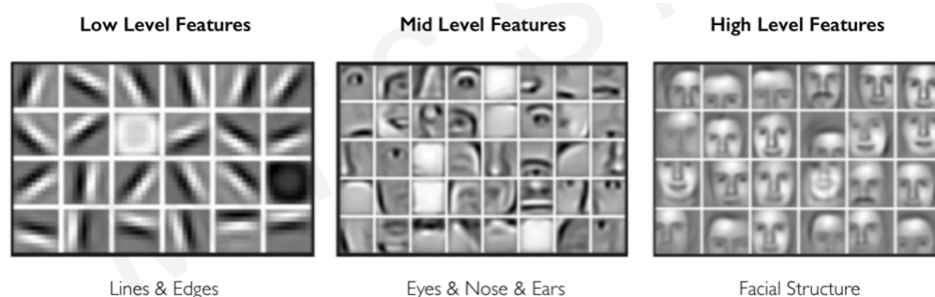
- **Trí tuệ nhân tạo:** mọi kỹ thuật cho phép máy tính có suy nghĩ hoặc hành động thông minh.
 - **Học máy:** khả năng học từ dữ liệu mà không cần đi kèm việc lập trình chính xác.

* **Học sâu:** sử dụng mạng nơ-ron nhân tạo để học từ dữ liệu

1.2 Vì sao cần học sâu?

Đối với đa phần các phương pháp học máy truyền thống, việc trích xuất đặc trưng dữ liệu, đôi khi còn gọi là pha tiền xử lý dữ liệu được tách riêng và làm thủ công. Dù chúng cũng tương đương với các bài toán học máy không giám sát đòi hỏi nhiều chi phí và thời gian. Thêm nữa, việc tiến hành tiền xử lý dữ liệu yêu cầu kinh nghiệm của người thực hiện trích chọn, thường là những người chuyên môn sâu trong lĩnh vực, bài toán cụ thể.

Với học sâu, việc trích chọn đặc trưng được thực hiện tự động đồng thời với quá trình huấn luyện của mạng nơ-ron giúp giảm thiểu các yếu tố chủ quan cũng như sự không tương thích giữa hai bước trích chọn đặc trưng phía trước và huấn luyện sau đó.



Hình 1.1: Đặc trưng trích chọn tự động bởi mạng nơ-ron học sâu trên bộ dữ liệu khuôn mặt. Từ trái qua phải, đặc trưng cấp thấp gồm đường cạnh, đặc trưng trung bình gồm bộ phận khuôn mặt, đặc trưng cấp cao gồm gương mặt hoàn chỉnh.

1.3 Tại sao tại thời điểm bây giờ mới bùng nổ các ứng dụng học sâu ?

Thật đáng ngạc nhiên là hầu hết các lý thuyết về học sâu hiện đại đều đã bắt nguồn từ thập niên 70' thậm chí 50' của thế kỷ trước, bạn đọc có thể nhớ lại 1950 là thời điểm máy tính điện tử Von Newman ra đời. Các lý thuyết về học sâu đa phần có liên quan mật thiết đến hình thành thiết bị điện tử cơ bản đến lý thuyết toán tối ưu hay đơn giản là ngay từ ban đầu mong muốn mô phỏng bộ não con người bằng cách dùng máy tính điện tử đã "manh nha" với các nhà khoa học. Hãy điểm lại lịch

sử các sự kiện liên quan đến học sâu trước khi nó thực sự bùng nổ vào thập niên 20' của thế kỷ 21 này.

- **Năm 1943:** Tế bào nơ ron nhân tạo xuất hiện
McCulloch, W; Pitts, W (1943). "A Logical Calculus of Ideas Immanent in Nervous Activity". Bulletin of Mathematical Biophysics. 5 (4): 115–133
- **Năm 1951:** Phương pháp Stochastic Gradient Descent được giới thiệu lần đầu.
Herbert Robbins. Sutton Monro (1951). "A Stochastic Approximation Method." Ann. Math. Statist. 22 (3) 400 - 407.
- **Năm 1958:** là mạng nơ ron nhân tạo có tham số học
Rosenblatt, F. (1958). "The perceptron: A probabilistic model for information storage and organization in the brain". Psychological Review. 65 (6): 386–408.
- **Năm 1986:** Giải thuật học lan truyền ngược được giới thiệu
Rumelhart, David E.; Hinton, Geoffrey E.; Williams, Ronald J. (1986). "Learning representations by back-propagating errors". Nature. 323 (6088): 533–536
- **Năm 1995:** Mạng nơ ron tích chập giới thiệu giải bài toán nhận diện chữ số viết tay
LeCun, Y.; Bottou, L.; Bengio, Y. & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE. 86(11): 2278 - 2324

... Nhưng phải đến năm 2012 với mạng nơ ron AlexNet giành chiến thắng thuyết phục trong cuộc thi ImageNet, nhà nghiên cứu mới quay lại tập trung các nghiên cứu về mạng nơ ron học sâu. Không những các nhóm nghiên cứu mà còn có cả các công ty công nghệ khổng lồ trong lĩnh vực phần cứng và phần mềm đầu tư cũng như cho ra các sản phẩm công nghệ dựa trên học sâu. Có 3 lý do đồng thời xuất hiện tại thời điểm này khiến cho sự bùng nổ của ứng dụng học sâu

- **Dữ liệu lớn:** Sự xuất hiện của mạng Internet và các mạng xã hội Wikipedia, Facebook, X tương tác cao, sử dụng nhiều hình ảnh e.g. Flickr, Instagram hay nhiều video e.g. YouTube, TikTok tạo nên kho dữ liệu khổng lồ dành cho học sâu
- **Phần cứng:** Kỷ nguyên phần cứng dành cho học sâu được thống trị bởi các hãng sản xuất các đồ họa (Graphical Processing Unit - GPU) nổi tiếng NVidia,

do khả năng song song hóa cao giúp tiết kiệm thời gian huấn luyện cũng như dự đoán mạng nơ ron. Các đồ họa GPU gần như là sự lựa chọn tất yếu cho các doanh nghiệp công ty muốn tham gia thị trường ứng dụng học sâu.

- **Phần mềm:** Đa phần hệ thống các thư viện phần mềm học sâu được "khởi phát" tại các trường đại học, sau đó được hỗ trợ và phát triển bởi các công ty công nghệ lớn dưới dạng mã nguồn mở (dành nghiên cứu) hoặc cung cấp dịch vụ (dành cho công ty).



Hình 1.2: Minh họa ba lý do khiến sự bùng nổ ứng dụng học sâu tại thời điểm hiện tại.

1.4 Nhắc lại một số kiến thức về học máy

Học Sâu là nhánh con của học máy, tại đó mô hình học máy được sử dụng là mạng nơ ron nhân tạo. Các kiến thức học máy (trong trường hợp bạn đọc đã có kiến thức, nhắc lại) giúp cho quá trình học học phần Học Sâu được thuận lợi hơn.

1.4.1 Học có giám sát

Học giám sát (supervised learning), thực hiện học ánh xạ dự đoán $f \in \mathcal{F}$ trong đó $\mathcal{F}$ là lớp các hàm dự đoán

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

Trong đó ký hiệu $\mathcal{X}$ và $\mathcal{Y}$ lần lượt là không gian dữ liệu đầu vào và đầu ra của bài toán học máy giám sát tương ứng, sử dụng tập dữ liệu huấn luyện

$$\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}\}$$

ít nhất thỏa mãn điều kiện

$$\begin{aligned} \hat{f} &\in \mathcal{F} \\ y_i &\approx \hat{f}(x_i) \quad \forall i \end{aligned}$$

Trong pha dự đoán tiếp theo, khi có dữ liệu x mới cần dự đoán, ta sử dụng hàm vừa tìm được

$$\hat{f}(x) = \hat{y} \in \mathcal{Y}$$

để xác định giá trị dự đoán tương ứng.

1.4.2 Tập huấn luyện và tập kiểm tra

Phần trước ta có thể cập đến tập dữ liệu huấn luyện

$$\mathcal{D} = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}\}$$

tại đó việc lựa chọn hàm $\hat{f}$ phải thỏa mãn điều kiện xấp xỉ tốt nhất trên mọi cặp (x_i, y_i) thuộc tập này. Tuy nhiên, trong pha dự đoán, khi dữ liệu x vốn không thuộc tập huấn luyện sẽ không đảm bảo ta có kết quả dự đoán "tốt". Thông thường, ta cần tạo thêm một tập dữ liệu kiểm tra

$$\mathcal{T} = \{(x_j, y_j) \in \mathcal{X} \times \mathcal{Y}\}$$

không tham gia vào quá trình huấn luyện để đánh giá $\hat{f}$ một cách độc lập và khách quan trên tập này.

1.4.3 Hiện tượng học chưa đủ và học quá

Hiện tượng được định nghĩa trực tiếp trên hai tập huấn luyện $\mathcal{D}$ và tập kiểm tra $\mathcal{T}$ như sau

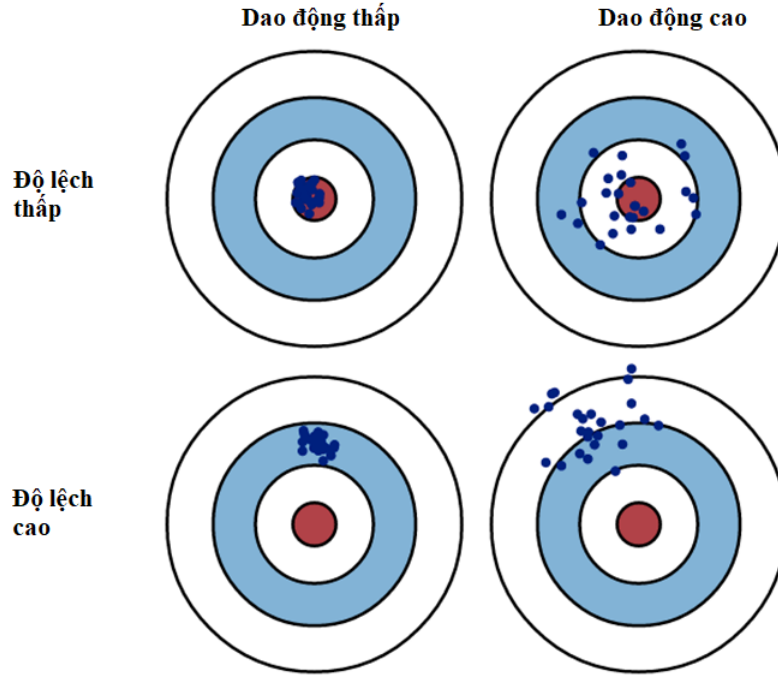
- Học chưa đủ (underfitting) : mô hình quá đơn giản để biểu diễn các tính chất dữ liệu. Sai số cao trên tập huấn luyện $\mathcal{D}$ và cả trên tập kiểm tra $\mathcal{T}$.
- Học quá (overfitting) : mô hình quá phức tạp dẫn tới học cả nhiễu dữ liệu. Sai số thấp trên tập huấn luyện $\mathcal{D}$ và cao trên tập kiểm tra $\mathcal{T}$.

1.4.4 Độ lệch (bias) và dao động (variance)

Độ lệch và dao động liên quan nhiều tính chất thống kê của ánh xạ dự đoán tìm được $\hat{f}$ trên không gian tập dữ liệu $\{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}\}$.

- Độ lệch (bias) : là cao nếu đa phần $\hat{f}(x_i)$ là khác biệt so với y_i , ngược lại, là thấp nếu đa phần $\hat{f}(x_i)$ là giống với y_i .
- Dao động (variance) : là cao nếu đa phần $\hat{f}(x_i)$ phân tán, bất định cao khi dự đoán, ngược lại, là thấp nếu đa phần $\hat{f}(x_i)$ hội tụ, ổn định khi dự đoán.

Học chưa đủ thường có độ lệch cao trên tập huấn luyện $\mathcal{D}$ lẫn tập kiểm tra $\mathcal{T}$ còn Học quá thường có độ lệch thấp trên tập huấn luyện $\mathcal{D}$ và độ lệch cao trên tập kiểm tra $\mathcal{T}$. Tất nhiên, ta luôn mong muốn tạo được $\hat{f}$ là độ lệch thấp và chỉ số dao động cũng thấp (xem minh họa Hình 1.3).



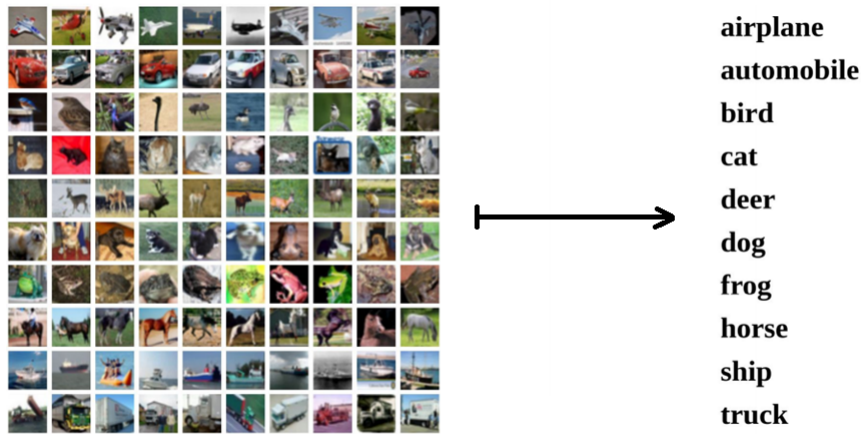
Hình 1.3: Minh họa chỉ số thông kê độ lệch và dao động. Dích đến của bài toán học máy giống như mục đích cung thủ. Nếu $\hat{f}$ tìm được có độ lệch thấp và dao động thấp nghĩa là đa phần dự đoán đúng và ổn định thì quá trình dự đoán thành công. Ngược lại, nếu $\hat{f}$ có độ lệch cao và dao động cao thì quá trình dự đoán không thành công.

1.4.5 Bài toán phân loại tuyến tính

Bài toán phân loại (classification problem) : là dạng bài toán kinh điển có nhiều ứng dụng trong lĩnh vực học máy. Trong trường hợp này ánh xạ dự đoán

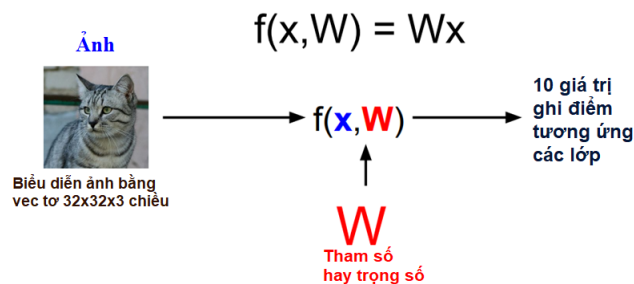
$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

trong đó $\mathcal{Y}$ là tập các lớp phân loại được định nghĩa trước.



Hình 1.4: Minh họa bài toán phân loại 10 lớp bộ dữ liệu CIFAR10, trong đó $\mathcal{Y} = \{\text{airplane, automobile, } \dots, \text{truck}\}$ được định nghĩa trước còn $\mathcal{X}$ là hình ảnh kích thước 32x32 gồm 3 lớp màu RGB. Bộ dữ liệu có 50K ảnh tập huấn luyện và 10K ảnh tập kiểm tra.

Thông thường, ta lựa chọn lớp dự đoán $f \in \mathcal{F}$ là tuyến tính, bài toán trở thành bài toán phân loại tuyến tính.



Hình 1.5: Minh họa lớp phân loại tuyến tính $f \in \mathcal{F}$ của bài toán CIFAR10 ở phía trên.

1.4.6 Hàm mục tiêu

Hàm mục tiêu : đôi khi còn gọi là hàm mất mát được định nghĩa trên tập dữ liệu huấn luyện $\mathcal{D}$ dùng để xác định chất lượng của hàm dự đoán $\hat{f}$ tìm được.

Giả sử chúng ta dùng ánh xạ dự đoán $f(x, W) = Wx$ (xem minh họa Hình 1.5) và lựa chọn được hàm mất mát tương ứng từng mẫu định nghĩa trên tập huấn luyện $\mathcal{D}$. Công thức hàm mục tiêu có thể biểu diễn theo công thức

$$L(W) = \frac{1}{m} \sum_{i=1}^m L_i(f(x_i, W), y_i)$$

trong đó $|\mathcal{D}| = m$.

Tùy thuộc vào hàm mục tiêu khác nhau, ta có giá trị mất mát được tính toán khác nhau trên từng mẫu, từng trường hợp cụ thể. Các bạn đọc cần đọc lại các kiến thức học phần cơ bản, đối chiếu từng tình huống, bài toán cụ thể để tính ra được các giá trị này.

1.4.7 Tối ưu hóa hàm mục tiêu

Để ước lượng tham số W^* làm cực tiểu hóa hàm mục tiêu

$$W^* = \arg \min L(W)$$

Giải thuật tụt dốc gradient (Descent Gradient -DG) là giải thuật tối ưu được dùng cho các bài toán học sâu dựa trên điều kiện tối ưu bậc một

1. $t \leftarrow 0, W^0 \leftarrow$ khởi tạo giá trị ngẫu nhiên
2. **Do**
3. $W^{t+1} \leftarrow W^t - \eta \times \nabla L(W^t)$
4. $t \leftarrow t + 1$
5. **While** (chưa hội tụ)

trong đó $\eta > 0$ gọi là tỷ lệ học còn $\nabla L(W^t)$ là đạo hàm bậc một của hàm mục tiêu tại thời điểm t . Thuật toán hội tụ khi $|W^{t+1} - W^t| < \epsilon$ với ϵ dương nhỏ cho trước.

Khi kích thước bộ dữ liệu huấn luyện m trở nên quá lớn, ta có thể xấp xỉ trên các bộ (batch) kích thước nhỏ thông thường N bằng 32/64 hoặc 128. Người ta gọi

cải tiến này là giải thuật tụt dốc gradient stochastic (Stochastic Descent Gradient - SDG), đa phần các thư viện học sâu đều cung cấp SDG làm mặc định.

1.5 Giới thiệu công cụ và môi trường

Hầu hết các thực viên học sâu đều sử dụng ngôn ngữ lập trình Python để sử dụng cũng như lập trình liên quan ứng dụng học sâu. Khuyến nghị, các bạn đọc nên sớm đọc thêm các tài liệu về ngôn ngữ lập trình chuyên dùng Học Máy laanc Học Sâu này.

1.5.1 Google Colab

Môi trường miễn phí dành cho đào tạo, nghiên cứu viết tắt của Google Colaboratory, là một dịch vụ cung cấp môi trường Jupyter notebook hoàn toàn trực tuyến. Nó cho phép người dùng tạo, chia sẻ và chỉnh sửa các tệp Jupyter notebook một cách dễ dàng mà không cần cài đặt bất kỳ phần mềm nào.

<https://colab.research.google.com/>

có ràng buộc về thời gian cũng như số lần truy cập tùy thuộc vào tài khoản người dùng có miễn phí hay không.

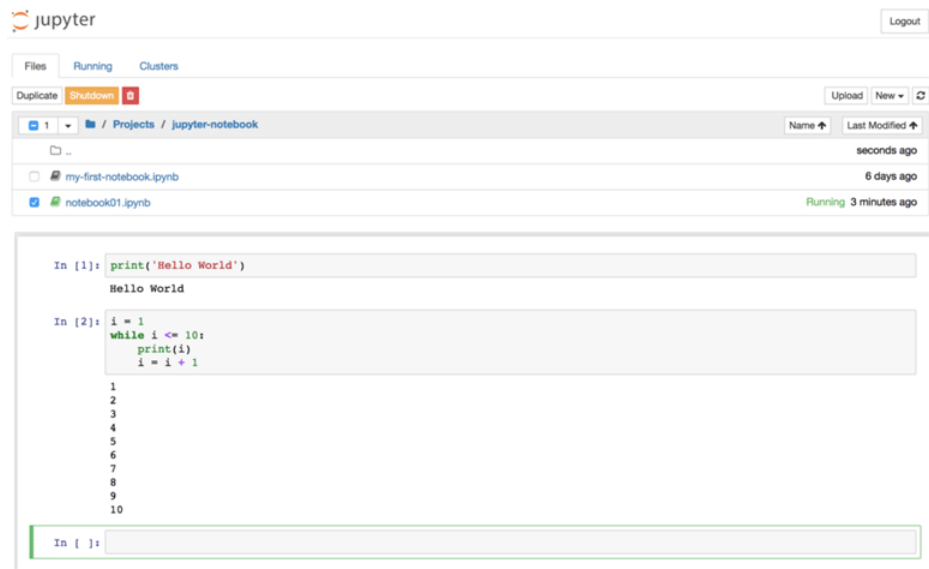


Hình 1.6: Giao diện phòng thí nghiệm Google COLAB

1.5.2 Jupyter Notebook

JupyterLab là một IDE dựa trên công nghệ Web tương tác dùng cho việc soạn thảo, lập trình, phân tích dữ liệu. Sự linh hoạt giao diện cho phép người dùng có thể cấu hình và sắp xếp luồng công việc trong khoa học dữ liệu, tính toán khoa học và học máy.

<https://www.dataquest.io/blog/jupyter-notebook-tutorial/>



Hình 1.7: Giao diện lập trình thực tuyến Jupiter

1.5.3 Các thư viện phổ biến

Có ba thư viện học sâu thường được sử dụng cho giáo dục và nghiên cứu liệt kê dưới đây

- TensorFlow : là thư viện mã nguồn mở dành cho học máy và trí tuệ nhân tạo. Nó có thể dùng với rất nhiều tác vụ nhưng đặc biệt tập trung vào huấn luyện và suy diễn dành cho mạng nơ ron sâu.
- Keras : là thư viện mã nguồn mở cung cấp giao diện Python dành cho mạng nơ ron nhân tạo. Keras phần mềm độc lập, tích hợp trong TensorFlow, và hỗ trợ hơn thế nữa.
- PyTorch : là thư viện học máy dựa theo thư viện Torch, dùng cho các ứng dụng như Thị Giác Máy Tính và Xử lý Ngôn ngữ Tự nhiên, gốc được phát

triển bởi Meta AI.



Hình 1.8: Ba thư viện học sâu phổ biến hiện nay

1.6 Bài tập

Bài tập 1.1. Học Sâu thường được định nghĩa nhánh con của lĩnh vực nào sau đây ?

1. Trí Tuệ Nhân Tạo
2. Thị Giác Máy Tính
3. Xử lý Ngôn Ngữ Tự Nhiên
4. Học Máy

Bài tập 1.2. Đặc điểm nổi bật của Học Sâu (Deep Learning) là gì ?

1. Có khả năng tự tối ưu nhờ nhiều phép biến đổi
2. Có khả năng tận dụng khả năng tính toán GPU
3. Có khả năng biến đổi các mô hình phức tạp về đơn giản
4. Có khả năng trích chọn đặc trưng tự động

Bài tập 1.3. Những lý do chính tạo nên sự bùng nổ của Học Sâu (Deep Learning) là gì ?

1. Dữ liệu lớn

2. Tính toán song song tốc độ cao thường dùng GPU
3. Thư viện phần mềm mã nguồn mở trong lĩnh vực
4. Cả ba lý do trên

Bài tập 1.4. Thư viện nào được dùng phổ biến hiện nay ?

1. Keras
2. TensorFlow
3. PyTorch

Bài tập 1.5. Ý nghĩa của Jupiter Notebook là gì ?

1. là một IDE dựa trên môi trường Web dùng để lập trình Python
2. là một trình soạn thảo văn bản dựa trên môi trường Web
3. là một trình dịch ngôn ngữ Python
4. là một bộ sinh mã máy ngôn ngữ Python

Chương 2

Giới thiệu về mạng nơ ron nhân tạo

2.1	Mạng nơ ron nhân tạo	25
2.2	Định lý xấp xỉ tổng quát	30
2.3	Giải thuật học lan truyền ngược	32
2.4	Bài tập	33

2.1 Mạng nơ ron nhân tạo

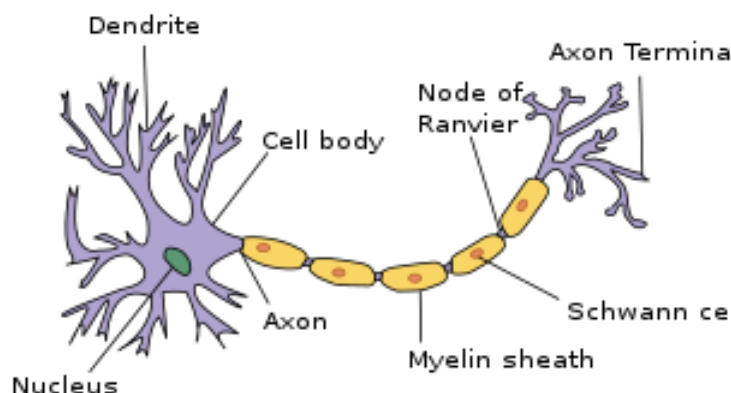
Dù đôi khi người đọc thường nói tắt là mạng nơ ron nhưng chính xác thì cần phải nói là các mạng nơ ron nhân tạo (artificial neural networks). Mục tiêu của mạng nơ ron nhân tạo đương nhiên là dùng để mô phỏng hoạt động mạng nơ ron sinh học tương ứng, hay bộ não người bằng máy tính điện tử. Như đã giới thiệu bài đọc chương 1, ý tưởng mô phỏng này đến từ rất sớm, McCulloch et Pitts, (1943) ¹ là đồng tác giả của tế bào nơ ron nhân tạo đầu tiên. Đến năm 1958, ² Rosenblatt xây dựng hoàn chỉnh mạng Perceptron có tham số học được cùng giải thuật học tương ứng dành cho bài toán phân lớp tuyến tính.

¹McCulloch, W; Pitts, W (1943). "A Logical Calculus of Ideas Immanent in Nervous Activity". Bulletin of Mathematical Biophysics. 5 (4): 115–133

²Rosenblatt, F. (1958). "The perceptron: A probabilistic model for information storage and organization in the brain". Psychological Review. 65 (6): 386–408.

2.1.1 Mạng nơ ron nhân tạo mô phỏng bộ não con người

Bộ não người được cấu tạo bởi khoảng 86 tỷ tế bào thần kinh, gọi là nơ ron (neuron), được kết nối sinh học thông qua tín hiệu điện, tất cả được chứa trong phần trên hộp sọ. Cũng như mọi tế bào sinh học, nó cũng có vòng đời được sinh ra, nuôi dưỡng, hoạt động, bị lão hóa, thải loại. Tất nhiên, đây là bộ phận quan trọng nhất trong cơ thể quyết định mọi hoạt động ý thức và vô thức. Trước hết, hãy quan sát hoạt động của một nơ ron này (xem minh họa Hình 2.1)



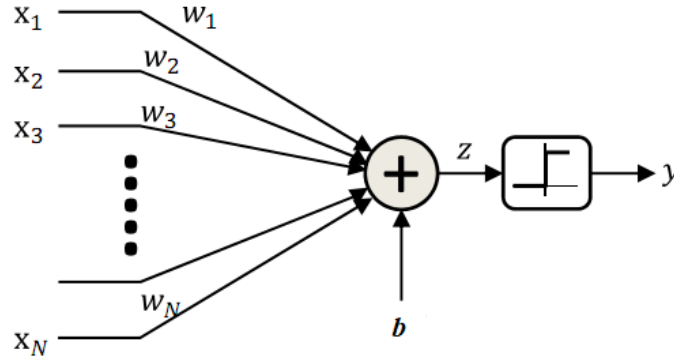
Hình 2.1: Minh họa các thành phần của nơ ron sinh học, nguồn Wiki

Ba thành phần chính của một tế bào thần kinh

- Tua thần kinh (Dendrites) : thu nhận tín hiệu từ các tế bào khác để truyền về thân tế bào
- Thân tế bào (Soma - Cell body) : tổng hợp các tín hiệu có được do tua thần kinh đem về
- Trục truyền (Axon) : truyền tín hiệu điện tổng hợp được từ thân tế bào đến các điểm tiếp xúc (axon termina), để kết nối với tua thần kinh của nơ ron khác.

trạng thái tế bào thần kinh được gọi là kích hoạt, tạo quá trình bắn xung (fire pulse) qua trục truyền, kết nối đến được các tế bào khác. Do là tín hiệu điện nên tốc độ bắn xung hay kích hoạt tế bào thần kinh có thể diễn ra liên tục nhưng có những nghiên cứu cho rằng đa phần tế bào thần kinh trong bộ não không hề có kết nối trong toàn bộ vòng đời của chúng.

Nơ ron nhân tạo được tạo bởi các thành phần điện tử (xem minh họa Hình 2.2) cũng gồm các thành phần như sau



Hình 2.2: Minh họa các thành phần của nơ ron nhân tạo của McCulloch et Pitts, (1943)

- Dãy tín hiệu vào $x_1, x_2, \dots, x_N$ dùng biểu diễn thông tin từ các tế bào thần kinh khác trước khi tiếp xúc tua thần kinh.
- Dãy trọng số $w_1, w_2, \dots, w_N$ kết hợp với tín hiệu vào thể hiện tính chất khác nhau của tua thần kinh khi tổng hợp thông tin về thân tế bào.
- Hàm tổng $\sum$ dùng mô phỏng chức năng tổng hợp thông tin tại thân tế bào cùng đại lượng mức b dùng xác định điều kiện bắn xung

$$z = \sum_{i=1}^N w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

- Trục truyền được mô tả bằng hàm xung *rời rạc* hai giá trị đầu ra $\{0, 1\}$ với giá trị 1 là trạng thái kích hoạt còn giá trị 0 là trạng thái không kích hoạt.

$$y = f(z) = \begin{cases} 1 & \text{nếu } z \geq 0 \\ 0 & \text{ngược lại} \end{cases}$$

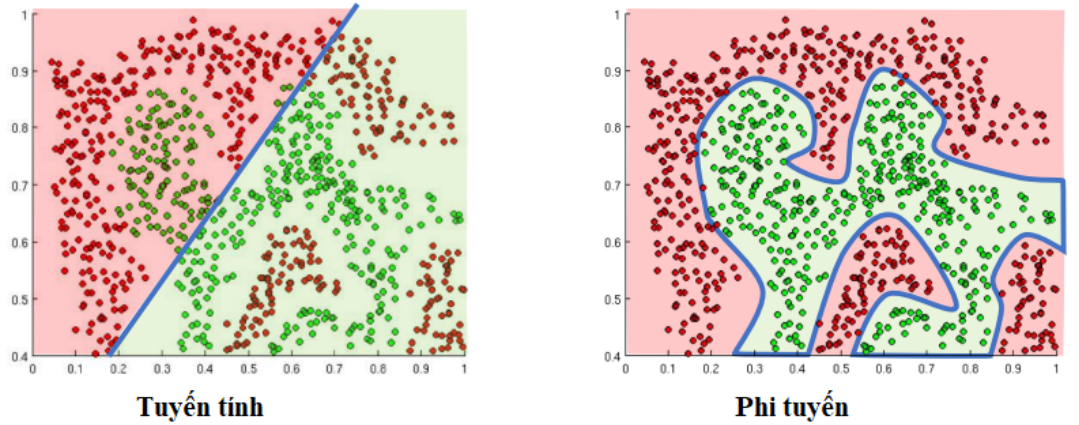
Nơ ron nhân tạo là thiết bị điện tử, được cung cấp năng lượng bởi điện, chịu ảnh hưởng yếu tố điều kiện hoạt động tác động vật lý tương ứng mọi các thiết bị điện tử khác. Hàm xung được gọi là hàm truyền (transfert function) nếu coi mặt chức năng hoặc hàm kích hoạt (activation function) nếu coi mặt mô phỏng. Trong thực tế, để "mềm hóa" tín hiệu ra của hàm này người ta thường dùng các hàm kích hoạt *liên tục* cho trong Bảng 2.1 được liệt kê dưới đây, mục đích đa phần để thông tin kết nối giữa nơ ron luôn thông suốt.

Vai trò của hàm kích hoạt quan trọng trong quá trình chuyển đổi tín hiệu đầu ra của nơ ron, nếu chúng ta dùng hàm tuyến tính (Linear) theo Bảng 2.1 nó sẽ tương

Tên (eng)	Công thức
Linear	$f(z) = z$
Sigmoid	$f(z) = \frac{1}{1+e^{-z}}$
Tanh	$f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
ReLU	$f(z) = \max(0, z)$
Leaky ReLU	$f(z) = \max(0.01z, z)$
Maxout	$f(z) = \max(\mathbf{w}_1^T \mathbf{x}_1 + b_1, \mathbf{w}_2^T \mathbf{x}_2 + b_2)$
ELU	$f(z) = \begin{cases} z & \text{nếu } z \geq 0 \\ \alpha(e^z - 1) & \text{ngược lại} \end{cases}$
ReLU6	$f(z) = \begin{cases} 0 & \text{nếu } z < 0 \\ z & \text{nếu } 0 \leq z < 6 \\ 6 & \text{ngược lại} \end{cases}$
swish	$f(z) = z \cdot \text{sigmoid}(\beta z) = \frac{z}{1+e^{-\beta z}}$
mish	$f(z) = z \cdot \text{tanhsoftplus}(z) = z \cdot \ln(1 + e^z)$

Bảng 2.1: Bảng các hàm kích hoạt hay dùng trong Học Sâu

đương tình huống bài toán phân loại tuyến tính $f \in \mathcal{F}$ đã giới thiệu bài đọc Chương 1, lúc này biên quyết định sẽ chỉ là đường thẳng. Tuy nhiên, trong tình huống biên quyết định không thẳng, ta có thể lựa chọn các hàm truyền phi tuyến (xem minh họa Hình 2.3).



Hình 2.3: Minh họa lựa chọn hàm kích hoạt tuyến tính và phi tuyến trong bài toán phân loại. Dữ liệu được gán hai nhãn đỏ và xanh lá, còn biên quyết định là đường tô màu xanh da trời.

2.1.2 Kiến trúc mạng nơ ron

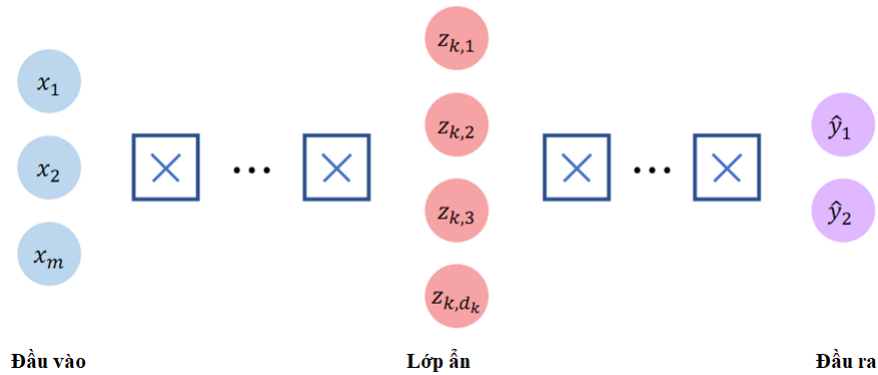
Rõ ràng mạng nơ ron không thể chỉ cấu tạo bởi một nơ ron, chúng ta cần sắp đặt chúng để tạo ánh xạ dự đoán mong muốn tương ứng bài toán cụ thể

$$f : \mathcal{X} \mapsto \mathcal{Y}$$

Việc sắp đặt này sẽ tạo nên cái gọi là *kiến trúc mạng nơ ron*, thể hiện toàn bộ quá trình suy dẫn từ không gian dữ liệu đầu vào $\mathcal{X}$ sang không gian dữ liệu đầu ra $\mathcal{Y}$. Thông thường để xây dựng kiến trúc mạng nơ ron ta dùng hai cách thức sắp đặt chính như sau

- Xếp chồng các nơ ron lên nhau để tạo nên lớp ẩn (hidden layer) hay đôi khi gọi là tầng ẩn
- Các tầng ẩn sẽ được sắp đặt tiếp nối nhau từ đầu vào $\mathcal{X}$ hướng đến (feed-forward) đầu ra $\mathcal{Y}$

Như vậy, dữ liệu đầu vào thông thường sẽ kết nối với các nơ ron tầng ẩn đầu tiên và giá trị hàm truyền của các nơ ron tầng cuối sẽ tạo ra giá trị đầu ra tương ứng. Mạng gồm nhiều nơ ron xếp chồng và có kết nối đầy đủ giữa hai tầng liên tiếp được gọi là mạng nơ ron perceptron đa lớp (multi-layer perceptron)



Hình 2.4: Minh họa kiến trúc mạng nơ ron perceptron đa lớp. Đầu vào $x_1, \dots, x_m$ kết nối đầy đủ với tầng ẩn đầu tiên còn thông tin ra $\hat{y}_1, \hat{y}_2$ là của lớp nơ ron cuối cùng. Lớp ẩn gồm các nơ ron xếp chồng đại diện bởi các giá trị tổng $z_{k,1}, z_{k,2}, \dots, z_{k,d_k}$ của chúng.

2.1.3 So sánh giữa bộ não người và mạng nơ ron

Đến thời điểm này, về cơ bản bạn đọc đã hình dung được nội dung học phần liên quan đến sử dụng mạng nơ ron nhân tạo dùng để xây dựng ánh xạ dự đoán dành cho bài toán học máy. Tuy nhiên, để chắc chắn hơn bạn có thể xem bảng so sánh lần nữa giữa mạng lưới thẳng kinh tạo nên bộ não con người và mạng nơ ron nhân tạo.

So sánh	Bộ não người	Mạng nơ ron
Thành phần	Tế bào sinh học	Thiết bị điện tử
Kiến trúc kết nối	Phức tạp	Đơn giản
Ý nghĩa	Điều khiển mọi hoạt động	Mô hình học máy tiên tiến

Bảng 2.2: Mọi vài điểm so sánh giữa mạng nơ ron nhân tạo và bộ não người.

Nhìn chung, sẽ còn một chặng đường rất "dài" để con người có thể tạo ra bộ não điện tử hoàn chỉnh có khả năng tương đương với bộ não sinh học.

2.2 Định lý xấp xỉ tổng quát

Định lý xấp xỉ tổng quát đóng vai trò *lý thuyết quan trọng* trong quá trình chuyển hóa lớp các bài toán tuyến tính như phân loại, hồi quy sang giải quyết lớp bài toán học máy phi tuyến khi sử dụng mạng nơ ron đa lớp ẩn. Định lý ³ được phát biểu như sau

Định lý xấp xỉ hàm tổng quát - Universal Function Approximators :
 Một mạng nơ ron có từ hai lớp ẩn trở lên với số lượng nơ ron đủ lớn có thể xấp xỉ bất kỳ hàm liên tục với độ chính xác tùy ý.

2.2.1 Ví dụ minh họa

Ý nghĩa của định lý cho phép chúng ta có thể xây dựng được bài toán học máy phi tuyến. Chẳng hạn, ta sẽ sử dụng mạng nơ ron một lớp ẩn gồm 6 nơ ron xếp chồng dùng hàm truyền ReLU (tham khảo thêm Bảng 2.1) có các tham số trọng như sau

³Hornik, Kurt; Stinchcombe, Maxwell; White, Halbert (January 1989). "Multilayer feedforward networks are universal approximators". *Neural Networks*. 2 (5): 359–366.

$$n_1(x) = \text{ReLU}(-5x - 7.7)$$

$$n_2(x) = \text{ReLU}(-1.2x - 1.3)$$

$$n_3(x) = \text{ReLU}(1.2x + 1)$$

$$n_4(x) = \text{ReLU}(-1.2x - 0.2)$$

$$n_5(x) = \text{ReLU}(2x - 1.1)$$

$$n_6(x) = \text{ReLU}(5x - 5)$$

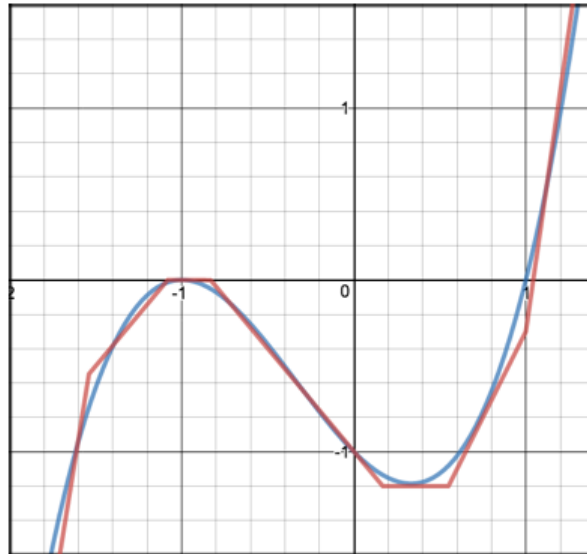
và giá trị xấp xỉ $Z(x)$ được tính bởi công thức

$$Z(x) = -n_1(x) - n_2(x) - n_3(x) + n_4(x) + n_5(x) + n_6(x)$$

dùng để xấp xỉ hàm liên tục $f(x)$, phi tuyến bậc ba sau đây

$$f(x) = (x + 1)^2(x - 1) = x^3 + x^2 - x - 1$$

Xem chi tiết kết quả theo Hình 2.5



Hình 2.5: Minh họa kết quả xấp xỉ của mạng nơ ron một lớp ẩn gồm sáu nơ ron. Đường thẳng $f(x)$ được vẽ màu xanh dương trong khi giá trị xấp xỉ $Z(x)$ được vẽ bởi các đoạn thẳng màu đỏ. Vẽ trong phạm vi $x \in [-2, 1.4]$ trên cùng hệ trục tọa độ.

Rõ ràng, nếu ta tăng số lượng nơ ron tương đương với việc tăng số đường thẳng xấp xỉ $Z(x)$ màu đỏ lên thì độ lỗi, phần chênh lệch giữa đường con màu xanh dương sẽ có thể giảm với ϵ nhỏ tùy ý. Nhắc lại, hàm bậc ba $f(x)$ là hàm liên tục mà ta có thể dùng mạng nơ ron xấp xỉ tuân theo định lý xấp xỉ hàm tổng quát.

2.2.2 Vì sao ta cần mạng nơ ron nhiều lớp ?

Thông qua ví dụ trên ta có các luận điểm ủng hộ cho mạng nơ ron nhiều lớp (deep networks) càng nhiều lớp thì mạng càng sâu.

- Mạng nơ-ron nhiều lớp (thậm chí chỉ cần duy nhất một lớp ẩn!) là hàm xấp xỉ tổng quát
- Mạng nơ-ron có thể biểu diễn hàm liên tục bất kỳ nếu nó đủ rộng (số nơ-ron trong một lớp đủ nhiều), đủ sâu (số lớp đủ lớn).
- Nếu muốn giảm độ sâu của mạng trong nhiều trường hợp sẽ phải bù lại bằng cách tăng chiều rộng lên lũy thừa lần! Hay nói cách khác, mạng nơ-ron một lớp ẩn có thể cần tới số lượng nơ-ron cao gấp lũy thừa lần so với một mạng nhiều tầng.
- Mạng nhiều lớp cần số lượng nơ-ron ít hơn rất nhiều so với các mạng nông (shallow networks) để cùng biểu diễn một hàm số giống nhau

Theo đó với mạng nơ ron sâu thì có độ rộng thấp hơn dẫn đến số lượng nơ ron ít hơn khi xấp xỉ cùng một hàm số liên tục, khi so sánh với các mạng nơ ron nông. Dẫn đến, mạng nơ ron nhiều lớp có nhiều giá trị hơn trong đa phần các trường hợp.

2.3 Giải thuật học lan truyền ngược

Về cơ bản mạng nơ ron là một mô hình học máy tiên tiến để học ánh xạ dự đoán $f(x, W)$, kết hợp các trọng số W theo đó ta có thể thực hiện ước lượng tham số, xem lại phần lý thuyết bài đọc Chương 1

$$W^* = \arg \min L(W) = \arg \min \frac{1}{m} \sum_{i=1}^m L_i(f(x_i, W), y_i)$$

Để tìm được trọng số tối ưu W^* khi cực tiểu hóa hàm mất mát trên, ta dùng giải thuật tụt dốc gradient (Descent Gradient)

Giải thuật tụt dốc gradient

1. Khởi tạo trọng số W từ phân bố chuẩn $\mathcal{N}(0, \sigma^2)$

2. **Repeat**

3. Tính đạo hàm $\frac{\partial L(W)}{\partial W}$

4. Cập nhật $W \leftarrow W - \eta \frac{\partial L(W)}{\partial W}$

5. **Until** (hội tụ)

6. **Return** W

Về cơ bản quá trình lặp Bước 3. và Bước 4. được thực hiện theo hai hướng

- **Hướng tiến** (Forward) : cần thực hiện tính toán hàm mất mát $L(W)$ tại thời điểm hiện tại.
- **Hướng lùi** (Backward) : cần tính đạo hàm lan truyền ngược (backpropagation) từ lớp ra về lại lớp vào $\frac{\partial L(W)}{\partial W}$ trước khi cập nhật lại chúng.

Các bạn đọc cần có những ví dụ cụ thể để hiểu về phương pháp lan truyền ngược, điều cần nhất là hiểu được cách thức hoạt động, suy dẫn tiến của mạng nơ ron, trước khi lan truyền ngược để tính được đạo hàm của hàm mất mát lại mỗi bước lặp.

2.4 Bài tập

Bài tập 2.1. Định nghĩa nào dưới đây là gần nhất với nơ ron nhân tạo ?

1. là mô phỏng tế bào thần kinh sinh học bằng thiết bị điện tử
2. là thành phần điện có trong tế bào thần kinh
3. là thiết bị điện tử được dùng trong lĩnh vực y sinh học
4. là thân tế bào thần kinh khi đang kích hoạt

Bài tập 2.2. Thành phần thân tế bào (Soma) được mô phỏng tương ứng với thành phần nào ?

1. là hàm truyền hay hàm kích hoạt
2. là thành phần tổng hợp

3. là các trọng số w trong công thức
4. là cả ba đáp án trên

Bài tập 2.3. Định nghĩa nào là gần nhất về Kiến trúc mạng nơ ron ?

1. là cách sắp đặt và kết nối của các nơ ron trong mạng
2. là số lượng các tầng có trong mạng học sâu
3. là giá trị các trọng số khi ước lượng
4. là độ rộng của tầng trong mạng

Bài tập 2.4. Ý nghĩa chính của định lý xấp xỉ hàm tổng quát ?

1. là có thể dùng mạng nơ ron xấp xỉ mọi hàm phi tuyến
2. là dùng mạng nơ ron nhiều tầng sâu có giá trị hơn
3. là cần lũy thừa số lần chiều rộng khi giảm độ sâu của mạng
4. là tất cả lý do trên

Bài tập 2.5. Giải thuật Descent Gradient có hai hướng cập nhật, hướng tiến dùng để ?

1. tính toán hàm mất mát tại thời điểm trước đó
2. tính toán đạo hàm của hàm mất mát tại thời điểm hiện tại
3. cập nhật các tham số tại thời điểm tiếp theo
4. cập nhật các tham số tại thời điểm hiện tại

Chương 3

Giới thiệu về mạng tích chập

3.1	Lịch sử về mạng nơ ron tích chập	35
3.2	Mạng nơ ron tích chập	38
3.3	Một số mạng nơ ron nổi tiếng	44
3.4	Tổng kết	45
3.5	Bài tập	46

3.1 Lịch sử về mạng nơ ron tích chập

Chúng ta sẽ điểm qua một chút lịch sử hình thành của mạng nơ ron tích chập theo các mốc thời gian quan trọng

- **Năm 1980** Mạng nơ ron tích chập được đề cập lần đầu bởi ¹ Fukushima theo đó tác giả đề nghị mô hình hóa bằng một mô hình dạng "bánh mỳ kẹp - sandwich" gồm các tế bào đơn giản Simple-cells kẹp với các tế bào phức tạp Complex-cells dựa trên các nghiên cứu sinh học liên quan đến vùng thị giác máy tính. Những nghiên cứu thị giác máy tính trước đó cũng đã đưa đến khái niệm về hệ thống các kết nối thần kinh từ võng mạc đến khu thần kinh thị giác gồm các kết nối thay đổi được (modifiable synapses) và không thay đổi được (unmodifiable synapses).

Kiến trúc Fukushima về cơ bản được dùng rất nhiều cho các ứng dụng học sâu đặc biệt liên quan đến lĩnh vực thị giá máy tính như hình ảnh, video ...

¹Fukushima, K. (1980). Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. Biological Cybernetics, 36, 193-202.

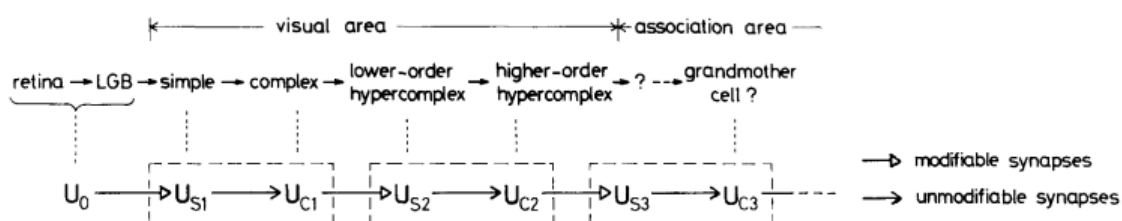
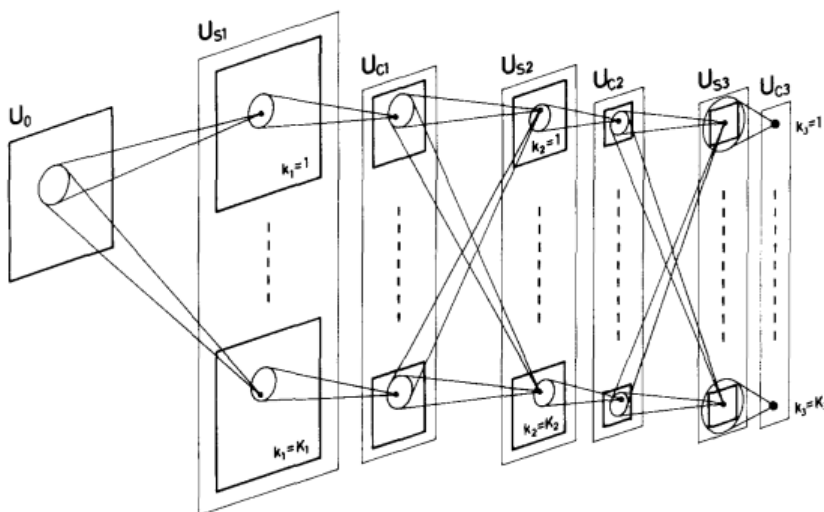


Fig. 1. Correspondence between the hierarchy model by Hubel and Wiesel, and the neural network of the neocognitron



Hình 3.1: Minh họa mạng nơ ron tích chập trích dẫn trong bài báo của Fukushima, theo đó các lớp tế bào Simple-cell và Complex-cell được kẹp thành cặp khi mô hình hóa khu thần kinh thị giác máy tính.

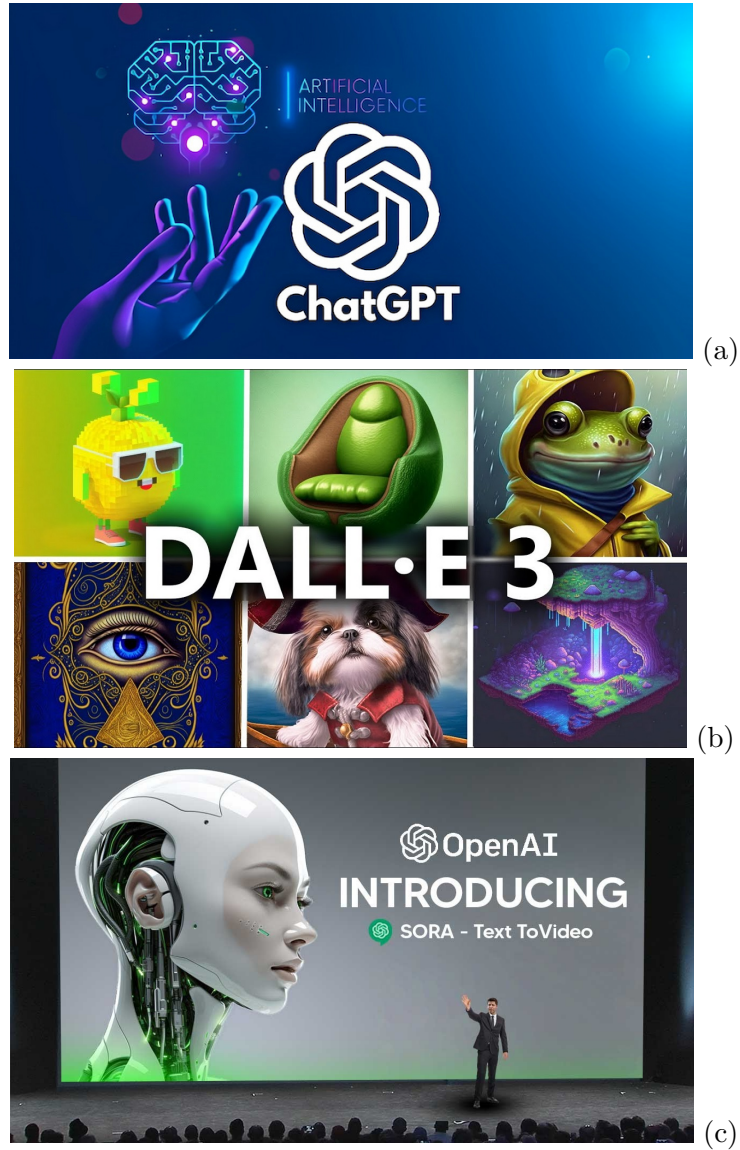
- **Năm 1989** LeCun, Y. cùng các đồng nghiệp ² đã dựa trên nghiên cứu của Fukushima để tạo nên một mạng nơ ron hoàn chỉnh giải quyết bài toán nhận dạng văn bản.

Mạng LeNet-5 đóng vai trò quan trọng về mặt lý thuyết lẫn thực hành, nó sử dụng giải thuật lan truyền ngược để học cũng như bộ dữ liệu chữ số viết tay MNIST khi thực nghiệm. Hầu hết các mạng nơ ron tích chập hiện nay đều chịu ít nhiều ảnh hưởng của mạng LeNet-5 về hướng tiếp cận và thiết kế kiến trúc của mạng nơ ron tích chập.

- **Năm 2012** Krizhevsky, Alex cùng các đồng nghiệp ³ đã dành chiến thắng

²LeCun, Y.; Bottou, L.; Bengio, Y. & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE. 86(11): 2278 - 2324

³Krizhevsky, Alex; Sutskever, Ilya; Hinton, Geoffrey E. (2017). "ImageNet classification with deep convolutional neural networks" (PDF). Communications of the ACM. 60 (6): 84-90.



Bảng 3.1: Bộ ba sản phẩm đến từ OpenAI. Theo thứ tự từ trên xuống dưới (a) ChatGPT ứng dụng văn bản chatbot (b) Dall-E sinh ảnh từ mô tả còn (c) Sora sinh đoạn video từ văn bản.

3.2 Mạng nơ ron tích chập

Mạng nơ ron tích chập hướng theo kiến trúc của Fukushima gồm các lớp nơ ron mới

- Lớp tích chập (Simple-Cells) : chứa các nơ ron tích chập kết nối địa phương

với các nơ ron tích chập của lớp trước đó, chứa tham số, tương ứng kết nối thay đổi được.

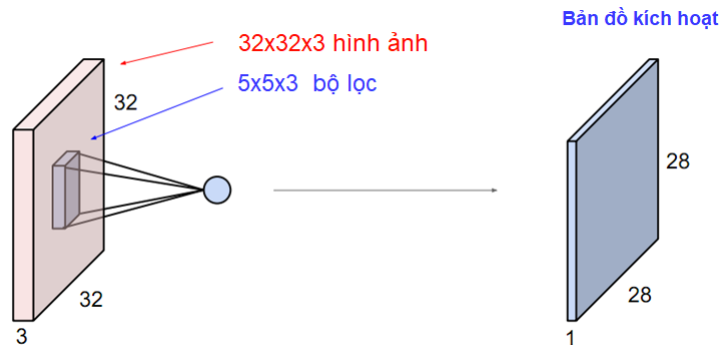
- Lớp giảm mẫu (Complex-Cells) : chứa các nơ ron giảm mẫu cũng kết nối địa phương với các nơ ron tích chập của lớp trước đó, không tham số, tương ứng các kết nối không thay đổi được.

3.2.1 Lớp tích chập

Trong phần này, ta sẽ tìm hiểu về mạng nơ ron tích chập. Khác với nơ ron perceptron thường được kết nối đầy đủ với các nơ ron ở tầng trước đó, sau khi xếp chồng tạo lớp ta thường biểu diễn chúng bởi một vec tơ cột. Nơ ron tích chập, đôi khi còn được gọi là bộ lọc (filter) hay nhân (kernel), chỉ có kết nối địa phương với lớp nơ ron tích chập trước đó. Lớp nơ ron tích chập thường được biểu diễn bởi ma trận ba chiều $[H, W, D]$ cũng có thể là kết quả sau khi xếp chồng bản đồ kích hoạt của các bộ lọc.

- Chiều cao H của lớp tích chập
- Chiều rộng W của lớp tích chập
- Độ sâu D của lớp tích chập

Để thực hiện lọc, nơ ron tích chập cần thực hiện quét từ trên xuống dưới, từ trái sang phải để sinh ra một bản đồ kích hoạt. Để đảm bảo điều này, độ sâu của bộ lọc phải luôn bằng độ sâu của lớp tích chập trước đó.



Hình 3.4: Minh họa mạng nơ ron tích chập kích thước 5x5x3 kết nối địa phương với lớp tích chập 32x32x3.

Do khi quét từ trên xuống dưới, từ trái sang phải của lớp tích chập trước đó, nơ ron tích chập cần có thêm bước nhảy hay hệ số trượt (stride) đi kèm với kích thước

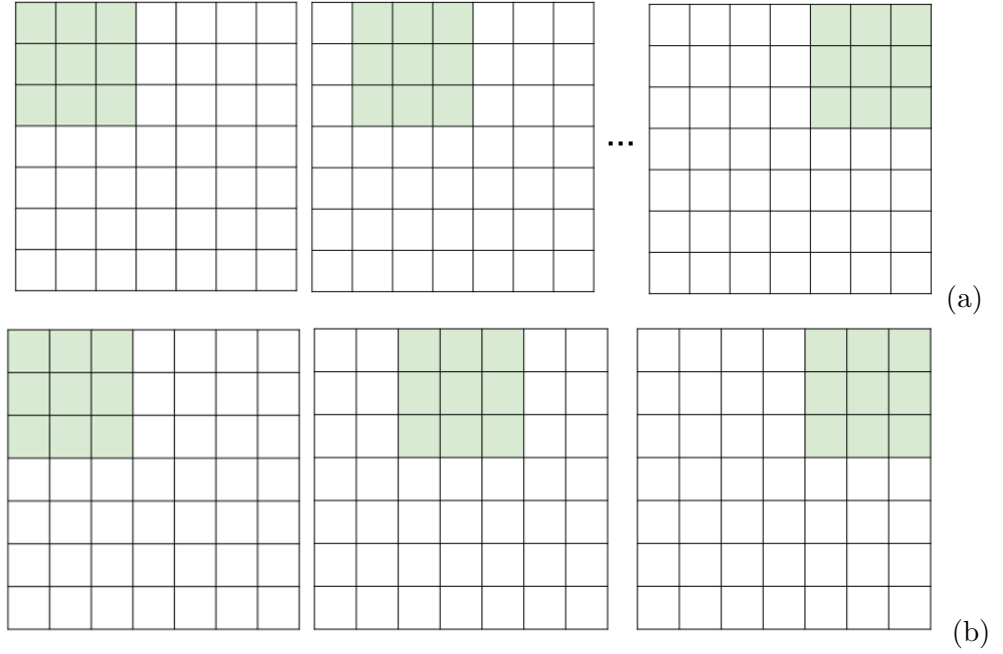
của bản thân nó $[H_f, W_f, D_f]$ thêm hệ số độ lệch b hay còn gọi là đại lượng mức. Kích thước bản đồ kích hoạt $[H_a, W_a]$ sẽ phụ thuộc vào kích thước lớp tích chập $[H, W, D]$ cùng kích thước bộ lọc $[H_f, W_f, D_f]$ và bước nhảy.

$$(H - H_f)/stride + 1 = H_a, \quad (W - W_f)/stride + 1 = W_a$$

theo ví dụ minh họa Hình 3.4 thì kích thước bản đồ kích hoạt sẽ là

$$(32 - 5)/1 + 1 = 28, \quad (32 - 5)/1 + 1 = 28$$

Nên bản đồ kích hoạt có chiều cao và rộng cùng là 28. Tất nhiên, nếu bước nhảy thay đổi thì kích thước của bản đồ kích hoạt cũng thay đổi theo dù kích thước lớp tích chập và bộ lọc là không đổi.



Bảng 3.2: Biểu diễn sự thay đổi của bước nhảy với kích thước lớp tích chập $7 \times 7 \times 1$ và kích thước bộ lọc $3 \times 3 \times 1$ (a) Bước nhảy 1 cho bản đồ kích hoạt 5×5 (b) Bước nhảy 2 cho bản đồ kích hoạt 3×3

Trong đa phần các trường hợp thì lớp tích chập và bộ lọc đều có chiều cao và chiều rộng bằng nhau. Ta có thể dùng công thức tính kích thước của bản đồ kích hoạt, bây giờ cũng là hình vuông đơn giản như sau

$$(N - F)/stride + 1$$

trong đó $N = H = W$ còn $F = H_f = W_f$.

Ta cũng có thể cho thêm viền không (zero padding) để bảo toàn kích thước hay đơn giản ta muốn lọc cả phần góc, cạnh của lớp tích chập vào. Do việc thêm viền cả trên, dưới lẫn bên phải, trái nên ta cần cộng thêm hai lần kích thước viền P khi tính kích thước bản đồ kích hoạt ở đầu ra (xem minh họa Hình 3.5)

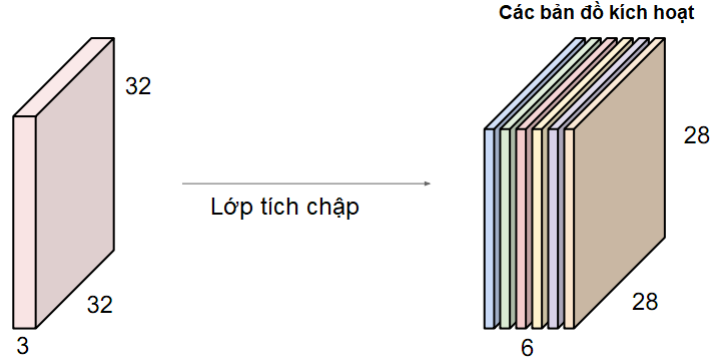
$$(N + 2P - F)/stride + 1$$

0	0	0	0	0	0			
0								
0								
0								
0								

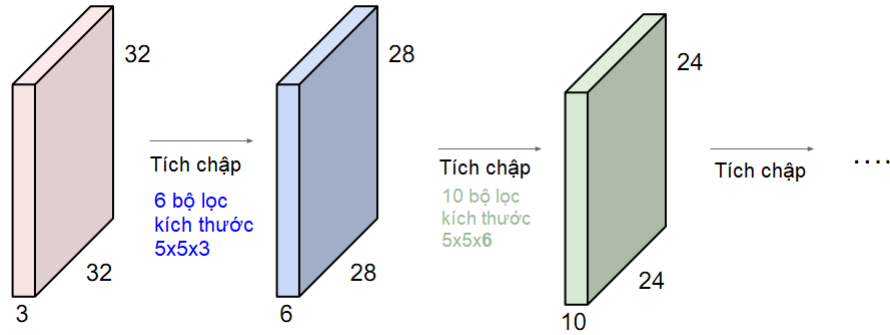
Hình 3.5: Minh họa mạng nơ ron tích chập sau khi thêm viền không.

Thêm các nơ ron tích chập, hay thêm các bộ lọc sẽ làm tăng số bản đồ kích hoạt. Điều này dẫn đến số lượng các nơ ron tích chập sẽ quyết định độ sâu của các bản đồ kích hoạt ở đầu ra và nó trở thành lớp tích chập đầu vào của tầng kế tiếp $[H, W, D]$ (xem minh họa Hình 3.6). Chú ý, mọi nơ ron đều có hàm kích hoạt, thường là ReLU, nên giá trị bản đồ kích hoạt là giá trị nhân tích chập đã truyền qua hàm này.

Tất nhiên, đây chỉ mới là ví dụ của một lớp tích chập. Nói nhiều lớn tích chập theo hướng truyền thẳng, ta sẽ có các lớp tích chập đa lớp được sắp đặt kế nhau (xem minh họa Hình 3.7). Thông thường, hàm kích hoạt ReLU được áp trực tiếp lên khối ma trận $[H, W, D]$ của lớp tích chập chứ không cần thiết kế riêng lẻ cho từng nơ ron tích chập. Tuy nhiên, theo thiết kế của Fukushima, ta đang thiếu lớp Complex-Cells là các tế bào kết nối không tham số.



Hình 3.6: Minh họa bản đồ kích hoạt sau khi thêm 6 nơ ron tích chập. Các bản đồ kích hoạt sẽ trở thành lớp tích chập kích thước 28x28x6.

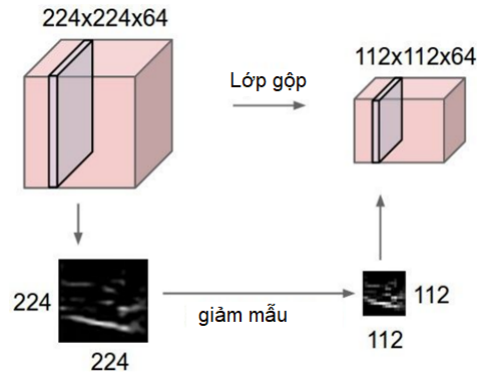


Hình 3.7: Minh họa mạng nơ ron tích chập đa lớp được sắp xếp liên tiếp.

3.2.2 Lớp giảm mẫu

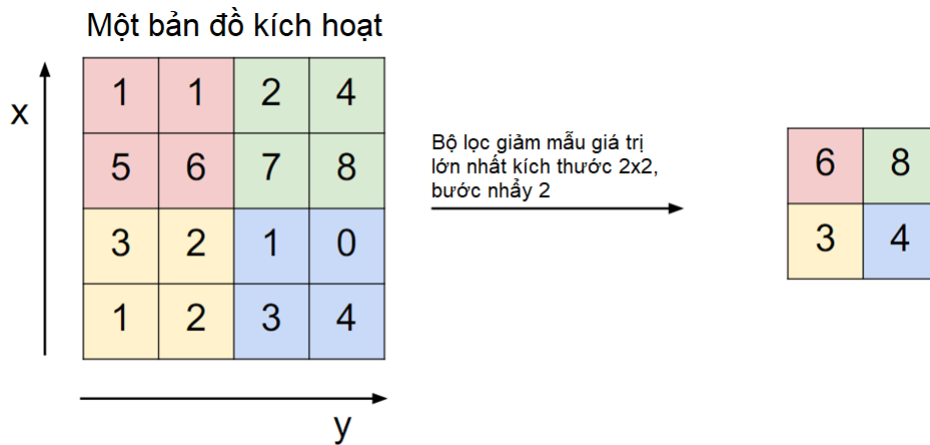
Lớp giảm mẫu làm giảm độ phân giải của khối dữ liệu để giảm bộ nhớ cũng như khối lượng tính toán, nó không có tham số nên không có kết nối có thể học. Nơ ron giảm mẫu hoạt động độc lập trên từng bản đồ kích hoạt có phạm vi hoạt động địa phương, cũng trượt từ trên xuống dưới, từ trái qua phải như nơ ron tích chập nhưng không có tham số (xem minh họa Hình 3.8). Hàm kích hoạt của nơ ron giảm mẫu có tác dụng gộp, thường là giá trị lớn nhất (max) hay giá trị trung bình (average), các giá trị trên bản đồ kích hoạt trong khi trượt. Lớp gộp đảm bảo tính chất bất biến đối các thay đổi như tịnh tiến (translation invariance) hoặc biến dạng (deformation invariance) đối với dữ liệu đầu vào.

Như đã trình bày ở trên, để xác định kích thước của lớp gộp, ta cũng sử dụng công thức chung $(N + 2P - F)/stride + 1$ trong đó thường $stride$ có giá trị lớn hơn 2. Tuy thuộc vào hàm kích hoạt của nơ ron giảm mẫu, ta xác định giá trị tương ứng



Hình 3.8: Minh họa vai trò lớp mạng nơ ron giảm mẫu, theo đó kích thước độ phân giải giảm từ 224x224 xuống 112x112. Hình ảnh độc lập trên bản đồ kích hoạt không bị biến dạng hay tịnh tiến sau khi giảm mẫu.

của lớp tích chập đầu ra. *Chú ý*, độ sâu của lớp giảm mẫu là không thay đổi do nơ ron giảm mẫu hoạt động độc lập trên từng bản đồ kích hoạt.

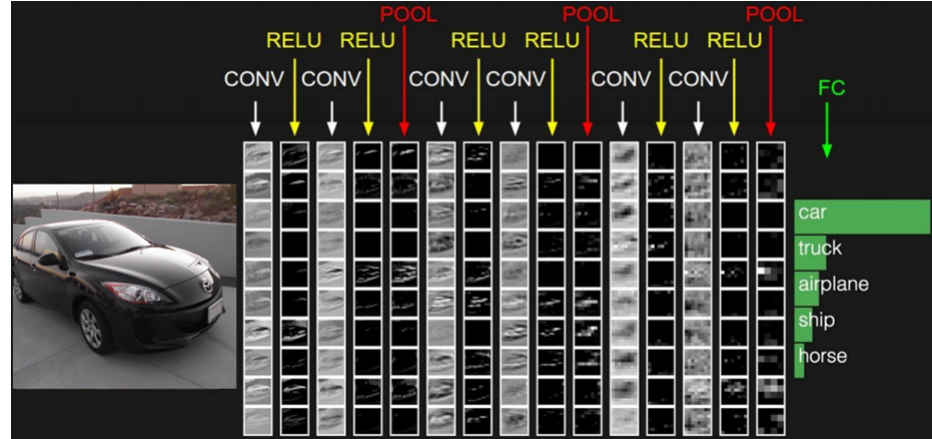


Hình 3.9: Minh họa giá trị của lớp gộp hàm giá trị lớn nhất với bản đồ kích hoạt đầu vào kích thước 4x4, bộ lọc kích thước 2x2, bước nhảy 2 cho ra tích chập đầu ra kích thước 2x2.

3.2.3 Kiến trúc "bánh mì kẹp"

Trong nghiên cứu của Fukushima (1980), các mạng nơ ron tích chập được thiết kế xen kẽ giữa lớp tích chập và lớp giảm mẫu. Kiến trúc "bánh mì kẹp" được xếp đặt vào phân đầu vào, nơi tiếp xúc với hình ảnh đầu tiên. Khối này với lớp tích chập có khả năng trích chọn tự động các đặc trưng hình ảnh của bộ dữ liệu thị giác máy

tính.



Hình 3.10: Minh họa kiến trúc mạng nơ ron tích chập có khả năng trích chọn tự động đặc trưng từ bộ dữ liệu hình ảnh.

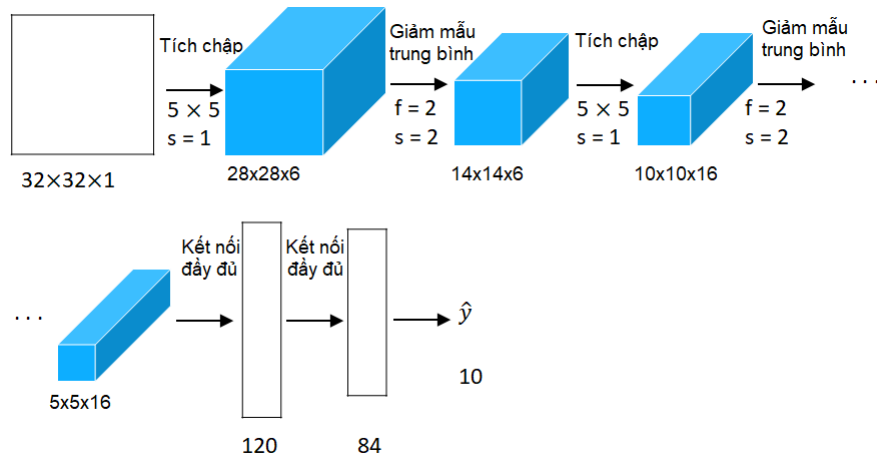
3.3 Một số mạng nơ ron nổi tiếng

Trong phần này chúng ta sẽ giới thiệu về các mạng nơ ron tích chập tiêu biểu liên quan đến bài toán phân lớp, các mạng nơ ron đều coi là các mạng nơ ron tham chiếu có tính chất *nền tảng* dùng để phát triển các ứng dụng cao cấp hơn như tìm kiếm thông tin, phân đoạn, phát hiện đối tượng trên ảnh. Như đã trình bày ở phần lịch sử mạng nơ ron, đây là khoảng thời gian rất dài nhiều thập kỷ trước khi mạng nơ ron học sâu có được ứng dụng phổ biến như hiện nay

3.3.1 LeNet-5

Là mạng nơ ron tích chập đầu tiên được thực sự hiện thực hóa, mạng LeNet-5 do Yann LeCun cùng các đồng sự thực hiện năm 1989 được coi là hình mẫu cho các mạng nơ ron tích chập sau này đặc biệt là mạng nơ ron AlexNet. Mạng có tất cả là 4 tầng nơ ron tích chập xen lớp giảm mẫu, tiếp đó là 2 tầng kết nối đầy đủ và cuối cùng là lớp đầu ra chứa 10 nơ ron (xem minh họa Hình 3.11). Toàn bộ mạng nơ ron tích chập có khoảng 60K tham số trong đó các nơ ron tích chập có hàm kích hoạt là $\text{sigmoid}(z)$ tiếp theo các nơ ron giảm mẫu đều dùng hàm trung bình (average) còn hàm kích hoạt tại các tầng kết nối đầy đủ là hàm $\text{tanh}(z)$, tham khảo lại Bảng 2.1 về các công thức của hàm kích hoạt tương ứng.

Số tham số của mạng nơ ron LeNet-5 là khá lớn ở thời điểm nó được tạo ra dẫn



Hình 3.11: Minh họa kiến trúc mạng nơ ron tích chập LeNet-5 dưới dạng khối ma trận ba chiều $[H, W, D]$ tô màu xanh nước biển, dùng cho bài toán nhận diện chữ số viết tay MNIST.

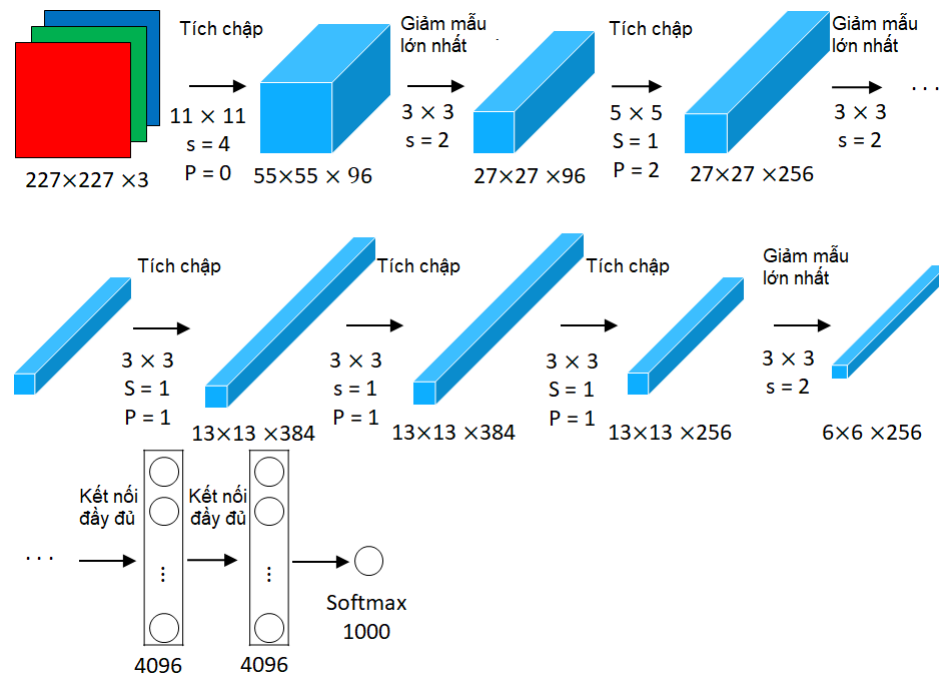
đến chi phí tính toán và lưu trữ là một thách thức. Để đảm bảo đạo hàm lan truyền ngược, LeCun và đồng sự đều sử dụng hàm liên tục và trơn để khả vi trong mọi tình huống khi huấn luyện.

3.3.2 AlexNet

Mạng nơ ron AlexNet tạo nên "địa chấn" khi giành chiến thắng trong cuộc thi ILSRC2012, bỏ xa các phương pháp tiếp cận thị giác máy tính truyền thống vào năm 2012. Chú ý, mạng nơ ron tích chập LeNet-5 xuất hiện từ hơn một thập kỷ trước đó. Một trong những mạng CNNs lớn nhất tại thời điểm đó tạo ra. Có trên 60 triệu tham số so với 60 ngàn tham số của mạng LeNet-5. các nơ ron tích chập có hàm kích hoạt là $ReLU(z)$ tiếp theo các nơ ron giảm mẫu đều dùng hàm lớn nhất (max) còn hàm kích hoạt tại các tầng kết nối đầy đủ là $ReLU(z)$. Mạng nơ ron AlexNet dùng hai card đồ họa NVIDIA GTX 580 3GB GPU để thực hiện quá trình huấn luyện trong khoảng 5 đến 6 ngày.

3.4 Tổng kết

Bạn đọc cần đọc hiểu thêm các kiến thức về các mạng nơ ron tích chập khác như VGG với stack layers, GoogleNet với Inception Layer hoặc ResNet với khái niệm bỏ kết nối trực tiếp (skip connections). Đếm được số tham số, tính được chi phí tính toán, chi phí bộ nhớ để có thể hiểu sâu về mạng nơ ron tích chập và vì sao phải



Hình 3.12: Minh họa kiến trúc mạng nơ ron tích chập AlexNet dưới dạng khối ma trận ba chiều $[H, W, D]$ tô màu xanh nước biển, dùng cho bài toán nhận diện 1000 lớp của bài toán trong cuộc thi ILSRC2012.

dùng các đồ họa GPU để song song hóa quy trình tính toán trong các mạng học sâu nhiều tham số nói chung.

3.5 Bài tập

Bài tập 3.1. Theo kiến trúc Fukushima (1980), các tế bào đơn giản (simple cells) được định nghĩa đúng nhất là ?

1. là các tế bào đơn giản có khả năng ước lượng tham số khi mô phỏng
2. là các tế bào phức tạp không có khả năng ước lượng tham số khi mô phỏng
3. là các tế bào sinh học liên quan đến trung khu thị giác máy tính
4. là các tế bào thần kinh không có khả năng kết nối

Bài tập 3.2. Theo kiến trúc Fukushima (1980), các tế bào phức tạp (complex cells) được mô phỏng đúng nhất là ?

1. là các tầng nơ ron giảm mẫu
2. là các tầng nơ ron tích chập
3. là các tầng nơ ron kết nối đầy đủ
4. là các tầng nơ ron hồi quy

Bài tập 3.3. Hãy cho biết kích thước của bản đồ kích hoạt đầu ra ? Biết rằng, ma trận tích chập đầu vào $224 \times 224 \times 3$, có số bộ lọc $f=128$, kích thước trường tiếp nhận 7×7 , bước nhảy $\text{stride}=1$ và không thêm viền $P=0$

Bài tập 3.4. Trong các ứng dụng học sâu sau, ứng dụng nào để tạo sinh các đoạn văn bản trả lời tự động từ các lời nhắc (prompts) ?

1. ChatGPT-4
2. Dall
3. Vec 3
4. Sora

Bài tập 3.5. Cho biết độ sâu (số tầng) của mạng AlexNet (2012) là bao nhiêu ?

1. 5 tầng
2. 6 tầng
3. 7 tầng
4. 8 tầng
5. Không có đáp án đúng

Chương 4

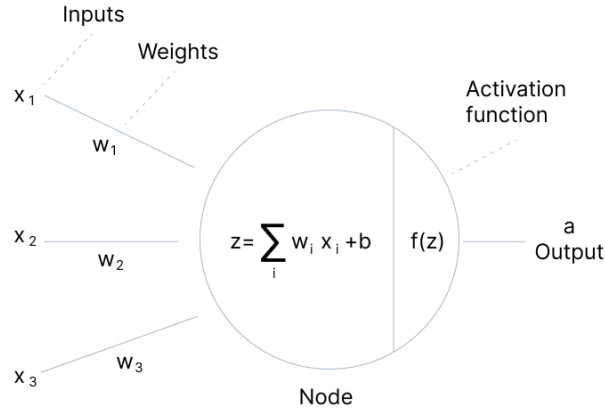
Huấn luyện mạng nơ-ron

4.1	Activation functions in Neural Networks	48
4.2	Tiền xử lý dữ liệu	56
4.3	Khởi tạo trọng số	58
4.4	Các kỹ thuật chuẩn hóa	62
4.5	Chiến lược thay đổi tốc độ học	68
4.6	Các thuật toán tối ưu cho mạng nơ-ron	72
4.7	Một số kỹ thuật chống overfitting	79
4.8	Làm giàu dữ liệu (data augmentation)	80
4.9	Kỹ thuật kết hợp nhiều mô hình (model ensemble) . . .	85
4.10	Kỹ thuật học tái sử dụng (transfer learning)	86

4.1 Activation functions in Neural Networks

Hàm kích hoạt được sử dụng để xác định đầu ra của các nơ ron trong mạng nơ ron. Nó quyết định liệu một nơ-ron có nên được kích hoạt hay không.

Cho dù chúng ta chồng bao nhiêu lớp ẩn trong mạng nơ-ron, nếu không có hàm kích hoạt, tất cả các lớp sẽ hoạt động theo cùng một cách vì hợp thành của hai hàm tuyến tính vẫn là một hàm tuyến tính. Mạng nơ-ron không có hàm kích hoạt về cơ bản chỉ là mô hình hồi quy tuyến tính. Mặt khác, việc áp dụng hàm kích hoạt phi tuyến dẫn đến tính phi tuyến của mạng nơ-ron nhân tạo và do đó dẫn đến tính phi tuyến của hàm xấp xỉ mạng nơ-ron này. Theo định lý về xấp xỉ hàm, một perceptron nhiều lớp chỉ có một lớp ẩn sử dụng hàm kích hoạt phi tuyến là một hàm xấp xỉ phổ quát.



Hàm kích hoạt thực hiện chuyển đổi phi tuyến đầu vào của mạng khiến nó có khả năng học và thực hiện các tác vụ phức tạp hơn. Do đó, chúng ta cần một chức năng kích hoạt.

Các hàm kích hoạt được chia ra theo miền giá trị hay đường cong của hàm.

4.1.1 Sigmoid

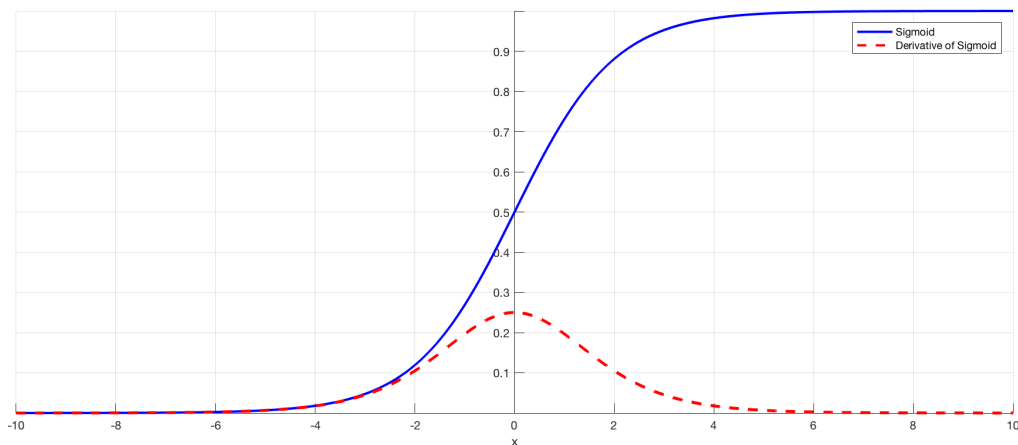
Hàm sigmoid biểu diễn về mặt toán học như sau:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Hàm sigmoid là hàm kích hoạt được sử dụng rộng rãi nhất từ khi bắt đầu học sâu. Đây là một hàm logistic, trơn với đường cong hàm trông giống hình chữ S và nhận giá trị trong khoảng từ 0 đến 1. Đạo hàm của một hàm sigmoid được biểu diễn bằng một hàm của chính nó và chúng ta có thể dễ dàng tìm thấy độ dốc của đường cong sigmoid tại hai điểm bất kỳ:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Trong hàm sigmoid, chúng ta có thể thấy đầu ra của nó nằm giữa khoảng mở $(0, 1)$ nên chúng ta có thể nghĩ đến xác suất. Hàm sigmoid đặc biệt được sử dụng trong các mô hình mà chúng ta phải dự đoán xác suất ở đầu ra vì xác suất của bất cứ điều gì chỉ tồn tại trong khoảng từ 0 đến 1 nên sigmoid là lựa chọn đúng đắn. Tuy nhiên, theo một nghĩa khác, hàm sigmoid có thể mô phỏng tốt tỉ lệ bắn xung (firing rate) của nơ-ron, trong đó ở giữa đường cong hàm, nơi có độ dốc tương đối lớn là vùng nhảy của nơ-ron còn ở hai phía có độ dốc rất thoải là vùng ức chế của nơ-ron.



Hình 4.1: Hình minh họa hàm sigmoid và đạo hàm của nó.

Mặc dù hàm sigmoid có một số ưu điểm tốt nhưng hàm này cũng có một số nhược điểm nhất định:

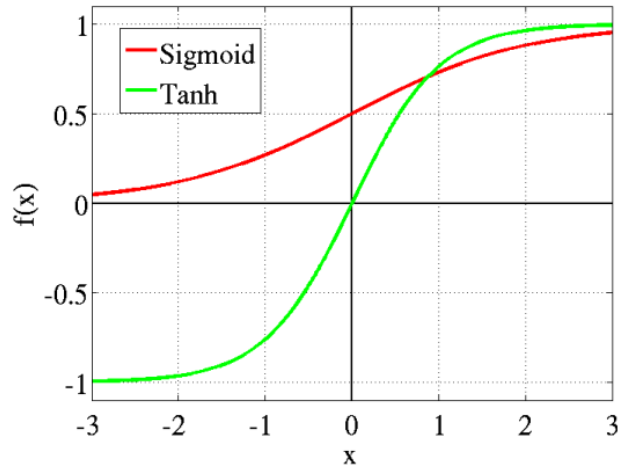
- Đầu tiên là gradient có thể suy biến trong hàm sigmoid. Một đặc tính rất không mong muốn của hàm là việc kích hoạt các nơ-ron bão hòa gần 0 hoặc gần 1. Đạo hàm của hàm sigmoid trở nên rất nhỏ đối với các vùng bão hòa này với các giá trị đầu vào âm lớn hoặc dương lớn. Trong trường hợp này, đạo hàm gần bằng 0 sẽ làm cho gradient của hàm mất mát rất nhỏ, ngăn cản việc cập nhật các trọng số và do đó ngăn cản toàn bộ quá trình học.
- Một đặc tính không mong muốn khác của kích hoạt sigmoid là đầu ra của hàm không có trung bình 0. Điều này thường làm cho việc huấn luyện mạng nơ-ron trở nên khó khăn và không ổn định hơn.
- Cuối cùng, hàm sigmoid phải thực hiện các phép toán hàm mũ làm cho tốc độ tính chậm hơn đối với máy tính.

4.1.2 Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

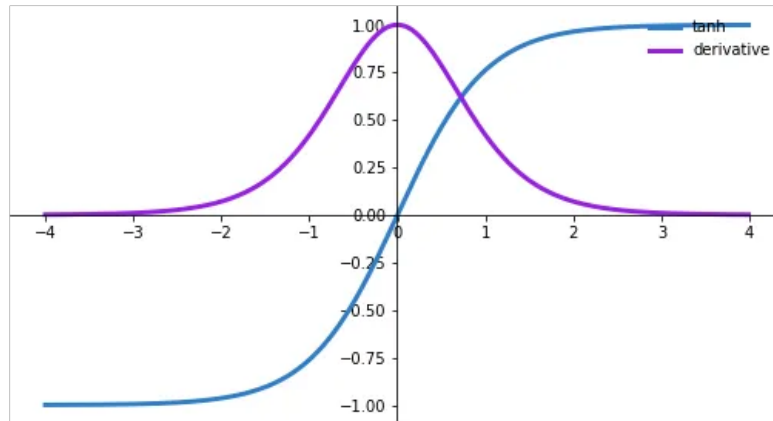
Hàm tanh cũng có đường cong hình chữ S tương tự như hàm sigmoid, nhưng nhận giá trị trong khoảng từ -1 đến 1 . Cả hai hàm kích hoạt sigmoid và tanh đều được sử dụng trong mạng truyền thẳng.

Giống như hàm sigmoid, các nơ-ron trong mạng bão hòa ở các giá trị âm và dương lớn, và trong vùng bão hòa này, đạo hàm của hàm sẽ tiến tới 0. Tuy nhiên,



Hình 4.2: Đồ thị hàm tanh và hàm sigmoid.

không giống như hàm sigmoid, đầu ra của nó có trung bình bằng 0. Vì vậy, trong thực tế, hàm tanh luôn được ưa chuộng hơn hàm sigmoid.

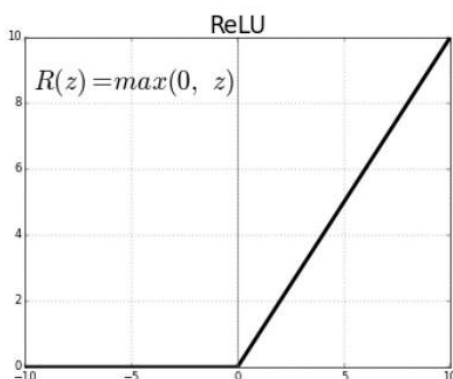


Hình 4.3: Hình minh họa hàm tanh và đạo hàm của nó.

Hàm tanh chủ yếu được sử dụng để phân loại giữa hai lớp. Ưu điểm là các đầu vào âm sẽ được ánh xạ âm mạnh và các đầu vào 0 sẽ được ánh xạ gần 0 trong biểu đồ tanh. Hàm này là khả vi và đạo hàm của một hàm tanh được biểu diễn bằng một hàm của chính nó.

4.1.3 ReLU (Rectified Linear Unit)

Hàm ReLU đã trở nên rất phổ biến trong những năm gần đây và được sử dụng trong hầu hết các mạng tích chập hoặc mạng sâu. Việc kích hoạt trong hàm này là tuyến tính đối với các giá trị đầu vào lớn hơn 0 và bất kỳ đầu vào âm nào được cung cấp cho hàm kích hoạt ReLU sẽ biến giá trị thành 0 ngay lập tức. Phạm vi của ReLU là từ 0 đến vô cùng.

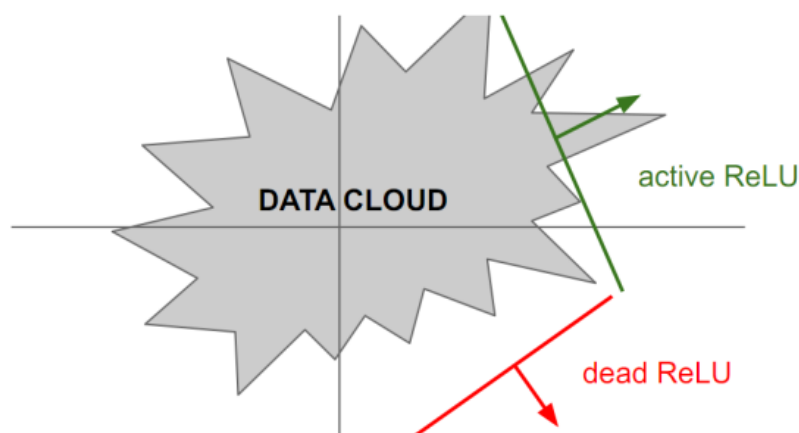


Một số ưu điểm và nhược điểm khi sử dụng ReLU:

- Trong thực tế, ReLU đã được chứng tỏ là có khả năng đẩy nhanh quá trình hội tụ của hàm mất mát theo phương pháp giảm gradient so với các hàm kích hoạt khác. Điều này là do tính chất tuyến tính, không bão hòa trong vùng dương của nó.
- Trong khi các hàm kích hoạt khác (tanh và sigmoid) yêu cầu các phép tính toán rất tốn kém về mặt tính toán như hàm mũ, v.v., thì ReLU, mặt khác, có thể được thực hiện đơn giản bằng cách đặt ngưỡng một vectơ giá trị tại 0.
- Thật không may, hàm ReLU cũng có vấn đề. Vì đầu ra của hàm này bằng 0 đối với các giá trị đầu vào dưới 0 nên các nơ-ron của mạng có thể trở nên rất yếu và thậm chí “chết” trong quá trình huấn luyện. Không nhất thiết nhưng có thể xảy ra hiện tượng khi các trọng số được cập nhật, các trọng số được điều chỉnh theo cách mà đối với một số nơ-ron nhất định của một lớp ẩn nào đó, đầu vào x luôn ở dưới 0. Điều này có nghĩa là các giá trị $f(x)$ của các nơ-ron này (với f là hàm ReLU) luôn bằng 0 ($\text{ReLU}(x) = 0$) và do đó không đóng góp gì cho quá trình huấn luyện. Điều này có nghĩa là gradient truyền ngược qua các nơ-ron được kích hoạt ReLU này cũng bằng 0 kể từ thời điểm đó. Chúng ta nói rằng các nơ-ron đã “chết”. Ví dụ, người ta thường quan sát

thấy rằng 20 – 50% toàn bộ mạng nơ-ron sử dụng hàm ReLU đã “chết”. Nói cách khác, những nơ-ron này không bao giờ được kích hoạt đối với toàn bộ tập dữ liệu được sử dụng trong quá trình huấn luyện.

Để tránh ReLU bị “văng” ra khỏi tập dữ liệu dẫn tới đầu ra luôn âm và không bao giờ được cập nhật trọng số nữa hay ReLU chết (Dying ReLU), chúng ta nên khởi tạo nơ-ron ReLU với bias dương bé (ví dụ 0.01) và cẩn thận hơn với tốc độ học để tránh ReLU bị chết.



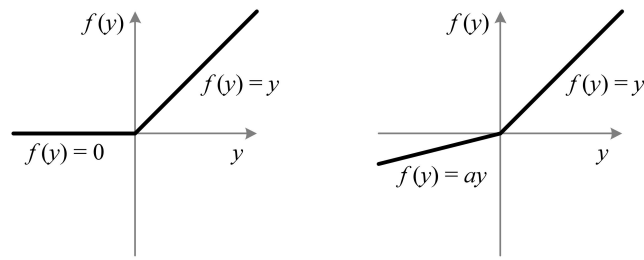
4.1.4 Leaky ReLU

Leaky ReLU không gì khác hơn là một phiên bản cải tiến của hàm kích hoạt ReLU. Như đã đề cập ở phần trước, việc sử dụng ReLU có thể “giết chết” một số nơ-ron trong mạng nơ-ron của chúng ta và những nơ-ron này có thể không bao giờ hoạt động trở lại.

Leaky ReLU được xác định để giải quyết vấn đề này. Không giống như ReLU “vanilla”, trong đó tất cả các giá trị $f(x)$ của nơ-ron bằng 0 đối với các giá trị đầu vào $x < 0$, trong trường hợp Leaky ReLU, chúng ta thêm một thành phần tuyến tính nhỏ vào hàm ReLU này.

Về cơ bản, chúng ta đã thay đường ngang cho các giá trị dưới 0 bằng đường tuyến tính không nằm ngang. Độ dốc của đường tuyến tính này có thể được đặt bằng tham số, được nhân với đầu vào. Phạm vi của Leaky ReLU là âm vô cùng đến dương vô cùng và giá trị của tham số độ dốc a là 0.01.

Ưu điểm của việc sử dụng Leaky ReLU khi thay đường ngang là chúng ta tránh được gradient bằng 0. Điều này là do trong trường hợp này, chúng ta không còn các nơ-ron “chết” luôn bằng 0 và do đó không còn đóng góp cho quá trình huấn luyện

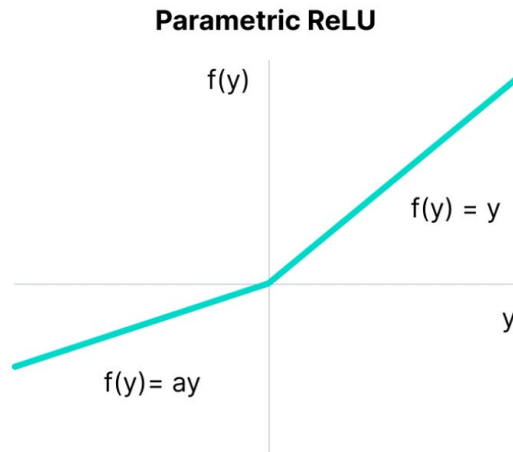


Hình 4.4: Đồ thị hàm ReLU với Leaky ReLU

nữa.

4.1.5 Parametric ReLU

ReLU tham số là một biến thể khác của ReLU nhằm giải quyết vấn đề gradient trở thành 0 đối với nửa bên trái của trục. Hàm này cung cấp độ dốc của phần âm của hàm với tham số a . Bằng cách thực hiện lan truyền ngược, giá trị thích hợp nhất của a sẽ được học.



Ở đây, a - siêu tham số là tham số độ dốc cho các giá trị âm. Hàm **ReLU tham số** được sử dụng khi hàm Leaky ReLU vẫn không giải quyết được vấn đề nơ-ron chết và thông tin thích đáng không được chuyển thành công sang lớp tiếp theo.

4.1.6 Softmax

Nói một cách đơn giản, hàm kích hoạt softmax buộc các giá trị của các nơ-ron đầu ra phải lấy các giá trị từ 0 đến 1, để chúng có thể biểu thị các giá trị xác suất trong khoảng $[0, 1]$.

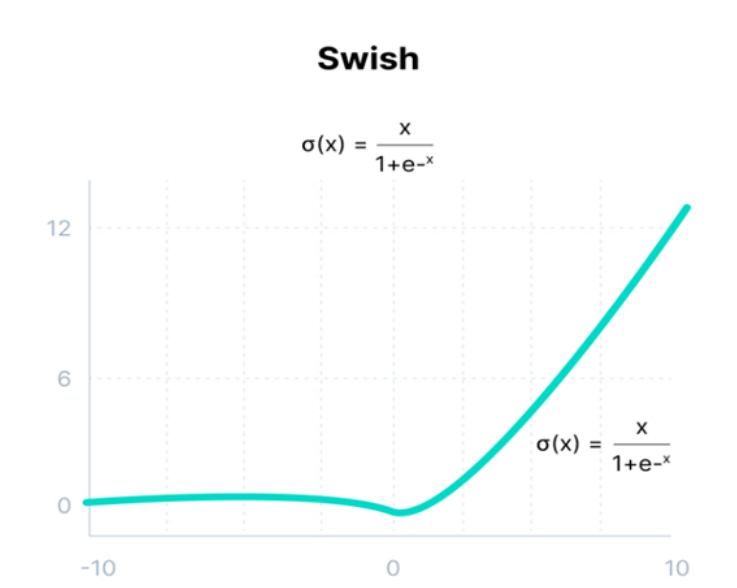
Một điểm khác cần xem xét là khi các đặc trưng đầu vào được phân loại thành các lớp khác nhau, các lớp này sẽ loại trừ lẫn nhau. Điều này có nghĩa là mỗi vectơ đặc trưng x chỉ thuộc một lớp. Hơn nữa, đối với các lớp loại trừ lẫn nhau, giá trị xác suất của tất cả các nơ ron đầu ra phải có tổng bằng một. Chỉ bằng cách này thì mạng nơ ron mới thể hiện được sự phân bố xác suất chính xác. May mắn thay, hàm softmax không chỉ buộc các đầu ra nằm trong phạm vi từ 0 đến 1 mà còn đảm bảo rằng tổng đầu ra của tất cả các lớp có thể cộng lại bằng 1.

Hàm Softmax được mô tả là sự kết hợp của nhiều sigmoid. Nó tính toán xác suất tương đối. Tương tự như hàm kích hoạt sigmoid, hàm Softmax trả về xác suất của từng lớp. Nó được sử dụng phổ biến nhất như một hàm kích hoạt cho lớp cuối cùng của mạng nơ-ron trong trường hợp phân loại nhiều lớp.

Về mặt toán học, nó có thể được biểu diễn dưới dạng:

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

4.1.7 Swish



Hàm này bị giới hạn bên dưới nhưng không bị chặn ở trên, tức là y tiến đến một giá trị không đổi khi x tiến đến âm vô cùng nhưng y tiến đến vô cùng khi x tiến đến vô cùng. Về mặt toán học nó có thể được biểu diễn dưới dạng:

$$f(x) = x \times \text{sigmoid}(x) = \frac{x}{1 + e^{-x}}$$

Swish luôn phù hợp hoặc vượt trội hơn hàm ReLU trên các mạng sâu được áp dụng cho các bài toán thách thức khác nhau như phân loại hình ảnh, dịch máy, v.v.

Tóm lại, việc chọn hàm kích hoạt phụ thuộc nhiều vào vấn đề đang cố gắng giải quyết và phạm vi giá trị đầu ra mà chúng ta mong đợi. Chẳng hạn, chúng ta cần chọn hàm kích hoạt đối với lớp đầu ra dựa trên loại bài toán dự đoán đang giải quyết - cụ thể là loại biến dự đoán của bài toán. Sau đây là một số quy tắc để chọn hàm kích hoạt cho lớp đầu ra của bài toán dự đoán:

1. Hồi quy: Hàm kích hoạt tuyến tính
2. Phân loại nhị phân: Hàm kích hoạt sigmoid
3. Phân loại nhiều lớp: Softmax
4. Phân loại đa nhãn: Sigmoid

Tiếp theo là một số lưu ý khi chọn hàm kích hoạt cho lớp ẩn của các mạng sâu:

1. Hàm kích hoạt ReLU chỉ nên được sử dụng trong các lớp ẩn.
2. Không nên sử dụng các hàm Sigmoid và Tanh trong các lớp ẩn vì chúng làm cho mô hình dễ gặp vấn đề hơn trong quá trình huấn luyện (gradients suy biến).
3. Hàm Swish được sử dụng trong mạng nơ-ron có độ sâu lớn hơn 40 lớp.
4. Đối với mạng CNN, chúng ta nên sử dụng hàm kích hoạt ReLU.
5. Đối với mạng RNN, chúng ta nên sử dụng hàm kích hoạt Tanh và/hoặc Sigmoid.

4.2 Tiền xử lý dữ liệu

Học sâu (DL) đã tạo ra những mô hình tốt nhất để giải quyết nhiều bài toán phức tạp như nhận dạng giọng nói, thị giác máy tính, dịch máy, v.v. Tuy nhiên, huấn luyện các mô hình DL là một thách thức khó khăn vì các mô hình hiện tại có cấu trúc phức tạp và phân bố dữ liệu qua các tầng sẽ thay đổi. Vì vậy, trong bài

viết này, chúng ta sẽ tìm hiểu về tầm quan trọng của việc chuẩn hóa và các phương pháp chuẩn hóa trong DL để áp dụng phù hợp với các bài toán trong thực tiễn.

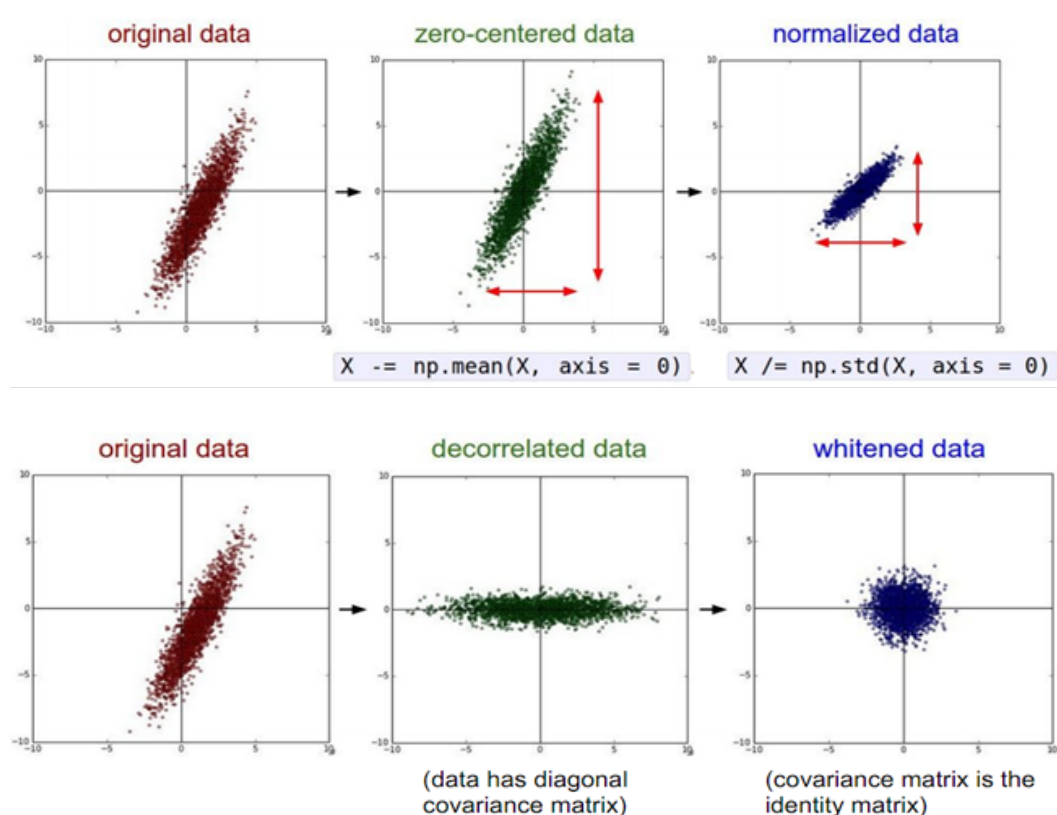
Trong học máy nói chung, các phương pháp chuẩn hóa đã được sử dụng để tiền xử lý dữ liệu đầu vào trên một thang đo chung. Ví dụ, một số phương pháp chuẩn hóa thường sử dụng trong học máy như: chuẩn hóa min-max, chuẩn hóa vectơ đơn vị, chuẩn hóa z-score, . . . Các phương pháp này đều cải thiện tính phù hợp của mô hình, đặc biệt khi các đặc trưng đầu vào khác nhau có phạm vi rất khác nhau (ví dụ: độ tuổi so với mức lương).

Trước khi chuẩn hóa dữ liệu đầu vào, các trọng số liên quan đến các đặc trưng/thuộc tính sẽ khác nhau rất nhiều vì giá trị đầu vào của chúng nằm ở các khoảng khác nhau lớn. Điều này dẫn đến một số trọng số lớn và một số trọng số nhỏ. Các trọng số lớn sẽ ảnh hưởng đến việc cập nhật trọng số trong quá trình lan truyền ngược lỗi khi huấn luyện. Vì sự phân bố không đồng đều của các trọng số, thuật toán có thể bị rơi vào vùng tối ưu cục bộ trước khi tìm đến vị trí tối ưu toàn cục. Do đó, để tránh việc thuật toán mất quá nhiều thời gian do sự dao động của hàm lỗi khi tối ưu, chúng ta cần chuẩn hóa các đặc trưng đầu vào về cùng tỉ lệ và phân phối. Điều này giúp mô hình có thể học nhanh hơn.

Trong học sâu, chuẩn hóa tầng đầu vào cũng được chứng tỏ là có lợi. Cụ thể, “làm trắng” các đặc trưng đầu vào để chúng không còn liên quan hay phụ thuộc vào nhau và có giá trị trung bình bằng 0 với phương sai đơn vị dẫn đến việc huấn luyện và hội tụ nhanh hơn (Desjardins, Simonyan, Pascanu, & Kavukcuoglu, 2015).

Các ví dụ về Chuẩn hóa dữ liệu đầu vào:

1. Biến đổi phân phối dữ liệu về kỳ vọng bằng 0: trừ tất cả mẫu dữ liệu cho mẫu trung bình và sau đó, biến đổi phân phối dữ liệu về độ lệch chuẩn đơn vị.
2. Sử dụng PCA hoặc Whitening dữ liệu.
3. Đối với bộ dữ liệu CIFAR10 gồm các ảnh kích thước 32x32x3, các mạng sâu đã thực hiện chuẩn hóa dữ liệu như sau:
 - Subtract the mean image (e.g. AlexNet)
(mean image = [32,32,3] array)
 - Subtract per-channel mean (e.g. VGGNet)
(mean along each channel = 3 numbers)
 - Subtract per-channel mean and Divide by per-channel std (e.g. ResNet)
(mean along each channel = 3 numbers)



4.3 Khởi tạo trọng số

Bạn có thể xây dựng các mô hình deep learning tốt hơn, huấn luyện nhanh hơn nhiều bằng cách sử dụng các kỹ thuật khởi tạo trọng số phù hợp. Mạng nơ-ron sẽ học các trọng số trong quá trình huấn luyện nhưng các trọng số khởi tạo của mạng đóng một vai trò quan trọng trong quá trình tối ưu hóa. Nó có thể mang lại lợi ích hay cản trở quá trình tối ưu hóa, trong đó, các trọng số tối ưu hội tụ nhanh như thế nào và hội tụ đến cực trị địa phương nào có thể phụ thuộc vào các giá trị khởi tạo trọng số này.

Mạng nơ-ron có thể chứa hàng trăm triệu tham số dẫn đến hàm mất mát của chúng thường rất phức tạp, nhiều chiều và không lồi với rất nhiều điểm cực trị. Vì vậy, điểm bắt đầu của các trọng số trên bề mặt hàm mất mát này sẽ xác định cực trị địa phương mà chúng hội tụ đến và một khởi tạo tốt hơn có thể mang lại mô hình tốt hơn.

Mạng sâu cũng có thể bị suy biến hoặc bùng nổ các gradient. Điều này là do thao tác chính được sử dụng để tính toán đạo hàm khi chúng ta truyền qua mô hình mạng là phép nhân ma trận. Do đó, một mạng gồm n lớp ẩn sẽ nhân n đạo hàm với

nhau.

Khi đạo hàm lớn, gradient sẽ tăng theo cấp số nhân khi chúng ta truyền qua mô hình mạng sâu cho đến khi nó bùng nổ. Ngược lại, khi đạo hàm nhỏ, gradient sẽ giảm theo cấp số nhân khi chúng ta truyền qua mô hình mạng sâu cho đến khi cuối cùng nó suy biến. Cả hai kịch bản đều dẫn đến cùng một kết quả (theo những cách khác nhau): mạng không thể học tốt.

Giải pháp một phần cho vấn đề này là phải cẩn thận hơn khi khởi tạo ngẫu nhiên các trọng số của mạng. Thủ tục này được gọi là khởi tạo trọng số. Mục tiêu cuối cùng của việc khởi tạo trọng số là ngăn không cho gradient bùng nổ hoặc suy biến, do đó, cho phép quá trình tối ưu hóa hiệu quả hơn. Nó không hoàn toàn giải quyết được vấn đề gradient suy biến/bùng nổ, nhưng nó là một lựa chọn thiết kế quan trọng và giúp ích rất nhiều khi xây dựng mạng sâu.

4.3.1 Khởi tạo các trọng số bằng 0 hay hằng số

Khi tất cả các trọng số được khởi tạo bằng 0 hay hằng số, các nơ-ron trong mỗi lớp sẽ học chính xác cùng một thứ. Điều này là do đầu ra của tất cả các nơ-ron ẩn trong mô hình sẽ có ảnh hưởng như nhau đến hàm mất mát, kết quả là gradient đối với các trọng số là giống nhau và các trọng số sẽ nhận cùng một cập nhật ở mỗi bước. Do đó, các nơ-ron sẽ phát triển một cách đối xứng khi quá trình huấn luyện diễn ra và chúng ta sẽ không thể phá vỡ sự đối xứng này.

Ưu điểm chính của việc sử dụng mạng nơ-ron so với các thuật toán học máy truyền thống là khả năng học các ánh xạ phức tạp từ không gian đầu vào sang đầu ra. Vì lý do này mà mạng nơ-ron được gọi là các bộ xấp xỉ hàm phổ quát. Các trọng số khác nhau của mạng cho phép các nơ-ron ở các lớp khác nhau học được các khía cạnh khác nhau của ánh xạ này. Tuy nhiên, khi các trọng số đi vào một nơ-ron không đổi thì tất cả các nơ-ron trong một lớp đều học những điều “giống nhau”. Một mô hình như vậy hoạt động kém trong thực tế.

4.3.2 Khởi tạo ngẫu nhiên

Việc gán trọng số với các giá trị ngẫu nhiên sẽ phá vỡ tính đối xứng. Nó tốt hơn khởi tạo bằng 0 hay hằng số nhưng không phải là không có một số vấn đề. Trọng số không được khởi tạo quá cao hoặc quá thấp vì:

- Nếu trọng số quá nhỏ thì phương sai của tín hiệu đầu vào bắt đầu giảm khi nó đi qua từng lớp trong mạng. Đầu vào cuối cùng giảm xuống giá trị thực sự thấp và không còn hữu ích nữa. Nếu chúng ta sử dụng hàm sigmoid làm

hàm kích hoạt thì chúng ta biết rằng nó gần như tuyến tính khi chúng ta tiến gần đến 0. Về cơ bản, điều này có nghĩa là sẽ không có bất kỳ sự phi tuyến tính nào. Nếu đúng như vậy thì chúng ta sẽ mất đi lợi thế của việc có nhiều lớp.

- Nếu trọng số quá lớn thì phương sai của dữ liệu đầu vào có xu hướng tăng nhanh theo từng lớp đi qua. Cuối cùng nó trở nên lớn đến mức thành vô dụng vì hàm sigmoid có xu hướng phẳng khi các giá trị trở nên lớn. Điều này có nghĩa là các hoạt động kích hoạt của chúng ta sẽ trở nên bão hòa và gradient sẽ bắt đầu tiến đến mức 0.

Khi khởi tạo kém, việc khởi tạo ngẫu nhiên có thể dẫn đến gradient suy biến/bùng nổ. Nếu chúng ta huấn luyện thuật toán lâu hơn, kết quả có thể được cải thiện nhưng việc khởi tạo các trọng số có giá trị ngẫu nhiên nhỏ/lớn sẽ làm chậm quá trình tối ưu hóa.

Việc khởi tạo mạng với trọng số phù hợp là rất quan trọng nếu bạn muốn mạng nơ-ron của mình hoạt động bình thường. Chúng ta cần đảm bảo rằng các trọng số nằm trong phạm vi hợp lý trước khi bắt đầu huấn luyện mạng. Đây là nơi sử dụng khởi tạo Xavier.

Khởi tạo Xavier, còn được gọi là khởi tạo Glorot, trở thành cách khởi tạo trọng số tiêu chuẩn khi các nút/nơ-ron trong mạng sử dụng hàm kích hoạt tanh hoặc sigmoid. Nó được đề xuất lần đầu tiên trong một bài báo của Xavier Glorot và Yoshua Bengio vào năm 2010. Mục tiêu của việc khởi tạo Xavier là khởi tạo các trọng số sao cho phương sai giữa các lớp trong mạng là như nhau. Điều này giúp ngăn chặn sự bùng nổ hoặc suy biến của gradient.

4.3.3 Khởi tạo Xavier

Việc chỉ định trọng số mạng trước khi chúng ta bắt đầu huấn luyện nên là một quá trình ngẫu nhiên nhưng chúng ta không biết gì về dữ liệu, vì vậy chúng ta không chắc chắn cách chỉ định trọng số phù hợp trong một trường hợp cụ thể. Một cách tốt là gán trọng số từ một phân bố Gaussian. Rõ ràng phân phối này sẽ có giá trị trung bình bằng 0 và một phương sai hữu hạn. Hãy xem xét một nơ-ron tuyến tính:

$$y = w_1x_1 + w_2x_2 + \dots + w_Nx_N + b$$

Với mỗi lớp đi qua, chúng ta muốn phương sai được giữ nguyên. Điều này giúp chúng ta giữ cho tín hiệu không bị bùng nổ đến giá trị cao hoặc suy biến về 0. Nói

cách khác, chúng ta cần khởi tạo các trọng số sao cho phương sai vẫn giữ nguyên đối với x và y . Quá trình khởi tạo này được gọi là khởi tạo Xavier.

Chúng ta tính phương sai của y :

$$\text{var}(y) = \text{var}(w_1x_1 + w_2x_2 + \dots + w_Nx_N + b)$$

Để tính phương sai của các số hạng bên trong dấu ngoặc đơn ở vế phải của phương trình trên, xét một số hạng tổng quát thì ta có:

$$\text{var}(w_ix_i) = E(x_i)^2\text{var}(w_i) + E(w_i)^2\text{var}(x_i) + \text{var}(w_i)\text{var}(x_i)$$

Ở đây, E là kỳ vọng của một biến nhất định, về cơ bản đại diện cho giá trị trung bình. Chúng ta đã giả định rằng đầu vào và trọng số đến từ phân phối Gaussian có giá trị trung bình bằng 0. Do đó số hạng E biến mất và chúng ta nhận được:

$$\text{var}(w_ix_i) = \text{var}(w_i)\text{var}(x_i)$$

Lưu ý rằng b là hằng số và do có phương sai bằng 0 nên nó sẽ biến mất. Hãy thay thế vào phương trình ban đầu:

$$\text{var}(y) = \text{var}(w_1)\text{var}(x_1) + \text{var}(w_2)\text{var}(x_2) + \dots + \text{var}(w_N)\text{var}(x_N)$$

Vì chúng đều được phân bố giống nhau nên chúng ta có thể viết dưới dạng:

$$\text{var}(y) = N \times \text{var}(w_i)\text{var}(x_i)$$

Vì vậy, nếu chúng ta muốn phương sai của y bằng x , thì số hạng $N \times \text{var}(w_i)$ phải bằng 1. Do đó: $N \times \text{var}(w_i) = 1$, có nghĩa là $\text{var}(w_i) = \frac{1}{N}$.

Chúng ta đã đến công thức khởi tạo Xavier. Chúng ta cần chọn các trọng số từ phân bố Gaussian với giá trị trung bình bằng 0 và phương sai là $\frac{1}{N}$, trong đó N chỉ định số lượng nơ-ron đầu vào. Đây là cách nó được triển khai trong thư viện Caffe. Trong bài báo gốc, các tác giả lấy trung bình số lượng nơ-ron đầu vào và số nơ-ron đầu ra. Vì vậy công thức trở thành:

$$\text{var}(w_i) = \frac{1}{N_{avg}} \text{ với } N_{avg} = \frac{N_{in} + N_{out}}{2}$$

Lý do họ làm điều này là để bảo toàn tín hiệu truyền ngược nhưng nó phức tạp hơn về mặt tính toán để thực hiện. Do đó, chúng ta chỉ lấy số lượng nơ-ron đầu vào trong quá trình triển khai thực tế.

4.3.4 Khởi tạo He

Người ta nhận thấy rằng quá trình khởi tạo Glorot hay Xavier không hoạt động đối với các mạng sử dụng kích hoạt ReLU do lan truyền ngược của gradient bị ảnh hưởng. Cụ thể, không giống như kích hoạt sigmoid và tanh, hàm ReLU ánh xạ tất cả đầu vào âm về 0 ($\text{ReLU}(x) = \max(0, x)$), không có giá trị trung bình bằng 0. Do đó, hàm ReLU trả đầu ra 0 cho một nửa phổ đầu vào, trong khi hàm kích hoạt tanh và sigmoid cho đầu ra khác 0 cho tất cả các giá trị trong không gian đầu vào.

Kaiming He và cộng sự. đã giới thiệu một kỹ thuật khởi tạo mới có tính đến điều này bằng cách đưa ra hệ số 2 khi tính toán phương sai, có nghĩa là:

$$\text{var}(y) = \frac{N}{2} \times \text{var}(w_i) \times \text{var}(x_i)$$

$$\text{nên } \text{var}(w_i) = \frac{2}{N} \text{ hay } \text{var}(w_i) = \frac{2}{N_{\text{avg}}}.$$

4.4 Các kỹ thuật chuẩn hóa

4.4.1 Batch Normalization (BN)

Gần đây, việc chuẩn hóa đã được mở rộng tới các tầng ẩn của các mạng sâu mà các kích hoạt của chúng có thể được xem như là đầu vào đối với tầng tiếp theo. Kiểu chuẩn hóa này thay đổi kích hoạt của các đặc trưng ẩn nằm trong một phạm vi nhất định hoặc có một phân phối nhất định, độc lập với các thống kê đầu vào hoặc các tham số mạng (Ioffe và Szegedy, 2015).

Trong khi cơ sở lý thuyết giải thích tại sao phương pháp chuẩn hóa này cho phép cải thiện hiệu năng đã gây ra nhiều tranh luận—ví dụ, giảm sự dịch chuyển hiệp phương sai (Ioffe và Szegedy, 2015), làm mịn bề mặt hàm loss (Santurkar, Tsipras, Ilyas, và Madry, 2018), tách độ dài và hướng của vectơ trọng số (Kohler và cộng sự, 2019), và đóng vai trò như một bộ hiệu chỉnh (Wu và cộng sự, 2019)—việc chuẩn hóa này hiện là một thành phần tiêu chuẩn của các kiến trúc mạng hiện đại và đã được gọi là một trong những đổi mới quan trọng nhất gần đây để tối ưu hóa các mạng sâu (Kohler và cộng sự, 2019).

BN là một trong những phương pháp chuẩn hóa phổ biến trong học sâu. Nó cho phép huấn luyện mạng nơ-ron sâu nhanh hơn và ổn định hơn bằng cách điều chỉnh phân bố đầu vào của các tầng ẩn trong quá trình huấn luyện. BN chuẩn hóa các kích hoạt trong mạng theo mini-batch được xác định trước đó. Đối với mỗi đặc trưng ẩn, BN tính trung bình và phương sai của đặc trưng đó trong một mini-batch. Sau đó,

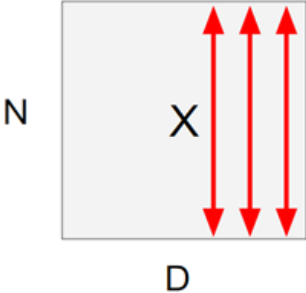
nó trừ đi giá trị trung bình và chia cho độ lệch chuẩn của mini-batch đó. Như vậy, chúng ta sẽ sử dụng BN như một tầng trong mạng sâu.

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{var}[x^{(k)}]}}$$

Về mặt trực quan, chúng ta biết rằng, trong gradient descent, mạng nơ-ron tính giá trị đạo hàm và cập nhật trọng số của nó dựa vào hướng đi của đạo hàm. Nhưng do các tầng được xếp chồng lên nhau, phân phối của dữ liệu đầu vào sẽ bị thay đổi dần do việc cập nhật trọng số của các tầng trước đó, làm cho phân phối của đầu vào của các tầng phía sau sẽ khác xa so với phân phối của đầu vào mạng. BN giúp ấn định lại phân phối của dữ liệu về phân phối chuẩn qua tất cả các tầng, dẫn tới tính chất phân phối của dữ liệu không thay đổi qua các tầng.

Ví dụ: Chúng ta muốn hàm kích hoạt có phân bố đều ra với trung bình bằng 0 và độ lệch chuẩn đơn vị theo kênh với mini-batch gồm có N ảnh đầu vào. Chúng ta biến đổi theo ý tưởng đó như sau:

Input: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is D}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is D}$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} \quad \text{Normalized x, Shape is N x D}$$

Tuy nhiên, ràng buộc kỳ vọng bằng 0 và độ lệch chuẩn đơn vị là quá chặt và có thể khiến mô hình bị underfitting. Chúng ta thực hiện nới lỏng cho mô hình bằng cách áp dụng một biến đổi tuyến tính (linear transformation) với hai tham số huấn luyện là γ và β khi tính output của tầng ẩn. Cụ thể output của tầng ẩn được tính như sau:

$$y = \gamma x + \beta$$

Bước này cho phép mô hình chọn được phân phối tối ưu cho từng tầng ẩn khi thay đổi hai tham số γ và β này, trong đó:

- γ giúp điều chỉnh phương sai phân phối

- β giúp điều chỉnh bias, dịch chuyển phân phối sang trái hay phải

Với mỗi lần lặp, mạng sẽ tính toán **trung bình** μ và **phương sai** σ^2 cho batch hiện tại. Sau đó nó huấn luyện γ và β bằng gradient descent và kết quả huấn luyện nếu $\gamma = \sigma$ và $\beta = \mu$ sẽ cho phép phục hồi lại hàm đồng nhất.

Khác với khi huấn luyện, chúng ta **không có batch đầy đủ để đưa vào mô hình và không thể tính kỳ vọng và phương sai theo lô dữ liệu (batch) lúc suy diễn**.

Để giải quyết vấn đề này, chúng ta tính $(\mu_{pop}, \sigma_{pop})$ với:

- μ_{pop} : ước lượng giá trị trung bình cho toàn bộ tập dữ liệu (population) được sử dụng huấn luyện.
- σ_{pop} : ước lượng giá trị độ lệch chuẩn cho toàn bộ tập dữ liệu (population) được sử dụng huấn luyện.

Hai giá trị này được tính toán sử dụng các giá trị $(\mu_{batch}, \sigma_{batch})$ của các mini-batch được tính trong quá trình huấn luyện và khi đó, BN đơn giản là một phép biến đổi tuyến tính.

Input: $x : N \times D$	$\mu_j =$ (Running) average of values seen during training	Per-channel mean, shape is D
Learnable scale and shift parameters: $\gamma, \beta : D$	$\sigma_j^2 =$ (Running) average of values seen during training	Per-channel var, shape is D
During testing batchnorm becomes a linear operator! Can be fused with the previous fully-connected or conv layer	$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$	Normalized x, Shape is N x D
	$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$	Output, Shape is N x D

Đến đây, chúng ta đã hiểu được BN có ý nghĩa quan trọng như thế nào trong lĩnh vực Học sâu. Tuy nhiên, BN có những tác dụng phụ và một số hạn chế mà chúng ta cần lưu tâm để tận dụng được nó:

- BN thực hiện lại các phép tính để chuẩn hóa qua các lần lặp, cho nên, về lý thuyết, chúng ta cần batch size đủ lớn để phân phối của mini-batch xấp xỉ phân phối của toàn bộ dữ liệu. Điều này gây khó khăn cho các mô hình đòi hỏi ảnh đầu vào có chất lượng cao như object detection, semantic segmentation,

... Việc huấn luyện với batch size lớn làm mô hình phải tính toán nhiều và chậm.

- Đối với batch size là 1, phương sai sẽ bằng 0 và BN sẽ không hoạt động. Đồng thời, nếu mini-batch nhỏ, nó sẽ tạo ra nhiễu và ảnh hưởng đến quá trình huấn luyện mô hình. Khi tính toán trên các hệ thống khác nhau, chúng ta cần đảm bảo batch size giống nhau vì các tham số sẽ khác nhau đối với các hệ thống khác nhau.
- BN phụ thuộc vào **trung bình** μ và **phương sai** σ^2 để chuẩn hoá giá trị kích hoạt. Vì thế mà kết quả đầu ra của BN sẽ bị ảnh hưởng bởi thống kê của batch hiện tại. Những sự biến đổi sẽ tạo thêm nhiễu phụ thuộc vào các dữ liệu đầu vào của batch hiện tại và rất có thể việc thêm nhiễu cũng sẽ giúp mạng sau khi huấn luyện tránh được overfitting.
- BN không hoạt động tốt với RNN. Lý do là RNN có các kết nối lặp lại với các timestamps trước đó và yêu cầu các giá trị γ và β khác nhau cho mỗi timestep, từ đó dẫn đến độ phức tạp tăng lên gấp nhiều lần, và gây khó khăn cho việc sử dụng BN trong RNN.
- Trong quá trình kiểm thử (test), BN không tính toán giá trị trung bình và phương sai của tập test mà sử dụng giá trị trung bình và phương sai được tính toán từ tập luyện, cụ thể, tầng BN sẽ sử dụng các giá trị trung bình và phương sai được tính toán từ trước trong dữ liệu huấn luyện để giúp không phải tính đi tính lại giá trị này.

BN có thể giải quyết một số vấn đề, như:

1. Chuẩn hóa dữ liệu mỗi đặc trưng giúp mô hình không thiên vị đối với các đặc trưng có giá trị cao hơn.
2. Giảm vấn đề *Internal Covariate Shift*, tức giảm sự biến đổi phân phối của dữ liệu trong quá trình huấn luyện.
3. Làm mịn bề mặt hàm loss trở nên mượt mà hơn, dễ tối ưu hóa.
4. Với bộ dữ liệu lớn, những cải thiện này càng quan trọng hơn là khi sử dụng những bộ dữ liệu nhỏ hơn như MNIST.
5. Thêm tầng BN cho phép chúng ta sử dụng **learning rate lớn hơn nhưng lại không hi sinh tính hội tụ của mô hình**.

6. Giới hạn độ lớn của gradients giúp quá trình tối ưu hóa nhanh chóng hơn.
7. Đóng góp một phần nhỏ vào việc Regularization của mô hình. Khi kích thước batch càng lớn thì tác động lên regularization càng ít đi do nhiều ảnh hưởng ít hơn.

Thêm tầng BN giúp giảm sự phụ thuộc lẫn nhau giữa các tầng (theo cách nhìn về sự ổn định phân phối) trong quá trình huấn luyện. Mà vì thế nên không cần xem xét tất cả tham số để hiểu về phân phối trong các tầng ẩn.

Do có BN, optimizer có thể thay đổi trọng số mạnh hơn mà không làm suy thoái các tham số đã được điều chỉnh trước đó của tầng ẩn khác. Điều này khiến việc điều chỉnh các siêu tham số (hyperparameter) dễ hơn rất nhiều!

4.4.2 Chuẩn hóa trọng số (Weight Normalization - WN)

WN là một phương pháp để cải thiện tính tối ưu của trọng số trong mạng nơ-ron. Nó tách độ dài của các vector trọng số khỏi hướng của chúng. WN được định nghĩa như sau:

$$w = \frac{g \times v}{\|v\|}$$

Việc tách các vector trọng số ra khỏi hướng của chúng giúp cải thiện tính tối ưu. Việc thực hiện WN có nhiều lợi ích, bao gồm:

1. Cải thiện điều kiện tối ưu và tăng tốc độ hội tụ của thuật toán gradient descent.
2. Áp dụng cho các mô hình hồi quy như LSTM và mô hình reinforcement learning.

4.4.3 Chuẩn hóa tầng (Layer Normalization - LN)

Trong khi BN dùng batch hiện tại để chuẩn hoá từng giá trị kích hoạt, LN thì dùng tất cả các tầng hiện tại. Nói cách khác, LN chuẩn hoá trên toàn bộ các đặc trưng của dữ liệu thay vì theo từng đặc trưng như BN. Điều này làm LN hiệu quả hơn với RNN vì RNN sử dụng phép tính lặp đi lặp lại với cùng một bộ trọng số cho các time-step. Vì vậy, chúng ta nên chuẩn hoá theo từng time-step một cách độc lập.

LN là một phương pháp chuẩn hóa tức thời đầu vào cho mỗi tầng trong mạng nơ-ron. Phương pháp này không phụ thuộc vào kích thước batch và phụ thuộc vào tầng do LN chuẩn hóa đầu vào trên các tầng thay vì chuẩn hóa từng mini-batch.

LN có một số lợi ích, như:

1. Dễ dàng áp dụng cho mô hình RNN bằng cách tính toán thống kê chuẩn hóa riêng biệt cho từng time-step.
2. Hiệu quả trong việc ổn định trạng thái ẩn trong các mạng hồi quy.

4.4.4 Chuẩn hóa mẫu (Instance Normalization - IN)

IN tương tự như LN, tuy nhiên, nó chuẩn hóa qua từng kênh trong từng ví dụ huấn luyện thay vì chuẩn hóa qua các đặc trưng đầu vào trong một ví dụ huấn luyện. IN độc lập với kích thước batch và cho phép áp dụng trong quá trình thử nghiệm mô hình trên các ảnh mới. IN có một số lợi ích, như:

1. Đơn giản hóa quá trình huấn luyện mô hình.
2. Có thể áp dụng trong quá trình thử nghiệm mô hình.

4.4.5 Chuẩn hóa nhóm (Group Normalization - GN)

GN là một giải pháp thay thế cho BN. Phương pháp này chia các kênh thành các nhóm và chuẩn hóa từng nhóm riêng biệt. GN độc lập với kích thước batch và ổn định trong nhiều loại kích thước batch. GN có một số lợi ích, như:

1. Thay thế BN trong một số bài toán Học sâu.
2. Dễ dàng triển khai.

Trong việc hiệu quả của chuẩn hóa, BN vẫn là phương pháp tốt nhất và được sử dụng rộng rãi. Tuy nhiên, khi mô hình của bạn gặp vấn đề khó hội tụ và chạy lâu, bạn có thể thử các phương pháp khác phù hợp với bài toán của bạn.

4.5 Chiến lược thay đổi tốc độ học

Các nhà nghiên cứu thường cho rằng các mô hình mạng nơ-ron rất khó huấn luyện. Một trong những vấn đề lớn nhất là số lượng lớn các siêu tham số cần xác định và tối ưu hóa như số lượng lớp ẩn, số nơ-ron đối với mỗi lớp ẩn, tốc độ học, hiệu chỉnh mô hình, ... v.v.

Việc điều chỉnh các siêu tham số này có thể cải thiện đáng kể các mô hình mạng nơ-ron. Việc xây dựng một mô hình mạng nơ-ron được coi là giải quyết vấn đề tối ưu hóa. Chúng ta cần tìm cực tiểu (toàn cục hoặc đôi khi cục bộ) của hàm mục tiêu bằng các phương pháp dựa trên gradient như giảm gradient.

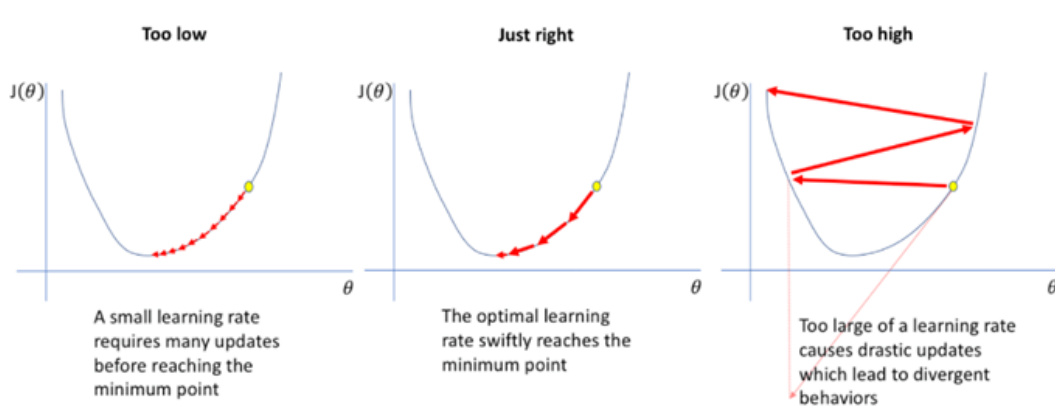
Trong số tất cả các siêu tham số, tốc độ học là một trong những yếu tố quan trọng nhất để đạt được hiệu năng tốt cho mô hình. Trong mục này, chúng ta sẽ khám phá tốc độ học và giải thích lý do tại sao chiến lược thay đổi tốc độ học trong quá trình huấn luyện mô hình lại quan trọng.

Tốc độ học (hoặc kích thước bước) được giải thích là mức độ thay đổi/cập nhật đối với trọng số mô hình trong quá trình huấn luyện lan truyền ngược lỗi. Là một siêu tham số về cấu hình, tốc độ học thường được chỉ định là giá trị dương nhỏ hơn 1.0. Trong lan truyền ngược, trọng số mô hình được cập nhật để giảm giá trị lỗi của hàm mất mát. Thay vì thay đổi trọng số bằng cách sử dụng toàn bộ giá trị lỗi trọng số tính được, chúng ta nhân nó với một số giá trị tốc độ học. Ví dụ: đặt tốc độ học thành 0.5 có nghĩa là cập nhật (thường trừ) các trọng số với một lượng bằng 0.5 nhân với các lỗi trọng số tính được (tức là các gradient hoặc sự thay đổi lỗi tổng thể ứng với trọng số).

Tốc độ học kiểm soát độ lớn của bước để trình tối ưu hóa đạt đến cực tiểu của hàm mất mát. Trong trường hợp tốc độ học không đổi, có nghĩa là, chúng ta khởi tạo một giá trị cho tốc độ học và không thay đổi nó trong quá trình huấn luyện. Chúng ta hãy quan sát ảnh hưởng của các giá trị tốc độ học khác nhau trong hình sau:

Các trường hợp về tốc độ học

- Với tốc độ học lớn (hình bên phải), thuật toán học nhanh nhưng cũng có thể khiến thuật toán dao động xung quanh hoặc thậm chí nhảy qua giá trị cực tiểu. Tệ hơn nữa, tốc độ học cao đồng nghĩa với việc cập nhật trọng số lớn, điều này có thể khiến trọng số bị tràn;
- Ngược lại, với tốc độ học nhỏ (hình bên trái), các cập nhật về trọng số nhỏ,



Hình 4.5: Các trường hợp về tốc độ học. Trái: Tốc độ học quá nhỏ, Giữa: Tốc độ học tốt, Phải: Tốc độ học quá lớn.

điều này sẽ hướng trình tối ưu hóa dần dần về phía cực tiểu. Tuy nhiên, trình tối ưu hóa có thể mất quá nhiều thời gian để hội tụ hoặc bị kẹt ở trạng thái ổn định hoặc cực tiểu cục bộ không mong muốn.

- Tốc độ học tốt là sự cân bằng giữa độ bao phủ và độ nhảy vọt (hình giữa). Nó không quá nhỏ để thuật toán của chúng ta có thể hội tụ nhanh chóng và nó không quá lớn để thuật toán của chúng ta không nhảy qua nhảy lại mà không đạt đến cực tiểu.

Mặc dù nguyên tắc lý thuyết về việc tìm ra tốc độ học thích hợp rất đơn giản (không quá lớn, không quá nhỏ), nhưng nói thì dễ hơn làm! Để giải quyết vấn đề này, các chiến lược thay đổi tốc độ học được đưa ra.

Chiến lược thay đổi tốc độ học là một khung được xác định trước để điều chỉnh tốc độ học giữa các chu kỳ hoặc các lần lặp khi thực hiện quá trình huấn luyện. Thông thường, chúng ta chọn tốc độ học ban đầu với giá trị lớn và giảm dần theo thời gian.

Đối với một quá trình huấn luyện, việc giảm dần tốc độ học thường là tốt. Trong giai đoạn đầu của huấn luyện, tốc độ học được đặt ở mức lớn để đạt được tập trọng số đủ tốt. Theo thời gian, các trọng số này được tinh chỉnh để đạt độ chính xác cao hơn bằng cách tận dụng tốc độ học nhỏ. Sau đây là một số chiến lược giảm tốc độ học có thể áp dụng:

- **Thay đổi tốc độ học tại một số thời điểm cố định.** Ví dụ, ResNets có thể giảm tốc độ học 10 lần tại các epochs 30, 60 và 90.

- Giảm theo cosin:

$$\alpha_t = \frac{1}{2}\alpha_0 \left[1 + \cos\left(\frac{t\pi}{T}\right) \right]$$

- Giảm tuyến tính:

$$\alpha_t = \alpha_0 \left(1 - \frac{t}{T} \right)$$

- Tỷ lệ nghịch căn bậc hai số epoch:

$$\alpha_t = \frac{\alpha_0}{\sqrt{t}}$$

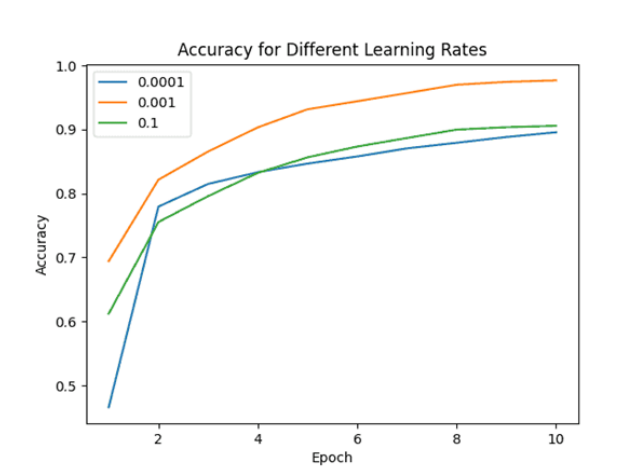
trong đó α_0 là tốc độ học ban đầu/khởi tạo, α_t là tốc độ học tại epoch t , T là tổng số epoch.

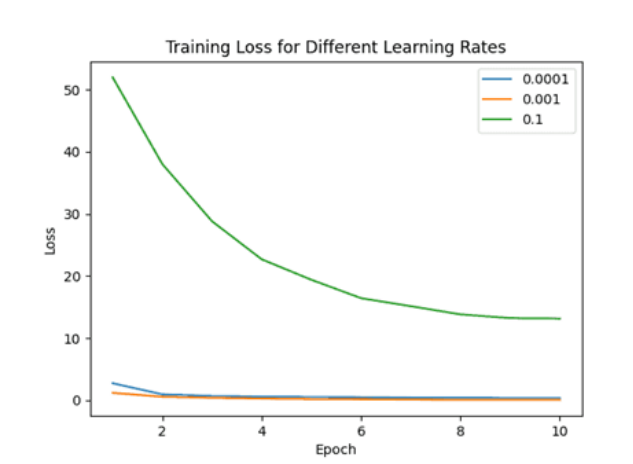
Chúng ta cũng có thể chọn sử dụng các trình tối ưu hóa như Adagrad, RMSProp, Adam,... để điều chỉnh tốc độ học của mỗi tham số trong mạng nơ-ron một cách thích nghi trong quá trình huấn luyện. Cụ thể, nó chia tốc độ học của từng tham số cho một đại lượng dựa trên lịch sử các gradient được tính toán cho tham số đó. Nói cách khác, các tham số có gradient lớn sẽ có tốc độ học nhỏ hơn, trong khi các tham số có gradient nhỏ sẽ có tốc độ học lớn hơn. Điều này giúp ngăn tốc độ học giảm quá nhanh đối với các tham số có tần suất cao và cho phép quá trình huấn luyện hội tụ nhanh hơn.

Các ví dụ về tốc độ học khác nhau

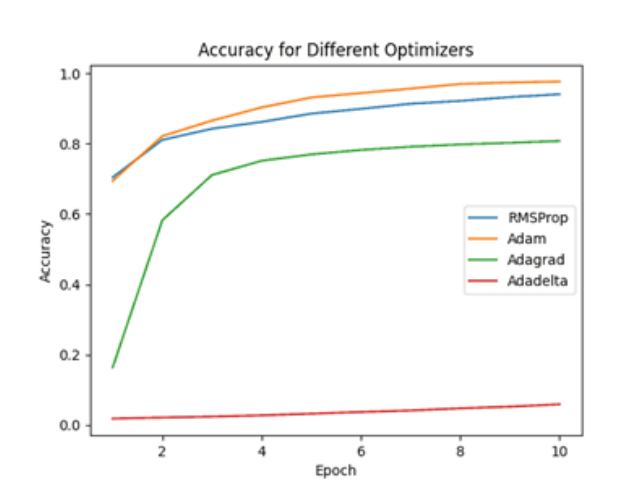
Để minh họa hiệu quả của việc sử dụng cùng một trình tối ưu hóa với tốc độ học khác nhau, chúng ta đã sử dụng thuật toán Adam để huấn luyện mạng nơ-ron nhận dạng các giống chó trong số 120 lớp.

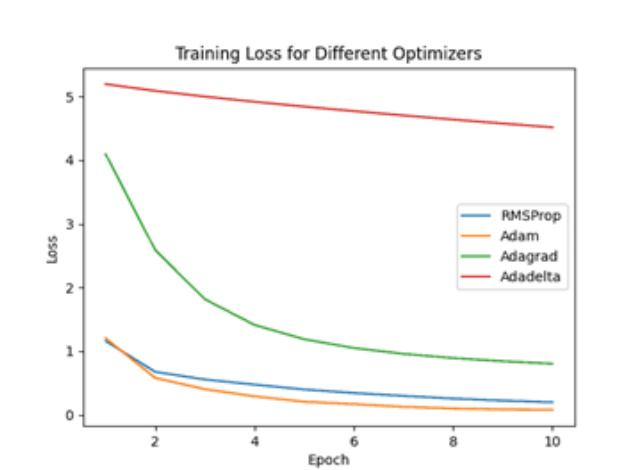
Chúng ta có thể dễ dàng nhận thấy ảnh hưởng của việc sử dụng ba tốc độ học khác nhau với cùng một trình tối ưu.





Chúng ta có thể thấy rõ tốc độ học 0.001 vượt trội hơn các kịch bản thử nghiệm khác như thế nào, chứng minh rằng trong trường hợp này, đó là giá trị tối ưu. Cuối cùng, chúng ta cũng so sánh hiệu năng huấn luyện bằng cách sử dụng cùng một kiến trúc mạng nơ ron với các cách tiếp cận khác nhau để xác định tốc độ học.





Cần nhấn mạnh rằng chúng tôi đã sử dụng cùng một giá trị cho tốc độ học trong tất cả các tình huống với các tham số được thiết lập theo giá trị mặc định của nó.

Để nhận biết các giống chó, trình tối ưu hóa Adam có độ chính xác cao hơn và hàm mất mát thấp hơn, nó được coi là chiến lược hoặc dự đoán đầu tiên để có được mô hình ưng ý.

Kết luận

Trong mục này, chúng ta đã thảo luận về các chiến lược khác nhau để cập nhật trọng số trong giai đoạn huấn luyện của mô hình học máy. Ví dụ này cho thấy tốc độ học hoặc chiến lược khác nhau có thể ảnh hưởng đáng kể đến kết quả đầu ra của ứng dụng trong giai đoạn huấn luyện và đánh giá.

4.6 Các thuật toán tối ưu cho mạng nơ-ron

Thuật toán tối ưu là phương pháp tối ưu hóa giúp cải thiện hiệu năng của mô hình học sâu. Các thuật toán tối ưu hóa hoặc trình tối ưu hóa này ảnh hưởng đến độ chính xác và tốc độ huấn luyện của mô hình học sâu. Trình tối ưu hóa là một hàm hoặc một thuật toán điều chỉnh các thuộc tính của mạng nơ-ron, chẳng hạn như trọng số và tốc độ học, giúp giảm mất mát tổng thể và cải thiện độ chính xác. Vấn đề chọn trọng số phù hợp cho mô hình là một nhiệm vụ khó khăn vì mô hình học sâu thường bao gồm hàng triệu trọng số. Vì vậy, việc hiểu các mô hình học máy để lựa chọn một thuật toán tối ưu hóa phù hợp cho ứng dụng của chúng ta là cần thiết đối với các nhà khoa học dữ liệu trước khi đi sâu vào lĩnh vực này.

Chúng ta có thể sử dụng các trình tối ưu hóa khác nhau để thay đổi trọng số và tốc độ học trong mô hình học máy của mình. Tuy nhiên, việc chọn trình tối ưu hóa

tốt nhất thường phụ thuộc vào ứng dụng. Khi mới bắt đầu, chúng ta thường thử tất cả các khả năng và chọn một khả năng cho kết quả tốt nhất. Điều này ban đầu có thể ổn, nhưng khi xử lý hàng trăm gigabyte dữ liệu, ngay cả một chu kỳ cũng có thể mất thời gian đáng kể. Vì vậy, việc chọn ngẫu nhiên một thuật toán là không thực tế.

Phần này sẽ đề cập đến nhiều trình tối ưu hóa học sâu khác nhau, chẳng hạn như Giảm gradient, Giảm gradient ngẫu nhiên, Giảm gradient ngẫu nhiên với momentum, Giảm gradient theo mini-batch, Adagrad, RMSProp, AdaDelta và Adam. Cuối cùng, chúng ta có thể so sánh các trình tối ưu hóa khác nhau và quá trình tối ưu dựa trên chúng.

4.6.1 Giảm gradient ngẫu nhiên (Stochastic Gradient Descent – SGD)

Giảm gradient (Gradient Descent - GD) là một trình tối ưu rất phổ biến mà hoạt động tốt cho hầu hết các mục đích. Tuy nhiên, nó cũng có một số nhược điểm sau:

- Việc tính toán gradient sẽ tốn kém nếu kích thước của dữ liệu lớn.
- Hơn nữa, GD hoạt động tốt chỉ đối với các hàm lồi, nhưng nó không biết phải di chuyển bao xa dọc theo gradient đối với các hàm không lồi.

Để giải quyết những thách thức mà các tập dữ liệu lớn đặt ra, chúng ta sử dụng phương pháp GD ngẫu nhiên, một cách tiếp cận phổ biến trong số các trình tối ưu hóa trong học sâu. Trong phương pháp GD ngẫu nhiên, thay vì sử dụng toàn bộ tập dữ liệu trong mỗi lần lặp, chúng ta chọn ngẫu nhiên các lô dữ liệu. Điều này ngụ ý rằng tại một thời điểm chỉ có một vài mẫu từ tập dữ liệu được xem xét nên cho phép tối ưu hóa hiệu quả hơn và khả thi về mặt tính toán hơn trong các mô hình học sâu.

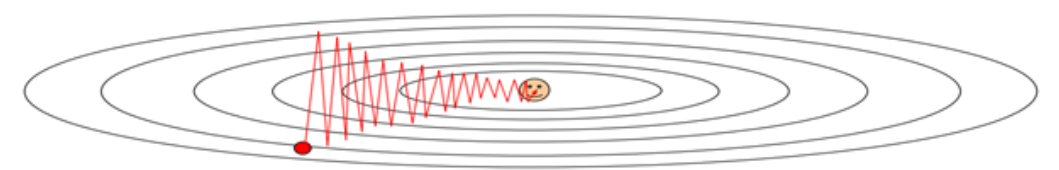
Ý tưởng cơ bản của SGD là lấy mẫu một tập con ngẫu nhiên nhỏ của dữ liệu huấn luyện, được gọi là mini-batch và tính gradient của hàm mất mát đối với các tham số mô hình chỉ sử dụng tập con đó. Gradient này sau đó được sử dụng để cập nhật các tham số. Quá trình này được lặp lại với một mini-batch ngẫu nhiên mới cho đến khi thuật toán hội tụ hoặc đạt đến tiêu chí dừng được xác định trước.

SGD có một số lợi thế so với GD, chẳng hạn như tốc độ hội tụ nhanh hơn và yêu cầu bộ nhớ thấp hơn, đặc biệt đối với các tập dữ liệu lớn. Nó cũng mạnh mẽ hơn đối với dữ liệu nhiễu và không ổn định, đồng thời có thể thoát khỏi điểm cực

tiểu cục bộ. Tuy nhiên, nó có thể yêu cầu nhiều lần lặp lại để hội tụ hơn GD và tốc độ học cần được điều chỉnh cẩn thận để đảm bảo sự hội tụ.

Một số vấn đề cần quan tâm khi sử dụng SGD:

- Khi hàm mục tiêu có số điều kiện lớn, có nghĩa là tỉ lệ giữa giá trị riêng lớn nhất và giá trị riêng nhỏ nhất của ma trận Hessian là lớn, hàm mục tiêu sẽ thay đổi nhanh theo một chiều và thay đổi chậm theo chiều khác. Điều này dẫn đến thuật toán hội tụ rất chậm do luôn nhảy từ bên này qua bên kia bề mặt hàm mục tiêu.



- Hơn thế nữa, hàm mục tiêu của các mô hình học sâu thường rất nhiều biến nên thường có nhiều điểm cực tiểu cục bộ và bề mặt của hàm thường xuất hiện điểm yên ngựa (saddle point). Khi đó, gradient là bằng 0 và thuật toán SGD sẽ bị tắc tại các điểm đó.
- Cuối cùng, trong SGD, do chúng ta không sử dụng toàn bộ tập dữ liệu mà sử dụng các lô của tập dữ liệu cho mỗi lần lặp, nên đường đi của thuật toán có nhiều nhiễu so với thuật toán GD. Do đó, SGD sử dụng số lần lặp cao hơn để đạt cực tiểu cục bộ. Do số lần lặp tăng lên nên thời gian tính toán tổng thể tăng lên. Nhưng ngay cả sau khi tăng số lần lặp, chi phí tính toán vẫn thấp hơn so với trình tối ưu hóa GD. Vì vậy, kết luận là nếu dữ liệu rất lớn và thời gian tính toán là một yếu tố thiết yếu, thì SGD ngẫu nhiên nên được ưu tiên hơn thuật toán GD.

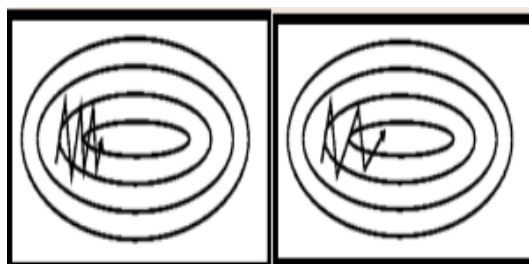
4.6.2 Giảm gradient ngẫu nhiên với momentum (SGD With Momentum)

Như đã thảo luận trong phần trước, chúng ta đã biết SGD có đường đi nhiễu hơn nhiều so với thuật toán GD khi sử dụng các trình tối ưu hóa trong học sâu. Do đó, nó đòi hỏi số lần lặp cao hơn đáng kể hơn để đạt được cực tiểu tối ưu và do đó, thời gian tính toán rất chậm. Để khắc phục vấn đề này, chúng ta sử dụng thuật toán SGD với momentum.

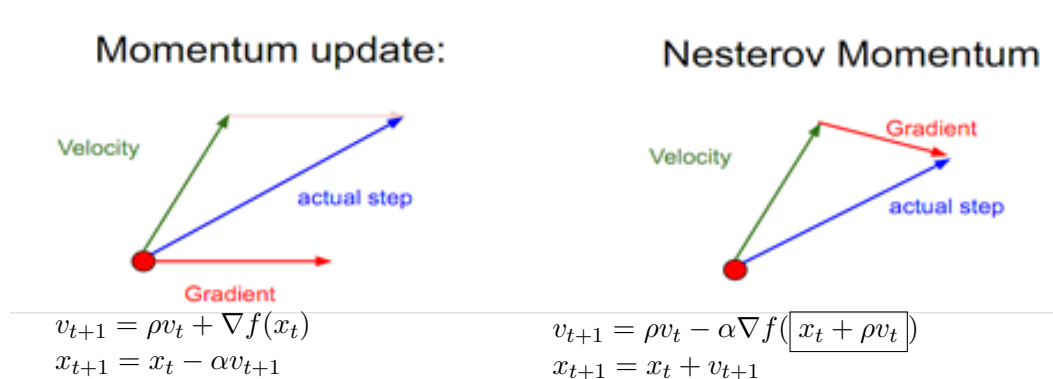
SGD dao động giữa các hướng của các gradient và cập nhật các trọng số tương ứng. Tuy nhiên, việc thêm một phần bản cập nhật trước đó vào bản cập nhật hiện

tại sẽ giúp quá trình cập nhật nhanh hơn. Cụ thể, trong SGD với momentum, chúng ta xây dựng một đại lượng “vận tốc” bằng trung bình dịch chuyển của gradients với lực ma sát ρ thường bằng 0.9 hoặc 0.99. Tại thời điểm ban đầu, ρ có thể thấp hơn do hướng di chuyển chưa rõ ràng, ví dụ $\rho = 0.5$.

Một điều cần được ghi nhớ khi sử dụng thuật toán này là tốc độ học cần giảm với một momentum cao.



Trong hình ảnh trên, phía bên trái hiển thị đường đi của thuật toán SGD và phía bên phải hiển thị SGD với momentum để đạt cực tiểu. Từ hình ảnh, chúng ta có thể so sánh đường đi được chọn bởi cả hai thuật toán và nhận ra rằng việc sử dụng momentum giúp đạt được sự hội tụ trong thời gian ngắn hơn. Chúng ta có thể nghĩ đến việc sử dụng momentum và tốc độ học lớn để khiến quá trình diễn ra nhanh hơn nữa. Nhưng hãy nhớ rằng khi tăng momentum, khả năng vượt qua cực tiểu tối ưu cũng tăng lên. Điều này có thể dẫn đến độ chính xác kém và thậm chí dao động nhiều hơn.



Nesterov Momentum

Thay vì đánh giá gradient ở vị trí hiện tại (hình tròn màu đỏ), chúng ta biết rằng momentum sẽ đưa chúng ta đến đầu mũi tên màu xanh lá cây. Do đó, với Nesterov Momentum, thay vào đó, chúng ta đánh giá gradient ở vị trí "nhìn trước" này.

Nesterov Momentum là một phiên bản hơi khác của bản cập nhật momentum

và hiện đang trở nên phổ biến trong thời gian gần đây. Nó có sự đảm bảo hội tụ về mặt lý thuyết mạnh hơn cho các hàm lồi và trong thực tế và cũng hoạt động tốt hơn một chút so với momentum chuẩn.

4.6.3 Adagrad (Adaptive Gradient Descent)

SGD là một kỹ thuật tối ưu hóa được sử dụng rộng rãi để huấn luyện các mô hình học máy, đặc biệt là mạng sâu. Tuy nhiên, SGD có một số hạn chế, đặc biệt khi xử lý các bề mặt tối ưu hóa phức tạp. Một thách thức đáng kể là việc lựa chọn tốc độ học chung cho các tham số của mô hình. Nếu tốc độ học quá cao, mô hình có thể vượt qua điểm cực tiểu và nếu quá thấp, quá trình huấn luyện có thể trở nên cực kỳ chậm và có thể bị kẹt ở điểm cực tiểu cục bộ hoặc điểm yên ngựa.

Thuật toán Adagrad điều chỉnh tốc độ học của mỗi tham số trong mạng nơ-ron một cách thích nghi trong quá trình huấn luyện. Cụ thể, nó chia tốc độ học của từng tham số cho một đại lượng dựa trên gradient lịch sử được tính toán cho tham số đó. Nói cách khác, các tham số có gradient lớn sẽ có tốc độ học nhỏ hơn, trong khi các tham số có gradient nhỏ sẽ có tốc độ học lớn hơn. Điều này giúp ngăn tốc độ học giảm quá nhanh đối với các tham số có tần suất cao và cho phép quá trình huấn luyện hội tụ nhanh hơn.

Thuật toán Adagrad đặc biệt hữu ích để xử lý dữ liệu thưa thớt, trong đó một số đặc trưng đầu vào có tần suất thấp hoặc bị thiếu. Trong những trường hợp này, Adagrad có thể điều chỉnh tốc độ học của từng tham số một cách thích nghi, cho phép xử lý dữ liệu thưa thớt tốt hơn.

Nhìn chung, Adagrad là một thuật toán tối ưu hóa mạnh mẽ có thể giúp đẩy nhanh quá trình huấn luyện mạng sâu và cải thiện hiệu năng của chúng.

Thuật toán Adagrad cập nhật các tham số theo các bước sau:

1. Tính gradient: $g_t = \nabla_{\theta} L(\theta)$, ở đây $L(\theta)$ là hàm mất mát.
2. Tích lũy bình phương đạo hàm: $E[g^2]_t = E[g^2]_{t-1} + g_t^2$, khởi tạo $E = 0$.
3. Tính tốc độ học thích nghi: $\eta_t = \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}}$, trong đó η là tốc độ học ban đầu và ϵ là một hằng số nhỏ để ngăn chặn chia cho 0 và thường đặt là 10^{-8} .
4. Cập nhật các tham số: $\theta_{t+1} = \theta_t - \eta_t \times g_t$.

Các bước này được lặp lại với mỗi tham số trong mạng cho đến khi hội tụ hoặc đã đạt đến số lần lặp cực đại.

Lợi ích của việc sử dụng Adagrad là không phải thay đổi tốc độ học một cách thủ công, trong đó, mỗi trọng số sẽ có tốc độ học riêng: “per-parameter learning rates” hoặc “adaptive learning rates”. Tốc độ học của mỗi trọng số tỉ lệ nghịch với tổng bình phương độ lớn đạo hàm riêng của hàm mục tiêu đối với trọng số đó ở các bước trước. Do vậy, tốc độ di chuyển theo hướng dốc được hãm dần và tốc độ di chuyển theo hướng thoải được tăng tốc.

Adagrad đáng tin cậy hơn các thuật toán GD và các biến thể của chúng, đồng thời đạt được sự hội tụ ở tốc độ cao hơn.

Một nhược điểm của trình tối ưu hóa AdaGrad là nó làm giảm tốc độ học quá mạnh và đơn điệu. Có thể có một thời điểm tốc độ học trở nên cực kỳ nhỏ do bình phương độ lớn đạo hàm riêng ở mẫu số tiếp tục tích lũy và tiếp tục tăng. Do tốc độ học nhỏ, mô hình cuối cùng không thể thu được nhiều tri thức hơn và độ chính xác của mô hình bị ảnh hưởng.

4.6.4 RMS Prop (Root Mean Square Propagation)

Giống như Adagrad, RMSProp điều chỉnh tốc độ học của từng tham số trong quá trình huấn luyện. Tuy nhiên, thay vì tích lũy tất cả các gradient trong quá khứ như Adagrad, RMSProp tính trung bình động của các gradient bình phương thông qua một hệ số suy giảm hàm mũ. Điều này cho phép thuật toán điều chỉnh tốc độ học trơn tru hơn và ngăn tốc độ học giảm quá nhanh.

Thuật toán RMSProp cũng sử dụng hệ số suy giảm này để kiểm soát ảnh hưởng của gradient trong quá khứ đến tốc độ học, cho trọng số cao hơn đối với các gradient gần đây và cho trọng số nhỏ hơn đối với các gradient xa hơn.

Một trong những ưu điểm chính của RMSProp so với Adagrad là nó có thể xử lý các mục tiêu không ổn định, trong đó hàm mục tiêu mà mạng nơ-ron đang cố gắng xấp xỉ thay đổi theo thời gian. Trong những trường hợp này, Adagrad có thể hội tụ quá nhanh, nhưng RMSProp có thể điều chỉnh tốc độ học cho phù hợp với hàm mục tiêu đang thay đổi.

Nhìn chung, RMSProp là một thuật toán tối ưu hóa mạnh có thể giúp đẩy nhanh quá trình huấn luyện mạng sâu và cải thiện hiệu năng của chúng, đặc biệt trong các tình huống mà hàm mục tiêu không ổn định.

Thuật toán RMSProp cập nhật các tham số theo các bước như Adagrad với bước tích lũy bình phương đạo hàm: $E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2$, ở đây β là hệ số suy giảm, thường được đặt là 0.9.

4.6.5 Adam (Adaptive Moment Estimation)

Adam là một thuật toán tối ưu hóa được sử dụng trong học máy và học sâu để tối ưu hóa việc huấn luyện mạng nơ-ron.

Adam kết hợp các khái niệm của momentum và RMSProp. Nó duy trì một giá trị trung bình động của các moment thứ nhất và thứ hai của gradient, tương ứng là giá trị trung bình và phương sai của các gradient. Giá trị trung bình động của moment thứ nhất, tương tự như hạng thức momentum trong các thuật toán tối ưu hóa khác, giúp trình tối ưu hóa tiếp tục di chuyển theo cùng một hướng ngay cả khi các gradient trở nên nhỏ hơn. Trung bình động của moment thứ hai, tương tự như hạng thức RMSProp, giúp trình tối ưu hóa xác định tốc độ học cho từng tham số dựa trên phương sai của các gradient.

Bằng cách kết hợp các tính chất tốt của các trình tối ưu khác, trình tối ưu hóa Adam đạt được tốc độ học thích nghi để có thể điều hướng trên bề mặt tối ưu một cách hiệu quả trong quá trình huấn luyện. Khả năng thích nghi này giúp hội tụ nhanh hơn và cải thiện hiệu năng của mạng nơ-ron.

Trong thực tế, Adam hiện được khuyến nghị sử dụng làm thuật toán mặc định và thường hoạt động tốt hơn RMSProp một chút. Tuy nhiên, chúng ta có thể thử SGD+Nesterov Momentum. SGD+Momentum thường tốt hơn Adam nhưng cần phải tinh chỉnh tốc độ học và lên chiến lược thay đổi tốc độ học hợp lý.

Tóm lại, SGD là một thuật toán rất cơ bản và hiện nay hầu như không được sử dụng trong các ứng dụng do tốc độ tính toán chậm. Một vấn đề nữa với thuật toán đó là tốc độ học không đổi với mỗi chu kỳ. Hơn nữa, nó không có khả năng xử lý tốt các điểm yên ngựa. Nhìn chung, Adagrad hoạt động tốt hơn so với SGD do tốc độ học được cập nhật thường xuyên. Nó tốt nhất khi được sử dụng để xử lý dữ liệu thưa thớt.

RMSProp hiển thị các kết quả tương tự với kết quả của thuật toán GD với momentum, nó chỉ khác ở cách các gradient được tính.

Cuối cùng là trình tối ưu hóa Adam kế thừa các đặc tính hay của RMSProp và các thuật toán khác. Kết quả của trình tối ưu hóa Adam nhìn chung tốt hơn mọi thuật toán tối ưu hóa khác, có thời gian tính toán nhanh hơn và yêu cầu ít tham số hơn để điều chỉnh. Vì tất cả những điều đó, Adam được khuyến dùng làm trình tối ưu hóa mặc định cho hầu hết các ứng dụng. Việc chọn trình tối ưu hóa Adam cho ứng dụng có thể mang lại xác suất tốt nhất để nhận được kết quả tốt nhất.

Nhưng cuối cùng, chúng ta cần biết rằng ngay cả trình tối ưu hóa Adam cũng có một số nhược điểm. Ngoài ra, có những trường hợp các thuật toán như SGD có thể

có lợi và hoạt động tốt hơn trình tối ưu hóa Adam. Vì vậy, điều quan trọng nhất là phải biết yêu cầu và loại dữ liệu đang xử lý để chọn thuật toán tối ưu hóa tốt nhất và đạt được kết quả vượt trội.

4.7 Một số kỹ thuật chống *overfitting*

4.7.1 Dropout: Một kỹ thuật hiệu chỉnh mạnh đối với mạng sâu

Mạng sâu (Deep Neural Networks - DNN) đã trở nên phổ biến trong các ứng dụng học máy khác nhau như nhận dạng hình ảnh, nhận dạng giọng nói và xử lý ngôn ngữ tự nhiên. Tuy nhiên, DNN thường gặp phải tình trạng quá khớp, xảy ra khi mô hình được huấn luyện để quá khớp với dữ liệu huấn luyện và không thể khái quát hóa đối với dữ liệu mới. Dropout là một kỹ thuật hiệu chỉnh giúp ngăn chặn việc quá khớp trong DNN.

Dropout là một kỹ thuật hiệu chỉnh trong đó nó loại bỏ ngẫu nhiên (tức là đặt thành 0) một số nơ-ron trong mạng nơ-ron trong quá trình huấn luyện. Ý tưởng đằng sau dropout là nó buộc mạng phải học các cách biểu diễn dư thừa của dữ liệu đầu vào. Bằng cách loại bỏ ngẫu nhiên các nơ-ron, mạng trở nên ít nhạy cảm hơn với trọng số cụ thể của từng nơ-ron riêng lẻ và mạnh mẽ hơn đối với dữ liệu đầu vào bị nhiễu. Dropout có thể được coi là việc huấn luyện một tập hợp gồm nhiều mạng nơ-ron, mỗi mạng có các tập nơ-ron khác nhau bị loại bỏ ngẫu nhiên.

Dropout được thực hiện trong giai đoạn huấn luyện của mạng nơ-ron. Trong quá trình huấn luyện, mỗi nơ-ron trong mạng được giữ lại với xác suất p hoặc bị loại bỏ với xác suất $1 - p$. Xác suất p là một siêu tham số có thể được điều chỉnh để đạt được mức độ hiệu chỉnh mong muốn. Thông thường, p được đặt trong khoảng từ 0.2 đến 0.5.

Trong bước truyền xuôi tín hiệu đầu vào của quá trình huấn luyện, mạng tính toán đầu ra bằng cách chỉ sử dụng các nơ-ron được giữ lại. Trong bước truyền ngược, các gradient chỉ được tính cho các nơ-ron được giữ lại. Trọng số của các nơ-ron bị loại bỏ không được cập nhật trong quá trình huấn luyện. Điều này tương đương với việc tạm thời loại bỏ các nơ-ron khỏi mạng.

Tại thời điểm kiểm thử (test), dropout bị tắt và toàn bộ mạng được sử dụng để tính toán đầu ra. Tuy nhiên, để đảm bảo rằng giá trị kỳ vọng của đầu ra là như nhau trong quá trình huấn luyện và kiểm tra, trọng số của các nơ-ron được giữ lại được tính theo xác suất giữ lại p .

Ưu điểm:

1. Giảm hiện tượng quá khớp: Dropout là một kỹ thuật hiệu chỉnh mạnh mẽ có thể làm giảm đáng kể tình trạng quá khớp trong DNN. Bằng cách loại bỏ ngẫu nhiên các nơ-ron trong quá trình huấn luyện, mạng học cách khái quát hóa dữ liệu mới tốt hơn.
2. Hội tụ nhanh hơn: Dropout có thể tăng tốc độ hội tụ của mạng trong quá trình huấn luyện. Bằng cách ngăn mạng khởi phụ thuộc quá nhiều vào trọng số cụ thể của từng nơ-ron riêng lẻ, dropout khuyến khích mạng học các cách biểu diễn mạnh mẽ hơn của dữ liệu đầu vào.
3. Hiệu suất tốt hơn: Dropout có thể cải thiện hiệu suất của DNN, đặc biệt khi dữ liệu huấn luyện bị hạn chế. Bằng cách huấn luyện một tập hợp nhiều mạng nơ-ron, dropout có thể nắm bắt được một vùng rộng hơn của các đặc trưng và tạo ra khả năng khái quát hóa tốt hơn.

Nhược điểm:

1. Tăng thời gian huấn luyện: Dropout đòi hỏi phải huấn luyện nhiều mạng nơ-ron, điều này có thể làm tăng đáng kể thời gian huấn luyện. Tuy nhiên, những tiến bộ gần đây về phần cứng và phần mềm đã khiến dropout trở nên thực tế hơn đối với các mạng nơ-ron quy mô lớn.
2. Khó khăn trong việc điều chỉnh siêu tham số: Dropout có một số siêu tham số cần được điều chỉnh, chẳng hạn như xác suất dropout và tốc độ học. Việc điều chỉnh các siêu tham số này có thể khó khăn và tốn thời gian.

Tóm lại

Mặc dù dropout có một số hạn chế nhưng ưu điểm của nó vượt xa những nhược điểm. Dropout đã trở thành một kỹ thuật tiêu chuẩn trong học sâu và được sử dụng rộng rãi trong các ứng dụng học máy khác nhau.

4.8 Làm giàu dữ liệu (data augmentation)

Độ chính xác của các mô hình học sâu phần lớn phụ thuộc vào chất lượng, số lượng và thông tin ngữ cảnh của dữ liệu huấn luyện. Tuy nhiên, sự khan hiếm dữ liệu là một trong những thách thức phổ biến nhất trong việc xây dựng các mô hình deep learning. Trong các ứng dụng học sâu thực tế, việc thu thập dữ liệu và ghi nhãn dữ liệu thường khá tốn kém và mất thời gian. Các công ty thường tận dụng một phương pháp chi phí thấp và hiệu quả - làm giàu dữ liệu để giảm sự phụ thuộc

vào việc thu thập và chuẩn bị các ví dụ huấn luyện, đồng thời xây dựng các mô hình AI có độ chính xác cao nhanh hơn.

Làm giàu dữ liệu là một quá trình tăng lượng dữ liệu một cách nhân tạo bằng cách tạo các điểm dữ liệu mới từ dữ liệu hiện có. Điều này bao gồm việc thêm các thay đổi nhỏ vào dữ liệu hoặc sử dụng một mô hình học máy để tạo các điểm dữ liệu mới trong không gian tiềm ẩn của dữ liệu gốc nhằm gia tăng tập dữ liệu.

Một câu hỏi có thể nảy sinh về sự khác biệt giữa dữ liệu tăng cường và dữ liệu tổng hợp.

- Dữ liệu tổng hợp: Khi dữ liệu được tạo ra một cách nhân tạo mà không sử dụng dữ liệu thực hay hình ảnh trong thế giới thực. Dữ liệu tổng hợp thường được tạo ra bởi một Mạng sinh dữ liệu.
- Dữ liệu tăng cường: Bắt nguồn từ dữ liệu gốc, áp dụng một số loại biến đổi nhỏ trên dữ liệu phụ thuộc vào kiểu của dữ liệu (chẳng hạn ảnh gốc với một số loại biến đổi hình học nhỏ như lật, dịch, xoay hoặc thêm nhiễu) nhằm tăng tính đa dạng của tập huấn luyện.

Ngày nay, có rất nhiều lo ngại về quyền riêng tư xoay quanh việc thu thập và sử dụng dữ liệu. Do đó, nhiều nhà nghiên cứu và công ty đã sử dụng các kỹ thuật tạo sinh dữ liệu tổng hợp để xây dựng bộ dữ liệu. Tuy nhiên, do những hạn chế như dữ liệu sinh không giống với dữ liệu gốc, dữ liệu tăng cường thường được ưu tiên hơn dữ liệu tổng hợp.

Các công ty có thể xây dựng bộ dữ liệu lớn bằng cách gia tăng bộ dữ liệu hiện có với kỹ thuật tăng cường dữ liệu nhằm cắt giảm chi phí cần thiết liên quan đến thu thập và chuẩn bị dữ liệu. Các phương pháp làm giàu dữ liệu được sử dụng rộng rãi trong các ứng dụng học sâu tiên tiến như phát hiện đối tượng, phân loại hình ảnh, nhận dạng hình ảnh, hiểu ngôn ngữ tự nhiên, phân đoạn ngữ nghĩa, v.v. Dữ liệu tăng cường đang cải thiện hiệu suất và kết quả của các mô hình học sâu bằng cách tạo ra các phiên bản mới và đa dạng cho tập dữ liệu huấn luyện.

Tuy nhiên, các phương pháp làm giàu dữ liệu này có những thách thức riêng sau:

- Chi phí đảm bảo chất lượng của các bộ dữ liệu tăng cường.
- Nghiên cứu và phát triển các mô hình học máy để tạo sinh các dữ liệu tổng hợp với các ứng dụng tiên tiến.
- Việc xác minh các kỹ thuật gia tăng hình ảnh như GAN là một thách thức.

- Việc tìm ra chiến lược làm giàu tối ưu cho dữ liệu là điều không hề đơn giản.
- Sự thiên vị cố hữu của dữ liệu gốc vẫn tồn tại trong dữ liệu gia tăng.

Các mô hình thị giác máy tính tiên tiến như RESNET (60M) và Inception-V3 (24M) có số lượng tham số khổng lồ để trích xuất được các đặc tính phức tạp trong các ảnh đầu vào. Các mô hình Xử lý ngôn ngữ tự nhiên (NLP) như BERT (340M) thậm chí còn có nhiều tham số hơn để có thể hiểu ngôn ngữ tự nhiên. Để xây dựng được một mô hình deep learning, chúng ta sẽ phải thu thập rất nhiều dữ liệu. Thật không may, đối với nhiều ứng dụng, chúng ta không có quyền truy cập vào lượng lớn dữ liệu. Làm giàu dữ liệu là một phương pháp để giải quyết vấn đề dữ liệu hạn chế. Khi làm giàu dữ liệu, chúng ta có thể chọn sử dụng một số kỹ thuật làm tăng lượng dữ liệu từ dữ liệu hiện có một cách nhân tạo để giải quyết vấn đề này.

Một số kỹ thuật tăng cường dữ liệu phổ biến trong Thị giác máy tính (CV)

1. Tăng cường theo vị trí

- Center Crop: Cắt hình ảnh đã cho ở giữa. Kích thước ảnh cắt là thông số do người dùng đưa ra.
- Random Crop: Cắt hình ảnh đã cho ở một vị trí ngẫu nhiên.
- Lật dọc ngẫu nhiên: Lật theo chiều dọc hình ảnh đã cho một cách ngẫu nhiên với xác suất nhất định.
- Lật ngang ngẫu nhiên: Lật ngang hình ảnh đã cho một cách ngẫu nhiên với xác suất nhất định.
- Random Rotation: Xoay hình ảnh theo một góc nào đó.
- Resize: Thay đổi kích thước của hình ảnh đầu vào theo kích thước nhất định.
- Random Affine: Thực hiện phép biến đổi affine ngẫu nhiên của tâm, giữ ảnh bất biến.

2. Tăng cường theo màu sắc

- Độ sáng: Một cách để tăng cường là thay đổi độ sáng của hình ảnh. Ảnh thu được trở nên tối hơn hoặc sáng hơn so với hình ảnh gốc.
- Độ tương phản: Độ tương phản được định nghĩa là mức độ tách biệt giữa vùng tối nhất và vùng sáng nhất của hình ảnh. Độ tương phản của hình ảnh cũng có thể được thay đổi.

- Độ bão hòa: Độ bão hòa là sự tách biệt giữa các màu của hình ảnh.

Một số kỹ thuật tăng cường dữ liệu phổ biến trong Xử lý ngôn ngữ tự nhiên (NLP)

Tăng cường dữ liệu ít phổ biến hơn trong NLP so với thị giác máy tính. Việc tự động hóa quá trình tăng cường dữ liệu văn bản là điều khó khăn do tính phức tạp của ngôn ngữ tự nhiên. Các phương pháp phổ biến để tăng dữ liệu trong NLP bao gồm:

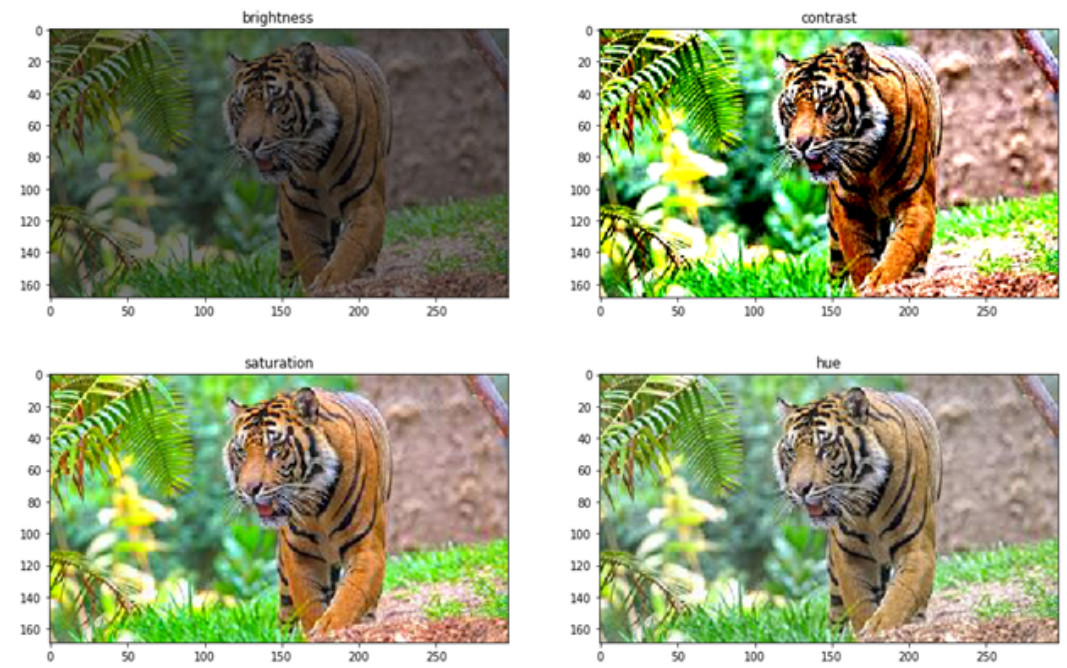
- Các thao tác tăng cường dữ liệu đơn giản và dễ dàng (EDA): Thay thế từ đồng nghĩa, chèn từ, hoán đổi từ và xóa từ
- Dịch ngược: Dịch lại văn bản từ ngôn ngữ đích trở lại ngôn ngữ gốc
- Nhúng từ theo ngữ cảnh

Một số mô hình nâng cao phổ biến để gia tăng dữ liệu

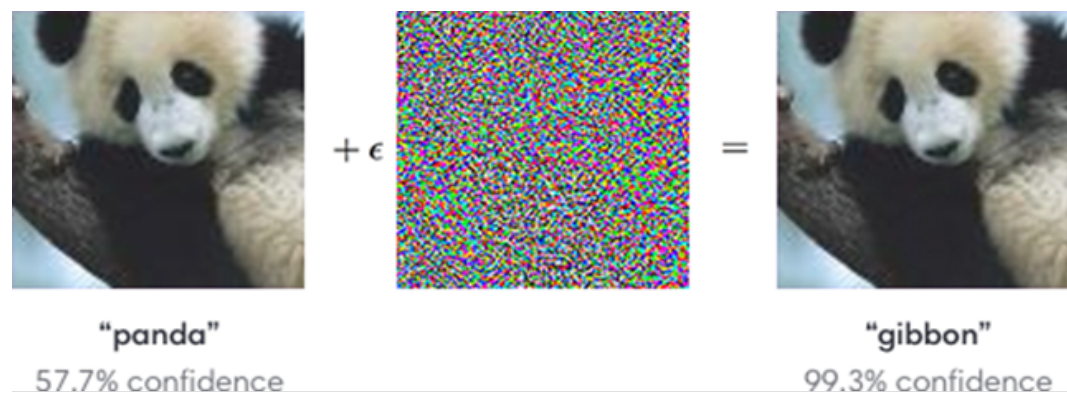
- Huấn luyện đối nghịch/Học máy đối nghịch: Các tấn công đối nghịch là những thay đổi không thể nhận thấy đối với hình ảnh (thay đổi cấp độ pixel) có thể thay đổi hoàn toàn dự đoán của mô hình. Để giải quyết vấn đề này, trong huấn luyện đối nghịch, hình ảnh được biến đổi cho đến khi mô hình học sâu bị đánh lừa và mô hình không phân tích chính xác dữ liệu.

Những hình ảnh được biến đổi hoặc tăng cường này được sử dụng trong các ví dụ huấn luyện để làm cho mô hình trở nên mạnh mẽ trước các tấn công đối nghịch.

- Mạng sinh dữ liệu (GAN): GAN được sử dụng rộng rãi để tạo hình ảnh tổng hợp trong miền mục tiêu. Các hình ảnh tổng hợp do GAN tạo ra được sử dụng làm hình ảnh gia tăng cho đầu vào của mô hình. Nhược điểm của việc sử dụng GAN là nó cần tiêu tốn nhiều tài nguyên và công sức.



Hình 4.6: Tăng cường màu sắc cho hình ảnh con hổ.



Hình 4.7: Hình ảnh của chú gấu trúc được tạo ra bằng cách thêm một chút nhiễu

Trong hình ảnh trên, chúng ta có thể thấy việc thêm một lượng nhiễu nhỏ vào hình ảnh có thể gây nhầm lẫn cho bộ phân loại AI và phân loại gấu trúc là vượt. Do đó, điều quan trọng là phải thêm những thay đổi như vậy vào tập dữ liệu huấn luyện để giải quyết các tấn công đối nghịch.

4.9 Kỹ thuật kết hợp nhiều mô hình (model ensemble)

Học kết hợp từ nhiều mạng nơ ron sâu dường như không phải là một lựa chọn dễ dàng vì nó có thể dẫn đến tăng chi phí tính toán rất nhiều và yêu cầu rất nặng về tài nguyên tính toán do việc thực hiện huấn luyện nhiều mạng nơ-ron sâu. Phần cứng hiệu năng cao cùng với khả năng tăng tốc của GPU vẫn có thể mất nhiều tuần để huấn luyện các mạng sâu này. Các kỹ thuật kết hợp ẩn/hiện nhiều mạng sâu đạt được mục tiêu trái ngược nhau trong đó một mô hình duy nhất được huấn luyện theo cách mà nó hoạt động giống như một sự kết hợp của huấn luyện nhiều mạng sâu mà không phát sinh chi phí bổ sung hoặc giữ chi phí bổ sung ở mức tối thiểu nhất có thể. Ở đây, thời gian huấn luyện của một sự kết hợp cũng giống như thời gian huấn luyện của một mô hình duy nhất. Trong các kết hợp ẩn, các tham số mô hình là được chia sẻ và sau khi hoàn thành huấn luyện, một mạng đơn đầy đủ được sử dụng tại thời điểm kiểm thử để xấp xỉ trung bình theo mô hình của các mô hình được kết hợp. Tuy nhiên, trong các kết hợp hiện, các tham số mô hình không được chia sẻ và đầu ra của kết hợp được lấy là sự tổ hợp của các dự đoán của các mô hình được kết hợp thông qua các cách tiếp cận khác nhau như bỏ phiếu theo đa số, tính trung bình, v.v.

- Dropout (Srivastava và cộng sự, 2014) tạo ra một mạng kết hợp bằng cách loại bỏ ngẫu nhiên các nút ẩn khỏi mạng trong quá trình huấn luyện của mạng. Trong thời gian kiểm thử, tất cả các nút ẩn trong mạng đều hoạt động.

Dropout cung cấp sự hiệu chỉnh của mạng để tránh overfitting và đưa vào tính thừa trong các vectơ đầu ra. Overfitting giảm đi vì nó huấn luyện số lượng mô hình theo cấp lũy thừa với trọng số được chia sẻ và cung cấp một kết hợp ẩn nhiều mạng sâu trong quá trình kiểm thử. Việc loại bỏ các đơn vị ẩn một cách ngẫu nhiên tránh được sự đồng thích ứng của các đơn vị ẩn bởi việc tạo ra sự hiện diện của một đơn vị cụ thể không đáng tin cậy. Mạng có sử dụng dropout lấy thời gian huấn luyện nhiều hơn 2 – 3 lần so với mạng nơ ron thường không sử dụng dropout.

Vì vậy, cần phải có sự cân bằng hợp lý giữa thời gian huấn luyện của mạng và overfitting.

- DropConnect (Wan và cộng sự, 2013) là sự tổng quát hóa của DropOut. Không giống như DropOut loại bỏ từng đơn vị ẩn đầu ra, DropConnect sẽ ngắt ngẫu nhiên từng kết nối và do đó, đưa vào tính thừa trong các trọng số của mô hình. Tương tự như DropOut, DropConnect tạo một kết hợp ẩn bằng cách loại bỏ các kết nối (đặt trọng số về 0) trong quá trình huấn luyện.

Cả Dropout và DropConnect đều có thời gian huấn luyện cao.

- Stochastic depth (Huang et al., 2016b) là một tiếp cận được đề xuất nhằm giảm bớt thời gian huấn luyện. Mạng sâu với độ sâu ngẫu nhiên nhằm mục đích giảm độ sâu mạng trong quá trình huấn luyện trong khi giữ nó không thay đổi trong quá trình kiểm thử mạng. Độ sâu ngẫu nhiên (Stochastic depth) là một cải tiến trên ResNet (He et al., 2016), trong đó các khối thặng dư (residual blocks) bị loại bỏ ngẫu nhiên trong quá trình huấn luyện và bỏ qua các kết nối khối chuyển đổi này theo các kết nối tắt (skip connections).
- Swapout (Singh và cộng sự, 2016) là sự tổng quát hóa của Dropout và Stochastic depth. Swapout liên quan đến việc loại bỏ các đơn vị ẩn riêng lẻ hoặc bỏ qua các khối một cách ngẫu nhiên. Một cách tiếp cận cụ thể đối với giảm thời gian thử nghiệm là chất lọc tri thức trong mạng (Hinton và cộng sự, 2015) để chuyển giao tri thức từ các sự kết hợp đến một mô hình duy nhất.

Tất cả các phương pháp nói trên cung cấp một tập hợp các mạng sâu bằng cách chia sẻ trọng số.

- Snapshot ensembling (Huang và cộng sự, 2017) phát triển một sự kết hợp hiện mà không có chia sẻ trọng số. Các tác giả đã khai thác các cực tiểu cục bộ tốt và xấu và đề xuất phương pháp tối ưu giảm gradient ngẫu nhiên (SGD) hội tụ M -lần tới các cực tiểu cục bộ dọc theo đường dẫn tối ưu hóa và lấy snapshot chỉ khi mô hình đạt tới điểm cực tiểu. Những snapshot này sau đó được kết hợp bằng cách lấy trung bình tại nhiều cực tiểu cục bộ để nhận dạng đối tượng. Thời gian huấn luyện của sự kết hợp này cũng giống như thời gian của mô hình đơn duy nhất đó. Đầu ra của kết hợp được lấy là giá trị trung bình của các đầu ra của các snapshot ở nhiều cực tiểu cục bộ.

Cho đến nay, đã có một số nỗ lực để tìm ra sự kết hợp hiện các mạng sâu trong đó các mô hình mạng sâu này không chia sẻ trọng số như *Random vector functional link network* (Malik và cộng sự, 2022).

4.10 Kỹ thuật học tái sử dụng (transfer learning)

Học chuyển giao là việc sử dụng lại mô hình được huấn luyện trước cho một bài toán mới có liên quan. Nó hiện rất phổ biến trong lĩnh vực học sâu vì nó có thể huấn luyện mạng sâu với lượng dữ liệu tương đối ít. Điều này rất hữu ích trong lĩnh vực khoa học dữ liệu vì hầu hết các bài toán trong thực tế thường không có hàng triệu điểm dữ liệu được gắn nhãn để huấn luyện các mô hình phức tạp như vậy.

Trong học chuyển giao, máy khai thác các tri thức thu được từ nhiệm vụ trước đó để cải thiện khả năng khái quát hóa cho nhiệm vụ khác. Các tri thức này có thể ở nhiều dạng khác nhau tùy thuộc vào bài toán và dữ liệu. Ví dụ, đó có thể là cách các mô hình được tạo ra để cho phép ta xác định các đối tượng mới dễ dàng hơn.

Ý tưởng chung của học chuyển giao là sử dụng tri thức mà mô hình đã học được từ một nhiệm vụ có nhiều dữ liệu huấn luyện được gắn nhãn sẵn trong một nhiệm vụ mới không có nhiều dữ liệu. Thay vì bắt đầu quá trình học từ đầu, chúng ta bắt đầu với các mẫu đã học được từ việc giải một nhiệm vụ liên quan. Cụ thể, chúng ta chuyển các trọng số mà mạng đã học được ở “nhiệm vụ A” sang “nhiệm vụ B” mới. Ví dụ: khi huấn luyện bộ phân loại để dự đoán liệu một hình ảnh có chứa đồ ăn hay không, bạn có thể sử dụng tri thức mà nó thu được trong quá trình huấn luyện để nhận dạng đồ uống.

Học chuyển giao chủ yếu được sử dụng trong các nhiệm vụ xử lý ngôn ngữ tự nhiên và thị giác máy tính do yêu cầu sức mạnh tính toán khổng lồ. Học chuyển giao không thực sự là một kỹ thuật học máy mà có thể được coi là một “phương pháp thiết kế” trong lĩnh vực này. Nó cũng không phải là một phần hay lĩnh vực nghiên cứu riêng biệt của học máy. Tuy nhiên, nó đã trở nên khá phổ biến khi khai thác các mạng nơ ron sâu đòi hỏi lượng dữ liệu và sức mạnh tính toán khổng lồ.

Học chuyển giao có một số lợi ích, nhưng ưu điểm chính là tiết kiệm thời gian huấn luyện, hiệu suất mạng nơ ron tốt hơn (trong hầu hết các trường hợp) và không cần nhiều dữ liệu.

Các cách tiếp cận học chuyển giao

1. Huấn luyện một mô hình để tái sử dụng nó

Giả sử bạn muốn giải quyết nhiệm vụ A nhưng không có đủ dữ liệu để huấn luyện mạng sâu. Một cách để giải quyết bài toán này là tìm nhiệm vụ B có liên quan với lượng dữ liệu dồi dào. Huấn luyện mạng sâu đối với nhiệm vụ B và sử dụng mô hình kết quả làm điểm bắt đầu để giải quyết nhiệm vụ A. Việc bạn cần sử dụng toàn bộ mô hình hay chỉ một vài lớp phụ thuộc rất nhiều vào nhiệm vụ bạn đang cố gắng giải quyết.

Nếu bạn có cùng thông tin đầu vào trong cả hai tác vụ, bạn có thể sử dụng lại mô hình và đưa ra dự đoán cho đầu vào mới. Ngoài ra, việc thay đổi và huấn luyện lại các lớp dành riêng cho các nhiệm vụ khác nhau và lớp đầu ra là một phương pháp để khám phá.

2. Sử dụng một mô hình đã được huấn luyện trước

Cách tiếp cận thứ hai là sử dụng một mô hình đã được huấn luyện trước cho

bài toán của bạn. Có bao nhiêu lớp để tái sử dụng và bao nhiêu lớp để huấn luyện lại tùy thuộc vào bài toán đó. Hiện có rất nhiều mô hình như vậy. Ví dụ, Keras cung cấp nhiều mô hình được huấn luyện trước có thể được sử dụng để học chuyển giao, dự đoán, trích xuất đặc trưng và tinh chỉnh. Bạn có thể tìm thấy những mô hình này cũng như một số hướng dẫn ngắn gọn về cách sử dụng chúng tại <https://keras.io/api/applications/>. Các mô hình phổ biến khác hiện có trên thị trường bao gồm AlexNet, VGG Model của Oxford và ResNet của Microsoft. Hơn nữa, một số mô hình được huấn luyện trước đã biết để giải quyết các bài toán liên quan đến NLP bao gồm Mô hình word2vec của Google và Mô hình GloVe của Stanford.

3. Trích xuất đặc trưng

Một cách tiếp cận khác là sử dụng deep learning để khám phá ra cách biểu diễn tốt nhất cho bài toán của bạn, nghĩa là tìm ra những đặc trưng thích đáng nhất. Cách tiếp cận này còn được gọi là học biểu diễn và thường có thể mang lại hiệu suất tốt hơn nhiều so với cách biểu diễn được thiết kế bằng tay.

Biểu diễn đã học sau đó cũng có thể được sử dụng cho các bài toán khác. Nên sử dụng các lớp đầu tiên để phát hiện ra cách biểu diễn phù hợp các đặc trưng và không sử dụng các lớp cuối hay đầu ra của mạng vì nó quá cụ thể cho nhiệm vụ. Cụ thể, hãy cấp dữ liệu của bạn vào mạng và sử dụng một trong các lớp trung gian làm lớp đầu ra. Lớp này sau đó có thể được hiểu là cách biểu diễn của dữ liệu thô.

Cách tiếp cận này chủ yếu được sử dụng trong thị giác máy tính vì nó có thể giảm kích thước tập dữ liệu của bạn, giúp giảm thời gian tính toán và làm cho nó phù hợp hơn với các thuật toán truyền thống.

Quá trình học chuyển giao

Quá trình học chuyển giao, về cơ bản, đều tuân theo các bước chính để hiện thực hóa sau đây:

- Truy cập các mô hình được huấn luyện trước: Các tổ chức có thể lấy các mô hình được huấn luyện trước từ bộ sưu tập thư viện mô hình của riêng họ hoặc các kho lưu trữ nguồn mở khác. Ví dụ: PyTorch Hub là kho lưu trữ mô hình được huấn luyện trước nguồn mở được thiết kế để tăng tốc quá trình nghiên cứu, từ tạo nguyên mẫu đến triển khai sản phẩm. Tương tự, TensorFlow Hub là một kho lưu trữ mở và thư viện ML có thể tái sử dụng với một số mô hình được huấn luyện trước có thể được sử dụng cho các tác vụ như nhúng văn bản, phân loại hình ảnh, v.v.
- Đóng băng các lớp: Một mạng nơ-ron điển hình có ba lớp: lớp đầu, lớp giữa và

lớp cuối. Trong học chuyển giao, các lớp đầu và giữa được giữ nguyên và chỉ các lớp cuối được huấn luyện lại để phương pháp có thể sử dụng dữ liệu được gắn nhãn của nhiệm vụ mà nó đã được huấn luyện trước đó. Việc đóng băng các lớp là cần thiết vì nó tránh việc khởi tạo lại các trọng số trong mô hình. Bước khởi tạo lại có thể khiến mô hình mất tất cả quá trình học trước đó.

- Huấn luyện các lớp mới: Sau khi đóng băng các lớp cần thiết, các lớp mới phải được thêm vào mô hình để đưa ra dự đoán mới trên tập dữ liệu mới nhất.
- Tinh chỉnh mô hình: Không cần tinh chỉnh mô hình cơ sở; tuy nhiên, nó có thể cải thiện hiệu suất tổng thể của mô hình. Quá trình này bao gồm việc bỏ việc đóng băng một số lớp của mô hình và sau đó huấn luyện lại nó ở tốc độ học tập thấp để xử lý tập dữ liệu mới.

Ứng dụng của học chuyển giao

Học chuyển giao là một công nghệ mới nổi có thể tìm thấy các ứng dụng trong nhiều lĩnh vực học máy khác nhau.

1. Xử lý ngôn ngữ tự nhiên (NLP)

Học chuyển giao củng cố các mô hình ML xử lý các nhiệm vụ NLP. Ví dụ: học chuyển giao có thể được sử dụng để huấn luyện các mô hình đồng thời nhằm phát hiện các thành phần ngôn ngữ khác nhau, phương ngữ, cụm từ hoặc từ vựng cụ thể.

Hơn nữa, học chuyển giao cho phép các mô hình thích ứng với nhiều ngôn ngữ. Điều này ngụ ý rằng các mô hình được huấn luyện bằng tiếng Anh có thể được huấn luyện lại và điều chỉnh cho phù hợp với các ngôn ngữ hoặc nhiệm vụ tương tự khác. Tri thức của các mô hình được huấn luyện trước với khả năng nhận dạng cú pháp ngôn ngữ có thể được chuyển sang các mô hình khác có thể dự đoán từ hoặc cụm từ tiếp theo, xem xét cấu trúc của các câu trước đó.

2. Thị giác máy tính (CV)

Thị giác máy tính cho phép các hệ thống rút ra ý nghĩa từ dữ liệu hình ảnh được cung cấp qua hình ảnh hoặc video. Các thuật toán ML huấn luyện trên các tập dữ liệu lớn (hình ảnh) và tự tinh chỉnh để có thể nhận dạng hình ảnh hoặc phân loại các đối tượng trong ảnh. Trong những trường hợp như vậy, học chuyển giao trở nên quan trọng vì nó kiểm soát các khía cạnh có thể tái sử dụng của thuật toán CV và chạy nó trên mô hình mới hơn.

Học chuyển giao có thể sử dụng các mô hình được tạo từ các tập dữ liệu huấn luyện lớn và áp dụng chúng cho các tập ảnh nhỏ hơn. Điều này có thể bao gồm việc xác định các cạnh sắc nét của vật thể trong bộ sưu tập hình ảnh được cung cấp.

Hơn nữa, các lớp xác định cụ thể các cạnh trong hình ảnh có thể được xác định và sau đó được huấn luyện dựa trên nhu cầu.

3. Mạng nơ ron

Mạng nơ ron là chìa khóa cho học sâu vì chúng được thiết kế để mô phỏng và tái tạo các chức năng não người. Việc huấn luyện mạng nơ ron đòi hỏi một lượng lớn tài nguyên do tính phức tạp của các mô hình mà chúng sử dụng. Do đó, học chuyển giao có thể được sử dụng ở đây để giảm nhu cầu tài nguyên, đồng thời làm cho toàn bộ quá trình hiệu quả hơn.

Một số tính năng có thể chuyển giao được chuyển từ mạng này sang mạng khác để tinh chỉnh quá trình phát triển mô hình. Khai thác tri thức qua các nhiệm vụ là hết sức quan trọng trong việc xây dựng mạng nơ ron sâu.

Tài liệu tham khảo

- Desjardins, G., Simonyan, K., Pascanu, R., & Kavukcuoglu, K. (2015). *Natural neural networks*. In Advances in Neural Information Processing Systems, 28, pp. 2071–2079.
- Ioffe, S., & Szegedy, C. (2015). *BN: Accelerating deep network training by reducing internal covariate shift*. In Proceedings of the 32nd International Conference on Machine Learning (pp. 448–456).
- Kohler, J., Daneshmand, H., Lucchi, A., Hofmann, T., Zhou, M., & Neymeyr, K. (2019). *Exponential convergence rates for BN: The power of length-direction decoupling in non-convex optimization*. In K. Chaudhuri & M. Sugiyamav (Eds.), Proceedings of Machine Learning Research (pp. 806–815).
- Hinton, G., Vinyals, O., Dean, J., 2015. *Distilling the knowledge in a neural network*. arXiv preprint arXiv:1503.02531.
- Huang, G., Sun, Y., Liu, Z., Sedra, D., Weinberger, K.Q., 2016b. *Deep networks with stochastic depth*. In: European Conference on Computer Vision. Springer pp. 646–661.
- Huang, G., Li, Y., Pleiss, G., Liu, Z., Hopcroft, J.E., Weinberger, K.Q., 2017a. *Snapshot ensembles: Train 1, get M for free*. arXiv preprint arXiv:1704.00109.
- Malik, A.K., Gao, R., Ganaie, M.A., Tanveer, M., Suganthan, P.N., 2022. *Random vector functional link network: recent developments, applications, and future directions*. arXiv preprint arXiv:2203.11316.
- Santurkar, S., Tsipras, D., Ilyas, A., & Madry, A. (2018). *How does BN help optimization?* In Advances in Neural Information Processing Systems, 31, 2483–2493.
- Singh, S., Hoiem, D., & Forsyth, D. (2016). *Swapout: Learning an ensemble of deep architectures*. Advances in Neural Information Processing Systems, 29.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R., Mele, B., Altarelli, G., 2014. *Dropout: a simple way to prevent neural networks from overfitting*. J. Mach. Learn. Res. 15, 1929–1958.
- Wan, L., Zeiler, M., Zhang, S., Cun, Y.L., Fergus, R., 2013. *Regularization of neural networks using DropConnect*. In: Dasgupta, S., McAllester, D. (Eds.), Proceedings of the 30th International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 28, PMLR, Atlanta, Georgia, USA, pp. 1058–1066. <http://dx.doi.org/10.1109/TPAMI.2017.2703082>.
- Wu, X., Dobriban, E., Ren, T., Wu, S., Li, Z., Gunasekar, S., ... & Liu, Q. (2020). *Implicit regularization and convergence for weight normalization*. Advances in Neural

Information Processing Systems, 33, 2835-2847.

Chương 5

Phần cứng và phần mềm cho học sâu

5.1	Tổng quan	93
5.2	Phần cứng cho Học sâu (Hardware for DL)	94
5.3	Các nền tảng lập trình cho Học sâu	96
5.4	Công cụ Tăng tốc và Nén mạng	98

5.1 Tổng quan

Bài giảng này cung cấp một cái nhìn toàn cảnh về hệ sinh thái hỗ trợ cho việc triển khai các mô hình học sâu. Chúng ta sẽ không chỉ tìm hiểu về các thuật toán, mà còn khám phá các công cụ và nền tảng giúp biến những ý tưởng lý thuyết thành các ứng dụng thực tế. Nội dung bao gồm ba phần chính:

1. **Phần cứng (Hardware):** Nền tảng vật lý nơi các phép toán khổng lồ của học sâu được thực thi.
2. **Phần mềm (Software):** Các thư viện và framework lập trình giúp chúng ta xây dựng và huấn luyện mô hình một cách hiệu quả.
3. **Công cụ tối ưu hóa:** Các kỹ thuật để nén và tăng tốc mô hình, giúp chúng có thể chạy trên các thiết bị có tài nguyên hạn chế.

5.2 Phần cứng cho Học sâu (Hardware for DL)

Việc lựa chọn phần cứng phù hợp là yếu tố quyết định đến tốc độ và khả năng huấn luyện các mô hình phức tạp.

5.2.1 So sánh CPU, GPU, và TPU

CPU (Central Processing Unit) - Bộ xử lý trung tâm

- **Thiết kế:** Gồm một vài nhân (cores) nhưng mỗi nhân có tốc độ rất cao và được tối ưu hóa cho các tác vụ phức tạp, đòi hỏi xử lý tuần tự (serial tasks). Ví dụ: quản lý hệ điều hành, chạy các ứng dụng văn phòng.
- **Điểm mạnh:** Rất linh hoạt, có thể xử lý nhiều loại tác vụ khác nhau. Tốc độ xử lý đơn luồng (single-thread performance) rất cao.
- **Vai trò trong Học sâu:** CPU vẫn đóng vai trò quan trọng trong việc quản lý toàn bộ hệ thống, tiền xử lý dữ liệu (data preprocessing), và điều phối các luồng công việc. Tuy nhiên, nó không hiệu quả cho việc huấn luyện (training) vì các phép toán trong mạng nơ-ron có thể song song hóa cao độ.
- **Ví dụ tương tự:** Hãy tưởng tượng CPU như một **vị bếp trưởng tài ba**. Ông có thể tự mình thực hiện một công thức nấu ăn phức tạp từ đầu đến cuối một cách hoàn hảo, nhưng tốc độ có giới hạn.

GPU (Graphics Processing Unit) - Bộ xử lý đồ họa

- **Thiết kế:** Ban đầu được thiết kế để xử lý đồ họa máy tính, GPU bao gồm hàng ngàn nhân xử lý nhỏ hơn và đơn giản hơn so với CPU.
- **Điểm mạnh:** Cực kỳ hiệu quả cho các tác vụ có thể chia nhỏ và thực hiện song song (parallel tasks). Phép nhân ma trận và vector, vốn là trái tim của các mạng nơ-ron, là một ví dụ điển hình.
- **Vai trò trong Học sâu:** Là "ngựa thồ" chính cho việc huấn luyện mô hình. Việc sử dụng GPU có thể tăng tốc độ huấn luyện lên hàng chục, thậm chí hàng trăm lần so với CPU.
- **Ví dụ tương tự:** GPU giống như một **đội quân phụ bếp**. Mỗi người chỉ làm một việc đơn giản (như thái rau, nhặt cỏ), nhưng khi hàng ngàn người cùng làm, họ có thể chuẩn bị nguyên liệu cho một bữa tiệc lớn trong thời gian cực ngắn.

TPU (Tensor Processing Unit) - Bộ xử lý Tensor

- **Thiết kế:** Là một mạch tích hợp chuyên dụng (ASIC - Application-Specific Integrated Circuit) được Google thiết kế đặc biệt cho các phép toán học sâu. Nó được tối ưu hóa ở cấp độ phần cứng cho các phép nhân ma trận (tensor operations).
- **Điểm mạnh:** Hiệu suất trên mỗi watt điện (performance-per-watt) cực kỳ cao cho các tác vụ huấn luyện và suy diễn (inference) mạng nơ-ron. Đặc biệt hiệu quả khi huấn luyện các mô hình khổng lồ trên quy mô lớn.
- **Vai trò trong Học sâu:** Là lựa chọn hàng đầu cho các doanh nghiệp và nhà nghiên cứu cần huấn luyện các mô hình tân tiến nhất (state-of-the-art) trên các bộ dữ liệu khổng lồ, thường được cung cấp qua các dịch vụ đám mây của Google (Google Cloud).
- **Ví dụ tương tự:** TPU là một **nhà máy chế biến thực phẩm tự động**. Nó không linh hoạt như bếp trưởng (CPU) hay đội quân phụ bếp (GPU), nhưng nó được thiết kế chỉ để làm một việc duy nhất—ví dụ, sản xuất xúc xích—with hiệu suất và tốc độ không đối thủ.

5.2.2 Thiết bị biên (Edge Devices)

Đây là các thiết bị phần cứng nhỏ gọn, tiêu thụ ít năng lượng, được thiết kế để chạy các mô hình học sâu ngay tại nơi dữ liệu được thu thập (ví dụ: trên camera, điện thoại, ô tô tự lái) thay vì gửi dữ liệu lên đám mây.

- **Tại sao cần thiết bị biên?**
 - **Độ trễ (Latency):** Xử lý tại chỗ cho kết quả gần như tức thì, rất quan trọng cho các ứng dụng thời gian thực như xe tự lái.
 - **Bảo mật (Privacy):** Dữ liệu nhạy cảm (ví dụ: hình ảnh từ camera an ninh) không cần phải rời khỏi thiết bị.
 - **Băng thông (Bandwidth):** Giảm chi phí và yêu cầu về băng thông mạng khi không phải liên tục gửi dữ liệu lên server.
 - **Độc lập (Offline capability):** Ứng dụng vẫn có thể hoạt động khi không có kết nối internet.
- **Các ví dụ nổi bật:**

- **NVIDIA Jetson Series (Jetson Nano, AGX Xavier):** Cung cấp sức mạnh của GPU NVIDIA trong một bo mạch nhỏ gọn, phù hợp cho robot, drone, và các hệ thống thị giác máy tính thông minh.
- **Google Coral (Dev Board, USB Accelerator):** Tích hợp chip Edge TPU của Google, chuyên dụng cho việc tăng tốc suy diễn TensorFlow Lite.
- **ARM ML Processors:** ARM, công ty thiết kế kiến trúc cho hầu hết các chip di động, cung cấp các thiết kế NPU (Neural Processing Unit) được tích hợp trực tiếp vào SoC (System-on-Chip) của điện thoại.

5.3 Các nền tảng lập trình cho Học sâu

Đây là các thư viện phần mềm cung cấp các công cụ và API để xây dựng, huấn luyện và triển khai mô hình một cách dễ dàng.

5.3.1 Sự phát triển và hợp nhất

Lịch sử các framework học sâu đã chứng kiến sự cạnh tranh và hợp nhất, từ các dự án học thuật (Theano, Torch, Caffe) đến các "ông lớn" được hậu thuẫn bởi các tập đoàn công nghệ. Hiện nay, cuộc chiến chủ yếu xoay quanh hai cái tên: **PyTorch** và **TensorFlow**.

5.3.2 PyTorch (do Facebook phát triển)

- **Triết lý cốt lõi: Đồ thị tính toán động (Dynamic Computation Graphs - Define-by-Run)**
 - Đồ thị tính toán được xây dựng "một cách linh hoạt" trong quá trình thực thi code. Mỗi khi bạn chạy code, một đồ thị mới được tạo ra.
 - **Ưu điểm:** Cảm giác rất "Pythonic", tự nhiên và linh hoạt. Việc debug trở nên cực kỳ đơn giản vì bạn có thể sử dụng các công cụ gỡ lỗi tiêu chuẩn của Python (như `print()` hoặc `pdb`) ở bất kỳ đâu trong code. Rất phù hợp cho nghiên cứu và các mô hình có cấu trúc thay đổi, ví dụ như mạng RNN xử lý các câu có độ dài khác nhau.
- **Các thành phần chính:**
 - **`torch.Tensor`:** Cấu trúc dữ liệu trung tâm, tương tự như NumPy array nhưng có thêm khả năng tăng tốc trên GPU.

- `torch.autograd`: Hệ thống tự động tính toán đạo hàm (gradient), là nền tảng cho việc huấn luyện mạng nơ-ron.
- `torch.nn`: Một module cấp cao chứa các lớp (layers), hàm lỗi (loss functions), và các khối xây dựng sẵn cho mạng nơ-ron.
- `torch.optim`: Chứa các thuật toán tối ưu hóa phổ biến như SGD, Adam, RMSprop.

5.3.3 TensorFlow (do Google phát triển)

- **Triết lý cốt lõi: Từ đồ thị tĩnh đến "Thực thi tức thì" (From Static to Eager)**
 - **TensorFlow 1.x (Đồ thị tĩnh - Define-and-Run)**: Bạn phải định nghĩa toàn bộ cấu trúc của đồ thị tính toán trước, sau đó biên dịch nó. Cuối cùng, bạn "đưa" dữ liệu vào và chạy phiên làm việc (session) trên đồ thị đã được tối ưu hóa này.
 - * **Ưu điểm**: Rất hiệu quả cho việc triển khai (deployment) vì đồ thị có thể được tối ưu hóa triệt để và chạy độc lập trên nhiều nền tảng (server, mobile) mà không cần code Python gốc.
 - * **Nhược điểm**: Khó debug, cảm giác không tự nhiên và công kênh cho việc thử nghiệm nhanh.
 - **TensorFlow 2.x (Eager Execution là mặc định)**: Để cạnh tranh với PyTorch, TF 2.x đã chuyển sang chế độ "thực thi tức thì", hoạt động tương tự như đồ thị động của PyTorch.
 - * Để lấy lại hiệu năng của đồ thị tĩnh, TF 2.x cung cấp decorator `@tf.function`. Hàm Python được "trang trí" bởi decorator này sẽ được tự động biên dịch thành một đồ thị tĩnh đã được tối ưu hóa. Điều này mang lại **sự kết hợp tốt nhất của cả hai thế giới**: linh hoạt trong phát triển và hiệu năng cao trong sản xuất.
- **Các thành phần chính**:
 - **Keras**: Là API cấp cao chính thức của TensorFlow. Keras cung cấp một giao diện cực kỳ thân thiện và dễ sử dụng để xây dựng mô hình, giúp che giấu đi sự phức tạp bên dưới của TensorFlow.
 - `tf.data`: Một API mạnh mẽ để xây dựng các đường ống (pipelines) nhập và tiền xử lý dữ liệu hiệu quả.

- **TensorBoard:** Công cụ trực quan hóa hàng đầu, cho phép theo dõi quá trình huấn luyện, xem kiến trúc mô hình, phân tích hiệu năng, v.v.

5.4 Công cụ Tăng tốc và Nén mạng

Sau khi huấn luyện xong một mô hình, nó thường quá lớn và chậm để chạy trên các thiết bị biên. Các công cụ này giúp giải quyết vấn đề đó.

5.4.1 Tại sao cần tối ưu hóa?

- **Kích thước mô hình (Model Size):** Các mô hình hiện đại có thể nặng hàng trăm MB hoặc cả GB, quá lớn cho bộ nhớ của điện thoại.
- **Tốc độ suy diễn (Inference Speed):** Thời gian để mô hình đưa ra dự đoán cần phải đủ nhanh cho các ứng dụng thời gian thực.
- **Tiêu thụ năng lượng (Power Consumption):** Rất quan trọng cho các thiết bị chạy bằng pin.

5.4.2 Các kỹ thuật chính

- **Lượng tử hóa (Quantization):** Giảm độ chính xác của các trọng số trong mô hình. Thay vì dùng số thực 32-bit (FP32), ta có thể chuyển sang số thực 16-bit (FP16) hoặc thậm chí là số nguyên 8-bit (INT8). Kỹ thuật này giúp giảm kích thước mô hình (lên đến 4 lần) và tăng tốc độ tính toán (đặc biệt trên phần cứng hỗ trợ INT8) với sự sụt giảm độ chính xác không đáng kể.
- **Tỉa cành (Pruning):** Loại bỏ các kết nối (trọng số) hoặc các nơ-ron không quan trọng (có giá trị gần bằng 0) ra khỏi mạng. Điều này tạo ra các ma trận trọng số thưa (sparse), giúp giảm kích thước và số lượng phép tính.
- **Chưng cất tri thức (Knowledge Distillation):** Dùng một mô hình lớn, phức tạp ("thầy" - teacher model) để huấn luyện một mô hình nhỏ hơn, gọn nhẹ hơn ("trò" - student model). Mô hình "trò" học cách bắt chước đầu ra của mô hình "thầy", qua đó thừa hưởng được hiệu năng cao trong một hình hài nhỏ gọn.

5.4.3 Các bộ công cụ phổ biến

- **TensorFlow Lite (TFLite):** Bộ công cụ của Google để chuyển đổi và tối ưu hóa mô hình TensorFlow cho thiết bị di động và nhúng. Nó tích hợp các kỹ thuật như quantization và cung cấp một trình thông dịch (interpreter) gọn nhẹ để chạy mô hình trên nhiều nền tảng (Android, iOS, Linux).
- **NVIDIA TensorRT:** Một nền tảng của NVIDIA để tối ưu hóa suy diễn học sâu trên các GPU của hãng. TensorRT phân tích mô hình đã huấn luyện và áp dụng các tối ưu hóa ở cấp độ thấp như hợp nhất các lớp (layer fusion), lựa chọn kernel tối ưu, và lượng tử hóa để đạt được thông lượng (throughput) cao nhất và độ trễ thấp nhất có thể.

Chương 6

Một số ứng dụng học sâu trong Thị giác máy

6.1	Giới thiệu tổng quan về Thị giác máy tính	100
6.2	Bài toán Phát hiện đối tượng (Object Detection)	102
6.3	Mạng đề xuất vùng (Two-Stage Detectors)	103
6.4	Mạng không đề xuất vùng (One-Stage Detectors)	105
6.5	Các kiến trúc nâng cao (Đọc thêm)	107
6.6	Giới thiệu bài toán phân đoạn ảnh	109
6.7	Hướng tiếp cận cho bài toán phân đoạn ảnh	110
6.8	Lớp tăng độ phân giải (Upsampling)	111
6.9	Hàm mục tiêu (Loss Functions)	112
6.10	Một số mạng phân đoạn ảnh tiêu biểu	114

6.1 Giới thiệu tổng quan về Thị giác máy tính

6.1.1 Thị giác máy tính là gì?

Thị giác máy tính (Computer Vision) là một lĩnh vực liên ngành của khoa học máy tính, tập trung vào việc làm cho máy tính có khả năng "nhìn", hiểu và diễn giải thông tin từ thế giới hình ảnh kỹ thuật số như ảnh và video. Như trong sơ đồ ở slide 4, thị giác máy tính là sự giao thoa của nhiều ngành khoa học khác nhau:

- **Khoa học Máy tính (Computer Science):** Cung cấp các thuật toán, lý thuyết, kiến trúc hệ thống.

- **Toán học (Mathematics):** Cung cấp nền tảng về đại số tuyến tính, xác suất, tối ưu hóa cho các thuật toán.
- **Vật lý (Physics):** Liên quan đến quang học (optics), cách ánh sáng tương tác với vật thể để tạo ra hình ảnh.
- **Sinh học và Tâm lý học (Biology & Psychology):** Nghiên cứu hệ thống thị giác của con người và các sinh vật khác để lấy cảm hứng, ví dụ như khoa học thần kinh (neuroscience) và khoa học nhận thức (cognitive sciences).
- **Kỹ thuật (Engineering):** Ứng dụng trong các lĩnh vực như robot, xử lý tín hiệu (ảnh, giọng nói, NLP).

Mục tiêu cốt lõi của thị giác máy tính là thu hẹp khoảng cách giữa những gì máy tính "nhìn thấy" và những gì con người "hiểu được". Đối với máy tính, một hình ảnh chỉ là một ma trận các con số (giá trị điểm ảnh). Mục tiêu của chúng ta là xây dựng một "cầu nối" để máy tính có thể diễn giải ma trận số này thành các khái niệm ngữ nghĩa (semantic meaning) mà con người hiểu, ví dụ như "một con tàu hỏa đã bị trật bánh khỏi nhà ga".

6.1.2 Tại sao nên học Thị giác máy tính?

Thị giác máy tính ngày càng trở nên quan trọng vì sự bùng nổ của dữ liệu hình ảnh và video từ khắp mọi nơi. Các ứng dụng của nó vô cùng đa dạng và hữu ích:

- **Tìm kiếm và tổ chức dữ liệu:** Các dịch vụ như Google Photos, Flickr sử dụng thị giác máy để tự động nhận dạng và phân loại ảnh.
- **Giám sát và an ninh:** Hệ thống camera an ninh có thể tự động phát hiện các hành vi bất thường.
- **Y tế và khoa học:** Phân tích ảnh y tế (chụp X-quang, MRI), ảnh viễn thám từ vệ tinh, ảnh thiên văn học để hỗ trợ chẩn đoán và nghiên cứu.
- **Thiết bị đo đạc và tái tạo 3D:** Thị giác máy có thể được dùng để đo đạc khoảng cách và tái tạo mô hình 3D từ các chuỗi hình ảnh 2D, ứng dụng trong robot, xe tự hành và lập bản đồ 3D các thành phố.
- **Tương tác Người-Máy:** Các thiết bị như Microsoft Kinect hay Sony EyeToy cho phép người dùng tương tác với máy tính thông qua cử chỉ cơ thể.
- **Nhận dạng và sinh trắc học:**

- **Phát hiện khuôn mặt (Face Detection):** Nhiều máy ảnh kỹ thuật số hiện nay có thể tự động xác định vị trí các khuôn mặt trong ảnh để tối ưu hóa việc lấy nét.
- **Nhận dạng khuôn mặt (Face Recognition):** Các hệ thống như Apple iPhoto hay FaceID của iPhone X có thể xác định danh tính của người trong ảnh.
- **Sinh trắc học (Biometrics):** Sử dụng các đặc điểm độc nhất của cơ thể như mống mắt (iris) hoặc vân tay để nhận dạng.
- **Thực tại tăng cường (Augmented Reality) và Thực tại ảo (Virtual Reality):** Chèn các vật thể ảo vào thế giới thực (AR) hoặc tạo ra một môi trường hoàn toàn ảo (VR).
- **Robotics và thám hiểm không gian:** Robot tự hành trên sao Hỏa sử dụng thị giác để chụp ảnh panorama, lập bản đồ 3D, phát hiện và tránh vật cản.

6.2 Bài toán Phát hiện đối tượng (Object Detection)

6.2.1 Các bài toán cơ bản trong Thị giác máy

Trong thị giác máy, có một số bài toán cốt lõi với mức độ phức tạp tăng dần:

1. **Phân loại ảnh (Image Classification):** Trả lời câu hỏi "Trong ảnh có gì?". Đầu ra là một nhãn duy nhất cho toàn bộ ảnh (ví dụ: "CAT").
2. **Phân loại và Định vị (Classification + Localization):** Ngoài việc phân loại, còn xác định vị trí của một đối tượng duy nhất bằng một hộp giới hạn (bounding box).
3. **Phát hiện đối tượng (Object Detection):** Phát hiện và định vị tất cả các đối tượng trong ảnh, có thể có nhiều đối tượng và nhiều loại khác nhau. Đầu ra là một danh sách các hộp giới hạn cùng với nhãn lớp cho mỗi hộp (ví dụ: "DOG, DOG, CAT").
4. **Phân đoạn theo ngữ nghĩa (Semantic Segmentation):** Phân loại từng điểm ảnh (pixel) trong ảnh thuộc về lớp nào (ví dụ: pixel này là cỏ, pixel kia là mèo). Nó không phân biệt các thực thể khác nhau của cùng một lớp.
5. **Phân đoạn theo thực thể (Instance Segmentation):** Nâng cao hơn semantic segmentation, nó không chỉ phân loại từng pixel mà còn phân biệt được các đối tượng riêng lẻ thuộc cùng một lớp.

6.2.2 Ứng dụng của bài toán Phát hiện đối tượng

Bài toán phát hiện đối tượng là nền tảng cho nhiều ứng dụng thực tế:

- **Giao thông thông minh:** Đếm và theo dõi các phương tiện (xe máy, ô tô, xe buýt), đo vận tốc, phân tích mật độ giao thông.
- **Phát hiện người:** Trong các hệ thống giám sát, phân tích đám đông hoặc bán lẻ.
- **Phát hiện khuôn mặt:** Ứng dụng trong việc điểm danh, bảo mật, và tương tác người-máy.
- **Phát hiện văn bản:** Trích xuất thông tin từ hóa đơn, tài liệu, biển số xe (OCR).
- **Nông nghiệp thông minh:** Robot có thể tự động phát hiện và thu hoạch nông sản như dâu tây.

6.3 Mạng đề xuất vùng (Two-Stage Detectors)

Các phương pháp này hoạt động theo hai giai đoạn: 1) đề xuất một số vùng có khả năng chứa đối tượng, và 2) phân loại và tinh chỉnh các vùng đó.

6.3.1 Tiếp cận quét cửa sổ (Sliding Windows)

Đây là một phương pháp kinh điển nhưng đã lỗi thời. Ý tưởng là trượt một cửa sổ (hộp) có kích thước cố định qua toàn bộ hình ảnh, từ trái sang phải, từ trên xuống dưới. Tại mỗi vị trí, một bộ phân loại (ví dụ như CNN) sẽ xác định xem vùng bên trong cửa sổ chứa đối tượng gì (mèo, nai, hay nền).

- **Nhược điểm:** Cực kỳ tốn kém về mặt tính toán vì phải xử lý hàng ngàn, thậm chí hàng triệu cửa sổ với các tỷ lệ và kích thước khác nhau.

6.3.2 R-CNN (Region-based Convolutional Neural Network)

Để giải quyết vấn đề của sliding windows, R-CNN đề xuất một cách tiếp cận thông minh hơn.

1. **Đề xuất vùng (Region Proposal):** Sử dụng một thuật toán riêng biệt như *Selective Search* để tạo ra khoảng 2000 vùng ứng viên (region proposals) có khả năng chứa đối tượng cao.

2. **Trích xuất đặc trưng:** Mỗi vùng đề xuất được "bóp méo" (warped) về một kích thước cố định và đưa qua một mạng CNN (ví dụ AlexNet) để trích xuất một vector đặc trưng có độ dài cố định.

3. **Phân loại và Hiệu chỉnh:**

- Một bộ máy vector hỗ trợ (SVM) riêng cho mỗi lớp được dùng để phân loại đối tượng trong vùng đó.
- Một bộ hồi quy tuyến tính (linear regressor) được huấn luyện để tinh chỉnh lại tọa độ của hộp giới hạn (bounding box) cho chính xác hơn.
- **Nhược điểm:** Rất chậm vì phải chạy CNN trên 2000 vùng đề xuất cho mỗi ảnh. Quá trình huấn luyện phức tạp, gồm nhiều giai đoạn (huấn luyện CNN, huấn luyện SVM, huấn luyện hồi quy bbox).

6.3.3 Fast R-CNN

Fast R-CNN cải tiến đáng kể tốc độ của R-CNN.

1. **Trích xuất đặc trưng một lần:** Thay vì xử lý từng vùng, toàn bộ ảnh được đưa qua mạng CNN **một lần duy nhất** để tạo ra một bản đồ đặc trưng (feature map).
2. **RoI Pooling:** Các vùng đề xuất từ Selective Search được chiếu (projected) lên bản đồ đặc trưng này. Lớp **RoI (Region of Interest) Pooling** sẽ trích xuất một vector đặc trưng có kích thước cố định từ mỗi vùng trên bản đồ đặc trưng, thay vì phải crop ảnh gốc.
3. **Đầu ra hợp nhất:** Từ vector đặc trưng RoI, mạng sử dụng các lớp kết nối đầy đủ (fully connected layers) để đồng thời thực hiện hai nhiệm vụ:
 - Phân loại đối tượng bằng một lớp softmax.
 - Hồi quy để hiệu chỉnh tọa độ hộp giới hạn.
- **Ưu điểm:** Nhanh hơn R-CNN rất nhiều và quá trình huấn luyện là end-to-end (ngoại trừ bước đề xuất vùng).

6.3.4 Faster R-CNN

Faster R-CNN là bước tiến cuối cùng, đưa cơ chế đề xuất vùng vào bên trong mạng nơ-ron, tạo ra một kiến trúc hợp nhất thực sự.

- **Mạng đề xuất vùng (Region Proposal Network - RPN):** Đây là cải tiến cốt lõi. Faster R-CNN giới thiệu một mạng con gọi là RPN, có nhiệm vụ học cách đề xuất các vùng chất lượng cao trực tiếp từ các bản đồ đặc trưng của CNN. RPN thay thế hoàn toàn cho Selective Search.
- **Hoạt động:** RPN trượt một cửa sổ nhỏ trên bản đồ đặc trưng và tại mỗi vị trí, nó đề xuất một số lượng k hộp giới hạn với các kích thước và tỷ lệ khác nhau (gọi là *anchor boxes*).
- **Kiến trúc hợp nhất:** RPN và mạng Fast R-CNN phía sau chia sẻ chung các lớp tích chập ban đầu. Toàn bộ hệ thống trở thành một mạng nơ-ron duy nhất, có thể huấn luyện end-to-end. Đây là một ví dụ điển hình của các bộ phát hiện đối tượng hai giai đoạn (two-stage object detector).

6.4 Mạng không đề xuất vùng (One-Stage Detectors)

Các phương pháp này bỏ qua giai đoạn đề xuất vùng riêng biệt và thực hiện việc định vị và phân loại trong một lần duy nhất. Chúng thường nhanh hơn nhưng có thể kém chính xác hơn so với các mạng hai giai đoạn.

6.4.1 YOLO (You Only Look Once)

YOLO là một trong những kiến trúc một giai đoạn nổi tiếng nhất.

- **Ý tưởng chính:** YOLO chia ảnh đầu vào thành một lưới ô $S \times S$ (ví dụ 7×7). Nếu tâm của một đối tượng rơi vào một ô lưới, ô đó sẽ chịu trách nhiệm phát hiện đối tượng đó.
- **Dự đoán:** Mỗi ô lưới sẽ dự đoán:
 1. B hộp giới hạn và một điểm tự tin (confidence score) cho mỗi hộp. Điểm tự tin được tính bằng $P(\text{object}) \times \text{IoU}_{\text{pred}}^{\text{truth}}$.
 2. C xác suất có điều kiện của các lớp đối tượng, $P(\text{Class}_i | \text{Object})$.
- **YOLOv1:** Sử dụng một kiến trúc CNN đầy đủ và cuối cùng là các lớp kết nối đầy đủ để tạo ra một tensor đầu ra có kích thước $S \times S \times (B \times 5 + C)$. Nhược điểm chính là khó phát hiện các vật thể nhỏ và có độ chính xác định vị không cao.
- **YOLOv2 (YOLO9000):** Cải tiến YOLOv1 với các kỹ thuật:

- **Anchor Boxes:** Thay vì dự đoán trực tiếp tọa độ của hộp, YOLOv2 dự đoán các độ dời (offsets) so với các hộp tham chiếu được định nghĩa trước gọi là anchor boxes. Các anchor box này có kích thước và tỷ lệ đa dạng, được tìm ra bằng thuật toán k-means trên tập dữ liệu huấn luyện.
- **Mạng hoàn toàn tích chập (Fully Convolutional):** Loại bỏ các lớp kết nối đầy đủ ở cuối, giúp mạng linh hoạt hơn với các kích thước ảnh đầu vào.
- **YOLOv3:** Cải tiến lớn nhất là khả năng phát hiện đa tỷ lệ.
 - **Phát hiện đa tỷ lệ:** Các phiên bản trước chỉ sử dụng bản đồ đặc trưng cuối cùng, gây khó khăn cho việc phát hiện vật nhỏ. YOLOv3 thực hiện dự đoán ở ba bản đồ đặc trưng có kích thước khác nhau. Điều này giúp nó phát hiện tốt cả vật thể lớn và nhỏ.
 - **Kiến trúc xương sống (Backbone):** Sử dụng một mạng sâu hơn gọi là Darknet-53, mạnh mẽ hơn so với các phiên bản trước.

6.4.2 SSD (Single Shot MultiBox Detector)

SSD là một kiến trúc một giai đoạn khác, cố gắng kết hợp ưu điểm của cả YOLO và các phương pháp dựa trên anchor.

- **Ý tưởng chính:** Thay vì chỉ dự đoán trên một bản đồ đặc trưng cuối cùng, SSD thực hiện dự đoán trên nhiều bản đồ đặc trưng ở các tầng khác nhau của mạng.
- **Hoạt động:**
 - Sử dụng một mạng xương sống (ví dụ VGG-16) và thêm vào các lớp tích chập phụ để tạo ra các bản đồ đặc trưng có độ phân giải giảm dần.
 - Trên mỗi bản đồ đặc trưng này, SSD sử dụng một tập các hộp mặc định (default boxes) với các tỷ lệ và kích thước khác nhau để dự đoán các độ dời và điểm tin cậy cho các lớp đối tượng.
 - Bản đồ đặc trưng ở các tầng đầu (độ phân giải cao) dùng để phát hiện vật nhỏ, trong khi các bản đồ ở tầng sau (độ phân giải thấp) dùng cho vật lớn.
- **Ưu điểm:** Đạt được sự cân bằng tốt giữa tốc độ và độ chính xác, thường nhanh hơn Faster R-CNN và chính xác hơn YOLOv1.

6.5 Các kiến trúc nâng cao (Đọc thêm)

6.5.1 RetinaNet và Focal Loss

Một trong những vấn đề lớn của các mạng một giai đoạn là sự mất cân bằng cực độ giữa số lượng anchor box là nền (negative, background) và số lượng anchor box thực sự chứa đối tượng (positive).

- **Vấn đề:** Hàng ngàn anchor box âm để phân loại có thể "lấn át" tín hiệu loss từ một vài anchor box dương trong quá trình huấn luyện, khiến mô hình không học được tốt.
- **Giải pháp - Focal Loss:** RetinaNet giới thiệu một hàm mất mát mới gọi là Focal Loss để giải quyết vấn đề này.

$$FL(p_t) = -(1 - p_t)^\gamma \log(p_t)$$

Trong đó p_t là xác suất dự đoán cho lớp đúng, và γ là một tham số điều chỉnh. Khi một mẫu được phân loại đúng với độ chắc chắn cao ($p_t \rightarrow 1$), hệ số $(1 - p_t)^\gamma$ sẽ tiến về 0, làm giảm đóng góp của mẫu đó vào tổng loss. Điều này giúp mô hình tập trung học từ các mẫu khó, bị phân loại sai. Nhờ Focal Loss, RetinaNet là mạng một giai đoạn đầu tiên vượt qua độ chính xác của các mạng hai giai đoạn hàng đầu tại thời điểm đó.

6.5.2 CornerNet: Phát hiện đối tượng bằng các cặp góc

CornerNet là một phương pháp một giai đoạn mới lạ, loại bỏ hoàn toàn khái niệm anchor box.

- **Ý tưởng chính:** Thay vì dự đoán hộp, CornerNet phát hiện một đối tượng bằng cách xác định một cặp điểm: góc trên-trái (top-left) và góc dưới-phải (bottom-right) của nó.
- **Hoạt động:** Mạng dự đoán hai bản đồ nhiệt (heatmaps), một cho tất cả các góc trên-trái và một cho tất cả các góc dưới-phải.
- **Embedding Vector:** Để ghép cặp các góc thuộc về cùng một đối tượng, mạng học một "embedding vector" cho mỗi góc được phát hiện. Các cặp góc có embedding vector gần giống nhau sẽ được nhóm lại để tạo thành một hộp giới hạn.

- **Corner Pooling:** Một lớp pooling đặc biệt được giới thiệu để giúp mạng xác định vị trí các góc tốt hơn bằng cách tổng hợp thông tin dọc theo các đường biên của vật thể.
- **Ưu điểm:** Loại bỏ sự cần thiết của việc thiết kế các anchor box và các siêu tham số liên quan, giúp đơn giản hóa kiến trúc.

6.5.3 CenterNet: Phát hiện đối tượng như các điểm

CenterNet tiếp tục xu hướng loại bỏ anchor box, đề xuất một cách tiếp cận còn đơn giản hơn.

- **Ý tưởng chính:** Mô hình hóa một đối tượng bằng một điểm duy nhất là tâm của hộp giới hạn.
- **Hoạt động:** Mạng dự đoán một bản đồ nhiệt cho các tâm đối tượng. Tại mỗi vị trí tâm được phát hiện, mạng cũng hồi quy các thuộc tính khác của đối tượng như kích thước (chiều rộng, chiều cao) và một độ dời nhỏ để tinh chỉnh vị trí tâm.
- **Ưu điểm:** Kiến trúc rất đơn giản, nhanh, và đạt được sự cân bằng tuyệt vời giữa tốc độ và độ chính xác, trở thành một trong những phương pháp hàng đầu cho việc phát hiện đối tượng thời gian thực.

6.6 Giới thiệu bài toán phân đoạn ảnh

Phân đoạn ảnh (Image Segmentation) là một trong những bài toán cốt lõi của lĩnh vực thị giác máy tính. Mục tiêu của nó không chỉ là nhận diện các đối tượng có trong ảnh mà còn là xác định vị trí chính xác của chúng ở cấp độ pixel. Nói cách khác, bài toán này thực hiện việc gán một nhãn (lớp đối tượng) cho từng pixel trong ảnh.

6.6.1 Các bài toán thị giác máy

Trong thị giác máy tính, có một chuỗi các bài toán từ đơn giản đến phức tạp để hiểu nội dung của một bức ảnh:

- **Phân loại ảnh (Image Classification):** Bài toán cơ bản nhất, trả lời câu hỏi "Trong ảnh có gì?". Mô hình sẽ đưa ra một hoặc nhiều nhãn cho toàn bộ bức ảnh.
- **Phân loại và Định vị (Classification + Localization):** Nâng cao hơn một bước, mô hình không chỉ xác định lớp của đối tượng chính mà còn vẽ một hộp giới hạn (bounding box) xung quanh nó.
- **Phát hiện đối tượng (Object Detection):** Mở rộng của bài toán định vị, áp dụng cho nhiều đối tượng trong cùng một ảnh. Mô hình sẽ xác định và vẽ bounding box cho tất cả các đối tượng mà nó nhận diện được.
- **Phân đoạn ngữ nghĩa (Semantic Segmentation):** Thay vì chỉ dùng bounding box, bài toán này phân loại từng pixel của ảnh vào một lớp nhất định (ví dụ: "cỏ", "mèo", "cây", "bầu trời"). Các đối tượng thuộc cùng một lớp không được phân biệt với nhau.
- **Phân đoạn thực thể (Instance Segmentation):** Đây là bài toán phức tạp nhất, kết hợp giữa phát hiện đối tượng và phân đoạn ngữ nghĩa. Nó không chỉ phân loại từng pixel mà còn phân biệt được các thực thể (instances) khác nhau của cùng một lớp đối tượng (ví dụ: "chó 1", "chó 2").

6.6.2 Phân đoạn ngữ nghĩa (Semantic Segmentation)

Đặc điểm chính của phân đoạn ngữ nghĩa là:

- Phân loại từng điểm ảnh trong ảnh.

- Không phân biệt các đối tượng riêng lẻ nếu chúng thuộc cùng một lớp. Ví dụ, tất cả các con bò trong ảnh sẽ được gán chung nhãn "bò".

6.6.3 Ứng dụng của phân đoạn ảnh

Phân đoạn ảnh có rất nhiều ứng dụng quan trọng trong thực tế:

- **Phân tích ảnh vệ tinh:** Dùng để nhận dạng các loại địa hình, công trình, thảm thực vật từ ảnh chụp từ trên cao, phục vụ cho quy hoạch đô thị, quản lý nông nghiệp và theo dõi biến đổi môi trường.
- **Xe tự hành:** Một trong những ứng dụng quan trọng nhất. Xe tự hành cần hiểu rõ môi trường xung quanh ở cấp độ pixel để nhận diện đường đi, làn đường, người đi bộ, các phương tiện khác và vật cản, từ đó đưa ra quyết định lái xe an toàn.
- **Y tế:** Phân đoạn ảnh y tế (ví dụ: MRI, CT scan, X-quang) để xác định vị trí, kích thước của các khối u, tổn thương hoặc các cơ quan nội tạng. Điều này hỗ trợ đắc lực cho bác sĩ trong việc chẩn đoán và lên kế hoạch điều trị.
- **Nhận dạng ký tự quang học (OCR):** Phân đoạn bố cục của một tài liệu, xác định các vùng chứa văn bản, tiêu đề, hình ảnh, bảng biểu trước khi đưa vào nhận dạng ký tự.

6.7 Hướng tiếp cận cho bài toán phân đoạn ảnh

6.7.1 Trượt cửa sổ (Sliding Window)

Một trong những cách tiếp cận đầu tiên là sử dụng kỹ thuật cửa sổ trượt:

1. Trượt một cửa sổ (patch) nhỏ trên toàn bộ ảnh đầu vào.
2. Tại mỗi vị trí, trích xuất vùng ảnh bên trong cửa sổ.
3. Đưa vùng ảnh này vào một mạng Nơ-ron tích chập (CNN) để phân loại pixel trung tâm của cửa sổ.
4. Lặp lại quá trình này cho đến khi tất cả các pixel trong ảnh đều được phân loại.

Nhược điểm: Phương pháp này cực kỳ **không hiệu quả** vì các vùng ảnh trích xuất bị trùng lặp rất nhiều, dẫn đến việc tính toán lại các đặc trưng một cách dư thừa và tốn kém.

6.7.2 Mạng Tích chập hoàn toàn (Fully Convolutional Network - FCN)

Để giải quyết vấn đề của cửa sổ trượt, các mạng tích chập hoàn toàn (FCN) đã được đề xuất. Ý tưởng chính là thiết kế một mạng CNN chỉ chứa các lớp tích chập, pooling và upsampling, loại bỏ hoàn toàn các lớp kết nối đầy đủ (fully connected layers) ở cuối.

- Mạng có thể nhận đầu vào là một ảnh với kích thước bất kỳ và tạo ra một bản đồ phân đoạn (segmentation map) có cùng kích thước không gian với ảnh đầu vào.
- **Thách thức:** Việc xử lý ảnh có độ phân giải cao ngay từ đầu đòi hỏi chi phí tính toán và bộ nhớ rất lớn.
- **Giải pháp:** Sử dụng kiến trúc "bộ mã hóa - bộ giải mã" (Encoder-Decoder).
 - **Encoder (Bộ mã hóa):** Gồm các lớp tích chập và gộp (pooling) để giảm dần độ phân giải không gian của bản đồ đặc trưng (feature map) và tăng dần chiều sâu (số kênh). Quá trình này giúp trích xuất các thông tin ngữ nghĩa cấp cao.
 - **Decoder (Bộ giải mã):** Lấy bản đồ đặc trưng có độ phân giải thấp từ Encoder và sử dụng các lớp tăng độ phân giải (upsampling) để khôi phục lại kích thước không gian ban đầu, tạo ra bản đồ phân đoạn chi tiết.

Cách tiếp cận này hiệu quả hơn nhiều vì các đặc trưng được tính toán và tái sử dụng trên toàn bộ ảnh chỉ trong một lượt truyền thẳng (single forward pass).

6.8 Lớp tăng độ phân giải (Upsampling)

Upsampling là quá trình biến một bản đồ đặc trưng có độ phân giải thấp thành một bản đồ có độ phân giải cao hơn. Đây là thành phần cốt lõi trong bộ giải mã của các mạng phân đoạn.

6.8.1 Lớp Unpooling

Đây là các phương pháp upsampling đơn giản và không có tham số cần huấn luyện.

- **Nearest Neighbor:** Mỗi giá trị đầu vào được sao chép để điền vào một vùng 2×2 (hoặc lớn hơn) trong bản đồ đầu ra.

- **Bed of Nails:** Mỗi giá trị đầu vào được đặt vào góc trên bên trái của một vùng 2×2 , các vị trí còn lại được điền bằng số 0.

6.8.2 Lớp Max Unpooling

Đây là một phương pháp "thông minh" hơn. Trong quá trình Max Pooling ở bộ mã hóa, ta không chỉ lấy giá trị lớn nhất mà còn **ghi nhớ vị trí** (chỉ số) của nó. Khi thực hiện Max Unpooling ở bộ giải mã, ta sử dụng các vị trí đã lưu này để đặt các giá trị trở lại đúng vị trí của chúng trong bản đồ đặc trưng có độ phân giải cao hơn. Các vị trí khác được điền bằng 0. Kỹ thuật này giúp bảo toàn thông tin không gian tốt hơn.

6.8.3 Tích chập chuyển vị (Transposed Convolution)

Đây là phương pháp upsampling **có tham số huấn luyện được**, do đó nó linh hoạt và hiệu quả hơn các phương pháp unpooling.

- **Tên gọi khác:** Thường bị gọi nhầm là Deconvolution, các tên gọi chính xác hơn bao gồm Upconvolution, Fractionally Strided Convolution, Backward Strided Convolution.
- **Cách hoạt động:** Nó không phải là phép toán ngược của tích chập thông thường. Thay vào đó, có thể hình dung nó như sau: mỗi pixel ở đầu vào sẽ nhân với một bộ lọc (filter) để tạo ra một vùng ở đầu ra. Các vùng đầu ra tạo bởi các pixel đầu vào khác nhau nếu có chồng lấp lên nhau thì sẽ được cộng lại. Vì bộ lọc có các trọng số cần huấn luyện, mạng có thể học được cách upsampling tối ưu cho bài toán.

Tóm lại, để giảm độ phân giải (downsampling), ta có thể dùng Max Pooling hoặc tích chập với sải chân (strided convolution). Để tăng độ phân giải (upsampling), ta có thể dùng các phương pháp Unpooling hoặc Tích chập chuyển vị.

6.9 Hàm mục tiêu (Loss Functions)

Hàm mục tiêu là thành phần định hướng quá trình huấn luyện của mô hình. Với bài toán phân đoạn, có nhiều loại hàm mục tiêu khác nhau.

6.9.1 Hàm mục tiêu dựa trên phân phối (Distribution-based)

Các hàm này xử lý bài toán như một bài toán phân loại trên từng pixel.

- **Cross-Entropy (CE):** Là hàm mất mát tiêu chuẩn cho bài toán phân loại đa lớp, được áp dụng độc lập cho từng pixel.

$$CE(p, \hat{p}) = - \sum_{c=1}^C p_c \log(\hat{p}_c)$$

Trong đó p_c là one-hot vector của nhãn thật và $\hat{p}_c$ là xác suất dự đoán của mô hình cho lớp c .

- **Weighted CE:** Một biến thể của CE để giải quyết vấn đề mất cân bằng lớp (class imbalance), bằng cách gán trọng số β_c lớn hơn cho các lớp ít phổ biến.

$$WCE(p, \hat{p}) = - \sum_{c=1}^C \beta_c p_c \log(\hat{p}_c)$$

- **Focal Loss:** Cải tiến từ CE, được thiết kế để giải quyết vấn đề mất cân bằng giữa các mẫu dễ và khó. Nó giảm giá trị mất mát đối với các mẫu đã được phân loại đúng một cách dễ dàng (xác suất cao), để mạng có thể tập trung hơn vào việc học các mẫu khó.

$$FL(p_t) = -(1 - p_t)^\gamma \log(p_t)$$

Trong đó p_t là xác suất dự đoán cho lớp đúng, và γ là tham số điều chỉnh.

6.9.2 Hàm mục tiêu dựa trên vùng (Region-based)

Các hàm này đánh giá sự trùng khớp giữa vùng dự đoán và vùng thật.

- **Dice Loss:** Dựa trên hệ số Dice (Dice Coefficient), một độ đo sự chồng chéo giữa hai tập hợp. Nó rất hiệu quả trong các trường hợp mất cân bằng lớp.

$$\text{Dice Coefficient} = \frac{2TP}{2TP + FP + FN} = \frac{2|X \cap Y|}{|X| + |Y|}$$

$$\text{Dice Loss} = 1 - \text{Dice Coefficient}$$

- **Tversky Loss:** Là một dạng tổng quát hóa của Dice Loss. Nó thêm các trọng số α và β vào False Positives (FP) và False Negatives (FN), cho phép điều chỉnh sự đánh đổi giữa precision và recall.

6.9.3 Hàm mục tiêu kết hợp (Compound Loss)

Thực tế cho thấy việc kết hợp các hàm mục tiêu khác nhau thường mang lại kết quả tốt hơn. Ví dụ, kết hợp Dice Loss (tốt cho cấu trúc vùng) và Cross-Entropy hoặc Focal Loss (tốt cho sự ổn định khi huấn luyện) là một lựa chọn phổ biến.

$$L_{total} = L_{CE} + L_{Dice}$$

6.9.4 Boundary Loss

Loại hàm mục tiêu này tập trung vào việc tối ưu hóa đường biên giữa các vùng phân đoạn. Thay vì so khớp vùng, nó tính toán khoảng cách giữa đường biên dự đoán (∂S) và đường biên thật (∂G). Điều này đặc biệt hữu ích để tạo ra các kết quả phân đoạn có đường viền sắc nét và chính xác.

6.10 Một số mạng phân đoạn ảnh tiêu biểu

6.10.1 FCN (Fully Convolutional Network)

Đây là một trong những kiến trúc tiên phong. Điểm nổi bật của FCN là việc sử dụng các **kết nối tắt (skip connections)**. Các kết nối này kết hợp thông tin từ các lớp nông (shallow layers - chứa đặc trưng chi tiết, độ phân giải cao) với các lớp sâu (deep layers - chứa đặc trưng ngữ nghĩa, độ phân giải thấp). Bằng cách cộng các bản đồ đặc trưng này lại, FCN có thể tạo ra các dự đoán vừa chính xác về mặt ngữ nghĩa, vừa sắc nét về mặt không gian. Ví dụ, FCN-8s kết hợp thông tin từ 3 tầng đặc trưng khác nhau cho kết quả tốt hơn FCN-16s và FCN-32s.

6.10.2 U-Net

U-Net là một kiến trúc cực kỳ nổi tiếng và có ảnh hưởng, đặc biệt trong lĩnh vực y tế.

- **Kiến trúc hình chữ U:** Nó có một bộ mã hóa (đường xuống) đối xứng với một bộ giải mã (đường lên).
- **Skip Connections mạnh mẽ:** Điểm cải tiến quan trọng của U-Net so với FCN là cách nó sử dụng kết nối tắt. Thay vì cộng, U-Net **nối (concatenate)** các bản đồ đặc trưng từ bộ mã hóa vào các bản đồ đặc trưng tương ứng trong bộ giải mã. Điều này cho phép bộ giải mã truy cập trực tiếp vào các đặc trưng cấp thấp, giúp nó tái tạo lại chi tiết không gian một cách chính xác hơn nhiều.

6.10.3 U-Net++

U-Net++ là một cải tiến của U-Net, giới thiệu các kết nối tắt được thiết kế lại, dày đặc và lồng vào nhau. Mục tiêu là để giảm khoảng cách ngữ nghĩa giữa các bản đồ đặc trưng của bộ mã hóa và bộ giải mã. Kiến trúc này cho phép mô hình học được các đặc trưng ở nhiều cấp độ chi tiết khác nhau, giúp cải thiện hiệu suất.

6.10.4 Mask R-CNN

Mask R-CNN là một kiến trúc cho bài toán **phân đoạn thực thể (instance segmentation)**.

- Nó mở rộng từ Faster R-CNN (một mô hình phát hiện đối tượng).
- Bên cạnh hai nhánh đầu ra của Faster R-CNN (một để phân loại lớp, một để tính chỉnh bounding box), Mask R-CNN thêm một nhánh thứ ba.
- Nhánh thứ ba này là một mạng FCN nhỏ, nhận đầu vào là mỗi vùng đề xuất (Region of Interest - RoI) và tạo ra một **mặt nạ (mask)** nhị phân cho đối tượng bên trong RoI đó.
- Một cải tiến quan trọng là lớp **RoIAlign**, giúp căn chỉnh các đặc trưng trích xuất một cách chính xác hơn so với RoIPooling, dẫn đến việc dự đoán mặt nạ chính xác hơn.

Chương 7

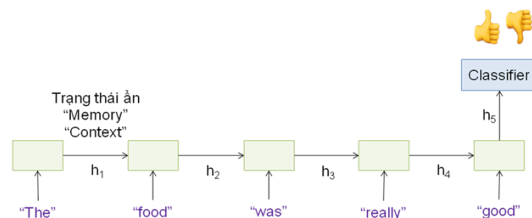
Mạng hồi quy

7.1	Bài toán dự đoán chuỗi	116
7.2	Mạng hồi quy thông thường	117
7.3	Lan truyền ngược theo thời gian (Backpropagation Through Time - BPTT)	121
7.4	Mạng LSTM và GRU	123

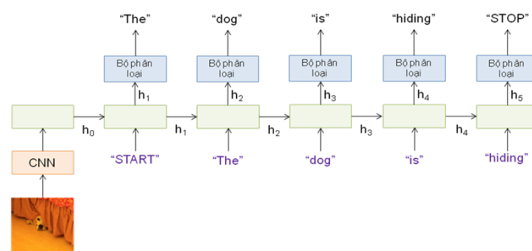
7.1 Bài toán dự đoán chuỗi

Trước đây, chúng ta chỉ tập trung vào các vấn đề/bài toán dự đoán với đầu vào có kích thước cố định. Với các bài toán dự đoán với đầu vào có kích thước thay đổi như các tài liệu/văn bản hay các dữ liệu âm thanh, chúng ta luôn cố gắng biểu diễn dữ liệu đầu vào dưới dạng véc tơ với số chiều xác định trước thông qua các kỹ thuật mã hóa hay trích chọn đặc trưng. Hầu hết các kỹ thuật này đạt được một biểu diễn có kích thước cố định của dữ liệu đầu vào trước khi lựa chọn một phương pháp hay mô hình để giải quyết bài toán đó. Tất nhiên, việc chuyển đổi từ một biểu diễn thô của dữ liệu sang một biểu diễn khác hay ánh xạ vào một không gian mới có thể làm mất nhiều thông tin hay ngữ nghĩa của dữ liệu gốc ban đầu khi các dữ liệu đầu vào là các chuỗi có kích thước khác nhau. Đặc biệt, đối với các bài toán dự đoán đầu ra dạng chuỗi có kích thước thay đổi, chúng ta không thể áp dụng các kỹ thuật trên một cách đơn giản để ánh xạ sang một không gian có số chiều cố định được. Mạng hồi quy (Recurrent Neural Network - RNN) là một giải pháp phù hợp cho các bài toán dự đoán với đầu vào và đầu ra là một chuỗi có kích thước thay đổi.

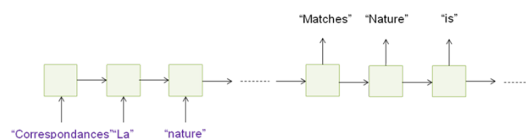
Ví dụ về một số bài toán dự đoán chuỗi trong thực tế:



Hình 7.1: Bài toán Phân loại cảm xúc: Nhiều đầu vào – một đầu ra



Hình 7.2: Bài toán Sinh mô tả bức ảnh: Một đầu vào – nhiều đầu ra

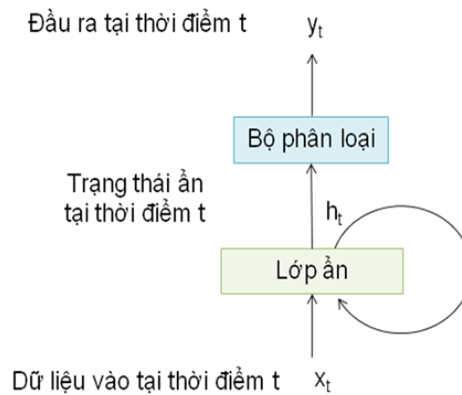


Hình 7.3: Bài toán Dịch máy: Nhiều đầu vào – nhiều đầu ra (hay còn gọi là sequence to sequence)

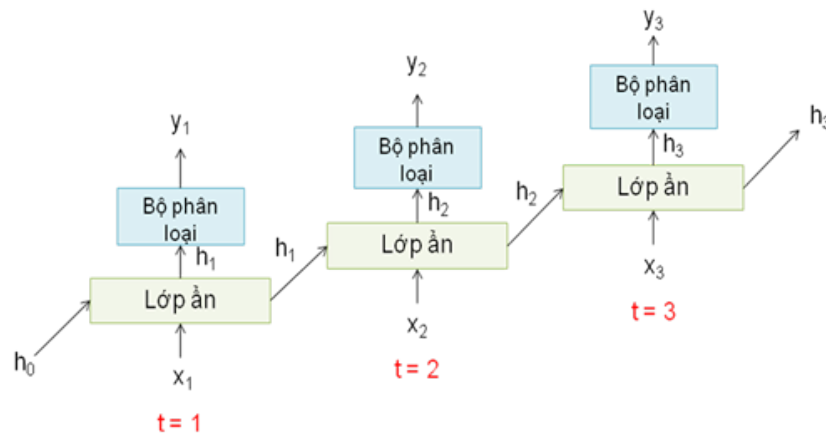
7.2 Mạng hồi quy thông thường

Trong các mạng nơ-ron truyền thống, tất cả các đầu vào và cả đầu ra là độc lập với nhau. Điều này có nghĩa là chúng không liên kết với nhau để tạo thành một chuỗi giàu thông tin và có ngữ nghĩa. Vì vậy, các mô hình mạng nơ-ron này không phù hợp trong rất nhiều bài toán với dữ liệu đầu vào hay dữ liệu đầu ra là một chuỗi các phần tử dữ liệu nối tiếp nhau. Ví dụ, nếu muốn đoán từ tiếp theo có thể xuất hiện trong một câu thì ta cũng cần biết các từ trước đó xuất hiện lần lượt thế nào? Ý tưởng chính của RNN là sử dụng chuỗi các thông tin học được trong dữ liệu dạng chuỗi. RNN được gọi là hồi quy (Recurrent) bởi lẽ chúng thực hiện cùng một tác vụ cho tất cả các phần tử của một chuỗi với đầu ra phụ thuộc vào cả các phép tính

trước đó. Nói cách khác, RNN có khả năng nhớ các thông tin được tính toán trước đó. Trên lý thuyết, RNN có thể sử dụng được thông tin của một văn bản rất dài, tuy nhiên, thực tế thì nó chỉ có thể nhớ được một số bước trước đó mà thôi. Về cơ bản một mạng RNN có dạng như sau:



Mạng hồi quy (RNN) là một mạng nơ-ron chứa một vòng lặp bên trong nó. Trong hình trên, RNN nhận một đầu vào tại thời điểm t là x_t , tiến hành xử lý và đưa ra đầu ra h_t . Điểm đặc biệt của RNN là nó sẽ lưu lại giá trị của h_t để sử dụng cho đầu vào tiếp theo. Có thể coi một mạng hồi quy là một chuỗi những mạng con giống hệt nhau, mỗi mạng sẽ truyền thông tin nó vừa xử lý cho mạng phía sau nó. Nếu ta tách từng vòng lặp xử lý trong RNN ra thành từng mạng con theo cách suy nghĩ như trên thì ta sẽ có một mạng có kiến trúc như sau:

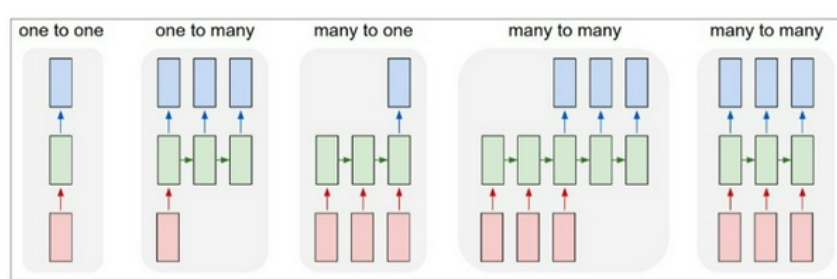


Hình 7.4: Mạng RNN duỗi theo các bước thời gian

Mạng RNN sau khi duỗi được xem như một mạng nơ-ron feed-forward lớn nhận

dữ liệu đầu vào là cả chuỗi dữ liệu, trong đó, các bản sao của RNN được tạo ra theo thời gian với các đầu vào khác nhau tại các bước thời gian khác nhau. Chuỗi đầu vào $x_1, \dots, x_t$ là những sự kiện xảy ra theo thứ tự thời gian. Những sự kiện này đều có mối liên hệ về thông tin với nhau và thông tin của chúng sẽ được giữ lại để xử lý sự kiện tiếp theo trong mạng hồi quy. Vì tính chất này, mạng hồi quy phù hợp cho những bài toán với dữ liệu đầu vào dưới dạng chuỗi và các sự kiện trong chuỗi có mối liên hệ với nhau. Vì vậy, mạng hồi quy có ứng dụng quan trọng trong các bài toán xử lý ngôn ngữ tự nhiên như: Dịch máy - **Neural Machine Translation**, Phân loại ngữ nghĩa - **Semantic classification**, Nhận dạng giọng nói - **Speech Recognition**,...

Một trong những điểm mạnh của mạng hồi quy là cho phép tính toán trên một chuỗi các vector. Dưới đây là tổng hợp các loại dự đoán chuỗi và các kiểu hoạt động của mạng hồi quy đối với dữ liệu chuỗi:



Mỗi hình chữ nhật là 1 vector và các mũi tên thể hiện các hàm biến đổi. Vector đầu vào có màu đỏ, vector đầu ra có màu xanh nước biển và vector trạng thái thông tin trao đổi giữa các mạng con có màu xanh lá cây. Từ trái sang phải ta có:

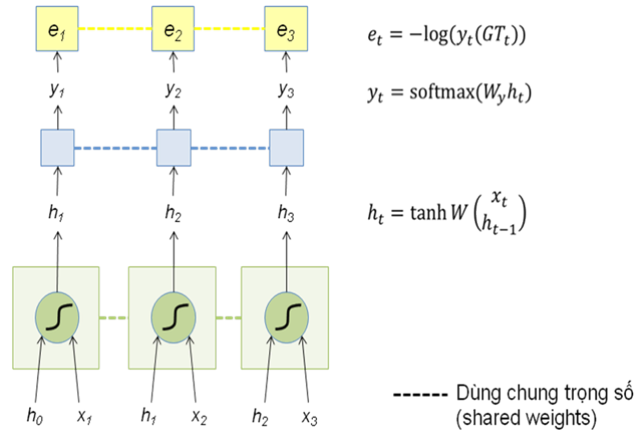
- Mạng neural kiểu Vanilla: Đầu vào và đầu ra có kích thước cố định (Bài toán Phân loại ảnh - **Image Classification**)
- Đầu ra có dạng chuỗi: Đầu vào cố định và đầu ra là một chuỗi các vector (Bài toán tạo tiêu đề/mô tả/chú thích cho ảnh - **Image Captioning**)
- Đầu vào có dạng chuỗi: Đầu vào là một chuỗi vector và đầu ra cố định (Bài toán phân tích cảm xúc - **Sentiment Analysis/Classification**)
- Đầu vào và đầu ra có dạng chuỗi: Bài toán Dịch máy - **Neural Machine Translation**
- Đầu vào và đầu ra có dạng chuỗi đồng bộ: Đầu vào và đầu ra là một chuỗi vector có độ dài bằng nhau (Bài toán phân loại video và gắn nhãn từng frame - **Video Classification**)

Có thể nhận thấy rằng độ dài các chuỗi đầu vào hay đầu ra tại mỗi trường hợp không bắt buộc phải cố định nhưng kích thước vector trạng thái thông tin trao đổi trong mạng neural là cố định. Giờ chúng ta sẽ đi sâu hơn vào phương thức hoạt động của mạng neural hồi quy.

Mạng hồi quy, tại bước thời gian t , nhận một vector đầu vào x_t và đưa ra vector đầu ra y_t . Để có thể lưu trữ được thông tin của các sự kiện trong quá khứ, mạng hồi quy lưu trữ trong chính nó một vector trạng thái ẩn h_t . Vector trạng thái này sẽ lưu giữ những thông tin của những sự kiện đã được xử lý bằng cách cập nhật lại giá trị mỗi khi một sự kiện mới được xử lý. Vector trạng thái ẩn được khởi tạo, kí hiệu là h_0 , là một vector 0. Hàm kích hoạt tanh là một hàm phi tuyến hyperbolic để đưa giá trị của từng phần tử về khoảng $[-1, 1]$. Các ma trận trọng số được dùng chung cho tất cả các bước thời gian.

Có thể nói, việc lan truyền xuôi trong mạng RNN tương đối đơn giản. Nó chỉ đòi hỏi chúng ta mở rộng mạng RNN theo từng bước thời gian một để thu được sự phụ thuộc giữa các biến mô hình và các tham số. Công thức toán học dùng để tính vector trạng thái h_t tại bước thời gian t như sau:

$$\begin{aligned} h_t &= f_W(x_t, h_{t-1}) \\ &= \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} \\ &= \tanh(W_x x_t + W_h h_{t-1}) \end{aligned}$$

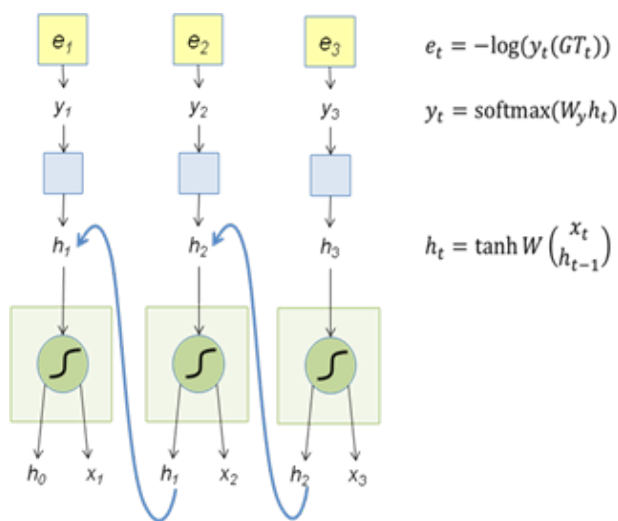


Hình 7.5: Lan truyền xuôi trong RNN

7.3 Lan truyền ngược theo thời gian (Backpropagation Through Time - BPTT)

Mạng hồi quy là một giải pháp phù hợp cho các bài toán dự đoán với đầu vào và đầu ra là một chuỗi có kích thước thay đổi. Tuy nhiên, kiến trúc RNN này có một nhược điểm đối với các chuỗi đầu vào có kích thước lớn. Nếu kích thước chuỗi đầu vào là rất lớn thì việc tính toán vector trạng thái ẩn h_t sẽ phải đi qua nhiều lớp tính toán tương ứng với các bước thời gian của chuỗi đầu vào. Trong quá trình lan truyền ngược để cập nhật các ma trận trọng số, chúng ta phải thực hiện *lan truyền ngược qua các bước thời gian* này.

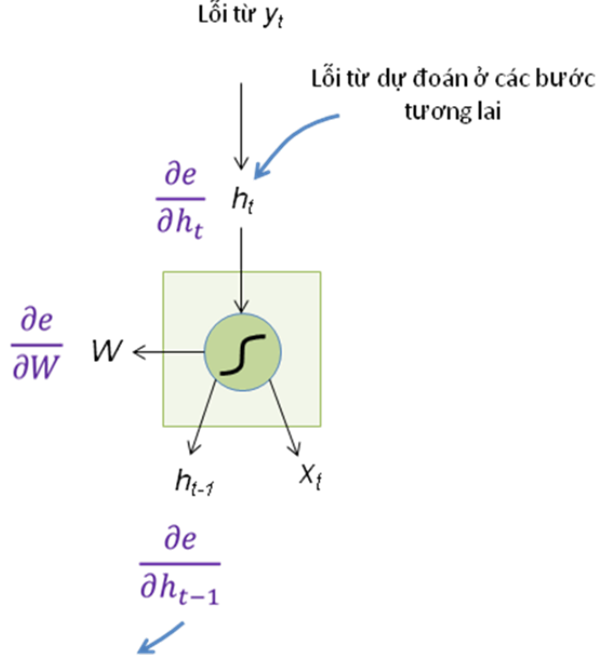
Lan truyền ngược theo thời gian (BPTT) thực chất là một ứng dụng cụ thể của lan truyền ngược trong mạng RNN để điều chỉnh hành vi của mạng hồi quy, các ma trận trọng số này sẽ được cập nhật trong quá trình huấn luyện.



Hình 7.6: Lan truyền ngược theo thời gian trong RNN

Áp dụng luật chuỗi đối với tính đạo hàm của hàm hợp khi lan truyền ngược để tính toán và lưu các giá trị gradient. Vì chuỗi có thể khá dài nên sự phụ thuộc trong chuỗi cũng có thể rất dài. Ví dụ, đối với một chuỗi gồm 1000 ký tự, ký tự đầu tiên cũng có thể ảnh hưởng tới ký tự ở vị trí 1000. Điều này không thực sự khả thi về mặt tính toán (cần quá nhiều thời gian và bộ nhớ) và nó đòi hỏi hơn 1000 phép nhân ma trận-vector trước khi thu được các giá trị gradient cần tính. Do tính đạo hàm phải đi qua nhiều lớp tính toán của vector h_t nên các giá trị **gradient** cũng sẽ lớn dần lên hay bị suy biến và việc cập nhật các trọng số không còn theo ý muốn và khiến mạng không ổn định. Đây là một quá trình chứa đầy sự bất định về mặt tính

toán và thống kê. Trong phần tiếp theo chúng ta sẽ làm sáng tỏ những gì sẽ xảy ra và cách giải quyết vấn đề này trong thực tế.



Hình 7.7: Tính đạo hàm để lan truyền ngược lỗi.

Trong quá trình BPTT, gradient đối với một trọng số của mạng RNN được tính tại mỗi bản sao của nó trong mạng duỗi (unfolded network), sau đó được cộng lại (hoặc tính trung bình) và được sử dụng để cập nhật trọng số mạng. Chúng ta có:

$$\begin{aligned}
 h_t &= \tanh(W_x x_t + W_h h_{t-1}) \\
 \frac{\partial e}{\partial W_h} &= \frac{\partial e}{\partial h_t} \odot (1 - \tanh^2(W_x x_t + W_h h_{t-1})) h_{t-1}^T \\
 \frac{\partial e}{\partial W_x} &= \frac{\partial e}{\partial h_t} \odot (1 - \tanh^2(W_x x_t + W_h h_{t-1})) x_t^T \\
 \frac{\partial e}{\partial h_{t-1}} &= W_h^T (1 - \tanh^2(W_x x_t + W_h h_{t-1})) \odot \frac{\partial e}{\partial h_t}
 \end{aligned}$$

Xem xét công thức cuối cùng ở trên, chúng ta dễ thấy gradient sẽ triệt tiêu khi tính tại các bước thời gian với khoảng cách rất xa nếu giá trị riêng lớn nhất của W_h nhỏ hơn 1. Hơn nữa, giá trị hàm tanh lớn sẽ tương ứng với gradient nhỏ (vùng bão hòa) và gradient cũng sẽ tiệm cận 0 và bị suy biến.

Tóm lại, lan truyền ngược theo thời gian chỉ là việc áp dụng lan truyền ngược

cho các mô hình chuỗi có trạng thái ẩn. Đối với các chuỗi đầu vào có kích thước lớn, việc tính đạo hàm tại các bước thời gian cách xa dẫn đến lũy thừa lớn của ma trận có thể làm các trị riêng tiêu biến hoặc tăng rất lớn, biểu hiện dưới hiện tượng tiêu biến hoặc bùng nổ gradient. Vì vậy, việc cắt xén là cần thiết để thuận tiện cho việc tính toán và ổn định các giá trị số. Cuối cùng, để tăng hiệu năng tính toán, tất cả các giá trị trung gian cần được lưu lại.

7.4 Mạng LSTM và GRU

Một điểm nổi bật của RNN chính là ý tưởng kết nối các thông tin phía trước để dự đoán cho hiện tại. Về mặt lý thuyết, rõ ràng là RNN có khả năng xử lý được các phụ thuộc xa (long-term dependencies). Thật không may là với khoảng cách càng lớn dần thì RNN bắt đầu không thể nhớ và học được nữa do hiện tượng suy biến gradient.

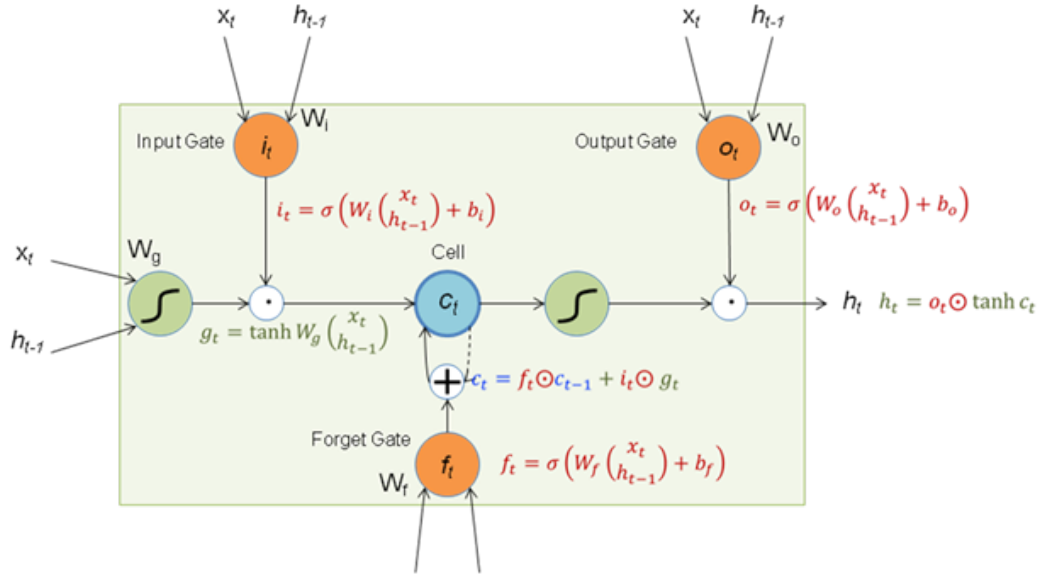
Vì vậy, các biến thể của mạng neural hồi quy đã ra đời để khắc phục vấn đề này: **LSTM - Long Short Term Memory** và **GRU - Gated Recurrent Unit**. Hai mạng này đã bổ sung thêm cơ chế loại bỏ những thông tin không cần thiết ra khỏi vector trạng thái ẩn h, từ đó đã khắc phục được một phần nhược điểm trên. Hai biến thể này được chú ý nhiều nhất và được sử dụng phổ biến nhất.

Mạng bộ nhớ dài-ngắn hạn (Long Short Term Memory networks), thường được gọi là LSTM - là một dạng đặc biệt của RNN, nó có khả năng học được các phụ thuộc xa. LSTM được giới thiệu bởi Hochreiter và Schmidhuber (1997), sau đó đã được cải tiến và phổ biến bởi rất nhiều người trong ngành. Nó hoạt động cực kì hiệu quả trên nhiều bài toán khác nhau nên dần đã trở nên phổ biến như hiện nay.

LSTM được thiết kế để tránh được vấn đề phụ thuộc xa (long-term dependency). Việc nhớ thông tin trong suốt thời gian dài là đặc tính mặc định của nó mà không cần phải huấn luyện để có thể nhớ được và không cần bất kì can thiệp nào.

Mọi mạng hồi quy đều có dạng là một chuỗi các mô-đun lặp đi lặp lại của mạng nơ-ron. Với mạng RNN thường, các mô-đun này có cấu trúc rất đơn giản, thường là một tầng với hàm kích hoạt tanh. LSTM cũng có kiến trúc dạng chuỗi như vậy, nhưng các mô-đun trong nó có cấu trúc khác với mạng RNN thường. Thay vì chỉ có một tầng mạng nơ-ron, chúng có tới 4 tầng tương tác với nhau một cách rất đặc biệt.

Chìa khóa của LSTM là trạng thái tế bào (cell state). Trạng thái tế bào là một dạng giống như băng truyền. Nó chạy xuyên suốt tất cả các nút mạng và chỉ thực hiện tương tác tuyến tính. Vì vậy mà các thông tin có thể dễ dàng truyền đi thông



suốt mà không sợ bị thay đổi.

Đặc biệt, LSTM có khả năng bỏ đi hoặc thêm vào các thông tin cần thiết cho trạng thái tế bào, chúng được điều chỉnh cẩn thận bởi các cổng (gates). Các cổng là nơi sàng lọc thông tin đi qua nó, chúng được kết hợp bởi một tầng mạng sigmoid và một phép nhân. Tầng sigmoid sẽ cho đầu ra là một số trong khoảng $[0, 1]$, thể hiện có bao nhiêu thông tin có thể được đi qua. Khi đầu ra là 0 thì có nghĩa là không cho thông tin nào qua cả, còn khi là 1 thì có nghĩa là cho tất cả các thông tin đi qua nó. Một LSTM gồm có 3 cổng như vậy để duy trì và điều hành trạng thái của tế bào.

Bước đầu tiên của LSTM là quyết định xem thông tin nào cần bỏ đi từ trạng thái tế bào. Quyết định này được đưa ra bởi tầng sigmoid - gọi là “tầng cổng quên” (forget gate layer). Nó sẽ lấy đầu vào là h_{t-1} và x_t rồi đưa ra kết quả là một số trong khoảng $[0, 1]$ cho mỗi phần tử trong trạng thái tế bào c_{t-1} . Đầu ra là 1 thể hiện rằng nó giữ toàn bộ thông tin lại, còn 0 chỉ rằng toàn bộ thông tin sẽ bị bỏ đi.

Bước tiếp theo là quyết định xem thông tin mới nào ta sẽ lưu vào trạng thái tế bào. Việc này gồm 2 phần. Đầu tiên là sử dụng một tầng sigmoid được gọi là “tầng cổng vào” (input gate layer) để quyết định phần tử nào sẽ cập nhật. Tiếp theo là một tầng tanh tạo ra một véc-tơ cho giá trị mới c_t nhằm thêm vào cho trạng thái. Trong bước tiếp theo, ta sẽ kết hợp 2 giá trị đó lại để tạo ra một cập nhật cho trạng thái cũ c_{t-1} thành trạng thái mới c_t . Trạng thái mới thu được này phụ thuộc vào việc ta quyết định cập nhật mỗi phần tử trạng thái ra sao.

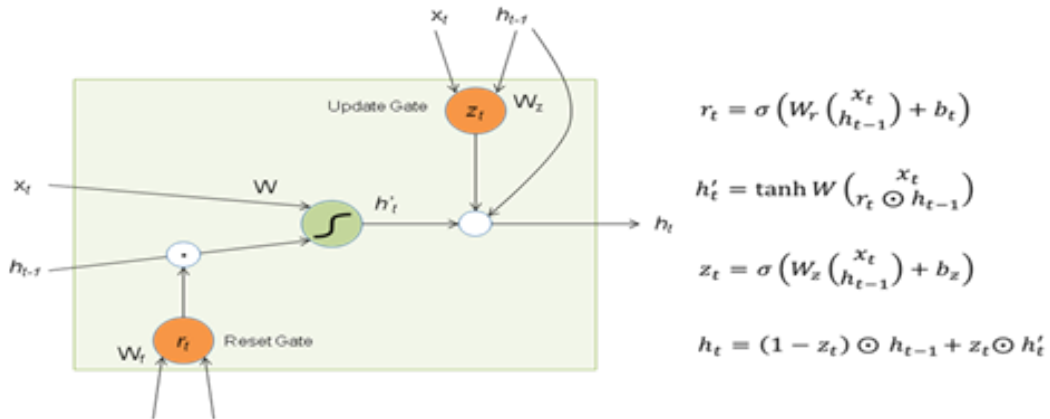
Cuối cùng, ta cần quyết định xem ta muốn đầu ra là gì. Giá trị đầu ra sẽ dựa vào trạng thái tế bào, nhưng sẽ được tiếp tục sàng lọc. Đầu tiên, ta chạy một tầng sigmoid - gọi là "tầng cổng ra" (output gate layer) để quyết định phần nào của trạng thái tế bào ta muốn xuất ra. Sau đó, chúng ta đưa trạng thái tế bào qua một hàm tanh để co giá trị nó về khoảng $[-1, 1]$ và nhân nó với đầu ra của cổng sigmoid để được giá trị đầu ra ta mong muốn.

Tóm lại, LSTM sử dụng thêm "cell" có bộ nhớ để tránh hiện tượng triệt tiêu gradient. Luồng gradient từ c_t tới c_{t-1} chỉ lan truyền ngược qua phép nhân và cộng từng phần tử, không đi qua phép nhân ma trận và hàm tanh nên tránh được hiện tượng triệt tiêu gradient.

Trên đây là một LSTM khá bình thường nhưng không phải tất cả các LSTM đều giống như vậy. Thực tế, các bài báo về LSTM đều sử dụng một phiên bản hơi khác so với mô hình LSTM chuẩn. Sự khác nhau không lớn, nhưng chúng giúp giải quyết phần nào đó trong cấu trúc của LSTM.

Một dạng LSTM phổ biến được giới thiệu bởi Gers và Schmidhuber (2000) được thêm các đường kết nối "peephole connections", làm cho các tầng cổng nhận được giá trị đầu vào là trạng thái tế bào.

Một biến thể khá thú vị khác của LSTM là Gated Recurrent Unit, hay GRU được giới thiệu bởi Cho, et al. (2014). Nó kết hợp các cổng quên và cổng ra thành một cổng "cổng cập nhật" (update gate). Nó cũng hợp trạng thái tế bào và trạng thái ẩn với nhau tạo ra một thay đổi khác. Kết quả là mô hình của ta sẽ đơn giản hơn mô hình LSTM chuẩn và ngày càng trở nên phổ biến.



Tài liệu tham khảo

1. Khóa cs231n của Stanford: <http://cs231n.stanford.edu>
2. Khóa cs244n của Stanford:
<http://web.stanford.edu/class/cs244n/slides/cs244n-2020-lecture06-rnnlm.pdf>
<http://web.stanford.edu/class/cs244n/slides/cs244n-2020-lecture07-fancy-rnn.pdf>
3. Training RNNs: <http://www.cs.toronto.edu/~rgrosse/csc321/lec10.pdf>

Chương 8

Một số ứng dụng học sâu trong xử lý ngôn ngữ tự nhiên

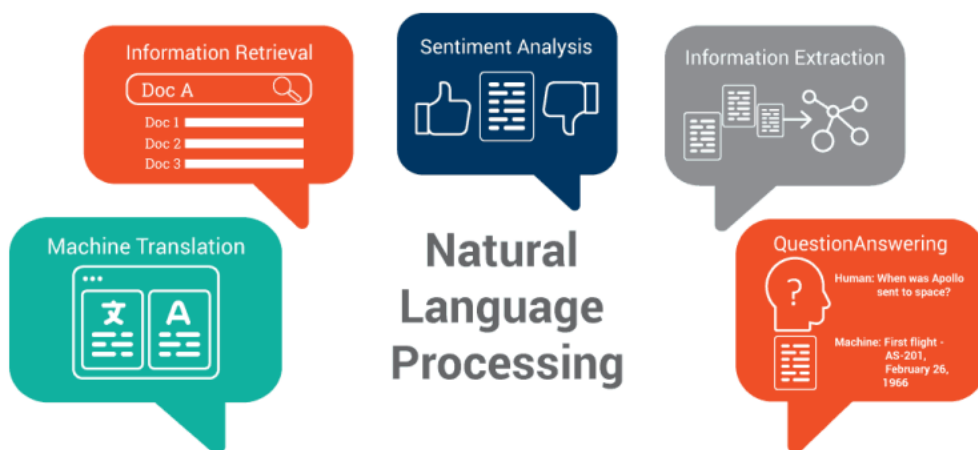
8.1	Tổng quan về xử lý ngôn ngữ tự nhiên	127
8.2	Biểu diễn từ và văn bản	129
8.3	Word2vec	129
8.4	Giới thiệu về bài toán dịch máy	135

8.1 Tổng quan về xử lý ngôn ngữ tự nhiên

Xử lý ngôn ngữ tự nhiên (XLNNTN) là một nhánh của trí tuệ nhân tạo liên quan đến sự tương tác giữa máy tính và ngôn ngữ của con người. Mục đích của XLNNTN là giúp máy tính có khả năng đọc, hiểu và rút ra ý nghĩa từ ngôn ngữ của con người.

Các mức phân tích trong XLNNTN bao gồm:

- Morphology (hình thái học): cách từ được xây dựng, các tiền tố và hậu tố của từ.
- Syntax (cú pháp): mối liên hệ về cấu trúc ngữ pháp giữa các từ và ngữ.
- Semantics (ngữ nghĩa): nghĩa của từ, cụm từ, và cách diễn đạt.
- Discourse (diễn ngôn): quan hệ giữa các ý hoặc các câu.



Hình 8.1: Một số ứng dụng quan trọng trong XLNNTN.

- Pragmatic (thực chứng): mục đích phát ngôn, cách sử dụng ngôn ngữ trong giao tiếp.
- World Knowledge (tri thức thế giới): các tri thức về thế giới, các tri thức ngầm.

XLNNTN có nhiều ứng dụng trong thực tiễn, các ứng dụng chính bao gồm:

- Nhận dạng giọng nói (speech recognition)
- Khai phá văn bản: Phân cụm văn bản; Phân lớp văn bản; Tóm tắt văn bản; Mô hình hóa chủ đề (topic modelling); Hỏi đáp (question answering)
- Gia sư ngôn ngữ (Language tutoring): Chỉnh sửa ngữ pháp/đánh vần
- Dịch máy (machine translation)

Ví dụ một số bài toán trong XLNNTN: Tách từ (tokenization) là bài toán chia văn bản thành các từ và các câu. Gán nhãn từ loại (part-of-speech tagging) là bài toán xác định từ loại của từng từ trong văn bản. Nhận diện thực thể có tên (Named entity recognition) là bài toán tìm kiếm và phân loại các thành phần trong văn bản vào những loại xác định trước như là tên người, tổ chức, địa điểm, thời gian, số lượng, giá trị tiền tệ... Phân tích cú pháp (syntactic parsing) Phân tích ngữ pháp của một câu cho trước theo các quy tắc ngữ pháp cho trước. Trích rút quan hệ (Relation extraction) là bài toán xác định quan hệ giữa các thực thể. Đây là bài toán phân tích ngữ nghĩa ở mức nông. Suy diễn logic là bài toán phân tích ngữ nghĩa ở mức sâu.

8.2 Biểu diễn từ và văn bản

Wordnet là một mạng lưới từ chứa các tập hợp từ đồng nghĩa (synset) và thể hiện các quan hệ bao hàm nghĩa (hypernymy). Việc xây dựng Wordnet cần nhiều công sức thủ công nên khả năng mở rộng của nó là giới hạn. Để sử dụng Wordnet một cách hiệu quả, cần phải xác định ngữ nghĩa của các từ trong các ngữ cảnh cụ thể. Việc tính toán độ tương đồng dựa trên wordnet không hiệu quả trong nhiều trường hợp.

Biểu diễn one-hot xây dựng biểu diễn từ như các kí hiệu rời rạc. Mỗi từ tương ứng với một vector có một thành phần duy nhất có giá trị 1 và các thành phần còn lại có giá trị 0. Độ dài của vector bằng độ lớn của từ điển. Nhược điểm chính của biểu diễn one-hot là không thể biểu diễn mối quan hệ đồng nghĩa hay gần nghĩa (vd hotel và model). Bên cạnh đó, việc lưu trữ và tính toán với biểu diễn one-hot cũng bị hạn chế khi kích thước từ điển lớn. Ví dụ với ngôn ngữ hàng ngày khoảng 20K từ, dịch máy 50K từ, khoa học vật liệu 500K từ, google web crawl 13M từ.

Biểu diễn dựa trên ngữ cảnh là việc biểu diễn từ dựa trên ngữ cảnh của nó. Ngữ cảnh của một từ là các từ thường xuất hiện của với từ này. Từ đó, ta có thể sử dụng nhiều ngữ cảnh khác nhau để xây dựng nên biểu diễn của một từ. Mỗi từ sẽ được biểu diễn bởi một vector dày sao cho nó biểu diễn được các từ thường hay xuất hiện trong ngữ cảnh của từ cần biểu diễn.

8.3 Word2vec

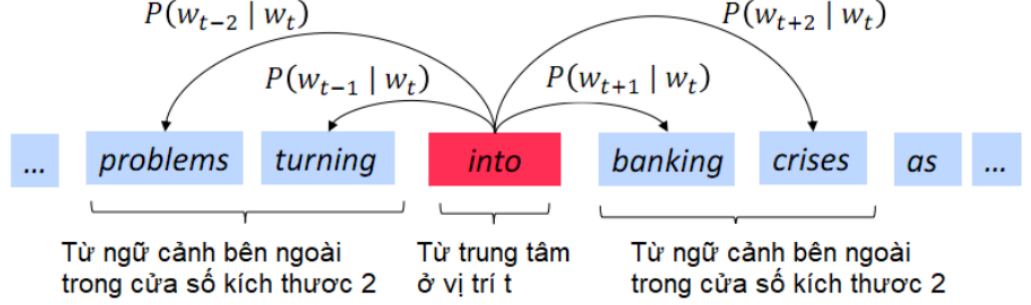
Word2vec (Mikolov et al. 2013) là phương pháp để học biểu diễn từ. Ý tưởng chính của phương pháp này là sử dụng một tập lớn nhiều văn bản (corpus). Mỗi từ trong tập từ vựng cố định được biểu diễn bằng một véctơ. Duyệt từng vị trí t trong văn bản, mỗi vị trí chứa từ trung tâm c và các từ ngữ cảnh bên ngoài o . Sử dụng độ tương đồng của các véctơ biểu diễn c và o để tính xác suất xuất hiện o khi có c (hoặc ngược lại). Tinh chỉnh véctơ từ để cực đại hóa xác suất này.

Xét khả năng sinh ra các từ trong ngữ cảnh khi biết từ trung tâm

$$L(\theta) = \prod_{t=1}^T \prod_{-m \leq j \leq m, j \neq 0} P(w_{t+j} | w_t; \theta) \quad (8.1)$$

Khi đó, ta xây dựng hàm mục tiêu là cực tiểu hóa negative log likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j} | w_t; \theta) \quad (8.2)$$



Hình 8.2: Ví dụ về dự đoán từ trung tâm trong mô hình Word2vec.

Để tính $P(w_{t+j}|w_t; \theta)$, ta sử dụng hai vector cho mỗi từ w . Khi w là từ trung tâm, ta sử dụng v_w , khi w là từ ngữ cảnh, ta sử dụng u_w . Khi đó, với từ trung tâm c và từ ngữ cảnh o , ta có

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)} \quad (8.3)$$

Khi đó, tham số của mô hình là tập hợp các vector u và v , $\theta \in R^{2dV}$. Tham số của mô hình có thể được huấn luyện bằng thuật toán SGD

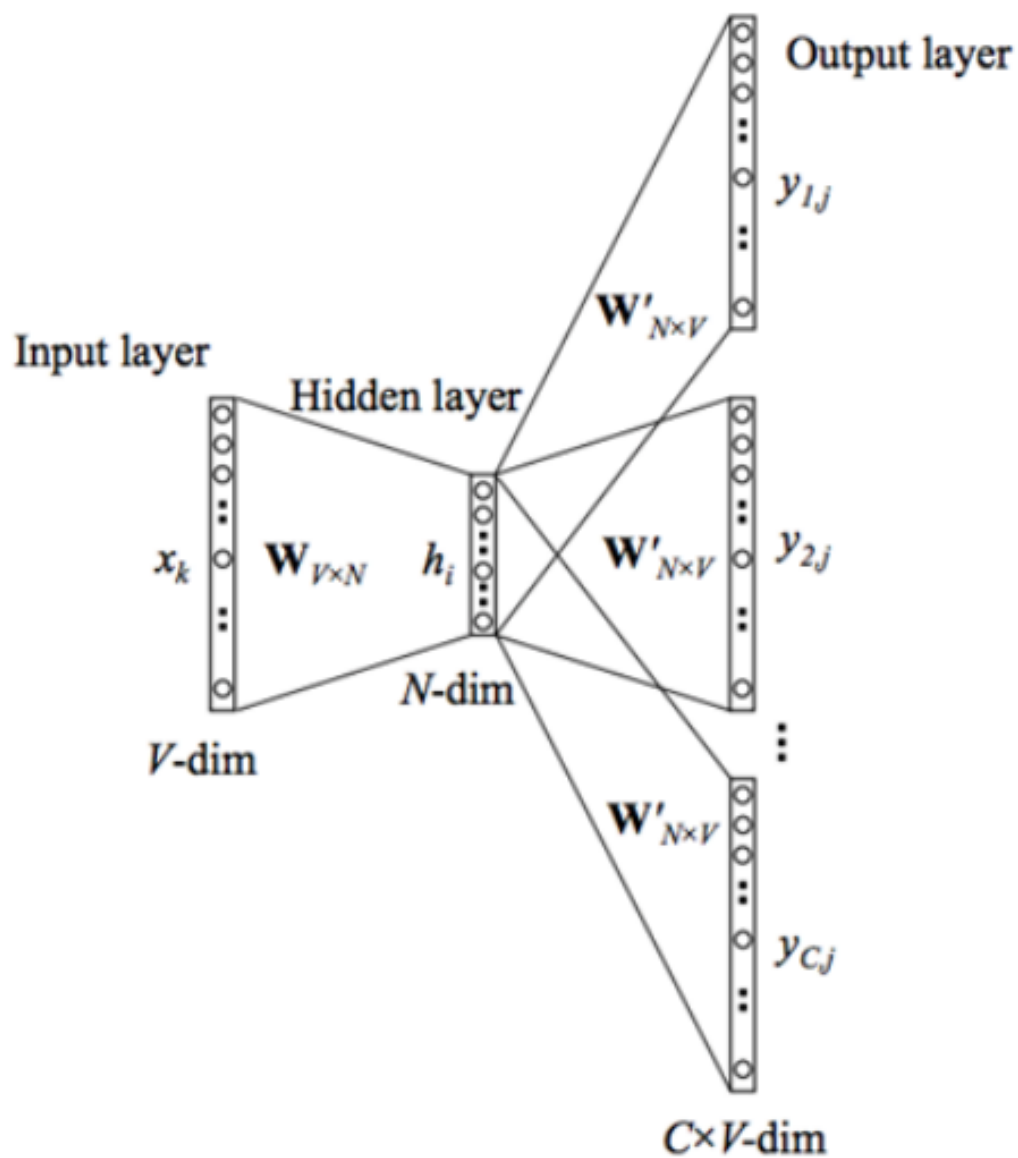
$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} J(\theta) \quad (8.4)$$

8.3.1 Mô hình skip-gram

Mô hình skip-gram bao gồm tầng đầu vào, tầng ẩn, và tầng đầu ra. Tầng đầu vào có V nơ-ron, nhận biểu diễn one-hot của từ trung tâm. Trọng số giữa tầng đầu vào và tầng ẩn là $W_{V \times N}$, trong đó hàng thứ k là biểu diễn từ trung tâm của từ thứ k trong từ điển. Tầng đầu ra có V nơ-ron, tương ứng với V từ trong từ điển. Trọng số giữa tầng ẩn và tầng đầu ra là $W'_{N \times V}$, trong đó cột thứ k là biểu diễn từ ngữ cảnh của từ thứ k trong từ điển.

Vấn đề với hàm negative log likelihood là việc tính toán mẫu số của xác suất $P(w_{t+j}|w_t; \theta)$ có độ phức tạp tính toán $O(V)$. Để giải quyết vấn đề này, kĩ thuật lấy mẫu âm (negative sampling) đã được đề xuất, trong đó xác suất $P(w_{t+j}|w_t; \theta)$ sẽ được xấp xỉ bởi

$$\log P(o|c; \theta) \sim \log \sigma(u_o^T v_c) + \sum_{k=1}^N E_{o' \sim P(w)} \log \sigma(u_{o'}^T v_c) \quad (8.5)$$



Hình 8.3: Kiến trúc mô hình Skipgram.

trong đó $P(w)$ là phân phối unigram của w . Phân phối này được xây dựng để lấy mẫu các từ dựa trên tần suất xuất hiện của từ đó.

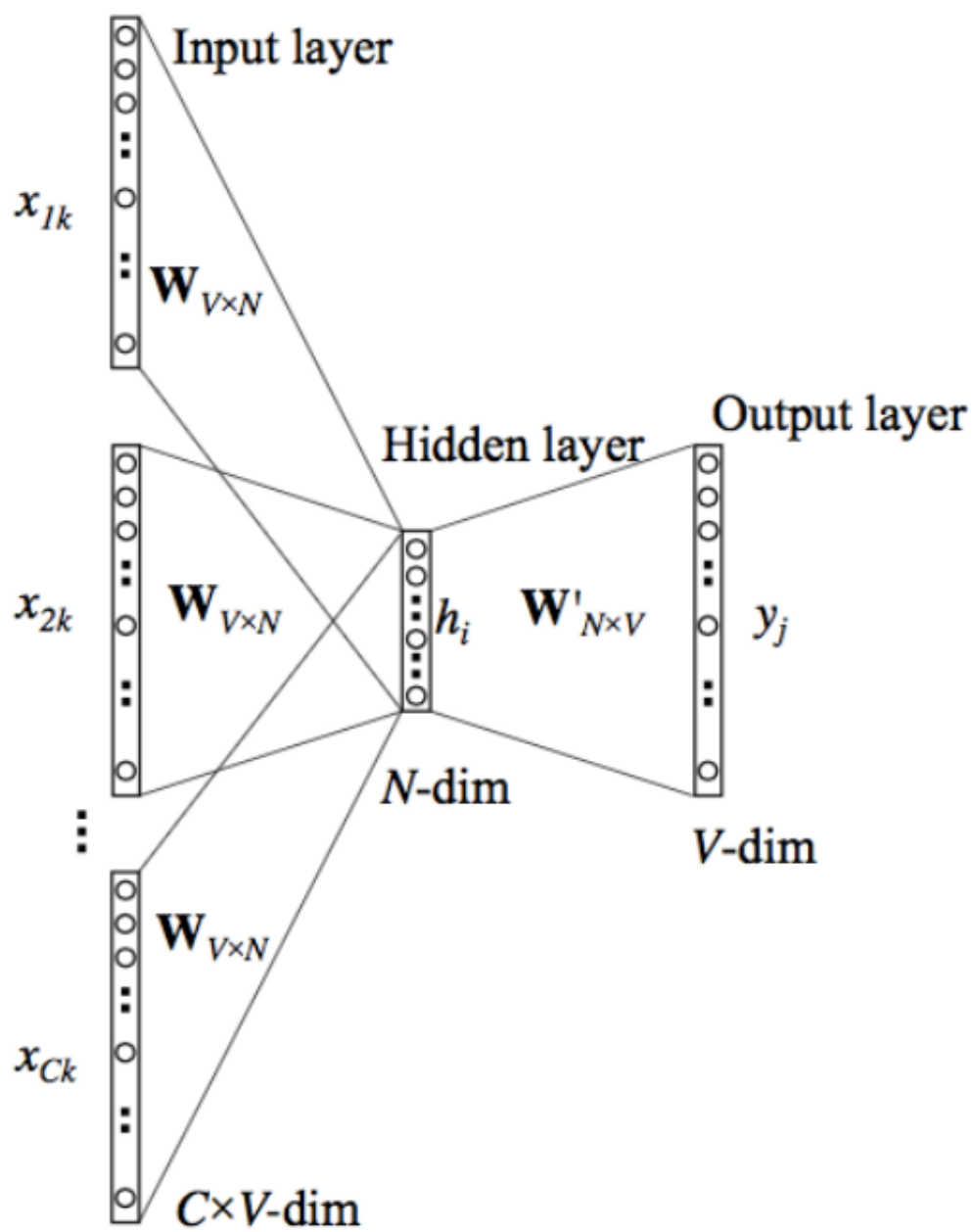
Ngược lại với skipgram, mô hình cbow sử dụng các từ trong ngữ cảnh để dự đoán từ trung tâm.

8.3.2 Mô hình glove

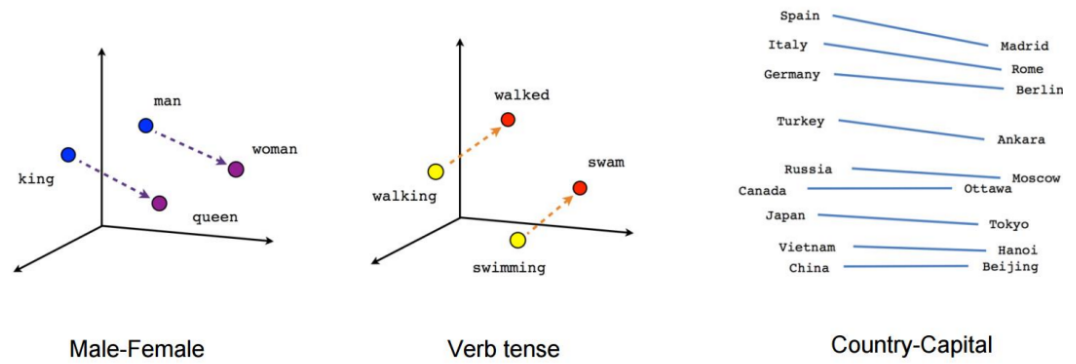
Ma trận đồng xuất hiện là một ma trận với số hàng và số cột bằng kích thước từ điển. Một ô (i,j) có giá trị là số lần từ i và từ j xuất hiện cùng nhau. Ma trận đồng xuất hiện có thể được sử dụng để sinh ra biểu diễn của các từ. Tương tự như biểu diễn one-hot, kích thước vector biểu diễn từ nhanh chóng bị tăng lên khi kích thước từ điển tăng. Khi đó, một giải pháp được đưa ra là áp dụng các kỹ thuật giảm chiều ma trận, nghĩa là chỉ xét các thành phần quan trọng nhất của ma trận để sinh ra biểu diễn thấp chiều. Glove chính là phương pháp kết hợp cách học trong word2vec với ma trận đồng xuất hiện để học biểu diễn từ.

$$J = \sum_{i,j=1}^V f(X_{ij})(w_i^T w'_j + b_i + b'_j + \log X_{ij}) \quad (8.6)$$

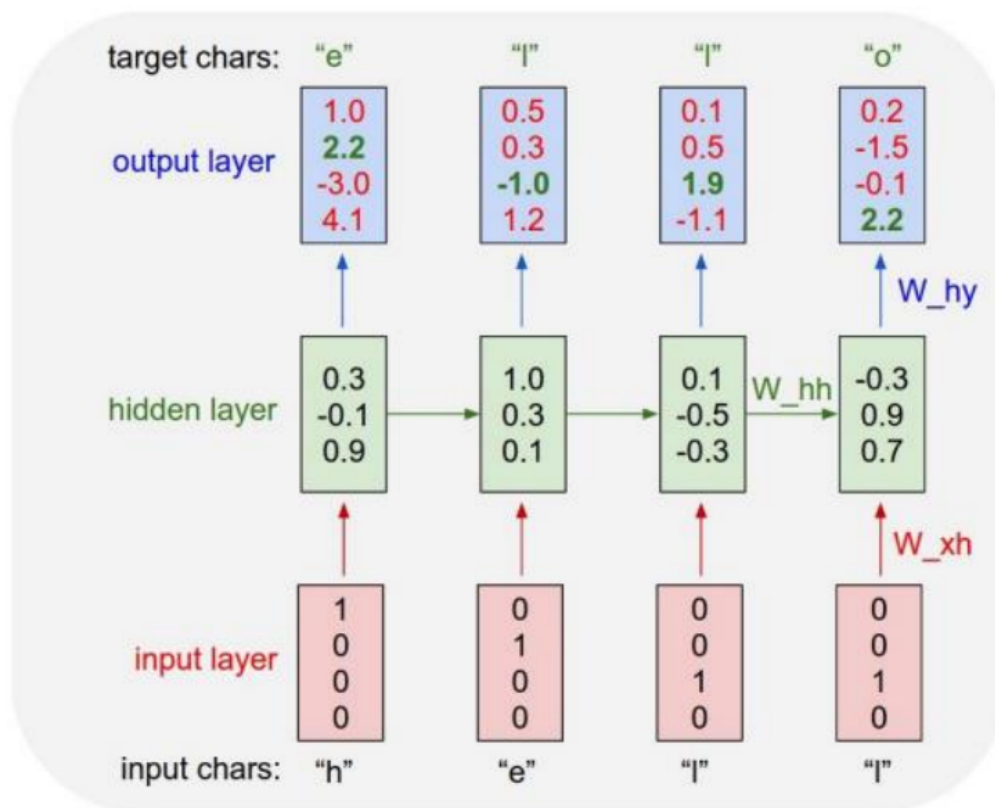
$$P(w) = 1 - \sqrt{\frac{t}{f(w)}} \quad (8.7)$$



Hình 8.4: Kiến trúc mô hình CBOW.



Hình 8.5: Một số kết quả trực quan hóa vector từ học được từ mô hình word2vec.



Hình 8.6: Kiến trúc mô hình Character RNN.

8.4 Giới thiệu về bài toán dịch máy

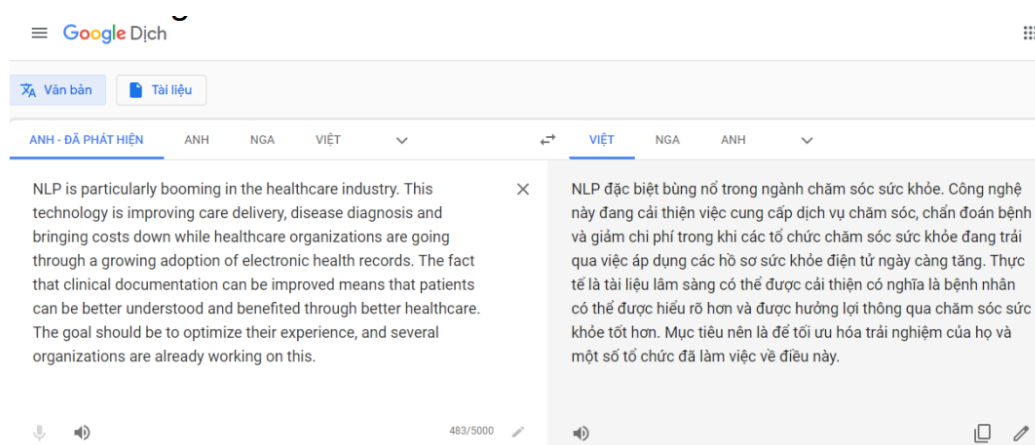
8.4.1 Mô hình Character RNN

Đây là một mô hình để sinh ra các kí tự tiếp theo từ các kí tự đầu vào. Mô hình gồm có một tầng đầu vào biểu diễn các kí tự dưới dạng one hot. Tầng ẩn gồm các nhân RNN được kết nối hồi quy với nhau. Tầng đầu ra biểu diễn các kí tự dưới dạng one hot. Hàm softmax được thực hiện trước tầng đầu ra để tính xác suất của các kí tự.

8.4.2 Giới thiệu về bài toán dịch máy

Dịch máy (MT) là thao tác dịch một câu x từ một ngôn ngữ (gọi là ngôn ngữ nguồn) sang một câu y trong ngôn ngữ khác (gọi là ngôn ngữ đích).

Bắt đầu từ những năm 1950 với việc dịch từ Nga sang Anh (xuất phát từ nhu



Hình 8.7: Ví dụ kết quả dịch tự động từ ứng dụng Google Translate.

cầu trong Chiến tranh lạnh). Hệ thống dịch chủ yếu theo quy tắc (rule-based), dùng từ điển để ánh xạ các từ tiếng Nga sang tiếng Anh.

Nhược điểm của phương pháp dựa trên luật là cần nhiều công sức thủ công, bao gồm từ điển ánh xạ từ, các luật chuyển đổi ngữ nghĩa và ngữ pháp, các luật hình thái từ. Các hệ thống dịch dựa trên luật thường có chất lượng thấp.

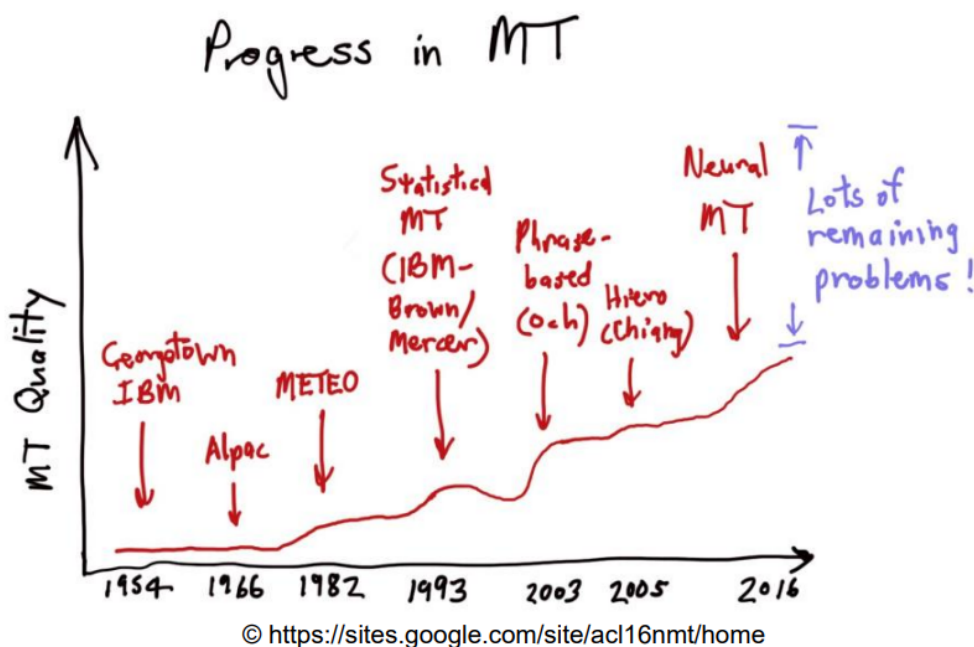
Dịch máy thống kê thực hiện việc học mô hình xác suất từ dữ liệu có nhãn. Mục tiêu chính là tìm kiếm câu phù hợp nhất ở ngôn ngữ đích dựa trên câu đầu vào ở ngôn ngữ nguồn.

Giả định ta cần giống các từ trong câu nguồn với các từ trong câu đích với vector giống a . Khi đó, mục tiêu là tìm cách giống sau cho cực đại hóa xác suất $P(s,a|t)$

Các hệ thống tốt nhất thiết kế theo cách này thường có kiến trúc phức tạp. Mỗi hệ thống bao gồm nhiều module nhỏ bên trong. Các hệ thống này dù vậy vẫn chưa đạt được hiệu năng dịch như con người. Bên cạnh đó, các hệ thống này vẫn cần nhiều xử lý thủ công như trích chọn đặc trưng hay việc sử dụng các tài nguyên từ bên ngoài. Chi phí bảo trì của các hệ thống này cao, khi chuyển sang cặp ngôn ngữ khác cần phải xây dựng lại từ đầu.

8.4.3 Mô hình dịch máy sâu

Mô hình dịch máy sâu (Neural Machine Translation - NMT) được thiết kế dựa trên kiến trúc chuỗi-tới-chuỗi (sequence-to-sequence). Kiến trúc này gồm hai khối mã hóa và giải mã. Khối mã hóa sinh ra thông tin mã hóa từ câu nguồn. Khối giải mã sinh ra câu đích dựa trên thông tin của câu nguồn. Bên cạnh bài toán dịch máy, mô hình seq2seq có thể sử dụng cho nhiều bài toán khác nhau như tóm tắt văn bản,



Hình 8.8: Các tiến bộ quan trọng trong lĩnh vực Dịch máy.

hội thoại, sinh mã nguồn...

Mô hình NMT mô hình hóa bài toán dịch máy như mô hình ngôn ngữ có điều kiện: Mô hình ngôn ngữ thực hiện việc dự đoán một từ dựa trên ngữ cảnh của các từ xung quanh. Mô hình ngôn ngữ có điều kiện thêm điều kiện dự đoán từ câu nguồn.

$$P(y|x) = P(y_1|x)P(y_2|y_1, x) \dots P(y_T|y_1, \dots, y_{T-1}, x) \quad (8.8)$$

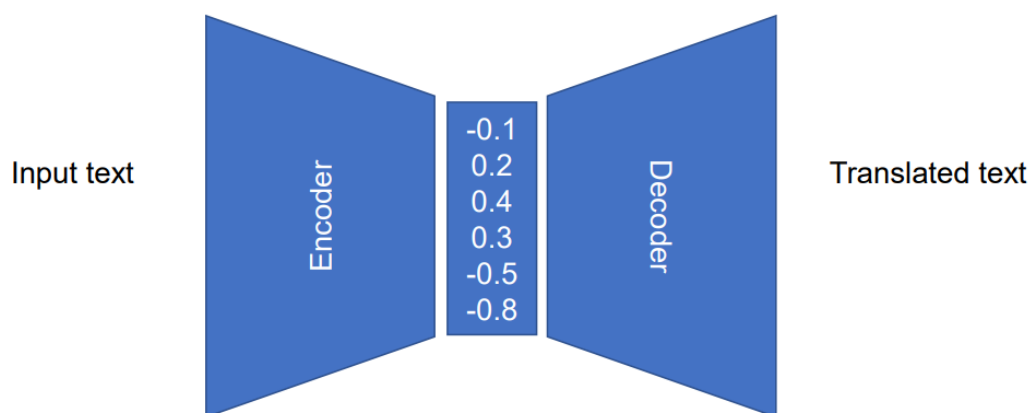
Tại bước giải mã, phương pháp đơn giản nhất là lấy từ có xác suất lớn nhất tại mỗi bước. Tuy nhiên, phương pháp tham lam này dễ dẫn đến sai sót ngay khi gặp lỗi tại một bước mà không thể quay lại sửa lỗi.

8.4.4 Độ đo BLEU

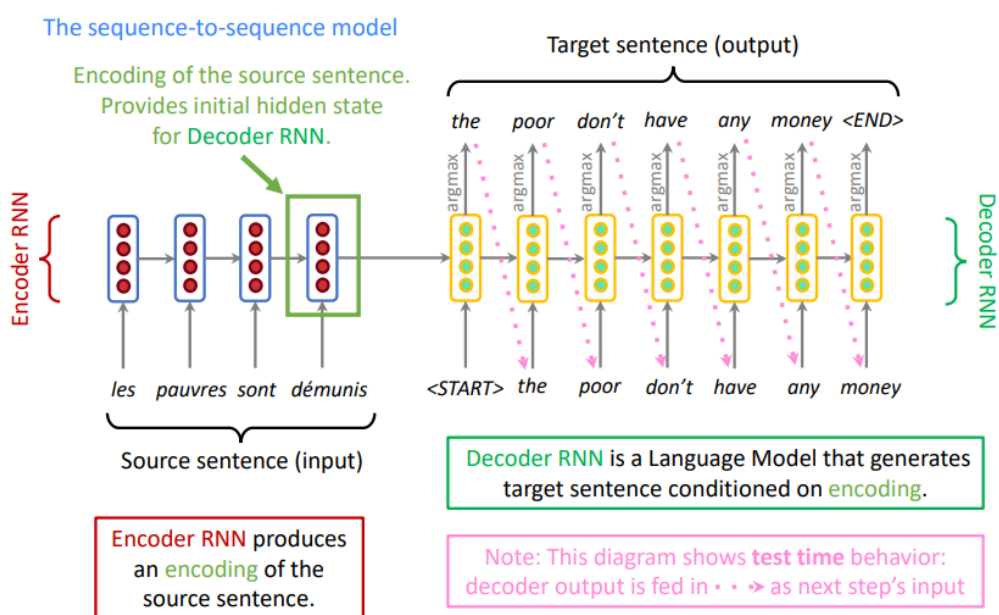
Độ đo BLEU được sử dụng đánh giá chất lượng của các hệ thống dịch. Câu dịch cho hệ thống sinh ra được so sánh với câu do người dịch dựa trên điểm BLEU. Độ đo này đo độ chính xác của các n-gram (từ 1 tới 4) và phạt các câu dịch quá ngắn.

$$BLEU = \left(1, \frac{\text{output_length}}{\text{reference_length}}\right) \left(\prod_{i=1}^4 \text{precision}_i\right)^{\frac{1}{4}} \quad (8.9)$$

- 30-40: Bản dịch có thể hiểu được - có chất lượng

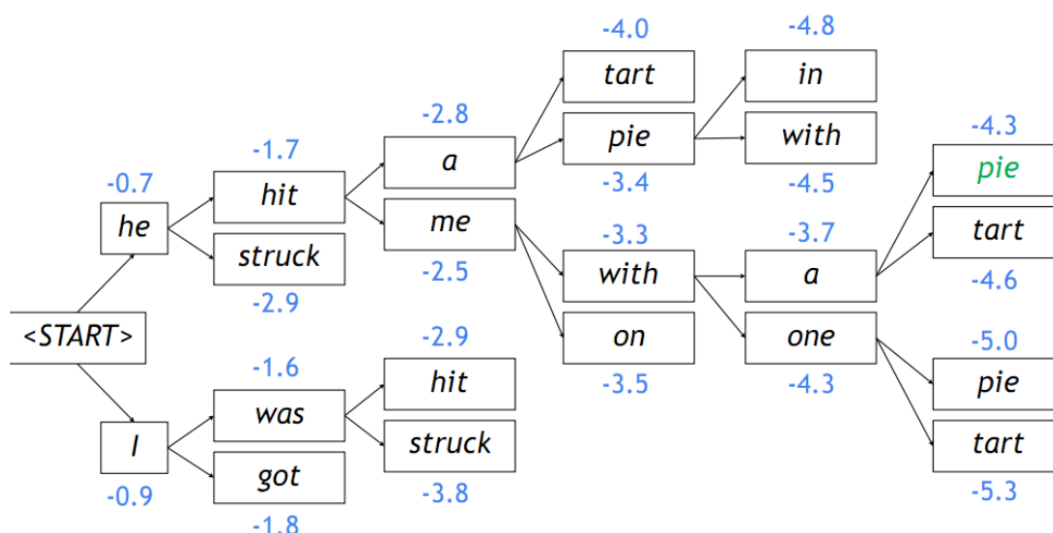


Hình 8.9: Kiến trúc mã hóa - giải mã.



© Tensorflow for deep learning research. Stanford

Hình 8.10: Kiến trúc sequence-to-sequence áp dụng trong bài toán dịch máy.



Hình 8.11: Một ví dụ của thuật toán tìm kiếm chùm với kích thước chùm $k = 2$.

- 40-50: Bản dịch chất lượng cao
- 50-60: Bản dịch chất lượng rất cao, đầy đủ, trôi chảy
- > 60 : Chất lượng tốt hơn con người

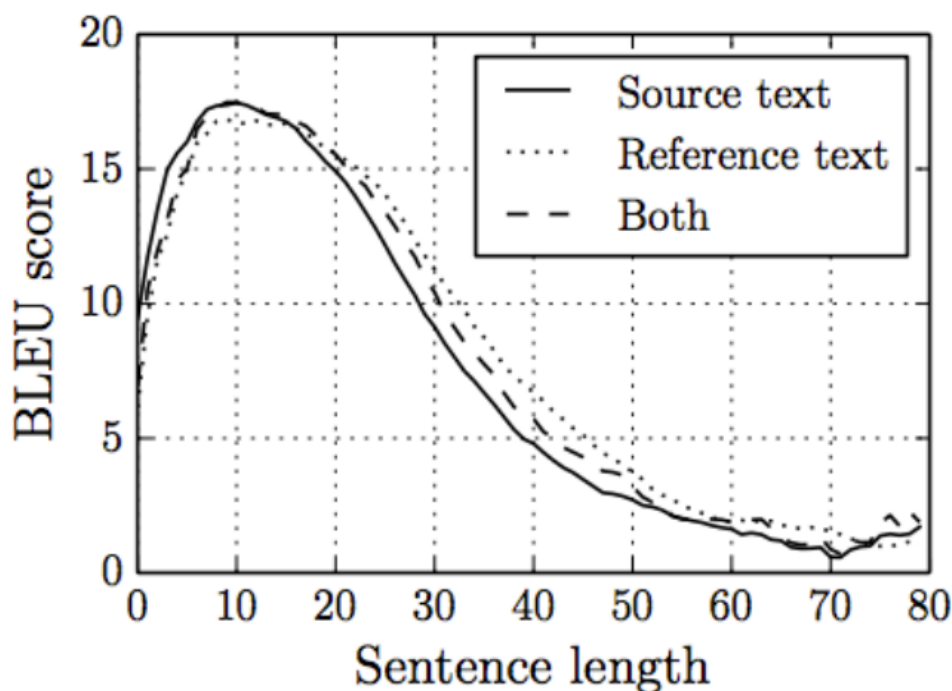
8.4.5 Giải mã

Ta mong muốn tìm được câu đích y với độ dài T nhằm cực đại hóa xác suất $P(y|x)$. Một cách vét cạn là ta có thể tính toán tất cả các khả năng và lựa chọn khả năng tốt nhất. Tuy nhiên, cách vét cạn này có độ phức tạp tính toán là $O(V^T)$ là không khả thi, trong đó V là kích thước từ vựng.

Ý tưởng của phương pháp tìm kiếm chùm (beam search) là tại mỗi bước tìm kiếm, ta duy trì k phương án bộ phận có xác suất cao nhất (gọi là các giả thuyết), trong đó k gọi là kích thước chùm (beam size). Mỗi giả thuyết y có điểm là log giá trị xác suất của nó. Mặc dù không đảm bảo tìm được lời giải tối ưu, phương pháp chùm có tính hiệu quả cao hơn nhiều so với phương pháp vét cạn.

Khi một giả thuyết sinh ra kí hiệu $\langle \text{END} \rangle$ là kí hiệu kết thúc câu, ta đặt giả thuyết đó vào trong tập các ứng viên. Thuật toán tìm kiếm chùm thường dừng lại khi hoặc là đạt đến T bước, hoặc là thu được n giả thuyết hoàn thành. Để không bị thiên vị vào các giả thuyết ngắn, ta có thể chuẩn hóa điểm của các giả thuyết theo độ dài.

So với SMT, NMT cho chất lượng dịch tốt hơn. Mô hình chỉ dùng một mạng duy



© On the Properties of Neural Machine Translation: Encoder-Decoder Approaches

Hình 8.12: Ảnh hưởng của độ dài đến hiệu năng dịch.

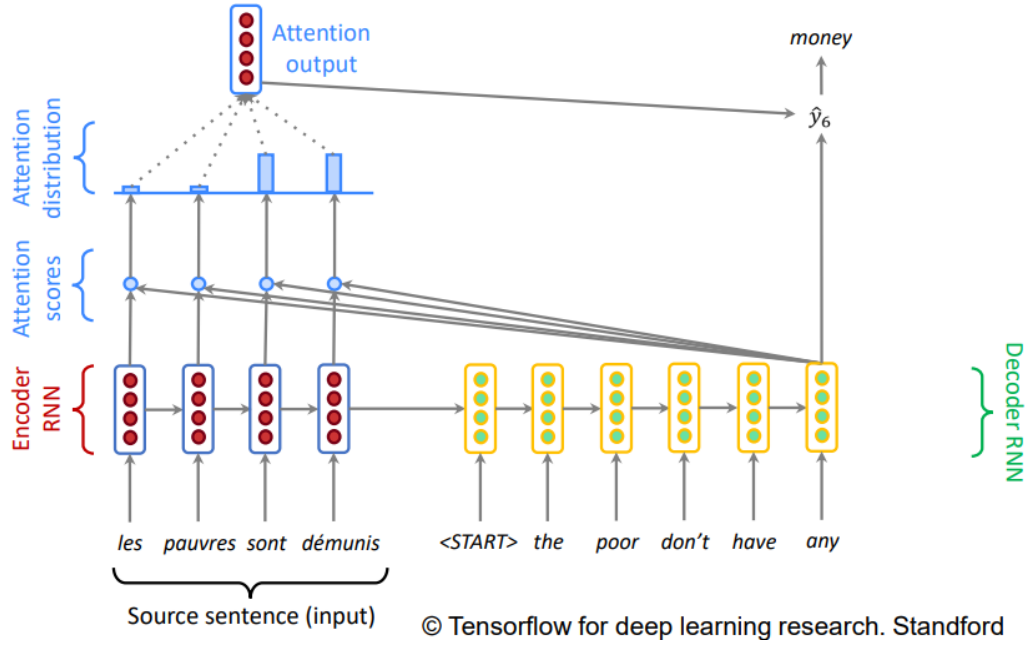
nhất để huấn luyện mà không cần nhiều module chồng kên. Mô hình cũng không cần thực hiện việc trích chọn đặc trưng một cách thủ công. Tuy nhiên, NMT có tính diễn giải thấp hơn và khó tùy chỉnh việc dịch theo một yêu cầu cụ thể.

8.4.6 Cơ chế chú ý

Mô hình seq2seq xảy ra hiện tượng thất cổ chai ở bộ mã hóa khi **toàn bộ** thông tin của câu đầu vào cần được mã hóa. Hiện tượng này dẫn đến khó khăn trong việc biểu diễn các phụ thuộc xa. Vì vậy, hiệu năng của các hệ thống dịch máy bị giảm đi đáng kể khi chiều dài của câu tăng lên.

Cơ chế chú ý được đề xuất để giải quyết vấn đề nút thắt cổ chai và tăng khả năng biểu diễn các phụ thuộc xa. Ý tưởng của cơ chế này là tại mỗi bước giải mã, chỉ tập trung sự chú ý vào các phần liên quan trong câu nguồn.

Giả sử ta có các trạng thái ẩn $h_1, h_2, \dots, h_N \in R^h$. Tại bước t , ta có trạng thái ẩn



Hình 8.13: Cơ chế chú ý trong mô hình seq2seq.

để giải mã $s_t \in R^h$. Điểm chú ý e^t cho bước này được tính toán như sau:

$$e^t = [s_t^T h_1, s_t^T h_2, \dots, s_t^T h_N] \in R^N \quad (8.10)$$

Để có phân phối xác suất, ta áp dụng softmax cho điểm chú ý này

$$\alpha^T = \text{softmax}(e^t) \in R^N \quad (8.11)$$

Khi đó, đầu ra của module chú ý là tổng chập của các trạng thái ẩn với trọng số trong α^t

$$a_t = \sum_{i=1}^N \alpha_i^t h_i \in R^h \quad (8.12)$$

Cuối cùng, ta ghép nối đầu ra của module chú ý với đầu ra của bộ giải mã tại thời điểm t và thực hiện tính toán như trong mô hình seq2seq cơ sở:

$$[a_t; s_t] \in R^{2h} \quad (8.13)$$

Như vậy, cơ chế chú ý đã giúp giải quyết vấn đề nút thắt cổ chai thông tin và cũng giúp làm giảm vấn đề triệu tiêu gradient. Bên cạnh đó, cơ chế này làm tăng tính diễn giải của mô hình dịch máy sâu khi nó được coi như một cơ chế giống hàng mềm dựa trên trọng số chú ý.

Chương 9

Các mô hình sinh

9.1	Tổng quan về Mô hình Sinh	142
9.2	Autoencoder: Nghệ thuật của sự nén và tái tạo	143
9.3	Variational Autoencoder (VAE): Kiến tạo không gian ẩn có ý nghĩa	145
9.4	Generative Adversarial Networks (GANs): Cuộc đối đấu sáng tạo	146

9.1 Tổng quan về Mô hình Sinh

9.1.1 Mô hình Sinh và Mô hình Phân biệt: Hai triết lý tiếp cận

Lĩnh vực học máy có hai trường phái chính để xây dựng các mô hình: **phân biệt (discriminative)** và **sinh (generative)**. Sự khác biệt giữa chúng không chỉ nằm ở kỹ thuật, mà còn ở triết lý tiếp cận vấn đề.

Một **mô hình phân biệt** học cách tìm ra **ranh giới quyết định (decision boundary)** giữa các lớp dữ liệu. Hãy tưởng tượng một người bảo vệ câu lạc bộ đêm. Anh ta không cần biết chi tiết về từng vị khách; anh ta chỉ cần biết một quy tắc đơn giản: "trên 18 tuổi thì được vào, dưới 18 tuổi thì không". Mô hình phân biệt cũng hoạt động như vậy. Với bài toán phân loại chó và mèo, nó chỉ cần học các đặc điểm đủ để phân biệt, ví dụ như "mèo có tai nhọn và mắt to, chó có mõm dài hơn". Về mặt toán học, nó tập trung vào việc mô hình hóa xác suất có điều kiện $P(y|x)$ – xác suất của nhãn y khi biết đầu vào x .

Ngược lại, một **mô hình sinh** theo đuổi một mục tiêu tham vọng hơn nhiều: nó

muốn **hiểu và tái tạo lại quá trình tạo ra dữ liệu**. Thay vì chỉ là người bảo vệ, nó muốn trở thành một họa sĩ chân dung bậc thầy. Để vẽ được một bức chân dung con mèo, người họa sĩ phải hiểu sâu sắc về giải phẫu của mèo: cấu trúc xương, hình dáng cơ bắp, kết cấu của bộ lông, cách ánh sáng phản chiếu trên mắt... Tương tự, một mô hình sinh học toàn bộ phân phối xác suất của dữ liệu, $P(x)$. Một khi đã hiểu được "bản chất" của dữ liệu, nó không chỉ có thể phân biệt (bằng cách tính $P(x|y)$ và sử dụng định lý Bayes), mà còn có thể **tạo ra (sinh)** những mẫu dữ liệu hoàn toàn mới.

9.1.2 Ứng dụng thực tiễn: Vượt ra ngoài ranh giới sáng tạo

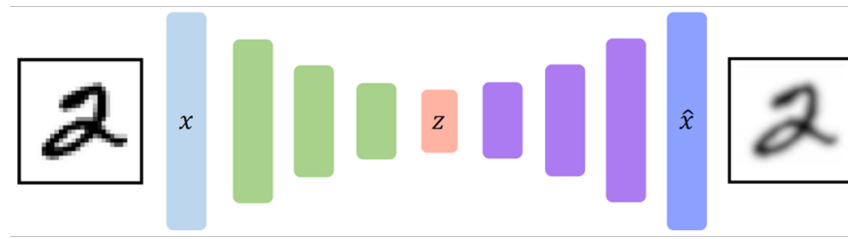
Sức mạnh của mô hình sinh đã và đang tạo ra những cuộc cách mạng trong nhiều lĩnh vực:

- **Sáng tạo nội dung đa phương tiện:** Các mô hình như DALL-E 3, Mid-journey (tạo ảnh từ văn bản), và Suno AI (tạo nhạc từ văn bản) đang dân chủ hóa khả năng sáng tạo. Giờ đây, bất kỳ ai cũng có thể tạo ra một bức tranh theo phong cách Van Gogh hay một bản nhạc jazz chỉ bằng vài dòng lệnh.
- **Khoa học và Kỹ thuật:** Trong ngành dược, các mô hình sinh có thể duyệt qua hàng triệu cấu trúc phân tử tiềm năng để tìm ra các ứng cử viên thuốc mới, rút ngắn thời gian nghiên cứu từ nhiều năm xuống còn vài tháng. Trong thiết kế vi mạch, chúng có thể đề xuất các cách bố trí linh kiện tối ưu hơn con người.
- **Mô phỏng và Tăng cường dữ liệu:** Việc thu thập dữ liệu trong các kịch bản hiếm hoặc nguy hiểm (ví dụ: tai nạn xe tự lái, sự cố lò phản ứng hạt nhân) là cực kỳ tốn kém và rủi ro. Mô hình sinh có thể tạo ra một thế giới mô phỏng (digital twin) siêu thực để huấn luyện các hệ thống AI trong một môi trường an toàn nhưng đầy đủ thử thách, giúp chúng đối phó tốt hơn với các "trường hợp rìa" (edge cases).

9.2 Autoencoder: Nghệ thuật của sự nén và tái tạo

9.2.1 Kiến trúc Encoder-Decoder và "Nút cổ chai" thông tin

Bộ tự mã hoá (Autoencoder - AE) [3] là một kiến trúc mạng nơ-ron tính tế, được thiết kế cho nhiệm vụ học không giám sát nhằm tìm ra các biểu diễn dữ liệu (data representations) hiệu quả. Cấu trúc của nó bao gồm hai thành phần đối xứng, được minh hoạ trong Hình 9.1:



Hình 9.1: Kiến trúc cơ bản của một AE. Nguồn: Jeremy Jordan's blog.

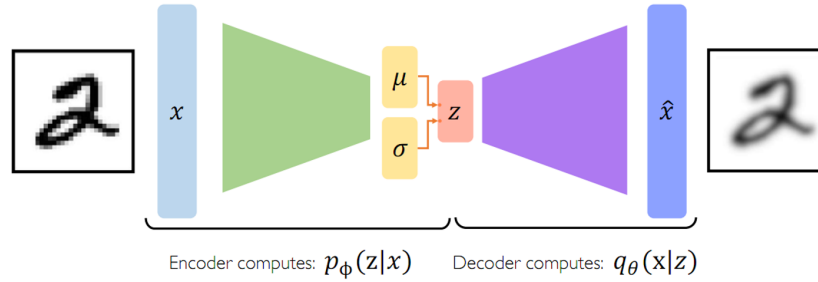
- **Bộ mã hóa (Encoder):** Nhận dữ liệu đầu vào x (ví dụ: một ảnh 784 pixel) và nén nó xuống một biểu diễn ẩn z có số chiều thấp hơn nhiều (ví dụ: một vector 32 chiều).
- **Bộ giải mã (Decoder):** Nhận biểu diễn ẩn z và cố gắng tái tạo lại dữ liệu đầu vào ban đầu $\hat{x}$ sao cho giống x nhất có thể.

Lớp ẩn z được gọi là "**nút cổ chai**" (**bottleneck**) vì nó buộc dòng thông tin phải đi qua một khe hẹp. Quá trình này ép mạng phải học cách chất lọc những thông tin tinh túy, cốt lõi nhất của dữ liệu và loại bỏ những chi tiết thừa, nhiều. Để có thể tái tạo lại ảnh chữ số '7' từ một vector nhỏ bé, mô hình phải học rằng các đặc trưng quan trọng là "một nét ngang ở trên và một nét chéo xuống dưới", thay vì ghi nhớ giá trị của từng điểm ảnh.

Mạng được huấn luyện bằng cách tối thiểu hóa **sai số tái tạo (reconstruction loss)**, thường là lỗi bình phương (MSE), $L(x, \hat{x}) = \|x - \hat{x}\|^2$, giữa ảnh gốc x và ảnh tái tạo $\hat{x}$.

9.2.2 Hạn chế của AE trong vai trò mô hình sinh

Mặc dù rất hữu ích cho việc giảm chiều dữ liệu và khử nhiễu, AE truyền thống lại thất bại trong việc sinh dữ liệu mới. Không gian ẩn của nó không có tính **liên tục (continuity)** hay **hoàn chỉnh (completeness)**. Các điểm dữ liệu sau khi mã hóa có thể tạo thành các cụm riêng biệt, và những "khoảng không" giữa các cụm này là vô nghĩa. Nếu ta chọn một điểm z ngẫu nhiên trong khoảng không đó và đưa vào bộ giải mã, kết quả $\hat{x}$ thu được thường là một hình ảnh mờ nhòe, không thể nhận dạng. AE chỉ biết cách giải mã những điểm gần với các điểm mà nó đã thấy trong quá trình huấn luyện.



Hình 9.2: Không gian ẩn của AE (trái) rời rạc, trong khi của VAE (phải) liên tục và có cấu trúc. Nguồn: aicurious.io.

9.3 Variational Autoencoder (VAE): Kiến tạo không gian ẩn có ý nghĩa

9.3.1 Giải pháp của VAE: Từ điểm đến phân phối

Variational Autoencoder (VAE) [2] ra đời để giải quyết triệt để hạn chế của AE. Thay vì mã hóa một đầu vào x thành một điểm duy nhất z , VAE mã hóa nó thành một **phân phối xác suất** trên không gian ẩn. Cụ thể, Encoder của VAE tạo ra hai vector: một vector trung bình μ và một vector log-phương sai $\log(\sigma^2)$. Hai vector này định nghĩa một phân phối chuẩn $N(\mu, \sigma^2)$, và vector ẩn z sẽ được **lấy mẫu (sampled)** từ phân phối này. Hình 9.2 cho một ví dụ minh họa.

Cách tiếp cận này buộc không gian ẩn phải có cấu trúc. Nó đảm bảo rằng không chỉ các điểm mã hóa có ý nghĩa, mà cả những điểm lân cận chúng cũng vậy. Điều này tạo ra một không gian ẩn "trơn" (smooth), nơi chúng ta có thể di chuyển từ từ và thấy sự biến đổi hình ảnh một cách hợp lý ở đầu ra.

9.3.2 Hàm mất mát của VAE

Sự thành công của VAE nằm ở hàm mất mát được thiết kế khéo léo, bao gồm hai thành phần:

$$\mathcal{L}_{VAE} = \underbrace{\mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x|z)]}_{\text{Mất mát tái tạo}} - \underbrace{D_{KL}(q_\phi(z|x) || p(z))}_{\text{Mất mát KL-Divergence}}$$

1. **Mất mát tái tạo:** Thành phần này có vai trò tương tự như trong AE, đảm bảo rằng dữ liệu được giải mã phải giống với dữ liệu gốc. Nó khuyến khích mô hình mã hóa càng nhiều thông tin về x vào z càng tốt.

2. **Mất mát KL-Divergence:** Đây là thành phần hiệu chỉnh (regularization term) và là trái tim của VAE. Nó đo lường "khoảng cách" giữa phân phối mà Encoder tạo ra, $q_\phi(z|x)$, và một phân phối tiên nghiệm (prior) cố định, thường là phân phối chuẩn đơn vị $N(0, 1)$. Bằng cách tối thiểu hóa khoảng cách này, nó "ép" tất cả các phân phối được mã hóa từ mọi điểm dữ liệu phải tụ tập lại gần gốc tọa độ và có hình dạng giống một quả cầu.

Quá trình huấn luyện VAE là một cuộc giằng co: Mất mát tái tạo cố gắng *đẩy* các phân phối của các dữ liệu khác nhau ra xa nhau để dễ phân biệt, trong khi mất mát KL lại *kéo* chúng lại gần nhau. Kết quả của sự cân bằng này là một không gian ẩn được tổ chức một cách đẹp đẽ, nơi các khái niệm tương tự nằm gần nhau.

9.3.3 Thủ thuật Đổi tham số (Reparameterization Trick)

Một thách thức kỹ thuật nảy sinh: bước lấy mẫu z từ $N(\mu, \sigma^2)$ là một thao tác ngẫu nhiên, do đó không khả vi và chặn đường lan truyền ngược của gradient. Giải pháp là **Thủ thuật Đổi tham số**:

$$z = \mu + \sigma \odot \epsilon, \quad \text{với } \epsilon \sim N(0, 1)$$

Sự ngẫu nhiên giờ đây được đưa vào từ bên ngoài thông qua ϵ . Gradient có thể tự do đi qua các phép toán cộng và nhân xác định để cập nhật μ và σ , từ đó huấn luyện được Encoder. Đây là một phát kiến then chốt giúp việc huấn luyện VAE trở nên khả thi.

9.4 Generative Adversarial Networks (GANs): Cuộc đối đầu sáng tạo

9.4.1 Lý thuyết trò chơi trong Học sâu

GAN [1] giới thiệu một kiến trúc mạng tính cách mạng, dựa trên lý thuyết trò chơi hai người có tổng bằng không (zero-sum game). Hai mạng nơ-ron được huấn luyện để đối đầu nhau:

- **Generator (G):** Một "nghệ sĩ làm hàng giả". Nó nhận một vectơ nhiễu ngẫu nhiên (latent code) z làm "nguồn cảm hứng" và cố gắng tạo ra dữ liệu giả $G(z)$ (ví dụ: ảnh giả) sao cho không thể phân biệt được với dữ liệu thật.
- **Discriminator (D):** Một "chuyên gia thẩm định". Nó được huấn luyện để phân biệt chính xác giữa dữ liệu thật x (từ tập huấn luyện) và dữ liệu giả.

$G(z)$. Đầu ra của D là một xác suất, $D(x)$, cho biết mẫu đầu vào là thật (gần 1) hay giả (gần 0).

Hàm mục tiêu của trò chơi này là:

$$V(D, G) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

Đối với D: D muốn tối đa hóa hàm này. Nó muốn $D(x)$ tiến tới 1 (nhận dạng đúng hàng thật) và $D(G(z))$ tiến tới 0 (nhận dạng đúng hàng giả). **Đối với G:** G muốn tối thiểu hóa hàm này. Nó không thể tác động vào hạng tử đầu tiên, nên nó cố gắng làm cho $D(G(z))$ tiến tới 1, tức là đánh lừa D tin rằng hàng giả là hàng thật. Như vậy quá trình huấn luyện cần giải bài toán minimax sau:

$$\min_G \max_D V(D, G)$$

Quá trình huấn luyện giống như một "cuộc chạy đua vũ trang": D trở nên tinh vi hơn trong việc phát hiện hàng giả, và G cũng phải liên tục cải tiến kỹ năng của mình để không bị phát hiện. Khi đạt đến điểm cân bằng Nash, G có thể tạo ra những mẫu hoàn hảo và D chỉ có thể đoán với xác suất 50%.

9.4.2 Những thách thức kinh điển trong huấn luyện GAN

- **Sụp đổ đỉnh (Mode Collapse):** Đây là vấn đề phổ biến nhất. G tìm ra một "điểm yếu" của D và chỉ tạo ra một hoặc một vài loại mẫu mà nó biết chắc sẽ lừa được D, thay vì học toàn bộ sự đa dạng của phân phối dữ liệu.
- **Gradient biến mất (Vanishing Gradients):** Nếu D quá giỏi, nó có thể dễ dàng phân biệt thật/giả. Gradient trả về cho G sẽ rất nhỏ, khiến G không học được gì thêm.
- **Huấn luyện không ổn định (Instability):** Sự cân bằng giữa G và D rất mong manh. Chúng có thể rơi vào vòng lặp phá hoại lẫn nhau thay vì cùng tiến bộ.

9.4.3 Các kiến trúc GAN nâng cao và ứng dụng

Conditional GAN (cGAN): Kiểm soát đầu ra

GAN gốc không cho phép ta kiểm soát nội dung được sinh ra. **Conditional GAN (cGAN)** giải quyết vấn đề này bằng cách đưa thêm thông tin điều kiện y (ví dụ: nhãn lớp) vào cả hai mạng.

- G nhận đầu vào là (z, y) và phải sinh ra ảnh thuộc lớp y .
- D nhận đầu vào là cặp (ảnh, nhãn) và phải trả lời câu hỏi: "Đây có phải là một ảnh thật của lớp này không?".

cGAN là nền tảng cho rất nhiều ứng dụng image-to-image translation, nơi đầu ra được kiểm soát bởi một đầu vào có cấu trúc.

Progressive GAN (ProGAN): Học từ thô đến tinh

Để tạo ảnh độ phân giải siêu cao (1024x1024), ProGAN đề xuất một phương pháp huấn luyện lũy tiến. Thay vì học tạo ảnh 1024x1024 ngay lập tức, mô hình bắt đầu với một nhiệm vụ dễ hơn nhiều: tạo ảnh 4x4. Sau khi ổn định, các lớp mới được thêm vào cả G và D để tăng gấp đôi độ phân giải lên 8x8, 16x16, và cứ thế tiếp tục. Cách tiếp cận "từ thô đến tinh" này giúp ổn định quá trình huấn luyện một cách đáng kể.

StyleGAN: Bậc thầy về phong cách

StyleGAN của NVIDIA đã tạo ra một cuộc cách mạng về chất lượng và khả năng kiểm soát. Nó giới thiệu một số cải tiến quan trọng:

- **Mạng ánh xạ (Mapping Network):** Thay vì đưa vector nhiễu z trực tiếp vào G, nó được đưa qua một mạng MLP để tạo ra một vector trung gian w . Điều này giúp "gỡ rối" không gian ẩn, làm cho nó ít bị vướng víu (less entangled) hơn.
- **Tiêm nhiễm phong cách (Style Injection):** Vector w được biến đổi và "tiêm" vào mỗi lớp của mạng sinh. Việc tiêm vào các lớp khác nhau sẽ kiểm soát các mức độ chi tiết khác nhau của hình ảnh:
 - **Lớp sớm (4x4, 8x8):** Kiểm soát đặc điểm thô (tư thế, hình dáng đầu).
 - **Lớp giữa (16x16, 32x32):** Kiểm soát đặc điểm chi tiết hơn (mắt, mũi, tóc).
 - **Lớp cuối (64x64 trở lên):** Kiểm soát chi tiết tinh vi (màu da, ánh sáng).
- **Đầu vào nhiễu ngẫu nhiên (Stochastic Noise):** Nhiễu được thêm vào mỗi lớp để kiểm soát các chi tiết ngẫu nhiên như tàn nhang, nếp nhăn, hay từng sợi tóc, làm cho ảnh trông thật hơn.

Hình 9.3: Ví dụ về trộn lẫn phong cách trong StyleGAN. Các hàng trên cùng (Source A) cung cấp phong cách cấp cao (tư thế, tóc), trong khi các cột bên trái (Source B) cung cấp phong cách cấp thấp (màu da, giới tính). Nguồn: Karras et al. (2019)

CycleGAN: Dịch ảnh không cần cặp dữ liệu

Làm thế nào để biến ngựa thành ngựa vằn, hay ảnh mùa hè thành mùa đông? Việc thu thập các cặp ảnh hoàn hảo (cùng một cảnh ở hai mùa khác nhau) là bất khả thi. **CycleGAN** giải quyết bài toán này bằng cách sử dụng **mất mát nhất quán theo chu kỳ (cycle consistency loss)**.

Nó sử dụng hai Generator ($G_{A \rightarrow B}$ và $G_{B \rightarrow A}$). Ý tưởng là: nếu bạn dịch một ảnh từ domain A sang B, rồi dịch ngược lại về A, bạn phải thu được ảnh gần giống ảnh gốc.

$$x \rightarrow G_{A \rightarrow B}(x) \rightarrow G_{B \rightarrow A}(G_{A \rightarrow B}(x)) \approx x$$

Điều này buộc các Generator phải giữ lại nội dung gốc của ảnh và chỉ thay đổi "phong cách" đặc trưng của domain đích, mở ra khả năng dịch ảnh cho các tập dữ liệu không cặp.

Chương 10

Các xu hướng mới trong học sâu

10.1 Transformer	150
10.2 Graph neural network	161
10.3 Nhúng đỉnh (Node Embedding)	166
10.4 Mạng đồ thị tự mã hoá (GAE)	169
10.5 GGNN	170
10.6 RGNN	170
10.7 Mạng nơ ron đồ thị tích chập (GCNN)	173
10.8 Thực nghiệm	176
Danh mục từ khóa	187

10.1 Transformer

10.1.1 Lời mở đầu

Trong những năm đầu của thời kỳ bùng nổ học sâu, các kiến trúc như mạng perceptron nhiều lớp (MLP), mạng nơ-ron tích chập (CNN), và mạng nơ-ron hồi quy (RNN) đã tạo ra các kết quả nổi bật. Mặc dù có nhiều cải tiến như các hàm kích hoạt ReLU, lớp dư (residual layers), chuẩn hóa hàng loạt (batch normalization), dropout, và các lịch trình học tự thích ứng, các kiến trúc cốt lõi vẫn là những phiên bản mở rộng của các ý tưởng cổ điển. Những thay đổi đáng kể chủ yếu đến từ sự gia tăng tài nguyên tính toán và lượng dữ liệu lớn hơn. Trong khi các mô hình CNN vẫn thống trị trong thị giác máy tính và RNN với LSTM là tiêu chuẩn trong xử lý ngôn ngữ tự nhiên (NLP), một sự thay đổi lớn đã xuất hiện với sự ra đời của Transformer.

Ngày nay, Transformer là kiến trúc chính cho hầu hết các nhiệm vụ NLP. Đối với bất kỳ nhiệm vụ mới nào, phương pháp tiếp cận mặc định là sử dụng một mô hình Transformer đã được huấn luyện trước (chẳng hạn như BERT, ELECTRA, RoBERTa, hoặc Longformer), điều chỉnh các lớp đầu ra cần thiết và tinh chỉnh mô hình trên dữ liệu cụ thể. Các mô hình ngôn ngữ lớn như GPT-2 và GPT-3 của OpenAI cũng dựa trên Transformer. Không chỉ trong NLP, Transformer còn trở thành tiêu chuẩn trong nhiều nhiệm vụ thị giác máy tính như nhận dạng hình ảnh, phát hiện đối tượng, phân đoạn ngữ nghĩa và siêu phân giải.

Ý tưởng chính của Transformer là cơ chế attention, ban đầu được phát triển như một sự cải tiến cho các mạng nơ-ron hồi quy encoder-decoder trong các ứng dụng sequence-to-sequence như dịch máy. Trong các mô hình sequence-to-sequence truyền thống, toàn bộ đầu vào được nén bởi encoder thành một vector cố định để đưa vào decoder. Cơ chế attention cho phép decoder xem xét lại chuỗi đầu vào tại mỗi bước giải mã, thay vì chỉ dựa vào một biểu diễn cố định. Năm 2017, Vaswani và các cộng sự đã đề xuất kiến trúc Transformer, loại bỏ hoàn toàn các kết nối hồi quy và thay vào đó sử dụng cơ chế attention đặc biệt để nắm bắt mọi mối quan hệ giữa các token đầu vào và đầu ra.

10.1.2 Tại sao cần cơ chế chú ý

RNN: Học ngữ cảnh tương tác tuyến tính giữa các từ trong chuỗi

Recurrent Neural Networks (RNNs) là một kiến trúc phổ biến để xử lý chuỗi dữ liệu như văn bản hoặc chuỗi thời gian. RNN xử lý dữ liệu tuần tự từ trái sang phải, mỗi bước nhận một từ và cập nhật trạng thái ẩn dựa trên trạng thái trước đó và từ hiện tại.

Hệ quả là, RNN gặp thách thức trong việc học ngữ cảnh tương tác giữa các từ trong chuỗi. Để hiểu đúng ngữ nghĩa của một từ, mô hình cần xem xét ngữ cảnh rộng hơn, bao gồm cả các từ trước và sau nó. Tuy nhiên, RNN gặp khó khăn khi khoảng cách giữa các từ tăng lên. Thông tin bị mất mát qua mỗi bước, làm giảm khả năng duy trì và truyền tải thông tin dài hạn. RNN cần $O(\text{độ dài dãy})$ bước để các từ cách xa nhau tương tác, làm giảm hiệu quả trong việc học các quan hệ dài hạn.

Hơn nữa, thứ tự tuyến tính của các từ không luôn phản ánh đúng cấu trúc ngữ nghĩa của câu. Các từ có thể cần tương tác trực tiếp bất kể khoảng cách của chúng. Ví dụ, một từ ở đầu câu có thể liên quan chặt chẽ với một từ ở cuối câu, nhưng RNN khó nắm bắt mối quan hệ này do tính tuần tự của nó. Điều này hạn chế hiệu quả của RNN trong việc hiểu ngữ cảnh toàn diện.

RNN: Thiếu Tính Song Song

RNNs gặp phải một vấn đề lớn là thiếu tính song song trong quá trình tính toán. Trong RNNs, cả quá trình tính toán xuôi (forward pass) và ngược (backward pass) đều cần $O(\text{độ dài dãy})$ phép toán, tức là số phép toán tỷ lệ thuận với độ dài của chuỗi đầu vào. Điều này xảy ra vì trạng thái ẩn tại mỗi bước trong chuỗi được tính dựa trên trạng thái ẩn của bước trước đó. Cụ thể:

Tính toán xuôi Tại mỗi bước, trạng thái ẩn h_t được cập nhật dựa trên trạng thái ẩn trước đó h_{t-1} và đầu vào hiện tại x_t .

Tính toán ngược Để cập nhật các trọng số của mô hình thông qua thuật toán lan truyền ngược (backpropagation), gradient tại mỗi bước cũng phải được tính toán theo trình tự ngược từ cuối chuỗi về đầu chuỗi.

Trong RNNs, các trạng thái ẩn tương lai không thể được tính toán trước khi các trạng thái ẩn quá khứ đã được xác định. Điều này có nghĩa là tại mỗi bước trong chuỗi, ta phải chờ đợi kết quả từ bước trước đó trước khi tiến hành tính toán tiếp. Kết quả là, GPUs không thể phát huy hết tiềm năng của mình trong việc tăng tốc các phép tính, vì các phép toán trong RNNs phụ thuộc lẫn nhau một cách tuần tự. Việc thiếu tính song song trong RNNs dẫn đến một nhược điểm nghiêm trọng: khó khăn trong việc huấn luyện trên các tập dữ liệu rất lớn.

10.1.3 Cơ chế tự chú ý

Cơ chế tự chú ý (self-attention) là một trong những đóng góp quan trọng nhất trong lĩnh vực học sâu trong những năm gần đây. Được giới thiệu lần đầu trong bài báo “Attention Is All You Need” của Vaswani et al. vào năm 2017, cơ chế này đã mang lại một bước đột phá trong việc xử lý chuỗi, đặc biệt là trong các tác vụ xử lý ngôn ngữ tự nhiên.

Cơ chế tự chú ý cho phép mô hình tập trung vào các phần khác nhau của chuỗi đầu vào khi tạo ra mỗi phần tử của chuỗi đầu ra. Điều này giúp mô hình nắm bắt được các phụ thuộc xa trong chuỗi một cách hiệu quả, vượt qua hạn chế của các kiến trúc truyền thống như RNN hoặc LSTM.

Trong kiến trúc Transformer, cơ chế tự chú ý được triển khai thông qua ba ma trận: Query (Q), Key (K), và Value (V). Mỗi ma trận này được tạo ra bằng cách nhân vector đầu vào với một ma trận trọng số được học. Cụ thể, nếu ta có một vector đầu vào x , các ma trận Q, K, V được tính như sau:

$$Q = xW^Q, \quad K = xW^K, \quad V = xW^V \quad (10.1)$$

trong đó W^Q, W^K, W^V là các ma trận trọng số được học trong quá trình huấn luyện.

Quá trình tính toán tự chú ý bao gồm ba bước chính. Bước đầu tiên là tính toán điểm tương đồng (similarity scores) giữa mỗi cặp vị trí trong chuỗi. Điểm này được tính bằng tích vô hướng giữa vector query tại một vị trí và vector key tại vị trí khác:

$$\text{score}(q_i, k_j) = q_i \cdot k_j^T \quad (10.2)$$

Sau đó, các điểm này được chuẩn hóa bằng cách chia cho căn bậc hai của kích thước của vector key (d_k) để tránh gradient quá lớn khi d_k lớn:

$$\text{scaled_score}(q_i, k_j) = \frac{q_i \cdot k_j^T}{\sqrt{d_k}} \quad (10.3)$$

Bước thứ hai là áp dụng hàm softmax để chuyển đổi các điểm này thành các trọng số chú ý. Hàm softmax biến đổi các điểm thành một phân phối xác suất, đảm bảo rằng tổng các trọng số bằng 1:

$$\text{attention_weight}_{ij} = \frac{\exp(\text{scaled_score}(q_i, k_j))}{\sum_k \exp(\text{scaled_score}(q_i, k_k))} \quad (10.4)$$

Bước cuối cùng là tính toán vector ngữ cảnh bằng cách lấy tổng có trọng số của các vector giá trị, sử dụng trọng số chú ý vừa tính được:

$$\text{context_vector}_i = \sum_j \text{attention_weight}_{ij} v_j \quad (10.5)$$

Vector ngữ cảnh này chứa thông tin từ toàn bộ chuỗi đầu vào, với trọng số phản ánh mức độ quan trọng của mỗi vị trí đối với vị trí hiện tại.

Cơ chế chú ý mang lại nhiều lợi ích như:

Tính toán song song Việc tính toán trọng số chú ý có thể được tính toán song song và đồng thời cho tất cả các từ trong chuỗi nhờ vào tính chất song song của phép nhân ma trận.

Khoảng Cách Tương Tác Xa Nhất: $O(1)$: Trong mạng transformer, khoảng cách tương tác giữa các từ không phụ thuộc vào độ dài chuỗi. Mọi từ trong chuỗi có thể tương tác trực tiếp với bất kỳ từ nào khác thông qua cơ chế chú ý, và điều này xảy ra đồng thời tại mọi lớp của mạng. Vì vậy, khoảng cách tương tác xa nhất chỉ là $O(1)$, nghĩa là bất kỳ từ nào cũng có thể liên kết trực tiếp với bất kỳ từ nào khác trong một bước duy nhất.

10.1.4 Cơ chế mã hóa vị trí thông qua hàm Sin

Một trong những thách thức khi sử dụng cơ chế tự chú ý (self-attention) trong mạng transformer là làm thế nào để mô hình có thể nhận biết được thứ tự của các từ trong chuỗi đầu vào. Khác với RNNs, vốn xử lý các từ theo thứ tự tuần tự và do đó, tự nhiên giữ lại thông tin về thứ tự, cơ chế tự chú ý không tự động xây dựng thông tin về thứ tự của các từ. Do đó, cần phải mã hóa thông tin này vào trong các vector keys, queries, và values.

Trong mạng transformer, một phương pháp phổ biến để mã hóa thông tin vị trí của các từ trong chuỗi đầu vào là sử dụng hàm sin và hàm cos. Đây là phương pháp được giới thiệu trong bài báo gốc về transformer của Vaswani và các cộng sự (2017). Cách tiếp cận này có một số ưu điểm và nhược điểm cần được xem xét kỹ lưỡng.

Cách Thức Hoạt Động

Thông tin vị trí được biểu diễn bằng cách sử dụng các hàm sin và cos với các tần số khác nhau cho mỗi chiều trong vector vị trí. Cụ thể, với mỗi vị trí i và mỗi chiều d trong vector vị trí, ta có các công thức sau:

$$\text{PE}_{(i,2k)} = \sin\left(\frac{i}{10000^{2k/d}}\right)$$

$$\text{PE}_{(i,2k+1)} = \cos\left(\frac{i}{10000^{2k/d}}\right)$$

Ở đây, i là vị trí của từ trong chuỗi, k là chỉ số của chiều trong vector vị trí, và d là số chiều của vector vị trí. Công thức này tạo ra một vector vị trí có cùng kích thước với vector embedding của từ. Mỗi chiều của vector vị trí được tính toán bằng cách sử dụng hàm sin cho các chỉ số chẵn và hàm cos cho các chỉ số lẻ.

Trong mô hình Transformer, các vector vị trí được tạo ra bởi phương pháp này được cộng trực tiếp vào các vector embedding của từ trước khi đưa vào các lớp self-attention:

$$\text{Input} = \text{WordEmbedding} + \text{PositionalEncoding}$$

Điều này cho phép thông tin về vị trí được truyền qua tất cả các lớp của mô hình mà không cần thay đổi cấu trúc của cơ chế attention.

Ưu Điểm

Tính Chu Kỳ Các hàm sin và cos có tính chu kỳ, điều này có nghĩa là các giá trị của chúng sẽ lặp lại sau một khoảng thời gian nhất định. Điều này có thể cho phép mô hình không phải quá phụ thuộc vào vị trí tuyệt đối của các từ, mà thay vào đó chú ý nhiều hơn đến các mẫu hình lặp lại và các mối quan hệ tương đối.

Khả Năng Ngoại Suy Do tính chất chu kỳ, các vector vị trí sinh ra từ hàm sin và cos có khả năng ngoại suy cho các chuỗi dài hơn. Điều này có nghĩa là nếu mô hình đã được huấn luyện trên các chuỗi ngắn, nó vẫn có thể áp dụng cho các chuỗi dài hơn mà không cần thay đổi cấu trúc mã hóa vị trí.

Tính bất biến tương đối Khoảng cách giữa hai vị trí bất kỳ có thể được biểu diễn như một hàm tuyến tính của các vector vị trí tương ứng. Điều này giúp mô hình dễ dàng học được các mối quan hệ tương đối giữa các vị trí.

Nhược Điểm

Không Học Được Các vector vị trí sinh ra từ hàm sin và cos là các giá trị cố định và không thể học được. Điều này có nghĩa là mô hình không thể điều chỉnh các vector vị trí này dựa trên dữ liệu đầu vào để cải thiện hiệu suất.

Ngoại Suy Không Thật Sự Hiệu Quả Mặc dù các hàm sin và cos có thể giúp mô hình ngoại suy cho các chuỗi dài hơn, nhưng khả năng này không thật sự hiệu quả trong mọi trường hợp. Khi chiều dài chuỗi đầu vào vượt quá giới hạn mà mô hình đã được huấn luyện, sự chính xác của mã hóa vị trí có thể giảm đi, dẫn đến hiệu suất của mô hình cũng bị ảnh hưởng.

10.1.5 Chú ý đa đầu (Multi-Headed Attention)

Với Chú ý đa đầu, mô hình có thể học và kết hợp nhiều loại mối quan hệ khác nhau giữa các từ trong chuỗi. Điều này giúp mô hình có cái nhìn toàn diện hơn về ngữ cảnh của chuỗi đầu vào. Chú ý đa đầu sử dụng nhiều “đầu” chú ý song song. Mỗi “đầu” này thực hiện cơ chế chú ý độc lập, cho phép mạng học và kết hợp nhiều loại mối quan hệ giữa các từ trong chuỗi đầu vào. Cơ chế multi-headed attention hoạt động theo các bước sau:

1. **Chia Không Gian Đặc Trưng:** Đầu tiên, các vector đầu vào (input embeddings) được chia thành nhiều phần nhỏ hơn. Giả sử chúng ta có h đầu chú ý

và mỗi vector đầu vào có kích thước d , mỗi phần nhỏ sẽ có kích thước d/h .

2. **Tính Toán Chú Ý Độc Lập:** Đối với mỗi đầu chú ý i (với $i = 1, 2, \dots, h$), chúng ta tính toán các vector query, key, và value từ các phần nhỏ này bằng cách sử dụng các ma trận trọng số có thể học được W_i^Q , W_i^K , và W_i^V :

$$\begin{aligned} Q_i &= XW_i^Q \\ K_i &= XW_i^K \\ V_i &= XW_i^V \end{aligned}$$

Trong đó X là ma trận đầu vào.

3. **Áp Dụng Self-Attention:** Sau đó, chúng ta áp dụng phép tính self-attention riêng biệt cho mỗi đầu:

$$\text{Attention}_i(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i$$

Trong đó d_k là kích thước của vector key, được sử dụng để chuẩn hóa tích vô hướng giữa query và key.

4. **Ghép Nối Kết Quả:** Kết quả từ tất cả các đầu chú ý được ghép nối (concatenate) lại để tạo ra một vector duy nhất có cùng kích thước với vector đầu vào ban đầu.
5. **Ánh Xạ Tuyến Tính:** Vector ghép nối này sau đó được đưa qua một phép biến đổi tuyến tính cuối cùng để tạo ra vector đầu ra:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Trong đó W^O là ma trận trọng số để kết hợp thông tin từ tất cả các đầu chú ý.

Cơ chế Chú ý đa đầu mang lại nhiều lợi ích quan trọng. Thứ nhất, nó cho phép mô hình học được nhiều biểu diễn khác nhau của mỗi quan hệ giữa các phần tử trong chuỗi đầu vào. Mỗi đầu chú ý có thể tập trung vào các khía cạnh khác nhau của thông tin, chẳng hạn như quan hệ ngữ pháp, ngữ nghĩa, hoặc cấu trúc câu. Điều này giúp mô hình có cái nhìn toàn diện hơn về ngữ cảnh của chuỗi đầu vào.

Ngoài ra, Chú ý đa đầu cũng giúp cải thiện tính ổn định và hiệu suất của quá trình huấn luyện. Bằng cách chia nhỏ không gian biểu diễn thành nhiều phần, mỗi đầu chú ý có thể học được các đặc trưng riêng biệt, giảm thiểu sự cạnh tranh giữa các đặc trưng khác nhau trong quá trình học.

10.1.6 Kiến trúc Transformer encoder

Thành phần Encoder của Transformer (Hình 10.1 bao gồm nhiều lớp (layers) giống nhau, thường từ 6 đến 12 lớp, tùy thuộc vào phiên bản và ứng dụng cụ thể của mô hình. Mỗi lớp encoder bao gồm:

1. Cơ chế tự chú ý đa đầu (Multi-Headed Self-Attention Mechanism)

Tự chú ý Ở mỗi lớp, đầu vào được chia thành nhiều “đầu” chú ý (attention heads). Với mỗi đầu, cơ chế tự chú ý tính toán các vector query, key, và value từ đầu vào. Các vector này sau đó được sử dụng để tính toán trọng số chú ý cho mỗi từ trong chuỗi, cho phép mô hình xem xét mọi từ trong ngữ cảnh của toàn bộ chuỗi.

Chú ý đa đầu Bằng cách sử dụng nhiều đầu chú ý song song, mô hình có thể học được nhiều loại mối quan hệ khác nhau trong dữ liệu. Kết quả từ mỗi đầu chú ý được ghép nối lại và chuyển qua một lớp ánh xạ tuyến tính để tạo ra đầu ra cuối cùng của cơ chế chú ý.

2. Mạng nơ ron tiến theo vị trí (Position-Wise Feed-Forward Neural Network)

Sau khi qua cơ chế tự chú ý, đầu ra được chuyển qua một mạng nơ-ron tiến tới bao gồm hai lớp tuyến tính với một hàm kích hoạt phi tuyến (thường là ReLU) ở giữa. Mạng này hoạt động trên từng vị trí của chuỗi độc lập, giúp tăng cường khả năng biểu diễn của mô hình.

3. Kết nối dư (Residual Connections) và chuẩn hoá lớp (Layer Normalization)

Kết nối dư Các kết nối phần dư được thêm vào giữa đầu vào của một thành phần và đầu ra của thành phần đó. Điều này giúp duy trì thông tin gốc và cải thiện quá trình lan truyền ngược gradient, làm cho việc huấn luyện sâu trở nên ổn định hơn.

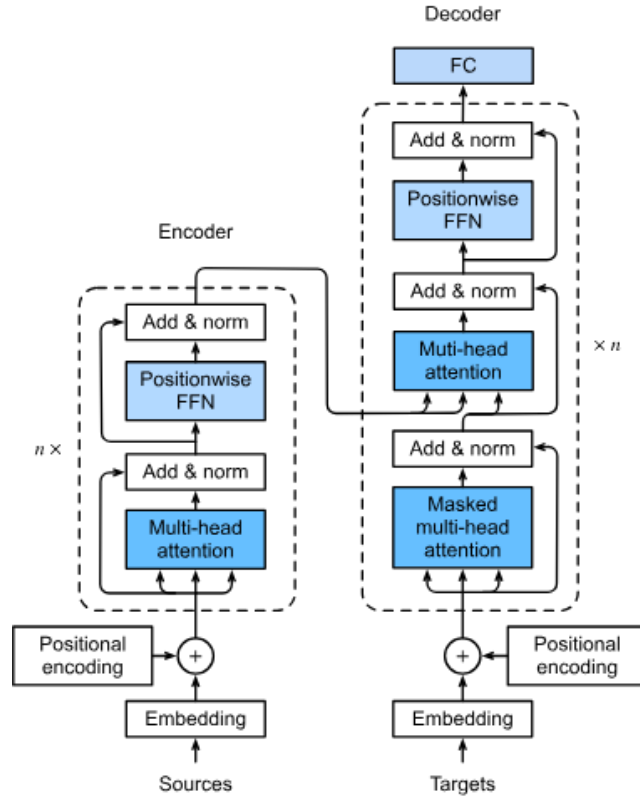
Chuẩn hoá lớp Chuẩn hóa lớp được áp dụng sau mỗi thành phần (cơ chế tự chú ý và mạng nơ-ron tiến) để chuẩn hóa đầu ra, giúp tăng cường sự ổn định và tốc độ hội tụ của mô hình.

Quá trình xử lý trong một lớp encoder có thể được tóm tắt bằng các phương trình sau:

$$\text{AttentionOutput} = \text{LayerNorm}(x + \text{MultiHeadAttention}(x))$$

$$\text{EncoderOutput} = \text{LayerNorm}(\text{AttentionOutput} + \text{FFN}(\text{AttentionOutput}))$$

Trong đó, x là đầu vào của lớp encoder.



Hình 10.1: Kiến trúc Transformer

10.1.7 Kiến trúc Transformer decoder

Kiến trúc decoder trong mô hình Transformer đóng vai trò quan trọng trong việc tạo ra chuỗi đầu ra dựa trên thông tin từ encoder và các phần tử đã được tạo ra trước đó. Khác với encoder, decoder không chỉ xử lý thông tin đầu vào mà còn phải tạo ra đầu ra tuần tự, đòi hỏi một cơ chế đặc biệt để đảm bảo tính nhân quả trong quá trình sinh chuỗi.

Cấu trúc cơ bản của decoder bao gồm một ngăn xếp các lớp giống nhau, mỗi lớp có ba thành phần chính: cơ chế tự chú ý có mặt nạ (masked self-attention),

cơ chế chú ý chéo (cross-attention), và mạng nơ-ron tiến theo vị trí (position-wise feed-forward network). Mỗi thành phần này đều được bổ sung bởi các kết nối dư (residual connections) và chuẩn hóa lớp (layer normalization) để tăng cường hiệu suất và ổn định quá trình huấn luyện.

Thành phần decoder bao gồm nhiều lớp tương tự nhau, mỗi lớp bao gồm các thành phần sau:

1. **Cơ chế tự chú ý đa đầu có mặt nạ** Cơ chế tự chú ý có mặt nạ là điểm đặc trưng của decoder. Trong quá trình sinh chuỗi đầu ra, tại mỗi bước, mô hình chỉ được phép sử dụng thông tin từ các từ đã được tạo ra trước đó. Để đạt được điều này, một ma trận mặt nạ được áp dụng lên ma trận điểm số chú ý (attention scores) trước khi thực hiện phép nhân softmax. Ma trận mặt nạ này có giá trị âm vô cùng (hoặc một giá trị âm rất lớn) tại các vị trí tương ứng với các từ trong tương lai, khiến cho xác suất chú ý đến các từ này trở nên gần như bằng không sau khi qua hàm softmax. Công thức toán học cho cơ chế tự chú ý có mặt nạ có thể được biểu diễn như sau:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V \quad (10.6)$$

Trong đó, M là ma trận mặt nạ, có giá trị 0 tại các vị trí được phép chú ý và $-\infty$ (hoặc một giá trị âm rất lớn) tại các vị trí khác.

2. **Cơ chế chú ý chéo đa đầu** Cơ chế chú ý chéo trong decoder có vai trò kết nối thông tin từ encoder với quá trình sinh chuỗi đầu ra. Trong cơ chế này, các vector query được tạo ra từ lớp tự chú ý của decoder, trong khi các vector key và value được lấy từ đầu ra của encoder. Điều này cho phép mô hình tập trung vào các phần quan trọng của chuỗi đầu vào khi tạo ra từng phần tử của chuỗi đầu ra.
3. **Mạng nơ-ron tiến theo vị trí** Mạng nơ-ron truyền thẳng theo vị trí trong decoder có cấu trúc tương tự như trong encoder. Nó bao gồm hai phép biến đổi tuyến tính với một hàm kích hoạt phi tuyến ở giữa, thường là ReLU. Mạng này giúp tăng cường khả năng biểu diễn phi tuyến của mô hình, cho phép nó học được các mối quan hệ phức tạp hơn giữa các phần tử trong chuỗi.
4. **Kết nối dư và chuẩn hoá lớp** Được áp dụng sau mỗi thành phần để cải thiện hiệu suất và ổn định của mô hình.

Quá trình sinh chuỗi đầu ra trong decoder thường sử dụng kỹ thuật sinh tuần tự có điều kiện (autoregressive generation). Tại mỗi bước, mô hình dự đoán phần

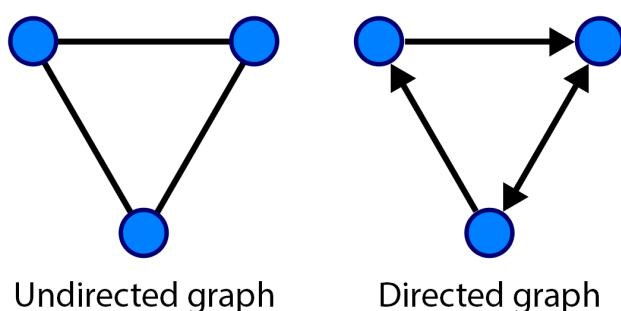
tử tiếp theo dựa trên đầu ra của encoder và các phần tử đã được sinh ra trước đó. Quá trình này tiếp tục cho đến khi gặp ký tự kết thúc hoặc đạt đến độ dài tối đa được chỉ định.

Việc sử dụng cơ chế tự chú ý có mặt nạ kết hợp với chú ý chéo cho phép decoder vừa duy trì tính nhất quán trong chuỗi đầu ra, vừa tận dụng hiệu quả thông tin từ chuỗi đầu vào. Điều này làm cho kiến trúc Transformer đặc biệt hiệu quả trong các tác vụ như dịch máy, tóm tắt văn bản, và sinh văn bản có điều kiện.

10.2 Graph neural network

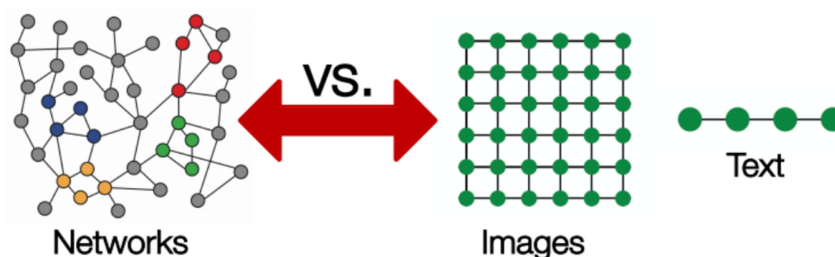
10.2.1 Khái niệm về đồ thị và Dữ liệu dạng đồ thị

Đồ thị là dạng dữ liệu trong đó chứa 2 thành phần chính: tập các đỉnh và tập cạnh. Mỗi đỉnh trong đồ thị thường biểu diễn một hay nhiều thông tin, ví dụ như con người, địa điểm hay một đồ vật, và mỗi cạnh biểu diễn mối quan hệ giữa các đỉnh đó. Các đồ thị có thể có hướng hoặc không.



Hình 10.2: Đồ thị không có hướng và có hướng

Dữ liệu dạng đồ thị mang lại khả năng biểu diễn cũng như phân tích các mối liên hệ phức tạp giữa các vật thể một cách hiệu quả. Tuy nhiên, việc xử lý các dữ liệu dưới dạng đồ thị gặp phải vấn đề do các đặc tính của dạng dữ liệu này. Một đồ thị tồn tại trong không gian phi Euclid thay vì 2D hay 3D, do đó khiến cho việc 'hiểu' dạng dữ liệu này khó khăn hơn. Để có thể mô phỏng cấu trúc đồ thị trong không gian 2D, ta cần phải sử dụng các công cụ giảm chiều. Bên cạnh đó, việc sử dụng các công cụ như CNN cũng khó khăn do CNN cần một kernel có cấu trúc cố định để đưa ra được đặc trưng trong khi các đồ thị không có dạng cụ thể như các cấu trúc dữ liệu khác (ảnh 2D, 3D, đồ thị sóng, v.v...), 2 đồ thị có thể có dạng mô phỏng khác nhau, tuy nhiên lại có chung một ma trận kề.



Hình 10.3: Mô phỏng cấu trúc dữ liệu đồ thị

10.2.2 Mạng nơ ron đồ thị(GNN)

Trong những năm gần đây, trí tuệ nhân tạo đương đại (AI), hoặc cụ thể hơn, học sâu (DL) đã thống trị bởi mạng nơ ron (NN). Các biến thể của mạng nơ ron đã được thiết kế nhằm tăng hiệu quả trong các lĩnh vực cụ thể, ví dụ mạng nơ ron tích chập (CNN) có khả năng vượt trội trong các vấn đề liên quan đến xử lý hình ảnh; mạng nơ ron hồi quy (RNN) trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP) và phân tích dữ liệu dãy thời gian. Các mạng nơ ron cũng đã được sử dụng như một khối trong các framework DL phức tạp hơn. Chúng được sử dụng như trong mạng đối nghịch tạo sinh (GAN), và như một thành phần trong mạng Transformer.

Mạng nơ ron đồ thị (GNN) đã trở nên phổ biến trong lĩnh vực AI nhờ khả năng độc đáo trong việc sử dụng dữ liệu dạng đồ thị - một dạng dữ liệu không có cấu trúc chặt chẽ, đồng thời cũng là một dạng dữ liệu đặc biệt tổng quát, có thể biểu diễn các dạng dữ liệu đầu vào khác như hình ảnh trong thị giác máy tính hay câu từ trong NLP. Thông thường, các thuật toán NN thường yêu cầu đầu vào có cấu trúc và chiều cụ thể. Lấy ví dụ, để phân loại tập dữ liệu MNIST, mạng CNN cần phải xây dựng một lớp đầu vào gồm 28×28 nơ ron, và các hình ảnh đầu vào theo đó cũng cần phải có kích thước 28×28 để đạt được yêu cầu về chiều. Chúng tôi hướng tới việc giải thích điểm mạnh và tính mới mẻ của GNN cho những người sử dụng AI thông qua việc đối chiếu, trình bày chi tiết các động cơ, khái niệm, toán học cũng như ứng dụng của các biến thể phổ biến và hiệu quả của GNN.

Trong phần này, chúng tôi sẽ giới thiệu một số mạng GNN phổ biến hiện nay.

Tên các loại mạng	Định nghĩa
Mạng đồ thị tích chập(Graph Convolutional Neural Network/GCN)	Một mạng nơ ron tương tự với mạng CNN truyền thống, thông qua việc kiểm tra các nút lân cận để học các đặc trưng của đồ thị. GNN tổng hợp các vector nút, truyền các kết quả tới một lớp dày hơn, và áp dụng phi tuyến tính bằng cách sử dụng các hàm kích hoạt. Nói ngắn gọn, nó bao gồm tích chập đồ thị, lớp tuyến tính và các hàm kích hoạt không tham gia học. Có 2 dạng chính của GCN: Mạng tích chập không gian và Mạng tích chập phổ.
Mạng đồ thị tự động mã hoá (Graph Auto-Encoder Network/-GAE)	Một mạng nơ ron tìm hiểu biểu diễn đồ thị sử dụng một bộ mã hoá và thử tái cấu trúc đồ thị đầu vào bằng bộ giải mã. Bộ mã hoá và giải mã kết nối thông qua một lớp cổ chai. Mạng này thường được sử dụng trong link prediction nhờ việc tự động mã hoá có khả năng xử lý tốt đối với cân bằng lớp.
Mạng đồ thị hồi quy (Recurrent Graph Neural Network/RGNN)	Một mạng nơ ron học mô hình khuếch tán tốt nhất, có khả năng xử lý đồ thị đa quan hệ trong đó một nút có nhiều mối quan hệ. Dạng mạng nơ ron này sử dụng các bộ điều chỉnh (regularizer) để cải thiện độ mượt và loại bỏ việc tham số hoá quá mức. RGNN sử dụng ít tính toán hơn nhằm đưa ra kết quả tốt hơn. Thường được sử dụng trong việc tạo văn bản, dịch máy, nhân dạng giọng nói, tạo mô tả hình ảnh, gắn thẻ hình ảnh và tóm tắt văn bản.
Mạng đồ thị cổng (Gated Graph Neural Network/GGNN)	Một mạng nơ ron có khả năng thực hiện các tác vụ với các phụ thuộc lâu dài tốt hơn so với RGNN. GGNN cải thiện RGNN bằng việc thêm cơ chế cổng: GGNN thêm các nút, cạnh và cổng thời gian vào các phụ thuộc dài hạn. Các cổng này nhằm nhớ hoặc quên thông tin ở các trạng thái khác nhau.

Hạn chế

Mặc dù có khả năng làm việc với đồ thị một cách hiệu quả, mạng GNN vẫn gặp phải một số hạn chế do cấu trúc mạng và dạng dữ liệu mà GNN phải xử lý:

1. Hầu hết các mạng nơ ron có thể xây dựng nhiều lớp để có thể cải thiện hiệu năng, trong khi các mạng GNN là mạng nông với hầu hết chỉ sử dụng 3 lớp. Điều này giới hạn việc đạt được hiệu suất như các state-of-the-art trên các bộ dữ liệu lớn.
2. Cấu trúc của các đồ thị thay đổi liên tục, khiến cho việc train mô hình trở nên khó khăn.
3. Việc triển khai mô hình vào sản phẩm sẽ phải đối mặt với vấn đề về mở rộng bởi các mạng này tiêu tốn khả năng tính toán lớn. Nếu ta sử dụng một đồ thị lớn và có cấu trúc phức tạp, việc mở rộng mạng GNN trong sản phẩm có thể gặp phải nhiều khó khăn.

Ứng dụng của mạng GNNs

Một số đặc điểm chung của mạng GNN như sau:

- **Phân loại đồ thị:** là phân loại toàn bộ đồ thị thành các loại khác nhau. Nó giống như phân loại hình ảnh, nhưng mục tiêu thay đổi thành miền biểu đồ. Các ứng dụng của phân loại biểu đồ rất nhiều và bao gồm từ việc xác định xem protein có phải là enzyme hay không trong tin sinh học, đến phân loại tài liệu trong NLP hoặc phân tích mạng xã hội.
- **Phân loại nút:** là xác định nhãn của các mẫu (được biểu thị dưới dạng nút) bằng cách xem nhãn của các nút lân cận. Thông thường, các bài toán thuộc loại này được huấn luyện theo cách bán giám sát, chỉ một phần của biểu đồ được dán nhãn.
- **Trực quan hóa đồ thị:** là một lĩnh vực của toán học và khoa học máy tính, điểm giao thoa giữa lý thuyết đồ thị hình học và trực quan hóa thông tin. Nó liên quan đến việc biểu diễn trực quan các biểu đồ cho thấy các cấu trúc và điểm bất thường có thể có trong dữ liệu và giúp người dùng hiểu được biểu đồ.
- **Dự đoán liên kết:** dựa vào thuật toán biểu diễn mối quan hệ giữa các thực thể trong biểu đồ và nó cũng cố gắng dự đoán liệu có mối liên hệ nào giữa hai thực thể hay không. Điều cần thiết trong mạng xã hội là suy ra các tương tác xã hội hoặc gợi ý những người bạn có thể có cho người dùng. Nó cũng đã được sử dụng trong các vấn đề về hệ thống gợi ý.

- **Phân cụm đồ thị:** đề cập đến việc phân cụm dữ liệu dưới dạng biểu đồ. Có hai dạng phân cụm riêng biệt được thực hiện trên dữ liệu biểu đồ. Phân cụm đỉnh tìm cách phân cụm các nút của biểu đồ thành các nhóm gồm các vùng được kết nối dày đặc dựa trên trọng số cạnh hoặc khoảng cách cạnh. Dạng phân cụm biểu đồ thứ hai coi các biểu đồ là các đối tượng được phân cụm và phân cụm các đối tượng này dựa trên sự giống nhau.

10.2.3 Một số thuật ngữ quan trọng

Đồ thị được định nghĩa bởi một tập các đỉnh và cạnh, biểu diễn bởi công thức $G = G(V, E)$. Về cơ bản, các đồ thị chỉ là một cách để mã hoá dữ liệu, theo đó, mỗi thuộc tính của đồ thị biểu diễn một vài yếu tố, hoặc khái niệm trong dữ liệu đó. Việc hiểu cách mà đồ thị có thể sử dụng thể biểu diễn các khái niệm phức tạp là chìa khoá trong việc hiểu rõ khả năng diễn tả của chúng cũng như tính tổng quát của một công cụ mã hoá.

Đỉnh thể hiện một đối tượng hay thực thể, có thể được biểu diễn thông thường bởi các thuộc tính định lượng được hay mối quan hệ của chúng đối với các đối tượng, thực thể khác. Ta coi một tập $|V|$ các đỉnh là V và đỉnh thứ v trong đó là v_i . Lưu ý rằng các đỉnh không nhất thiết phải đồng nhất về mặt cấu trúc.

Cạnh thể hiện và tính chất hoá mối quan hệ giữa các đối tượng hay thực thể. Một cạnh có thể được biểu diễn thông qua 2 đỉnh. Ta coi một tập $|E|$ các cạnh là E và mỗi cạnh đơn lẻ giữa đỉnh thứ i và j là e_{ij} .

Vùng lân cận là một đồ thị con (subgraph) trong đồ thị, biểu diễn một nhóm các đỉnh với cạnh khác nhau. Thông thường, vùng lân cận $\mathcal{N}_{v_i}$ xung quanh và bao gồm đỉnh v_i , các cạnh kề ($e_{ij} = 1$), và các đỉnh nối trực tiếp với đỉnh v_i . Các vùng lân cận có thể phát triển từ một đỉnh bằng cách xét các đỉnh đang nối với vùng lân cận hiện tại thông qua cạnh.

Đặc trưng là một thuộc tính có thể định lượng được mô tả đặc điểm của một hiện tượng đang trong quá trình theo dõi. Trong lĩnh vực đồ thị, đặc trưng này có thể được sử dụng để mô tả rõ hơn đặc điểm của đỉnh và cạnh. Lấy ví dụ về đồ thị mô tả một mạng xã hội, ta có thể có các đặc trưng của mỗi người (được tính là một đỉnh) để định lượng tuổi tác, độ phổ biến hay việc sử dụng mạng xã hội của người đó. Tương tự, ta có thể có đặc trưng đối với mỗi mối quan hệ (cạnh) để định lượng xem hai người biết nhau rõ như thế nào, hay mối quan hệ của họ là gì (người thân, đồng nghiệp, v.v...).

Biểu diễn nhúng (Embeddings) là cách biểu diễn các đặc trưng dưới dạng nén. Thông qua việc giảm các vector đặc trưng lớn liên quan tới các đỉnh, cạnh thành các biểu diễn dưới chiều không gian thấp, ta có thể phân loại chúng bằng các

mô hình bậc thấp (tức là, ta có thể làm cho dữ liệu có thể phân tách tuyến tính). Một tiêu chí quan trọng về chất lượng của nhúng là liệu các điểm trong không gian gốc có thể giữ được sự tương đồng trong không gian nhúng hay không. Các biểu diễn nhúng có thể được tạo ra (hoặc học được) từ các đỉnh, cạnh, vùng lân cận hay từ đồ thị. Nhúng cũng có thể được xem như là biểu diễn, mã hoá, các vector ẩn hoặc các vector đặc trưng bậc cao tùy thuộc vào ngữ cảnh.

Kiểu đầu ra thay đổi tùy thuộc vào miền của vấn đề cần giải quyết. Quá trình forward pass của một mạng GNN có thể được coi là 2 quá trình chính: chuyển đổi đồ thị đầu vào thành các biểu diễn nhúng có ích, thực hiện các tác vụ sau đó (ví dụ như phân loại) trên các biểu diễn nhúng, chuyển đổi các biểu diễn đó thành đầu ra có ích. Ta xác định được 3 loại đầu ra thường thấy sau đây.

1. **Đầu ra cấp đỉnh** yêu cầu dự đoán (ví dụ như một lớp riêng biệt hoặc giá trị hồi quy) cho mỗi đỉnh của đồ thị trước.
2. **Đầu ra cấp cạnh** yêu cầu dự đoán cho mỗi cạnh trong đồ thị.
3. **Đầu ra cấp đồ thị** yêu cầu dự đoán cho mỗi đồ thị. Ví dụ: dự đoán các tính chất của các đồ thị phân tử hoá học.

10.3 Nhúng đỉnh (Node Embedding)

10.3.1 Giới thiệu

GNN được dùng để học về biểu diễn của các nút trong đồ thị hay nói cách khác là node embedding. Khi áp dụng các thuật toán học máy, học sâu đã biết vào dữ liệu dạng đồ thị chính là tính chất không có cấu trúc cố định (đa dạng về topo). Do đó ý tưởng về cách để trích xuất thông tin từ dữ liệu đồ thị (cụ thể ở đây là các nút) về dạng cấu trúc cố định để có thể sử dụng chúng trong các downstream task: dự đoán, phân loại,...

10.3.2 Khái niệm

Như đã đề cập phía trên, Embedding (hay biểu diễn nhúng) là biểu diễn các đặc trưng của dữ liệu nén. Bằng cách embedding, ta có thể giảm chiều dữ liệu của vector đặc trưng của các đỉnh và cạnh, ánh xạ chúng sang 1 không gian latent D-dims. Ở đây nút embedding là ánh xạ thông tin đặc trưng của từng nút sang vector trong không gian latent. Ngoài ra còn có graph embedding là ánh xạ thông tin đặc

trung của cả graph sẽ được đề cập sau. Một số thuật ngữ dùng thay thế embedding: encoding, representation, latent vector, high-level feature vector.

10.3.3 Các đặc điểm

Node Embedding có ba đặc điểm chính.

Thứ nhất, giảm chiều dữ liệu. Dữ liệu dạng graph là rất lớn và rất phức tạp cả về số lượng và số chiều. Bằng việc giảm chiều dữ liệu và giữ lại những đặc trưng chính trong cấu trúc giúp cho việc áp dụng các thuật toán học máy hiệu quả hơn.

Thứ hai, diễn giải và biểu diễn dữ liệu. Việc diễn giải đồ thị của con người chưa bao giờ là dễ dàng đặc biệt là các đồ thị có quy mô lớn. Do đó việc nén dữ liệu lại giúp con người có thể dễ dàng hiểu được các quy luật, mối quan hệ và biểu diễn trên đồ thị không gian 2D.

Thứ ba, độ tương đồng của nút. Một tính chất của nút Embedding chính là 2 nút càng gần nhau trong không gian latent sẽ càng giống nhau về mặt vai trò trong đồ thị ban đầu. Từ đó ta có thể dễ dàng thực hiện các bài toán quen thuộc như dự đoán, phân loại, phân cụm,...

10.3.4 Cách tiếp cận kỹ thuật

Có rất nhiều giải thuật Embedding nhưng nói chung đều dựa trên mục tiêu: các nút ở gần nhau trong không gian latent sẽ có các “vị trí” trong đồ thị tương tự nhau. Ví dụ như những người có cùng sở thích thường sẽ có sự tương đồng về người theo dõi trên facebook. Từ đó, ta có thể lập nên một đồ thị mạng theo dõi trên mạng xã hội để dự đoán xu hướng,... Tóm lại, chúng ta cần tìm hàm ENC là ánh xạ từ tập các đỉnh sang không gian latent, hàm similarity là độ tương đồng trong đồ thị. Ngoài ra ta cũng cần hàm DEC là độ tương đồng trong không gian nén, ở đây đơn giản chọn là hàm nhân ma trận.

Khái quát hóa bài toán, ta có ánh xạ $ENC : V \rightarrow \mathbf{R}^D$. Xét 2 nút u, v bất kỳ thì $ENC(u) = z_u, ENC(v) = z_v$. Khi đó mục tiêu bài toán là $similarity(u, v) \approx z_u^T \cdot z_v$

10.3.5 Shallow Encoder

Một cách tiếp cận đơn giản là sử dụng bảng tra cứu embedding:

$$ENC(v) = z_v = Z.v$$

trong đó, $Z \in \mathbf{R}^{d \times |V|}$, là ma trận mà mỗi cột là một nút embedding và $v \in \mathbf{I}^{|V|}$, là một dạng index vector chỉ vị trí của vector v trong ma trận Z

10.3.6 Random Walk

Random Walk là một quá trình mà ta đi từ một nút bất kỳ và chọn ngẫu nhiên một nút kề với nó và thăm nút đó. Chú ý rằng có thể thăm lại chính nút vừa đi qua. Trong bài này sẽ ký hiệu $P(v|z_u)$ là xác suất thăm nút v trong một random walk bắt đầu từ nút u . Từ đó chúng ta có thể định nghĩa độ tương đồng giữa 2 nút như sau: $similarity(u, v)$ là xác suất để nút u và v cùng xuất hiện trong một random-walk qua đồ thị. Tổng quát, quá trình embedding diễn ra như sau: đầu tiên cần tính xác suất thăm nút v trong 1 random-walk từ đỉnh u sử dụng chiến lược R (ký hiệu $P_R(v|u)$), sau đó tối ưu hóa hàm mã hóa ENC để xấp xỉ độ tương đồng Random-walk và độ tương đồng trong không gian latent.

10.3.7 Node2Vec

Ý tưởng đơn giản để chạy một random-walk là chạy ngẫu nhiên với độ dài cố định từ mỗi nút. Tuy nhiên vấn đề là như vậy thì cách tính toán có phần nghiêm ngặt. Để linh hoạt hơn trong việc chọn đường đi chúng ta sẽ sử dụng thêm hai trọng số p, q (biased walk), trong đó p là khả năng quay lại 1 nút trước đó trên 1 random walk và q là khả năng bước đi gần hay xa so với nút ban đầu. Nhờ thế chúng ta có thể thay đổi góc nhìn về đồ thị là tổng quát hoặc cục bộ. Hai chiến lược di chuyển cơ bản trên đồ thị là BFS, di chuyển theo chiều rộng và DFS, di chuyển theo chiều ngang. Trong khi BFS cho cái nhìn cục bộ thì DFS cho cái nhìn toàn cục. Như vậy, tham số q đã nhắc ở trên có thể coi là tỷ lệ giữa việc dùng BFS hay DFS.

10.3.8 Hạn chế của các mô hình Node Embedding

Nói chung, các mô hình đều hướng tới việc ánh xạ nút trong đồ thị sang không gian vector latent và các nút tương tự nhau, có context giống nhau sẽ gần nhau.

Tuy nhiên vẫn có một số hạn chế:

Thứ nhất là các bài toán có domain lớn, có sự cập nhật về các nút, các liên kết mới trong đồ thị thì sẽ không thể ánh xạ chúng. Mô hình Word2vec có ý tưởng tương tự cũng không có node embedding cho mỗi từ nằm ngoài từ điển cho trước.

Thứ hai là sử dụng thông tin là các nút lân cận nhau để xây dựng mô hình, chứ chưa sử dụng các thông tin khác như node feature hay thuộc tính của từng nút trong đồ thị. Ví dụ là bài toán về social network thì ngoài kết nối giữa người với người thì còn thông tin của mỗi người như tuổi tác, việc làm,... cũng rất quan trọng.

10.4 Mạng đồ thị tự mã hoá (GAE)

10.4.1 Mạng tự mã hoá (AutoEncoder)

AutoEncoder là một mạng nơ ron nhân tạo có khả năng học không giám sát hiệu quả các biểu diễn của dữ liệu đầu vào mà không cần tới nhãn, thông qua việc mã hoá đầu vào, sau đó giải mã để có thể biểu diễn lại đầu ra giống với ban đầu, từ đó học các thuộc tính, đặc trưng cần thiết của đầu vào ban đầu. Các mô hình tự mã hoá được áp dụng nhiều trong các bài toán liên quan tới việc tái tạo hình ảnh, giảm chiều dữ liệu và các mô hình học tập trung, ví dụ như huấn luyện một tập các khuôn mặt để tạo ra khuôn mặt mới. Kiến trúc của một mạng tự mã hoá gồm có 3 phần chính:

- **Bộ mã hoá:** Bao gồm các convolutional blocks, theo sau là các pooling modules nhằm nén đầu vào thành các phần nhỏ hơn, được gọi là bottleneck.
- **Bottleneck:** Module chứa biểu diễn của đầu vào sau khi đi qua bộ mã hoá, có kích thước nhỏ nhất và mang theo những đặc trưng, tri thức của đầu vào. Việc bottleneck nhỏ sẽ giảm rủi ro overfitting đáng kể, tuy nhiên nếu kích thước không đủ lớn sẽ không thể lưu trữ được các đặc trưng cần thiết, gây khó khăn cho bộ giải mã.
- **Bộ giải mã:** Sau khi đầu vào được nén lại thông qua bottleneck, module này sẽ thực hiện việc giải mã các biểu diễn và thử tái cấu trúc lại dữ liệu từ dạng mã hoá, so sánh đầu ra của decoder với đầu vào ban đầu.

10.4.2 Ứng dụng mạng tự mã hoá trong miền đồ thị

Như đã nêu trên, các mạng tự mã hoá thường bao gồm 2 bước, đầu tiên mã hoá các dữ liệu đầu vào, sau đó giải mã các biểu diễn đã được nén lại này để tái cấu trúc lại dữ liệu đó. Sau đó, mạng AE được huấn luyện để tối thiểu hoá mất mát khi thực hiện tái cấu trúc, trong đó hàm mất mát được tính thông qua đầu vào với công thức đơn giản $Loss_{AE} = \|X - \hat{X}\|^2$, với X là một đầu vào và $\hat{X}$ là đầu vào được tái cấu trúc. Điểm khác biệt giữa các mạng AE và GAE nằm ở việc xử lý dạng dữ liệu đầu vào, yêu cầu mạng GAE cần phải thay đổi cách mã hoá và giải mã để có thể đưa vào và ra dữ liệu dạng đồ thị. Một phương pháp điển hình đó là thay thế bộ mã hoá với một mạng CGNN, và thay đổi bộ giải mã với một cách thức mà có thể tái tạo được cấu trúc của đồ thị.

Với một hàm mất mát được thiết kế tốt, ta có thể thực hiện việc học end-to-end

(cho mô hình học trực tiếp từ đầu vào mà không thông qua feature engineering hay các bước khác) để tối ưu hoá việc mã hoá và giải mã, nhằm đạt được cân bằng giữa tính nhạy cảm đối với đầu vào và tính khái quát.

10.5 GGNN

Mạng Nơ-ron Đồ Thị Cổng (GGNN) là một sự điều chỉnh của Mạng Nơ-ron Đồ Thị (GNN) được thiết kế đặc biệt để xử lý các đầu ra không tuần tự. Khác với GNN, sử dụng ràng buộc ánh xạ điểm cố định để đảm bảo độc lập của các khởi tạo ban đầu, GGNN tích hợp nhãn đỉnh như đầu vào bổ sung, được gọi là chú thích đỉnh. Những chú thích này được sử dụng để đánh dấu các đỉnh đặc biệt trong đồ thị, chẳng hạn như các đỉnh nguồn và đích trong các nhiệm vụ như dự đoán khả năng tiếp cận giữa các đỉnh.

10.6 RGNN

10.6.1 Vấn đề đối với dữ liệu dạng tuần tự (Sequence data)

Khi thực hiện các tác vụ đối với dữ liệu dạng tuần tự như một câu dài, chuỗi thời gian hay các tín hiệu giọng nói, các mô hình mạng nơ ron thường gặp phải vấn đề do sự độc lập, không phụ thuộc giữa các lớp với nhau, trong khi các dữ liệu loại này thường liên hệ tới các điểm dữ liệu trước hoặc sau nó. Điều này thể hiện rõ qua các đặc tính quan trọng của dạng dữ liệu tuần tự:

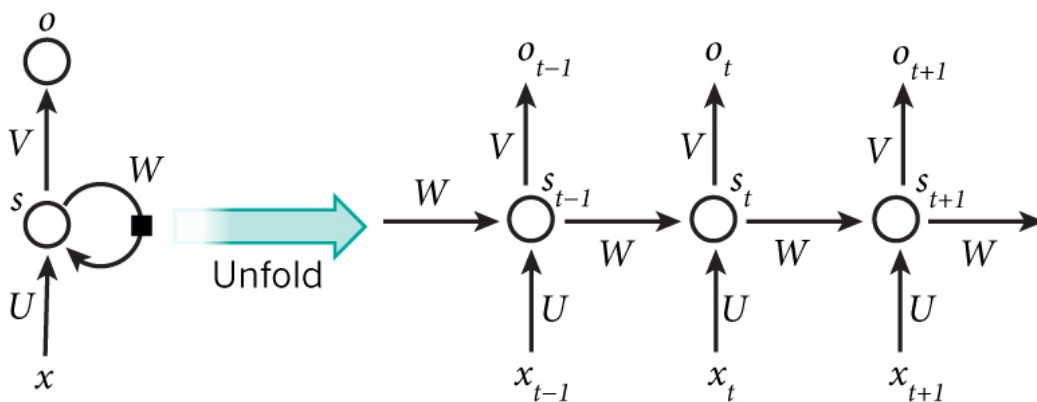
- **Tính thứ tự:** Các dữ liệu tuần tự thường mang ý nghĩa khác nhau tùy thuộc vào vị trí được sắp xếp của các điểm dữ liệu trong đó, việc thay đổi vị trí giữa các điểm dữ liệu có thể dẫn đến việc ý nghĩa của dữ liệu bị thay đổi hoàn toàn. Lấy ví dụ, trong một câu hoàn chỉnh, thứ tự của các từ quyết định ý nghĩa của câu đó.
- **Các mối quan hệ tuần tự và tạm thời:** Dữ liệu tuần tự thường thể hiện một mối quan hệ tuần tự hay tạm thời, tức là, các điểm dữ liệu thường được thu thập theo một thứ tự và theo dõi trong một khoảng thời gian nhất định như các dữ liệu về chuỗi thời gian như giá cổ phiếu.

Điều này đã đặt ra một vấn đề lớn đối với các mạng nơ ron trước đó, khi mà các lớp trong mô hình không thể hiểu được các dữ liệu tuần tự và cần một giải pháp về việc 'nhớ' để có thể xử lý được dạng dữ liệu này.

10.6.2 Mạng nơ ron hồi quy (RNNs)

Ý tưởng chính để ra đời được mạng nơ ron hồi quy chính là sử dụng một bộ nhớ chính để có thể lưu lại thông tin từ các bước tính toán trước đó, dựa vào nó để có thể đưa ra được dự đoán chính xác cho bước hiện tại. Một mô hình RNN có thể được thể hiện như hình 10.4.

Một mô hình RNN thường được biểu diễn tổng quát thông qua ảnh bên trái,



Hình 10.4: Mô tả tổng quan về mạng RNNs

và được unfold thành một mạng đầu đủ như bên phải (việc unfold phụ thuộc vào độ dài dữ liệu mà ta quan tâm tới). Các công thức điều khiển quá trình tính toán trong một RNN như sau:

- x_t là đầu vào tại bước thời gian t . Ví dụ, x_1 có thể là một vector one-hot tương ứng với từ thứ hai của một câu.
- s_t là trạng thái ẩn tại bước thời gian t . Đây là "bộ nhớ" của mạng. s_t được tính toán dựa trên trạng thái ẩn trước đó và đầu vào tại bước hiện tại:

$$s_t = f(Ux_t + Ws_{t-1})$$

Hàm f thường là một hàm phi tuyến như tanh hoặc ReLU. - s_{-1} , được yêu cầu để tính trạng thái ẩn đầu tiên, thường được khởi tạo bằng toàn số không. - o_t là đầu ra tại bước t . Ví dụ, nếu chúng ta muốn dự đoán từ tiếp theo trong một câu thì nó sẽ là một vector xác suất trên từ điển của chúng ta.

$$o_t = \text{softmax}(Vs_t)$$

Có một vài điều cần lưu ý ở đây:

Có thể coi trạng thái ẩn s_t là bộ nhớ của mạng. s_t lưu trữ thông tin về những gì đã xảy ra trong tất cả các bước thời gian trước đó. Đầu ra tại bước o_t được tính

toán hoàn toàn dựa trên bộ nhớ tại thời điểm t . Trong thực tế, điều này hơi phức tạp hơn vì s_t thường không thể lưu trữ thông tin từ quá nhiều bước thời gian trước.

Khác với một mạng nơ-ron sâu truyền thống, sử dụng các tham số khác nhau tại mỗi lớp, một RNN chia sẻ cùng một bộ tham số (U, V, W) cho tất cả các bước. Điều này phản ánh thực tế là chúng ta đang thực hiện cùng một nhiệm vụ tại mỗi bước, chỉ với các đầu vào khác nhau. Điều này giúp giảm đáng kể tổng số tham số mà chúng ta cần học.

Sơ đồ trên có đầu ra tại mỗi bước thời gian, nhưng tùy thuộc vào nhiệm vụ, điều này có thể không cần thiết. Ví dụ, khi dự đoán cảm xúc của một câu, chúng ta có thể chỉ quan tâm đến đầu ra cuối cùng, không phải cảm xúc sau mỗi từ. Tương tự, chúng ta có thể không cần đầu vào tại mỗi bước thời gian. Đặc điểm chính của một RNN là trạng thái ẩn của nó, lưu trữ một số thông tin về một chuỗi.

10.6.3 Mạng nơ-ron đồ thị hồi quy

Áp dụng cách tiếp cận giống như mạng RNNs, ta thực hiện xây dựng các hàm chuyển tiếp đệ quy để xử lý đồ thị thay cho các lớp ẩn lặp lại, cho phép ta thực hiện tính toán dựa trên thông tin từ đồ thị trước đó. Ngoài ra, tương tự như RNNs sử dụng cùng một bộ trọng số qua các bước thời gian để giảm số lượng tham số và tăng khả năng tổng quát hóa, RGNNs sử dụng cùng một hàm chuyển tiếp (transition function) tại mỗi đỉnh trong mỗi bước lặp. Điều này giúp mô hình có thể áp dụng nhất quán trong toàn bộ đồ thị bất kể cấu trúc hay kích thước của nó. Hàm chuyển tiếp của mạng RGNNs được mô tả như sau: Đối với một biểu diễn nhúng thứ k tại điểm i h_i^k phụ thuộc vào số lượng của các thuộc tính

- lượng đặc trưng của nút trung tâm v_i .
- lượng đặc trưng của các cạnh kề e_{ij} .
- lượng đặc trưng của các nút lân cận v_j
- biểu diễn nhúng của các nút lân cận trong lần lặp lại trước đó h_j^{k-1}

Để áp dụng đệ quy hàm chuyển tiếp đã học để tính toán các biểu diễn liên tục, hàm f cần có một số lượng biến đầu vào và đầu ra cố định. Có hai giải pháp đơn giản, giải pháp thứ nhất là thiết lập "kích thước lân cận tối đa" và sử dụng vector rỗng khi xử lý các đỉnh không có hàng xóm. Giải pháp thứ hai là tổng hợp tất cả các đặc điểm lân cận theo một cách không phụ thuộc vào hoán vị, đảm bảo mỗi lân cận trong đồ thị được biểu diễn bởi một vector đặc điểm cố định. Phương trình cho

việc này có thể được biểu diễn như sau:

$$h_i^k = \sum_{j \in N_{v_i}} f(v_{F_i}, e_{F_{ij}}, v_{F_j}, h_j^{k-1}) \text{ trong đó } h_i^0 \text{ được xác định khi khởi tạo.}$$

Hàm chuyển tiếp có thể được áp dụng lặp đi lặp lại cho đến khi đạt được một biểu diễn ổn định cho tất cả các đỉnh trong đồ thị. Biểu thức này có thể được hiểu là việc truyền "thông điệp", hay các đặc điểm, khắp đồ thị; trong mỗi lần lặp, biểu diễn h_i^k phụ thuộc vào các đặc điểm và biểu diễn của các đỉnh lân cận.

Sau khi đã có được các biểu diễn nhúng có ích quanh các nút của đồ thị, ta cần biểu diễn ra được các đầu ra có nghĩa nhằm thực hiện các tác vụ. Lúc này, hàm đầu ra g sẽ thực hiện lấy các biểu diễn hội tụ của đồ thị $G(V, E)$ và tạo ra đầu ra đã nói. Trên thực tế, hàm đầu ra g cũng sử dụng mạng feed-forward để thực hiện giống như hàm dịch chuyển f .

10.7 Mạng nơ ron đồ thị tích chập (GCNN)

10.7.1 Mạng nơ ron tích chập (CNN)

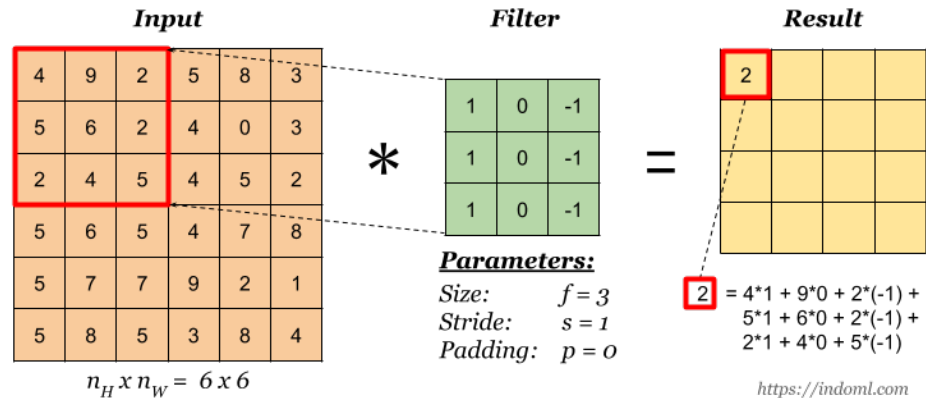
Các mạng tích chập đã đạt tới mức state-of-the-art về mặt hiệu suất trong các tác vụ dự đoán liên quan tới hình ảnh. Thông qua việc tích chập một bộ ma trận các trọng số đã được học với hình ảnh đầu vào, mạng CNN có thể trích xuất được các đặc trưng được quan tâm dựa trên hình dạng trực quan của chúng - bất kể vị trí của chúng trong hình ảnh. Và do hình ảnh được coi là một trường hợp của đồ thị, một phép tích thập tổng quát có thể được xây dựng đối với đồ thị, từ đó đem tới một hướng đi mới cho GNN, đem lại các đặc tính mong muốn dưới đây tới mạng GNN:

- **Tính địa phương:** Các trọng số học được để trích xuất đặc trưng cần được định vị địa phương, tức là chúng chỉ xem xét thông tin trong một khu vực lân cận nhất định. Điều này đảm bảo rằng các đặc trưng được học là có thể áp dụng đồng nhất trên toàn bộ đồ thị đầu vào, không phụ thuộc vào vị trí cụ thể của các nút trong đồ thị.
- **Khả năng mở rộng:** Việc học các bộ trích xuất đặc trưng này cần phải có khả năng mở rộng, nghĩa là số lượng tham số có thể học được không phụ thuộc vào số lượng nút $|V|$ trong đồ thị. Các toán tử nên có thể xếp chồng lên nhau để xây dựng mô hình từ nhiều lớp độc lập, thay vì yêu cầu lặp đi lặp lại cho đến khi hội tụ như các mô hình RGNN. Độ phức tạp tính toán cũng nên được giới hạn ở mức có thể.

- **Khả năng giải thích:** Hoạt động tích chập nên được cơ sở hóa trên một số diễn giải toán học hoặc vật lý, và cơ chế hoạt động của nó nên dễ hiểu. Điều này không chỉ giúp người dùng hiểu rõ cách thức mô hình làm việc mà còn hỗ trợ trong việc phát triển và điều chỉnh mô hình.

Phép tích chập

Ta định nghĩa một phép tích chập là một toán tử trong đó đầu ra được lấy ra từ hai đầu vào thông qua phép hợp nhất hay phép tổng, thể hiện việc giá trị này bị thay đổi bởi giá trị còn lại như thế nào.



Hình 10.5: Một ví dụ về phép tích chập

Phép tích chập trong mạng CNN bao gồm hai ma trận đầu vào, một ma trận từ kết quả của hàm kích hoạt của lớp trước đó, ma trận còn lại là một ma trận trọng số học $W \times H$, được 'trượt' qua ma trận kích hoạt, tổng hợp từng vùng $W \times H$ bằng một tổ hợp tuyến tính đơn giản. Ma trận thứ hai này được coi như một bộ lọc, có khả năng trích xuất các đặc trưng từ ma trận đầu vào trước đó. Thông qua phép tích chập, mạng CNN có thể thu thập được một ma trận mới, gọi là **feature map**.

Tại sao không thể sử dụng CNN trên đồ thị

CNN có thể được sử dụng để giúp máy tính "nhìn" và thực hiện các nhiệm vụ như phân loại hình ảnh, nhận dạng hình ảnh hoặc phát hiện vật thể. Đây là những lĩnh vực mà CNN được ứng dụng phổ biến nhất. Khái niệm cốt lõi đằng sau CNN là việc giới thiệu các lớp tích chập và lớp gộp ẩn. Điều này nhằm mục đích xác định các đặc trưng địa phương qua một tập hợp các trường thụ cảm dưới dạng bộ lọc. Khi

CNN hoạt động trên các hình ảnh dạng lưới hai chiều, chúng ta sẽ trượt cửa sổ toán tử tích chập qua bề mặt hình ảnh, và tính toán một hàm số dựa trên cửa sổ trượt đó. Sau đó, kết quả được truyền qua nhiều lớp lần lượt. Nút trung tâm của pixel trung tâm sẽ tổng hợp thông tin từ các vùng lân cận xung quanh cũng như chính nó để tạo ra một giá trị mới.

Tuy nhiên, việc áp dụng CNN lên đồ thị gặp phải nhiều khó khăn do kích thước đồ thị không cố định và topô phức tạp, dẫn đến việc không có tính địa phương không gian. Thêm vào đó, vấn đề thứ tự của các nút cũng không cố định. Nếu lần đầu tiên chúng ta đánh nhãn các nút là A, B, C, D, E và lần sau là B, D, A, E, C, thì đầu vào của ma trận trong mạng sẽ thay đổi. Đồ thị là bất biến với thứ tự nút, do đó chúng ta muốn có cùng kết quả bất kể thứ tự sắp xếp các nút như thế nào.

10.7.2 Mạng nơ ron đồ thị tích chập

Ý tưởng về GraphSAGE

GraphSAGE (Hamilton et al, NIPS 2017) là một kỹ thuật học biểu diễn cho đồ thị động. Nó có thể dự đoán việc nhúng một nút mới mà không cần quy trình đào tạo lại. Để làm điều này, GraphSAGE sử dụng phương pháp học quy nạp. Nó tìm hiểu các hàm tổng hợp có thể tạo ra việc nhúng nút mới, dựa trên các tính năng và vùng lân cận của nút. Bất kể chức năng tổng hợp là gì, GraphSAGE hoạt động bằng cách tính toán các phần nhúng dựa trên đỉnh trung tâm và tập hợp các lân cận của nó (xem Công thức 10.7). Bằng cách bao gồm đỉnh trung tâm, nó đảm bảo rằng các đỉnh có các lân cận gần giống nhau sẽ có các phần nhúng khác nhau. GraphSAGE kể từ đó đã được các khung công tác khác vượt trội hơn về các điểm chuẩn được chấp nhận, nhưng khung này vẫn có tính cạnh tranh và có thể được sử dụng để khám phá khái niệm về trình tổng hợp đã học.[?]

$$h_i^k = \sigma(\mathbf{W}_{concat}(h_i^{k-1}, \text{aggregate}(h_j^{k-1} \forall j \in \mathcal{N}_{v_i}))) \quad (10.7)$$

Mạng đồ thị tích chập (GCN)

Một trong những phương pháp tích chập không gian phổ biến nhất là Mạng tích chập đồ thị (GCNs), tạo ra các phần nhúng bằng cách tính tổng các đặc điểm được trích xuất từ mỗi đỉnh lân cận và sau đó áp dụng tính phi tuyến tính. Những phương pháp này có khả năng mở rộng cao, cục bộ và hơn nữa, chúng có thể được ‘xếp chồng lên nhau’ để tạo ra các lớp trong CGNN. Mỗi tính năng này được chuẩn hóa dựa trên quy mô lân cận tương đối của đỉnh hiện tại và đỉnh lân cận, do đó đảm

bảo rằng các phần nhúng không 'bùng nổ' quy mô trong quá trình chuyển tiếp.

$$h_i^k = \sigma\left(\sum_{j \in \mathcal{N}_{v_i}} \frac{\mathbf{W}h_j^{k-1}}{\sqrt{|\mathcal{N}_{v_i}| |\mathcal{N}_{v_j}|}}\right) \quad (10.8)$$

Mạng chú ý đồ thị (GAT)

Mạng chú ý đồ thị (GATs) mở rộng GCN: thay vì sử dụng kích thước của các vùng lân cận để đánh giá tầm quan trọng của v_i đối với v_j , họ ngầm tính toán trọng số này dựa trên sản phẩm được chuẩn hóa của một cơ chế chú ý. Trong trường hợp này, cơ chế chú ý là phụ thuộc vào sự nhúng của hai đỉnh và cạnh giữa chúng. Các đỉnh bị hạn chế chỉ có thể quan tâm đến các đỉnh lân cận, do đó định vị được các bộ lọc. GATs được ổn định trong quá trình đào tạo bằng cách sử dụng sự chú ý và chính quy hóa nhiều đầu và được coi là ít tổng quát hơn MPNNs. Mặc dù GATs giới hạn cơ chế chú ý tới vùng lân cận trực tiếp, khả năng mở rộng sang các biểu đồ lớn không được đảm bảo vì các cơ chế chú ý có độ phức tạp tính toán tăng bậc hai với số đỉnh đang được xem xét.

$$h_i^k = \sigma\left(\sum_{j \in \mathcal{N}_{v_i}} a_{ij} \mathbf{W}h_j^{k-1}\right), \text{ trong đó } a_{ij} = \frac{e^{att}(h_i^{k-1}, h_j^{k-1}, e_{ij})}{\sum_{l \in \mathcal{N}_{v_i}} e^{att}(h_i^{k-1}, h_l^{k-1}, e_{il})} \quad (10.9)$$

Thú vị thay, tất cả các phương pháp này đều xem xét thông tin từ vùng lân cận trực tiếp và các nhúng trước đó, tổng hợp thông tin này theo một cách đối xứng nào đó, áp dụng các trọng số đã học để tính toán các đặc trưng phức tạp hơn, và 'kích hoạt' các kết quả này theo một cách nào đó để tạo ra một biểu diễn nhúng mà nắm bắt các mối quan hệ phi tuyến.

10.8 Thực nghiệm

Trong phần này, chúng tôi sẽ thực hiện việc xây dựng một cơ sở dữ liệu dạng đồ thị thông qua Neo4j, đồng thời xây dựng một mạng GCN và thử nghiệm tác vụ phân loại đỉnh (node classification) trên tập dữ liệu này.

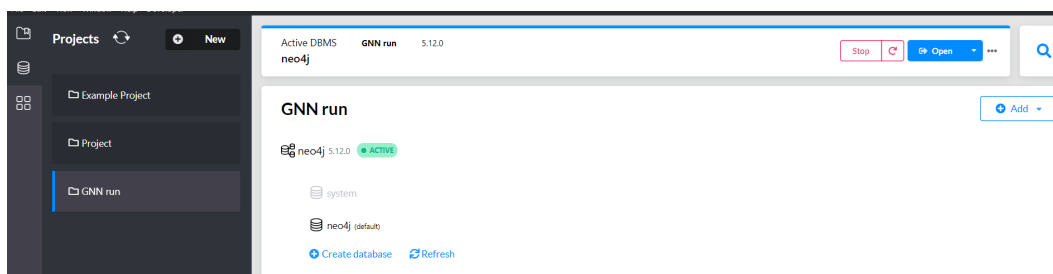
Chúng ta sẽ sử dụng tập dữ liệu *ogbn-arxiv*. Tập dữ liệu ogbn-arxiv là một đồ thị có hướng, biểu diễn mạng trích dẫn giữa tất cả các bài báo arXiv về Khoa học Máy tính được được lập chỉ mục bởi MAB. Mỗi nút là một bài báo arXiv và mỗi cạnh có hướng thể hiện một bài báo đang trích dẫn một bài báo khác. Mỗi bài báo là một vectơ đặc trưng 128 chiều thu được bằng cách tính trung bình mã nhúng của tiêu đề và phần tóm tắt của nó. Việc nhúng các từ riêng lẻ được tính toán bằng cách

chạy mô hình Skip-gram trên kho ngữ liệu MAG. Khi cung cấp một ánh xạ từ ID bài báo MAG sang dạng văn bản của tiêu đề và tóm tắt tại đây. Ngoài ra, tất cả các bài báo cũng được liên kết với năm mà bài báo tương ứng được xuất bản. Sau đó, xử lý dữ liệu và đưa vào hệ cơ sở dữ liệu thông qua framework Neo4j Graph Data Science và cài đặt một mạng GCN với framework PyTorch Geometric(PyG). Các bước thực hiện như sau:

- Sinh mẫu đại diện cho cơ sở dữ liệu đồ thị bằng cách áp dụng phép chiếu của Graph Data Science. Sử dụng thuật toán random walk with restarts (RWR) để lấy mẫu ngẫu nhiên các nút và cạnh từ đồ thị gốc, đảm bảo mẫu giữ được tính đặc trưng của đồ thị ban đầu.
- Chuyển đổi mẫu đồ thị đã tạo thành các DataFrame của Python để dễ dàng xử lý và phân tích dữ liệu. Điều này bao gồm việc xuất thông tin về các nút và cạnh của mẫu đồ thị.
- Xây dựng và huấn luyện một mô hình GCN trên mẫu đồ thị. Mô hình GCN sẽ học cách biểu diễn và phân loại các nút dựa trên đặc trưng của chúng và cấu trúc đồ thị.
- Sau khi huấn luyện, đánh giá hiệu suất của mô hình GCN trên tập dữ liệu kiểm thử. Sử dụng các chỉ số đánh giá như độ chính xác để xem mô hình phân loại các nút trong đồ thị mẫu như thế nào.

Trước khi bắt đầu, ta cần cài đặt hệ quản trị cơ sở dữ liệu từ Neo4j tại đây.

Sau khi cài đặt, khởi động Neo4j Desktop, tạo một project mới và tạo một database tên 'neo4j', đặt mật khẩu cho database và khởi động.



Hình 10.6: Giao diện Neo4j

Chuẩn bị dữ liệu

Bước 1: Sau khi khởi động database, tạo một thư mục graph_data bao gồm 2 file __init__.py và data_import.py. Trong đó, file data_import được viết như sau:

data_import.py

```
from typing import Tuple

from graphdatascience import GraphDataScience
from numpy.typing import ArrayLike
from ogb.nodeproppred import NodePropPredDataset
import pandas as pd
import numpy as np
from pandas import DataFrame

    %pip install graphdatascience python-dotenv ogb
    import pandas as pd
import numpy as np
from graphdatascience import GraphDataScience
from dotenv import load_dotenv
import os
from graph_data.data_import import get_ogbn_arxiv_data, chunks,
from numpy.typing import ArrayLike
```

Bước 2: Kết nối tới database đã được khởi động phía trên. Thông qua hàm get_ogbn_arxiv_data để thực hiện kéo dữ liệu trang web của stanford vào 2 data frame paper_source_df và citation_source_df. Trong đó paper_source_df chứa dữ liệu về các bài viết và citation_source_df chứa dữ liệu về các chú thích.

```

1 paper_source_df, citation_source_df = get_ogbn_arxiv_data(map_txt_properties=True, node_id_col_name='ogbld')
2 paper_source_df

```

Downloading <http://snap.stanford.edu/ogb/data/nodepropred/arxiv.zip>
Downloaded 0.08 GB: 100% [██████████] 81/81 [10:57<00:00, 8.12s/it]
Extracting dataset/arxiv.zip
Loading necessary files...
This might take a while.
Processing graphs...
100% [██████████] 1/1 [00:00<?, ?it/s]
Saving...

	paperId	title	abstract	ogbld	textEmbedding	year	subjectId	subjectLabel
0	630234	spreadsheets on the move an evaluation of mobi...	The power of mobile devices has increased dram...	104447	[0.010079000145196915, -0.028968000784516335, ...	2011	6	arxiv cs hc
1	803423	multi view metric learning for multi view vide...	Traditional methods on video summarization are...	15858	[-0.004714000038802624, 0.24545599520206451, ...	2014	16	arxiv cs cv
2	1102481	big data analytics in future internet of things	Current research on Internet of Things (IoT) m...	107156	[-0.015197999775409698, 0.10176599770784378, ...	2013	5	arxiv cs dc
3	1532644	machine learner for automated reasoning 0.4 an...	Machine Learner for Automated Reasoning (MaLAR...	141536	[-0.04284000024199486, -0.058010999113321304, ...	2014	24	arxiv cs lg
4	1810480	cryptographic hardening of d sequences	This paper shows how a one-way mapping using m...	82077	[-0.12808799743652344, -0.058931998908519745, ...	2011	4	arxiv cs cr
...	...	...	...	...	...	...	...	...
169338	3012554562	federated visual classification with real worl...	Federated Learning enables visual models to be...	109796	[-0.04787899926304817, 0.08063100278377533, -0...	2020	24	arxiv cs lg
169339	3012555423	kernel quantization for efficient network comp...	This paper presents a novel network compressio...	79475	[-0.11253000050783157, 0.07338699698448181, -0...	2020	24	arxiv cs lg

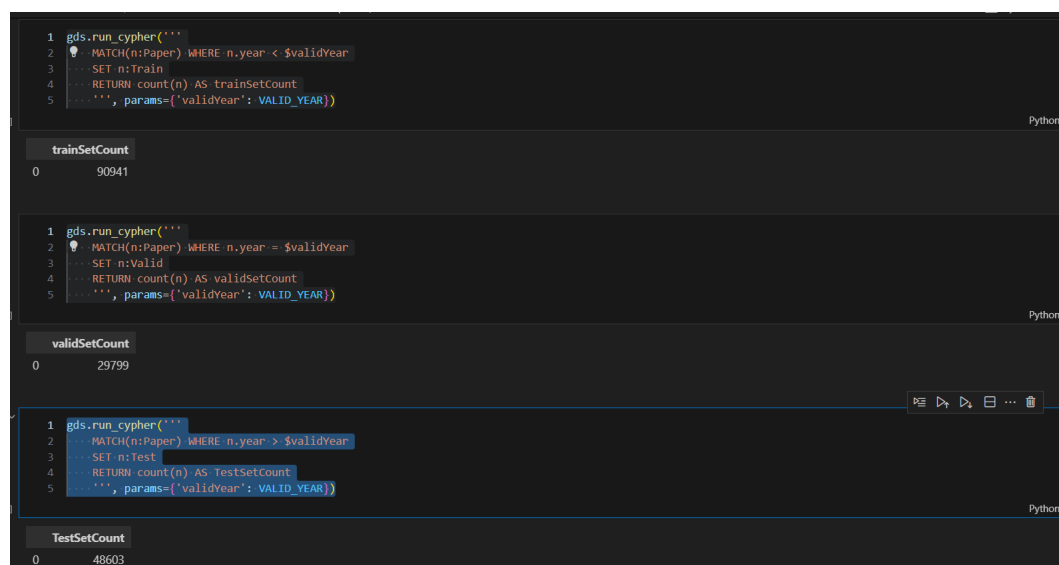
Hình 10.7: Kéo dữ liệu và liệt kê thử dữ liệu trong data frame paper_source_df

```

# Use Neo4j URI and credentials according to our setup
gds = GraphDataScience(
    'YOUR NEO4J URI',
    auth=(YOUR NEO4J USERNAME,
          YOUR NEO4J PASSWORD),
)

```

Bước 3: Ta chạy các câu truy vấn trên gds để thực hiện load dữ liệu từ các dataframe lên neo4j, thực hiện chia ra các bộ train, test và valid.



Hình 10.8: Thực hiện load dữ liệu tới neo4j

```

gds.run_cypher('CREATE INDEX year_ind .....
#TRAIN SET
gds.run_cypher(''
MATCH(n:Paper) WHERE n.year < $validYear
SET n:Train
RETURN count(n) AS trainSetCount
'', params={'validYear': VALID_YEAR})
#VALID SET
gds.run_cypher(''
MATCH(n:Paper) WHERE n.year = $validYear
SET n:Valid
RETURN count(n) AS validSetCount
'', params={'validYear': VALID_YEAR})
#TEST SET
gds.run_cypher(''
MATCH(n:Paper) WHERE n.year > $validYear
SET n:Test
RETURN count(n) AS TestSetCount
'', params={'validYear': VALID_YEAR})
gds.close()

```

Sau khi chuẩn bị dữ liệu, ta bắt đầu việc cài đặt và huấn luyện mô hình.

Bước 3.1: Đầu tiên, ta thực hiện cài các package và kết nối với database, thiết lập các biến cần thiết.

```

import pandas as pd
import numpy as np
from graphdatascience import GraphDataScience
from dotenv import load_dotenv
import os
import torch
from torch_geometric.data import Data
import torch.nn.functional as F
from torch_geometric.nn import GCNConv

load_dotenv('db-credentials.env', override=True)

# Use Neo4j URI and credentials according to our setup
gds = GraphDataScience(
    'YOUR NEO4J URI',
    auth=(YOUR NEO4J USERNAME,
          YOUR NEO4J PASSWORD))

# Necessary if you enabled Arrow on the db –
# AuraDS
gds.set_database("neo4j")
PROJ_NAME = 'proj'

RANDOM_SEED = 7474
VALID_YEAR = 2018
SAMPLE_RATIO = 0.34

```

Bước 3.2: Tiếp đến, ta thực hiện phép chiếu trên đồ thị tên PROJ_NAME sử dụng các nút Train, Valid, Test và quan hệ 'CITES' từ dữ liệu đã được chuẩn bị, đồng thời trích xuất các đặc tính 'textEmbedding', 'subjectID' và 'year'. Sau đó tạo một sample từ đồ thị đã chiếu tới, sử dụng thuật toán random walk.

```

%%time
g, _ = gds.graph.project(PROJ_NAME, ['Train', 'Valid', 'Test'], ...
                        nodeProperties=['textEmbedding', 'subjectId'])

#drop if the project is exist
SAMPLE_PROJ_NAME = PROJ_NAME + '_sample'
if gds.graph.exists(SAMPLE_PROJ_NAME)['exists']:
    gds.graph.get(SAMPLE_PROJ_NAME).drop()

%%time
g_sample, _ = gds.alpha.graph.sample.rwr(
    SAMPLE_PROJ_NAME, g,

```

```
samplingRatio=SAMPLE_RATIO,
restartProbability=0.05, nodeLabelStratification=True,
concurrency=1, randomSeed=RANDOM_SEED)
```

Bước 3.3: Tạo topology từ các quan hệ trên `g_sample`, truyền các thuộc tính của các nút tới các dataframe, với mỗi thuộc tính một bảng riêng. Sau đó, chỉnh lại ID các thuộc tính thành các số nguyên tuần hoàn, nhằm đánh chỉ số cho tensor trong PyG, đồng thời ánh xạ lại các thuộc tính với các ID đã được chỉnh sửa lại trên dataframe topology.

```
raw_sample_topology_df = gds.beta.graph.relationships.stream(g_
raw_sample_node_df = gds.graph.nodeProperties.stream(
g_sample,
[ 'subjectId ', 'year ', 'textEmbedding ' ],
separate_property_columns=True,)
sample_node_df = raw_sample_node_df.reset_index().rename(column
sample_topology_df = (raw_sample_topology_df
.merge(sample_node_df[[ 'neo4jNodeId ', 'nodeId ' ]], how='left ',
left_on='sourceNodeId ', right_on='neo4jNodeId ')
.drop(columns=[ 'sourceNodeId ', 'neo4jNodeId ' ])
.rename(columns={ 'nodeId ': 'sourceNodeId ' })
.merge(sample_node_df[[ 'neo4jNodeId ', 'nodeId ' ]], how='left ',
left_on='targetNodeId ', right_on='neo4jNodeId ')
.drop(columns=[ 'targetNodeId ', 'neo4jNodeId ' ])
.rename(columns={ 'nodeId ': 'targetNodeId ' })))
```

Bước 3.4: Trích xuất các chỉ số tensor của cạnh từ dataframe topology. Tensor này đóng vai trò định hình các liên kết đồ thị trong PyG. Ta chia tensor này ra 2 phần, `x` và `y` trận đồ thị trong đó mỗi hàng là một đặc trưng của một nút và `y` là vector target trong đó mỗi phần tử là một nhãn của nút.

```
sample_topology = sample_topology_df.by_rel_type()
edge_index = torch.tensor(sample_topology[ 'CITES' ], ...)

x = torch.tensor(np.stack(sample_node_df[ 'textEmbedding ' ]), ...)
y = torch.tensor(np.stack(sample_node_df[ 'subjectId ' ]), ...)
```

Bước 3.5: Tiếp đến, ta xây dựng object data chứa các dữ liệu bao gồm đặc trưng của nút, các nhãn và cấu trúc đồ thị và mask dữ liệu ứng với từng bộ train, test và valid.

```
data = Data(x=x, y=y, edge_index=edge_index)
data.train_mask = torch.tensor(np.stack(sample_node_df.year < V
```

```
data.val_mask = torch.tensor(np.stack(sample_node_df.year == VALID_YEAR))
data.test_mask = torch.tensor(np.stack(sample_node_df.year > VALID_YEAR))
```

Xây dựng mô hình

Bước 1: Như đã đề cập ban đầu, ta cài đặt một mô hình mạng GNN 2 lớp sử dụng các lớp GCNConv từ PyG. //

```
class GCN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = GCNConv(data.num_node_features, 72)
        self.conv2 = GCNConv(72, num_classes)

    def forward(self, data):
        x, edge_index = data.x, data.edge_index

        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, training=self.training)
        x = self.conv2(x, edge_index)

        return F.log_softmax(x, dim=1)
```

Bước 2: Ta thực hiện chạy huấn luyện trên tập dữ liệu sample có được bên trên. Đầu tiên, ta khởi tạo một thuật toán tối ưu Adam với learning rate=0.01. Quá trình huấn luyện sẽ diễn ra trong 200 epoch. Sau đó, ta kiểm tra đánh giá mô hình trên bộ dữ liệu test.

```
#OPTIMIZER INIT
optimizer = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=5e-5)
#MODEL TRAIN
model.train()
for epoch in range(200):
    optimizer.zero_grad()
    out = model(data)
    train_loss = F.nll_loss(out[data.train_mask], data.y[data.train_mask])
    valid_loss = F.nll_loss(out[data.val_mask], data.y[data.val_mask])
    test_loss = F.nll_loss(out[data.test_mask], data.y[data.test_mask])
    valid_loss.backward()
    optimizer.step()

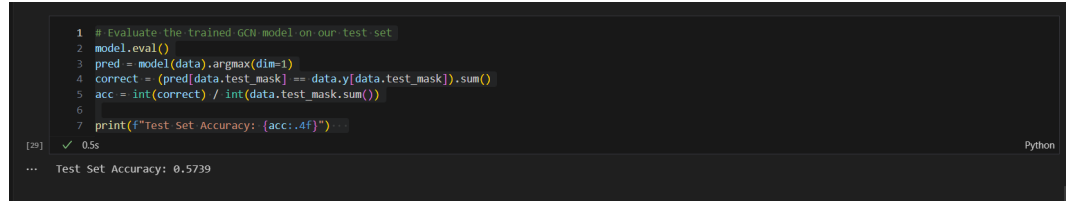
model.eval()
```

```

pred = model(data).argmax(dim=1)
correct = (pred[data.test_mask] == data.y[data.test_mask]).sum()
acc = int(correct) / int(data.test_mask.sum())

print(f"Test Set Accuracy: {acc:.4f}")

```



```

1 # Evaluate the trained GCN model on our test set
2 model.eval()
3 pred = model(data).argmax(dim=1)
4 correct = (pred[data.test_mask] == data.y[data.test_mask]).sum()
5 acc = int(correct) / int(data.test_mask.sum())
6
7 print(f"Test Set Accuracy: {acc:.4f}")

```

[29] ✓ 0.5s Python

... Test Set Accuracy: 0.5739

Hình 10.9: Kết quả đánh giá trên tập dữ liệu sample

Bước 3: Sau khi đánh giá trên tập sample, ta sẽ chạy mô hình trên toàn bộ dữ liệu ban đầu và kiểm tra tính chắc chắn của mô hình đối với dữ liệu lớn hơn.

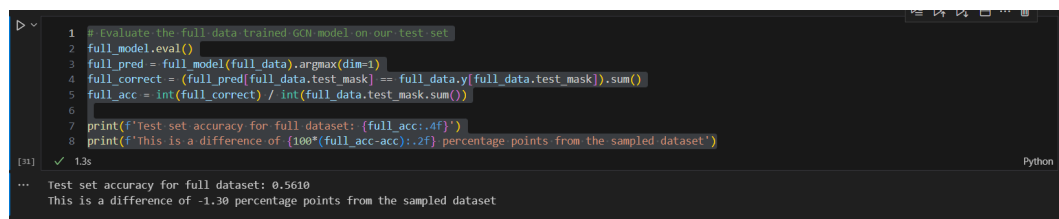
```

raw_topology_df = gds.beta.graph.relationships.stream(g)

raw_node_df = gds.graph.nodeProperties.stream(
    g,
    [ 'subjectId ', 'year ', 'textEmbedding' ],
    separate_property_columns=True,
)

print(f'Test set accuracy for full dataset: {full_acc:.4f}')
print(f'This is a difference of {100*(full_acc-acc):.2f} ')

```

```
1 # Evaluate the full_data trained GCN model on our test set
2 full_model.eval()
3 full_pred = full_model(full_data).argmax(dim=1)
4 full_correct = (full_pred[full_data.test_mask] == full_data.y[full_data.test_mask]).sum()
5 full_acc = int(full_correct) / int(full_data.test_mask.sum())
6
7 print(f'Test set accuracy for full dataset: {full_acc:.4f}')
8 print(f'This is a difference of {100*(full_acc-acc):.2f} percentage points from the sampled dataset')
```

[31] ✓ 1.3s Python

... Test set accuracy for full dataset: 0.5610
This is a difference of -1.30 percentage points from the sampled dataset

Hình 10.10: Kết quả đánh giá trên tập dữ liệu đầu đủ

Như vậy, thông qua phần thực nghiệm này ta đã thành công xây dựng một mạng GCN 2 lớp có khả năng thực hiện tác vụ phân loại với độ chính xác xấp xỉ 55% trên tập dữ liệu ogbn-axiv.

Tài liệu tham khảo

- [1] Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27, 2014.
- [2] Diederik P Kingma and Max Welling. Auto-encoding variational {Bayes}. In *Int. Conf. on Learning Representations*, 2014.
- [3] Pascal Vincent, Hugo Larochelle, Isabelle Lajoie, Yoshua Bengio, Pierre-Antoine Manzagol, and Léon Bottou. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *Journal of Machine Learning Research*, 11(12), 2010.

